

Go for C Programmers

Learn Go Fast by Building on What You Already Know

Written for dwang, with Devin as guide

2026

Contents

Preface	11
What this book is	11
The one big promise	11
The seven biggest shifts from C (your mental map)	11
A few things C programmers must <i>unlearn</i>	12
How the book is organized	12
Conventions	12
 Chapter 1 — Why Go for a C Programmer	 13
Where Go comes from	13
The Go philosophy (and how it differs from C's)	13
What Go removes (the relief)	14
What Go adds (the new tools)	14
Your first program: “Hello, World”	14
A slightly bigger taste	15
How a Go program is built and run	16
Is Go fast? (an honest answer)	17
When <i>not</i> to choose Go	17
Key takeaways	17
Watch out (gotchas for C programmers)	18
Interview questions	18
Try it	18
 Chapter 2 — Installing Go and the go Command	 19
Installing Go	19
Where Go keeps its files: GOROOT, GOPATH, GOBIN	19
The one tool: an overview	21
Running and building: run, build, install	21
Formatting and checking: gofmt, go fmt, go vet	22
Dependencies and modules: go mod, go get, go work	23
Inspecting and configuring: go env, go doc, go list, go version	24
Housekeeping and extras: go clean, go generate, go tool	25
The build cache and the module cache	25

Cross-compilation recap	26
Daily workflow	26
Key takeaways	26
Watch out (gotchas for C programmers)	26
Interview questions	27
Try it	27
Chapter 3 — Program Structure: Packages, Imports, and Visibility	27
A directory is a package	28
How a directory tree maps to packages	28
Imports	29
package main and func main	30
Exported vs unexported: capitalization is visibility	30
init and package initialization order	31
Circular imports are forbidden	32
How Go finds a package (brief)	33
Naming conventions	33
Key takeaways	33
Watch out (gotchas for C programmers)	33
Interview questions	34
Try it	34
Chapter 4 — Types, Variables, and Constants	34
Declaring variables	35
The basic types	36
Zero values	38
No implicit conversions	38
Constants	39
Named types vs type aliases	41
Printing and inspecting types	42
Key takeaways	43
Watch out (gotchas for C programmers)	43
Interview questions	43
Try it	44
Chapter 5 — Control Flow	44
if	44
for: the only loop	45
range	48
switch	50
goto and labels	51
A note on select	52
Key takeaways	52
Watch out (gotchas for C programmers)	52
Interview questions	53
Try it	53
Chapter 6 — Functions	54

Declaring a function	54
Multiple return values	55
Named return values and the naked return	56
Variadic functions	56
Functions are values	58
Closures	58
defer : cleanup that always runs	59
Recursion	64
Key takeaways	64
Watch out (gotchas for C programmers)	65
Interview questions	65
Try it	66
Chapter 7 — Pointers	66
The basics: <code>&</code> , <code>*</code> , and <code>*T</code>	66
The big difference: no pointer arithmetic	67
new (T): allocate a zeroed value	68
nil and the defined panic	68
Pass-by-value, always	69
Struct pointers: no <code>-></code> , just	69
Returning the address of a local is safe	70
When to use a pointer (and when not to)	71
unsafe.Pointer (briefly)	72
Key takeaways	72
Watch out (gotchas for C programmers)	72
Interview questions	73
Try it	73
Chapter 8 — Arrays, Slices, and Strings	74
Arrays: fixed size, and a value type	74
Slices: the workhorse	74
Slicing: sub-slices share memory	76
append: grow a slice (carefully)	77
copy: duplicate elements	79
Common slice operations	80
Passing slices to functions	80
Multidimensional data: slices of slices	81
Strings: immutable UTF-8 bytes	81
Key takeaways	84
Watch out (gotchas for C programmers)	84
Interview questions	84
Try it	85
Chapter 9 — Maps	85
Creating a map	86
Basic operations	86
A missing key returns the zero value	87
The zero value of a map is nil — and writing to it panics	87

Iteration order is randomized on purpose	88
Keys must be comparable	89
Maps are reference-like	89
Maps are not safe for concurrent use	90
The set idiom (Go has no built-in set)	90
Map values are not addressable	90
Key takeaways	91
Watch out (gotchas for C programmers)	92
Interview questions	92
Try it	93
Chapter 10 — Structs and Methods	93
Declaring a struct	93
Creating struct values	94
Memory layout: just like C	95
Comparing and copying structs	96
Methods	96
Methods on any named type	97
Constructors: a convention, not a keyword	98
Embedding: composition, not inheritance	98
Key takeaways	100
Watch out (gotchas for C programmers)	100
Interview questions	101
Try it	101
Chapter 11 — Interfaces	101
What an interface is	102
Implicit satisfaction: no <code>implements</code> keyword	102
An interface value is a <code>(type, value)</code> pair	103
The empty interface: <code>any</code>	103
Interfaces you will meet everywhere	104
Method sets: which types satisfy an interface	106
The nil-interface trap (the #1 interface bug)	107
The compile-time satisfaction check	108
Embedding interfaces	108
Key takeaways	108
Watch out (gotchas for C programmers)	109
Interview questions	109
Try it	110
Chapter 12 — Errors	110
An error is just an interface	110
The core idiom: check <code>err</code> immediately	111
Creating errors	112
Wrapping errors: adding context	113
<code>panic</code> , <code>recover</code> , and <code>defer</code>	115
Don't ignore errors	119
Key takeaways	119

Watch out (gotchas for C programmers)	120
Interview questions	120
Try it	121
Chapter 13 — Goroutines and the Scheduler	121
A goroutine is a cheap thread	121
Goroutines vs OS threads	123
The scheduler: G, M, and P	123
When does a goroutine yield? Scheduling points	125
The life of a goroutine	125
Stack growth	126
Goroutine leaks	126
Data races: shared memory is not safe by default	127
Key takeaways	128
Watch out (gotchas for C programmers)	128
Interview questions	129
Try it	129
Chapter 14 — Channels and select	130
Making, sending, and receiving	130
Unbuffered channels: a rendezvous	130
Buffered channels: a small queue	132
Closing a channel	133
Direction types in APIs	134
select: waiting on several channels	134
Deadlocks	137
Share memory by communicating	137
Key takeaways	138
Watch out (gotchas for C programmers)	139
Interview questions	139
Try it	140
Chapter 15 — Synchronization and context	140
Two toolboxes: channels vs the <code>sync</code> package	140
<code>sync.Mutex</code> : the lock you already know	141
<code>sync.WaitGroup</code> : wait for a set of goroutines	144
<code>sync.Once</code> : run something exactly once	145
<code>sync.Pool</code> : reuse allocations (brief)	146
<code>sync/atomic</code> : lock-free counters and flags	146
The Go memory model, in practice	147
<code>context.Context</code> : cancellation, deadlines, and request values	148
Key takeaways	150
Watch out (gotchas for C programmers)	151
Interview questions	151
Try it	152
Chapter 16 — Concurrency Patterns	152
The race detector: your most important tool	152

Pipelines	154
Fan-out and fan-in	155
Worker pool with bounded concurrency	157
Bounding parallelism with a semaphore channel	158
<code>errgroup</code> : goroutines that can fail	159
Rate limiting	160
Graceful shutdown: context + <code>WaitGroup</code>	161
Common mistakes	162
Recap: share memory by communicating	162
Key takeaways	163
Watch out (gotchas for C programmers)	163
Interview questions	163
Try it	164
Chapter 17 — Memory and the Garbage Collector	164
Stack vs heap	164
Escape analysis: the compiler decides stack or heap	165
The garbage collector	167
You cannot <code>free</code> , but you can help the GC	169
Measuring memory	170
Value semantics reduce GC pressure	171
Key takeaways	171
Watch out (gotchas for C programmers)	172
Interview questions	172
Try it	173
Chapter 18 — Packages and Modules	173
A module is a tree of packages	173
Anatomy of <code>go.mod</code>	173
Import paths	174
Adding and updating dependencies	175
Integrity and proxies	175
Special directories and directives	176
Project layout	177
Key takeaways	178
Watch out (gotchas for C programmers)	178
Interview questions	179
Try it	179
Chapter 19 — Generics	179
Life before generics	180
Type parameters	181
Constraints: describing the allowed types	182
Generic types	184
Generics vs interfaces: pick the right tool	185
The standard generic helpers	186
A short performance note	187
Key takeaways	187

Watch out (gotchas for C programmers)	188
Interview questions	188
Try it	189
Chapter 20 — A Tour of the Standard Library	189
Batteries included	189
<code>fmt</code> — formatted I/O	190
<code>strings</code> — text without <code><string.h></code>	191
<code>strconv</code> — numbers and strings	191
<code>bytes</code> — the <code>[]byte</code> twin of <code>strings</code>	192
<code>unicode</code> and <code>unicode/utf8</code>	192
<code>io</code> — the <code>Reader</code> and <code>Writer</code> interfaces	193
<code>bufio</code> — buffered I/O and scanning	193
<code>os</code> — talking to the operating system	194
<code>time</code> — dates, durations, and timers	194
<code>sort</code> and the generic <code>slices.Sort</code>	196
<code>encoding/json</code> — JSON in the standard library	196
<code>errors</code> — working with error values	197
<code>math</code> , <code>math/rand/v2</code> , and <code>crypto/rand</code>	197
<code>regexp</code> — regular expressions	198
<code>log</code> and structured <code>log/slog</code>	198
<code>flag</code> — command-line flags	199
<code>context</code> — cancellation and deadlines	199
The generic helpers: <code>slices</code> , <code>maps</code> , <code>cmp</code>	199
Key takeaways	200
Watch out (gotchas for C programmers)	200
Interview questions	201
Try it	201
Chapter 21 — Testing	201
C has no standard test framework; Go does	202
Tests live beside the code	202
Your first test	203
Table-driven tests with subtests	204
Running tests	204
Benchmarks	205
Fuzzing	206
Example functions are tests <i>and</i> documentation	207
Setup and teardown	207
<code>testdata/</code> and golden files	208
Test doubles via interfaces (no mocking framework)	208
Parallel tests	210
A note on <code>testify</code>	210
Key takeaways	210
Watch out (gotchas for C programmers)	211
Interview questions	211
Try it	212

Chapter 22 — Tooling: Format, Vet, Lint, Race, Profile, Debug	212
One toolbox instead of many	212
Formatting: <code>gofmt</code> , <code>go fmt</code> , <code>goimports</code>	212
Vetting: <code>go vet</code>	213
Linting: <code>staticcheck</code> and <code>golangci-lint</code>	213
The race detector	214
Profiling with <code>pprof</code>	214
The execution tracer: <code>go tool trace</code>	216
The debugger: Delve (<code>dlv</code>)	216
Compiler insight: <code>-gcflags=-m</code> and <code>go tool objdump</code>	217
<code>go generate</code>	217
<code>govulncheck</code> — vulnerability scanning	217
<code>gopls</code> — the language server	218
Key takeaways	218
Watch out (gotchas for C programmers)	218
Interview questions	218
Try it	219
Chapter 23 — Building Web Services with <code>net/http</code>	219
The smallest server	220
Handlers: the <code>http.Handler</code> interface	220
Modern routing: method and wildcard patterns	221
The request and the response	222
A complete JSON REST API	222
Middleware	226
Context and timeouts	227
Graceful shutdown	228
The HTTP client	229
How a request flows	230
Key takeaways	231
Watch out (gotchas for C programmers)	231
Interview questions	232
Try it	232
Chapter 24 — Building Command-Line Tools	233
Arguments: <code>os.Args</code>	233
The flag package	234
Subcommands (git-style)	236
Reading standard input	237
Configuration precedence	238
Handling Ctrl-C	238
A complete tool: <code>minigrep</code>	239
Popular libraries (and when you need them)	241
Distributing your tool	242
Key takeaways	242
Watch out (gotchas for C programmers)	242
Interview questions	243
Try it	243

Chapter 25 — Idioms and Style	243
The gofmt culture: one canonical format	244
Naming	244
Error-handling style	245
Accept interfaces, return concrete types	246
Design so the zero value is useful	247
The comma-ok idiom	248
Constructors: the NewX convention	248
The functional options pattern	248
Composition over inheritance	250
Avoid premature abstraction	250
Libraries return errors; they do not panic	251
Keep concurrency simple and document ownership	251
Pass <code>context.Context</code> first; never store it	251
Where the rules are written down	252
Key takeaways	252
Watch out (gotchas for C programmers)	252
Interview questions	253
Try it	253
Chapter 26 — Gotchas for C Programmers (the checklist)	253
Arrays, slices, and strings	254
Maps	256
Numbers	256
Declarations, scope, and visibility	258
Control flow	258
Functions and defer	260
Pointers and interfaces	261
Concurrency	262
Serialization	263
Key takeaways	263
Watch out (the one-line checklist)	264
Interview questions	264
Chapter 27 — Interview Questions and Answers	265
Language basics	265
Slices and strings	266
Maps	267
Structs and interfaces	268
Errors	269
Concurrency — goroutines and channels	269
Concurrency — synchronization and patterns	270
Runtime and memory	271
Modules and tooling	272
Predict the output	273
Design questions	276
Chapter 28 — Capstone Projects	277

Project 1 — a wc-style line counter	277
Project 2 — a JSON REST API for tasks	283
Project 3 — a bounded concurrent fetcher	293
Key takeaways	300
Watch out (gotchas for C programmers)	300
Interview questions	300
Appendix A — C to Go Cheat Sheet	301
Types	301
Declarations	302
Operators and keywords that differ	303
Control flow	303
Memory	304
Strings	304
C header to Go package	305
Idioms	306
The go command	307
Handy one-liners	308
Appendix B — Glossary	308
B	308
C	308
D	309
E	309
F	309
G	309
H	310
I	310
M	310
N	311
P	311
R	311
S	311
T	312
V	312
W	312
Z	313
References and Further Reading	313
Official documentation	313
Books	313
Practice	314
Tools	314
Community	314
Especially for C programmers	314

Preface

What this book is

This is a textbook for one specific reader: an **experienced C programmer** who needs to learn **Go** quickly and well. It does not assume you know any other language. It does assume you are comfortable with the things C teaches deeply: pointers, memory, the stack and the heap, structs, and how source files become a running program.

That C background is worth a lot. Go was created by people who wrote C and C++ for decades (some of them helped create Unix and C itself). Go *feels* like a cleaned-up systems language. So instead of starting from zero, this book starts from C and shows you the differences.

The writing is deliberately simple. Sentences are short. Every new word is explained the first time it appears. The goal is that the **ideas** are the only hard part — never the language the book is written in.

The one big promise

You already know most of Go. Variables, functions, structs, pointers, loops, integers, and the compile-and-run model are all here and behave much like C.

What is left to learn is a small set of *new ideas* and a small set of *changed rules*. This book is organized around exactly those. If we do our job, Go should feel less like a new language and more like “C, with the annoying parts removed and a few superpowers added.”

The seven biggest shifts from C (your mental map)

Keep these in mind. The whole book is really an expansion of this list.

1. **No manual memory management.** There is no `malloc` or `free`. A *garbage collector* frees memory for you. You still care about pointers and layout — you just stop tracking ownership and lifetimes by hand. (Chapter 17)
2. **Slices replace raw arrays + length.** The thing you reach for is not `T*` plus a separate `int len`. It is a *slice* (`[]T`): a small 3-field header (pointer, length, capacity) that the language manages and the GC frees. This is the #1 source of surprises for C programmers. (Chapter 8)
3. **No pointer arithmetic.** Pointers exist (`*T`, `&x`), but you cannot do `p + 1`. To walk memory, you index a slice. This removes a giant class of bugs. (Chapter 7)
4. **Errors are ordinary values, not exceptions.** A function that can fail returns an `error` as its last result, and you check it with `if err != nil`. There is no `try/catch`. (Chapter 12)
5. **Concurrency is built into the language.** `go f()` starts a lightweight thread (a *goroutine*). *Channels* pass data between goroutines safely. This is the main reason many teams pick Go. (Chapters 13–16)
6. **Interfaces are implicit.** A type satisfies an interface just by having the right methods — there is no `implements` keyword. Think of it as a type-safe, automatic version of a C struct full of function pointers (a *vtable*). (Chapter 11)

7. **One official style, enforced by tools.** `gofmt` formats every Go file the same way, so all Go code looks alike. Capitalization decides visibility (public vs private). The toolchain (build, test, format, vet) is one command: `go`. (Chapters 2 and 22)

If those seven sink in, everything else is detail.

A few things C programmers must *unlearn*

- **Zero values are real and useful.** A freshly declared variable is never garbage; it is set to a well-defined “zero” (0, "", `false`, or `nil`). Go code relies on this. (Chapter 4)
- **== on a struct compares all fields.** Assigning a struct copies it. There is no implicit reference like a C++ reference; if you want to share, pass a pointer.
- **Capitalization is not style — it is meaning.** `Name` is exported (public); `name` is not (package-private, like `static`). (Chapter 3)
- **There are no implicit numeric conversions.** You must write `int64(x)` explicitly. The compiler will not quietly mix `int` and `int64`. (Chapter 4)

How the book is organized

The chapters build on each other, but you do not have to read them all before writing code. The **Start Here** part (Chapters 1–7) plus the slices chapter (Chapter 8) is enough to be dangerous. The `README.md` lays out a **fast-track** reading order; follow it, then read the rest as your work demands.

Two reference sections are meant to stay open on your desk:

- **Appendix A — the C → Go cheat sheet:** a translation table for nearly every construct, plus the commands you’ll type all day.
- **Chapter 26 — gotchas:** the traps that catch C programmers, in one checklist.

Conventions

You will see short **callout boxes**. Each one always means the same thing:

Mental model. A quick analogy to build intuition.

C vs Go. A direct comparison with C.

Watch out. A common trap or surprise. Read these carefully.

Rule of thumb. Advice you can apply right away.

Deep dive. An optional, deeper aside. Safe to skip on the fast track.

Try it. A tiny exercise.

Code blocks are labeled by language — `go`, `c`, or `sh` (a shell command you type in a terminal). Most chapters end with **Key takeaways**, a **Watch out** checklist, and a few **Interview questions** with answers, so the book doubles as interview preparation.

Let’s begin with *why* Go exists, and what it feels like coming from C.

Chapter 1 — Why Go for a C Programmer

What you’ll learn. Why Go was created, how its philosophy compares to C’s, exactly what Go removes and adds relative to C, and how to read and run your first Go program. By the end you will understand the *shape* of a Go program and why it looks the way it does.

Where Go comes from

Go was created at Google around 2007 by Robert Griesemer, Rob Pike, and Ken Thompson. Those last two names matter to a C programmer: **Ken Thompson co-created Unix and the B language (C’s ancestor), and Rob Pike was a long-time Unix and systems person.** Go is what experienced C and C++ programmers built when they got to start over.

They had three complaints about large C++ codebases:

1. **Builds were slow.** A big C/C++ program could take many minutes to compile, largely because of how header files (`#include`) are re-parsed again and again.
2. **The language was huge and complex.** C++ had grown many features; reading someone else’s code was hard.
3. **Concurrency was painful.** Threads, locks, and callbacks were bolted on by libraries, not designed into the language.

Go is the answer to those three pains: **fast compiles, a small simple language, and concurrency built in** — while staying a *compiled, statically typed, native* language like C, not an interpreted one like Python.

Mental model. Go is “C for the network age”: compiled to a native binary, statically typed, and close to the machine — but with garbage collection, a huge standard library, and concurrency as a first-class feature.

The Go philosophy (and how it differs from C’s)

C’s philosophy is “trust the programmer”: the language is small and gives you sharp tools, including ones you can cut yourself with (undefined behavior, manual memory, pointer arithmetic).

Go’s philosophy is “**simplicity and one obvious way.**” The designers removed features on purpose. Some things that exist in C/C++ are deliberately *missing* from Go, because leaving them out makes large codebases easier to read and safer.

Topic	C	Go
Language size	Small core, but undefined behavior everywhere	Small core, defined behavior, few surprises
Who frees memory	You do (<code>malloc/free</code>)	A garbage collector does
Pointer arithmetic	Yes	No (use slices to walk memory)
Header files / preprocessor	Yes (<code>#include</code> , <code>#define</code>)	None (packages instead)
Build system	You write Makefiles	Built in (<code>go build</code>)
Formatting style	Up to you / the team	One official style (<code>gofmt</code>)
Error handling	Return codes / <code>errno</code>	error values, checked explicitly

Topic	C	Go
Concurrency	Library (pthreads)	Language (go, channels)
Generics	Macros / <code>void *</code>	Real generics (since Go 1.18)
Compile speed	Often slow on big projects	Very fast

Watch out. Go’s simplicity is sometimes *frustrating* at first. There is no `while`, no `?:` ternary, no exceptions, and (until recently) no generics. This is intentional. The payoff is that almost all Go code looks and reads the same, which matters enormously on a team.

What Go removes (the relief)

Coming from C, you get to stop doing several tedious or dangerous things:

- **Manual memory management.** No `malloc/free`, no “who owns this pointer?”, no use-after-free, no double-free, no leak hunts with Valgrind. (Chapter 17)
- **Pointer arithmetic and buffer overruns.** You index slices, which know their own length and are bounds-checked. A huge class of CVEs simply cannot happen.
- **Header files and the preprocessor.** No `#include`, no include guards, no `#define` macros, no separate declaration and definition. (Chapter 3)
- **Makefiles for building.** `go build` understands your project from the imports.
- **Most undefined behavior.** Integer sizes are fixed, evaluation order is defined, and uninitialized variables get a defined *zero value* instead of garbage.

What Go adds (the new tools)

In exchange, Go gives you features that C does not have built in:

- **Garbage collection (GC).** Memory is freed automatically. (Chapter 17)
- **Goroutines and channels.** Cheap concurrency: `go f()` starts a lightweight thread; channels pass values between them safely. (Chapters 13–16)
- **Slices, maps, and strings as first-class types.** Dynamic arrays, hash maps, and UTF-8 strings are built into the language, not bolted on. (Chapters 8–9)
- **Interfaces.** Type-safe polymorphism without inheritance. (Chapter 11)
- **A rich standard library.** HTTP servers, JSON, crypto, compression, and more ship with the language. You can build a real web service with zero dependencies.
- **One integrated tool.** `go` builds, tests, formats, vets, fetches dependencies, and more. (Chapter 2)

Your first program: “Hello, World”

Here it is in C and in Go, side by side.

```
/* hello.c - compile: cc hello.c -o hello && ./hello */
#include <stdio.h>

int main(void) {
    printf("Hello, World\n");
    return 0;
}
```

```
// hello.go - run: go run hello.go
package main

import "fmt"

func main() {
    fmt.Println("Hello, World")
}
```

Line by line, with the C version as our reference:

- **package main** — Every Go file belongs to a *package* (a named group of files). The special package **main** means “this builds into a runnable program,” and its **main** function is the entry point. There is no equivalent line in C; the file just *is* the translation unit.
- **import "fmt"** — Bring in the **fmt** (“format”) package from the standard library, Go’s rough equivalent of `<stdio.h>`. But notice: **you import a package, not a header file**. There is no separate declaration to keep in sync.
- **func main() { ... }** — A function named **main** that takes nothing and returns nothing. Note the keyword **func** and that the **type comes after the name** (more on that in Chapter 4). Unlike C, **main** does not return **int**; to exit with a status code you call **os.Exit(code)**.
- **fmt.Println("Hello, World")** — Call the **Println** (“print line”) function from the **fmt** package. It adds the newline for you. The capital P in **Println** means the function is *exported* (public) — a rule we’ll meet in Chapter 3.

Watch out. Go is strict in ways C is not. An **unused import** or an **unused local variable** is a *compile error*, not a warning. This feels harsh on day one and wonderful by week two: dead code never accumulates.

Run it without a separate compile step:

```
go run hello.go           # compiles to a temp binary and runs it
```

Or build a real, standalone binary (like **cc -o**):

```
go build hello.go         # produces ./hello, a single static executable
./hello
```

That binary is **statically linked** by default: it has no external **.so** dependencies and will run on another machine of the same OS/architecture with nothing else installed. This is one of Go’s most loved features for shipping software.

A slightly bigger taste

This program shows a few Go ideas at once: a slice (dynamic array), a **for/range** loop, multiple return values, and the zero value. Read the comments; we cover each in detail later.

```
package main

import "fmt"

// sum returns the total and the count of the numbers passed in.
// Note: TWO return values. This is normal and idiomatic in Go.
```

```

func sum(nums []int) (total int, count int) {
    for _, n := range nums { // range yields index, value; we ignore the index with _
        total += n
    }
    return total, len(nums)
}

func main() {
    nums := []int{3, 1, 4, 1, 5, 9} // a slice literal; no size, no malloc
    total, count := sum(nums)
    fmt.Printf("sum=%d count=%d avg=%.2f\n", total, count, float64(total)/float64(count))
}

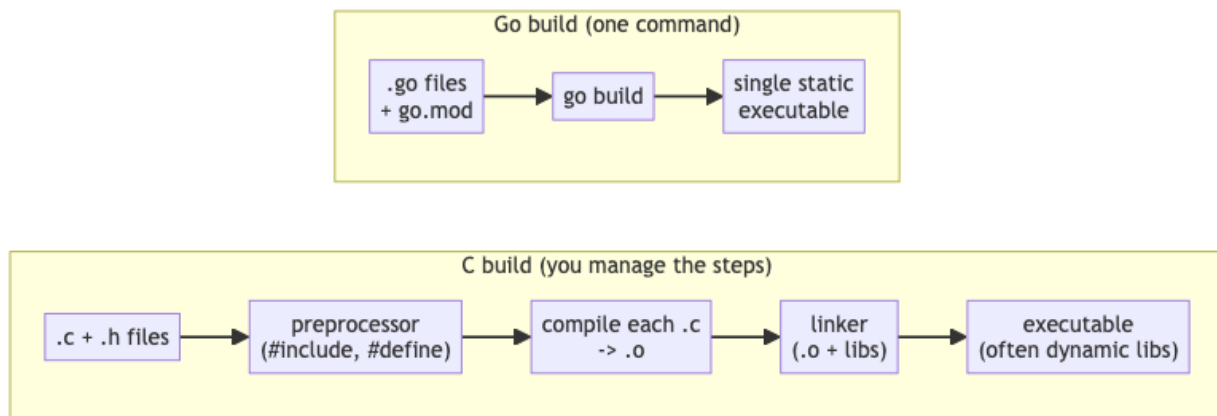
```

Things to notice compared to C:

- `nums := []int{...}` declares **and** initializes a slice. The `:=` operator means “declare a new variable and infer its type.” No size, no `malloc`, no `free`.
- `range` walks the slice and gives you each (index, value) pair. The blank identifier `_` throws away the index we don’t use.
- `sum` returns **two values**. The caller receives both with `total, count := sum(nums)`.
- `float64(total)` is an **explicit conversion**. Go will not divide an `int` by an `int` and silently give you a float, nor mix `int` and `float64`. You convert on purpose.

How a Go program is built and run

A C build has phases you manage: preprocess, compile each `.c` to an object file, then link the objects and libraries into a binary. In Go, the `go` tool does all of this from your imports — there is nothing to wire up by hand.



The same idea as a plain-text diagram (this is the ASCII style you’ll see when a picture is overkill):

```

C:  source  preprocess  compile (.o)  link  a.out  (you script this)
Go: source                go build                binary  (one step)

```

A few practical consequences:

- **Fast compiles.** Go does not re-parse headers for every file, and it tracks dependencies precisely, so even large projects build in seconds.

- **Easy cross-compilation.** Set two environment variables and you get a binary for another OS/CPU, no cross-toolchain to install:

```
GOOS=linux GOARCH=arm64 go build -o myapp .    # build a Linux/arm64 binary on a Mac
```

- **One binary to ship.** No “install these libraries first.” Copy the file and run.

Is Go fast? (an honest answer)

Go compiles to **native machine code**, like C. It is much faster than interpreted languages (Python, Ruby) and usually within a small factor of C for typical server and tool workloads. Programs start instantly and use memory predictably.

The trade-offs versus C:

- **Garbage collection** adds small, occasional pauses (usually well under a millisecond in modern Go) and uses some extra memory. For servers and tools this is a great deal; for hard real-time control loops it can matter. (Chapter 17)
- **Bounds checking** on slice access costs a tiny amount, which the compiler often removes when it can prove the index is safe.
- You give up the absolute lowest-level control (no manual memory layout tricks without the `unsafe` package, which you rarely need).

Rule of thumb. If you would have reached for C, C++, or a scripting language to write a **network service**, a **CLI tool**, or a **systems daemon**, Go is very often the better default in 2026: nearly C-like speed, far fewer footguns, and you ship a single binary.

When *not* to choose Go

Be honest about the edges. Go is a poor fit when you need:

- **Hard real-time guarantees** or sub-microsecond, jitter-free control (GC pauses, however small, are still pauses).
- **Tiny embedded targets** (a few KB of RAM, no OS) — that is still C’s domain (though TinyGo exists for some microcontrollers).
- **Manual control of memory layout and lifetime** down to the byte for a performance-critical kernel or allocator — C, C++, or Rust fit better.
- **A large existing C/C++ codebase** you must extend in place (though Go can call C via `cgo` when needed).

For almost everything else a backend or infrastructure engineer does day to day — APIs, microservices, CLIs, network daemons, data pipelines, DevOps tooling — Go is an excellent, productive choice. That is why Docker, Kubernetes, Prometheus, Terraform, and much of modern cloud infrastructure are written in Go.

Key takeaways

- Go was made by C/Unix veterans to fix three pains: slow builds, language complexity, and bolted-on concurrency.

- Go keeps C’s good parts (compiled, statically typed, native, close to the machine) and removes the dangerous, tedious parts (manual memory, pointer arithmetic, headers, undefined behavior).
- In return you get garbage collection, goroutines/channels, slices/maps/strings, interfaces, a big standard library, and one integrated `go` tool.
- A Go program is a set of *packages*; `package main` with a `main` function builds into a single, statically linked binary.
- `go run` runs code directly; `go build` produces a standalone executable.

Watch out (gotchas for C programmers)

- **Unused imports and unused local variables are compile errors**, not warnings.
- **`main` does not return `int`**. Use `os.Exit(code)` to set an exit status.
- **No implicit numeric conversions**. `float64(x)` and friends are required.
- The minimalism (no `while`, no ternary, no exceptions) is intentional; do not fight it.

Interview questions

Q: Why was Go created, and what problems does it solve? A: To address slow build times, language complexity, and awkward concurrency in large C++ codebases at Google. It provides fast compilation, a small and readable language, built-in concurrency (goroutines and channels), garbage collection, and a rich standard library, while remaining a compiled, statically typed, native language.

Q: How is Go different from C in terms of memory management? A: C requires manual `malloc/free` and has pointer arithmetic. Go uses a garbage collector to free memory automatically and forbids pointer arithmetic, eliminating whole classes of bugs (use-after-free, double-free, buffer overruns from pointer math). You still use pointers, but you do not manage lifetimes by hand.

Q: What does it mean that Go produces a “statically linked binary,” and why is that useful? A: By default `go build` bundles the runtime and all Go dependencies into one executable with no external shared-library requirements. You can copy that single file to another machine of the same OS/architecture and run it with nothing else installed — which makes deployment and containerization very simple.

Q: Name two things Go deliberately leaves out that C or C++ has, and why. A: Examples: pointer arithmetic (removed to prevent memory-safety bugs), the preprocessor/header files (replaced by packages to speed builds and avoid declaration drift), exceptions (replaced by explicit **error** values for clearer control flow), and implicit numeric conversions (removed to prevent silent bugs).

Try it

1. Save the “Hello, World” program and run it with `go run hello.go`.
2. Then `go build hello.go` and inspect the binary: `file ./hello` and `ls -lh ./hello`. Notice it is a native executable of a few megabytes (that size is the Go runtime baked in).
3. Cross-compile it for Linux: `GOOS=linux GOARCH=amd64 go build -o hello-linux hello.go`, then run `file ./hello-linux` to confirm the target.

Chapter 2 — Installing Go and the go Command

What you’ll learn. How to install Go and verify it, where Go keeps its files (GOROOT, the module cache, GOBIN), and a complete tour of the `go` command — `run`, `build`, `install`, `test`, `fmt`, `vet`, `mod`, `get`, `work`, `env`, `doc`, and the rest — each with a one-line purpose and a real example.

In C, the toolchain is a kit of separate parts you assemble yourself: a compiler (`gcc/clang`), a linker (`ld`), a build driver (`make`), a formatter (`clang-format`), a static checker, and some way to manage libraries. You wire them together with a `Makefile`.

Go ships **one** program — `go` — that does all of that. There is no `Makefile`, no separate formatter to install, no dependency manager to bolt on. Learning the subcommands of `go` is most of what “learning the Go toolchain” means.

Installing Go

Pick one of these. All of them give you the same `go` command.

Homebrew (macOS or Linux). The simplest path if you already use Homebrew:

```
brew install go
```

Official installer (macOS / Windows). Download the `.pkg` (macOS) or `.msi` (Windows) from <https://go.dev/dl/> and run it. It installs Go and adds it to your `PATH`.

Official tarball (Linux). Download, remove any old install, and unpack into `/usr/local`:

```
curl -LO https://go.dev/dl/go1.26.4.linux-amd64.tar.gz
sudo rm -rf /usr/local/go
sudo tar -C /usr/local -xzf go1.26.4.linux-amd64.tar.gz
export PATH=$PATH:/usr/local/go/bin # add this line to ~/.profile or ~/.bashrc
```

Then verify the install:

```
go version
# go version go1.26.4 darwin/arm64
```

That single line tells you the toolchain version (`go1.26.4`) and the host platform (`darwin/arm64` — macOS on Apple silicon). If `go version` prints a version, you are done; there is nothing else to configure to start building.

Deep dive. Since Go 1.21 the toolchain can manage *itself*: if a project’s `go.mod` asks for a newer Go than you have, the `go` command downloads and runs that exact version automatically. `GOTOOLCHAIN` controls this (`auto` by default; `local` forbids downloads). This is why teams no longer argue over “which Go version is everyone on.”

Where Go keeps its files: GOROOT, GOPATH, GOBIN

Three locations matter. You usually do not set any of them by hand, but you must know what they are.

- **GOROOT** is where Go *itself* lives: the compiler, the linker, the runtime, and the **standard library** source. On this machine it is `/opt/homebrew/Cellar/go/1.26.4/libexec`. The `go`

tool finds this on its own. Think of it as the rough equivalent of `/usr/lib/gcc/...` plus the system headers in `/usr/include` — the parts that come *with* the compiler.

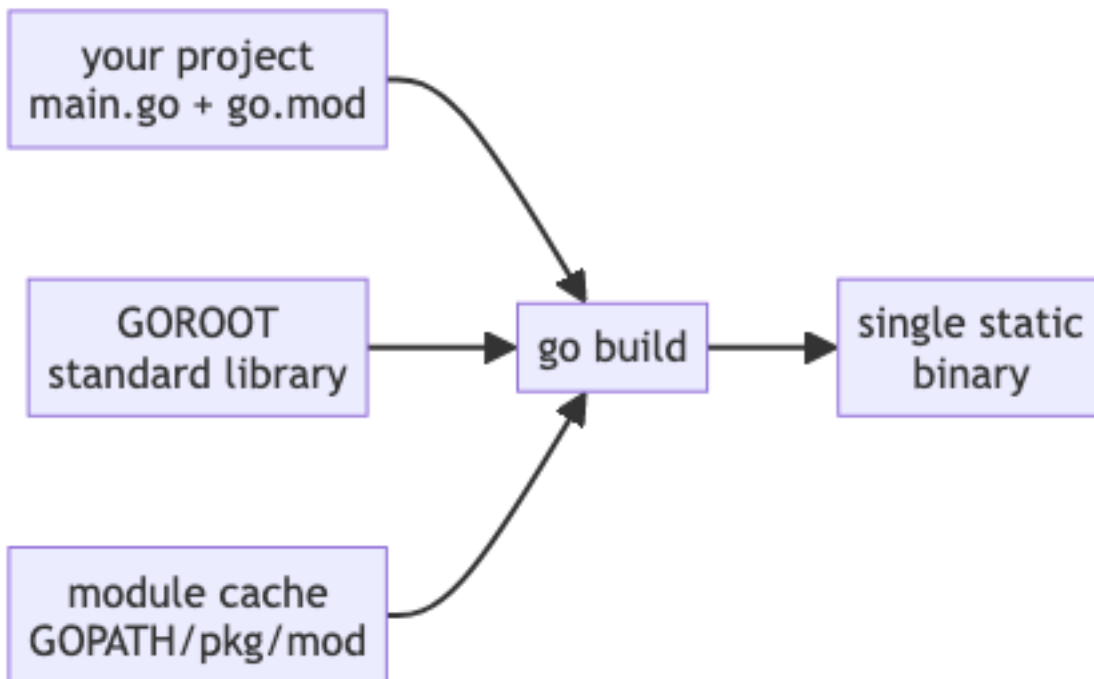
- **GOPATH** defaults to `$HOME/go`. Its modern job is to be a **cache and bin directory**, not a place for your source:
 - `$GOPATH/pkg/mod` — the **module cache**: the downloaded source of every third-party dependency, shared across all your projects.
 - `$GOPATH/bin` — where `go install` puts the executables it builds.
- **GOBIN** is where `go install` writes binaries. If it is empty (the common case), binaries go to `$GOPATH/bin`. Put that directory on your `PATH` so the tools you install are runnable:

```
export PATH=$PATH:$(go env GOPATH)/bin
```

Watch out. Old tutorials (pre-2018) say your code *must* live under `$GOPATH/src/...`. **That is no longer true.** Since Go modules (Go 1.11), your project can live in any directory you like; `GOPATH` is now just the cache and bin location. If a guide tells you to `mkdir -p $GOPATH/src/...`, ignore it.

C vs Go. `GOROOT` is like your compiler's own install tree (`/usr/lib/gcc`, `/usr/include`). The module cache (`$GOPATH/pkg/mod`) is like `/usr/include` and `/usr/lib` for *third-party* code — but per-user, versioned, and filled automatically instead of by `apt install libfoo-dev`.

Here is how the pieces fit together when you build:



The same idea as a plain-text map of the filesystem:

```
GOROOT  /opt/.../go/...      <- the compiler, runtime, and std library
GOPATH  ~/go
```

```

    pkg/mod/...      <- downloaded dependencies (the module cache)
    bin/...          <- binaries from `go install` (put on PATH)
GOCACHE ~/Library/Caches/go-build <- compiled object cache (speeds up builds)

```

```

your code: ~/projects/myapp/    <- ANYWHERE; just needs a go.mod
            go.mod
            *.go

```

`go build` reads the `import` lines in your `.go` files, pulls the standard library from `GOROOT` and any dependencies from the module cache, and links everything into one binary. You never list those inputs in a `Makefile`; the imports *are* the build description.

The one tool: an overview

Every command below is a subcommand of `go`. Here is the whole map; the rest of the chapter explains each one with an example.

Command	One-line purpose
<code>go run</code>	Compile and run in one step (no binary left behind).
<code>go build</code>	Compile to a binary (or just check that code compiles).
<code>go install</code>	Compile and install a binary into <code>GOBIN</code> .
<code>go test</code>	Build and run tests and benchmarks.
<code>go fmt / gofmt</code>	Reformat code to the one official style.
<code>go vet</code>	Report suspicious code the compiler still accepts.
<code>go mod</code>	Manage the module file (<code>init</code> , <code>tidy</code> , <code>download</code> , ...).
<code>go get</code>	Add, upgrade, or remove a dependency in <code>go.mod</code> .
<code>go work</code>	Manage a multi-module workspace.
<code>go env</code>	Read and set the toolchain's configuration.
<code>go doc</code>	Show documentation in the terminal.
<code>go list</code>	List packages or modules and their metadata.
<code>go clean</code>	Remove build artifacts and caches.
<code>go version</code>	Print the Go version (of the tool or of a binary).
<code>go generate</code>	Run code generators marked in source.
<code>go tool</code>	Run lower-level tools (<code>pprof</code> , <code>trace</code> , <code>objdump</code> , ...).

Running and building: `run`, `build`, `install`

`go run` compiles your code to a temporary binary, runs it, and throws the binary away. It is for quick iteration, like running a script.

```

go run .           # build and run the package in the current directory
go run ./cmd/server # build and run the package at that path
go run main.go     # also works for a single file

```

`go build` compiles and produces a binary, but does **not** run it. This is the closest match to `cc -o app *.c`.

```

go build .           # build current package; exe named after the directory
go build -o myapp .  # choose the output name explicitly (like cc -o)
go build ./cmd/server # build a specific package

```

The default executable name is the *directory* name (so a `main` package in `~/projects/myapp` builds to `./myapp`). Use `-o` to pick another name or output directory.

The pattern `./...` means “this package and every package below it.” It is the idiom for “do this to the whole project”:

```
go build ./...    # compile every package in the module
```

Watch out. When `./...` matches **more than one** package, `go build` compiles them only to *check that they build* and writes **no** executables. When it matches a single `main` package, it does write the binary. So `go build ./...` in a multi-package project is a fast “does everything still compile?” check, not a way to collect all your binaries — use `go install` for that.

`go install` compiles a `main` package and copies the resulting binary into `GOBIN` (or `$GOPATH/bin`), so it lands on your `PATH`.

```
go install .      # build the current command and put it on your PATH
```

`go install` is also how you install **command-line tools** written in Go. Give it a package path with a version suffix (`@latest`, `@v1.2.3`):

```
go install golang.org/x/tools/cmd/stringer@latest
go install honnef.co/go/tools/cmd/staticcheck@latest
```

The `pkg@version` form builds in isolation: it ignores your project’s `go.mod`, so installing a tool never changes your project’s dependencies.

C vs Go. `go run` is “compile-and-run” with nothing left over — there is no equivalent single C command; you would script `cc x.c -o /tmp/x && /tmp/x`. `go build` is `cc -o`. `go install` is roughly `cc -o tool && cp tool ~/bin/` — build and place it where the shell can find it.

`go test` builds and runs the tests in the current packages. We cover it in depth in Chapter 21 — Testing; the one line to remember now is:

```
go test ./...    # run all tests in the project
```

Formatting and checking: `gofmt`, `go fmt`, `go vet`

Go has exactly one official source format, and a tool that applies it. Style debates simply do not happen.

`gofmt` reformats Go source: tabs for indentation, consistent spacing, braces on the same line.

```
gofmt -l .        # -l: just LIST files that are not formatted (good for CI)
gofmt -w .        # -w: rewrite files in place
```

`go fmt` is a thin wrapper that runs `gofmt -l -w` on whole packages, taking package patterns instead of file paths:

```
go fmt ./...      # format every package in the project
```

`go vet` is a static analyzer. It reports code that compiles but is probably wrong: a `Printf` format string that does not match its arguments, a struct tag with a typo, a lock copied by value, an unreachable statement, and more.

```
go vet ./...
```

C vs Go. `gofmt` is `clang-format`, except there is nothing to configure — one style for the whole language. `go vet` is the compiler's `-Wall -Wextra` plus a linter, but it ships in the box. Run both before every commit; most teams enforce them in CI.

Dependencies and modules: `go mod`, `go get`, `go work`

A **module** is a collection of packages versioned together, described by a `go.mod` file at its root. (Modules and `go.sum` are covered fully in Chapter 18 — Packages and Modules; here we focus on the commands.)

`go mod` manages that file. Its subcommands:

Subcommand	What it does	Example
<code>init</code>	Create a new <code>go.mod</code> for this project.	<code>go mod init example.com/myapp</code>
<code>tidy</code>	Add missing requirements, drop unused ones, fix <code>go.sum</code> .	<code>go mod tidy</code>
<code>download</code>	Pre-download modules into the cache.	<code>go mod download</code>
<code>verify</code>	Check cached modules match <code>go.sum</code> (not tampered).	<code>go mod verify</code>
<code>why</code>	Explain why a package is in your build.	<code>go mod why golang.org/x/text</code>
<code>graph</code>	Print the full module dependency graph.	<code>go mod graph</code>
<code>edit</code>	Edit <code>go.mod</code> from a script (no text editor).	<code>go mod edit -go=1.26</code>

The two you will type most are `init` (once per project) and `tidy` (after you add or remove imports):

```
mkdir myapp && cd myapp
go mod init example.com/myapp  # creates go.mod
# ... write code that imports things ...
go mod tidy                    # sync go.mod/go.sum with your imports
```

`go get` changes your dependencies. Since Go 1.17 its job is *only* to edit the module requirements in `go.mod` and download source — it **no longer builds or installs binaries** (that moved to `go install`).

```
go get example.com/pkg          # add it (or move to its latest version)
go get example.com/pkg@v1.4.2   # pin an exact version
go get -u ./...                 # upgrade dependencies to newer minor/patch
go get example.com/pkg@none     # remove the dependency
```

Watch out. If you learned Go years ago, `go get` used to download *and* install a tool into your bin. That is gone. Today: **`go get` = manage dependencies in `go.mod`; `go install pkg@version` = install a tool.** Mixing these up is the single most common command-line confusion for returning users.

go work sets up a **workspace**: a way to develop several modules at once without publishing them or editing each `go.mod`. Use it when you are changing a library and an app that depends on it side by side.

```
go work init ./app ./lib    # create go.work listing two local modules
go work use ./tool          # add another local module to the workspace
```

With a `go.work` file present, builds in `./app` use your local `./lib` instead of a downloaded version — handy for editing both at the same time.

C vs Go. `go.mod` is your dependency manifest (there is no real C equivalent; you would track versions by hand or rely on the system package manager). `go mod tidy` is “figure out exactly which libraries this code needs and pin them.” A workspace (`go work`) is like building several local libraries from a shared checkout instead of against the installed `.so` versions.

Inspecting and configuring: `go env`, `go doc`, `go list`, `go version`

go env prints the toolchain’s configuration. With no argument it prints everything; with names it prints just those values:

```
go env GOPATH GOOS GOARCH
go env -w GOFLAGS=-mod=vendor # persist a setting for future commands
go env -u GOFLAGS             # undo that persisted setting
```

Variables worth knowing:

Variable	Meaning
GOOS / GOARCH	Target operating system / CPU for the build.
GOMODCACHE	Path of the module cache (default <code>\$GOPATH/pkg/mod</code>).
GOCACHE	Path of the build (object) cache.
GOPROXY	Where modules are downloaded from (default <code>proxy.golang.org</code>).
GOFLAGS	Flags added to every <code>go</code> command automatically.
GOBIN	Where <code>go install</code> writes binaries.

C vs Go. `go env` replaces the pile of environment variables and `Makefile` settings you would manage in C (`CC`, `CFLAGS`, `LDFLAGS`, `PKG_CONFIG_PATH`). `go env -w` writes them to a config file (`go/env`) so they persist without editing your shell profile.

go doc shows documentation right in the terminal, straight from the source comments:

```
go doc fmt                # package summary for fmt
go doc fmt.Println        # docs for one function
go doc -src strings.Trim # also show the source
```

The website <https://pkg.go.dev> is the same documentation, rendered for the browser, for every public module.

go list reports metadata about packages and modules — useful in scripts:


```

go list ./...           # list the import paths of all packages here
go list -m all          # list this module and all its dependencies

go version prints the tool's version, and can also tell you which Go built a given binary:

go version              # the installed toolchain
go version ./myapp      # which Go compiled this executable

```

Housekeeping and extras: `go clean`, `go generate`, `go tool`

`go clean` removes build outputs and, with flags, the caches:

```

go clean                # remove build artifacts in the current package
go clean -cache         # wipe the build (object) cache
go clean -modcache      # wipe the entire module cache (forces re-download)

```

`go generate` runs commands written as special comments in your source (`//go:generate ...`). It is never run automatically; you invoke it on purpose, usually to regenerate generated code:

```
go generate ./...
```

`go tool` runs the lower-level programs that ship with Go. The performance and debugging tools live here:

```

go tool pprof cpu.prof  # analyze a CPU/memory profile
go tool trace trace.out # explore an execution trace
go tool objdump -s main ./app # disassemble a function

```

These are the heart of Go's profiling story; Chapter 22 — Tooling covers `pprof`, `trace`, the race detector, and friends in detail.

The build cache and the module cache

Two caches make Go builds fast and repeatable. You rarely touch them, but you should know they exist.

- **Build cache** (GOCACHE, e.g. `~/Library/Caches/go-build`): compiled package objects and test results. Because Go caches per-package compilation, a second build only recompiles what changed. This is why even a clean checkout builds in seconds and a rebuild is near-instant.
- **Module cache** (GOMODCACHE, default `$GOPATH/pkg/mod`): the *source* of every downloaded dependency, at every version you have used, shared across all your projects. It is read-only and verified against `go.sum`, so two projects that need the same library version reuse one copy.

C vs Go. The build cache is like `ccache` — but built in and on by default, so you never set it up. The module cache is like a per-user, versioned `/usr/include` + `/usr/lib` for third-party code, filled automatically by the build instead of by `apt/dnf`. Builds are **incremental** without any `Makefile` rules describing what depends on what; the tool derives that from the imports.

Cross-compilation recap

Because the Go toolchain contains everything and links statically by default, building for another OS or CPU is two environment variables — no cross-toolchain to install (you do not need a separate `aarch64-linux-gnu-gcc`):

```
GOOS=linux GOARCH=arm64 go build -o app-linux-arm64 .
GOOS=windows GOARCH=amd64 go build -o app.exe .
go tool dist list          # show every GOOS/GOARCH pair Go supports
```

Set `CGO_ENABLED=0` to force a pure-Go build with no `libc` dependency — the result runs on a bare Linux container with nothing else installed.

Daily workflow

In practice you type a handful of commands all day. Here they are in one box:

```
go run .          # try the program right now
go build ./...    # does the whole project still compile?
go test ./...     # run all the tests
gofmt -w .        # (or `go fmt ./...`) format everything
go vet ./...      # catch suspicious code
go mod tidy       # sync dependencies after changing imports
go get pkg@latest # add or upgrade a dependency
go doc somepkg    # look up an API without leaving the terminal
```

Master those eight and you can work in any Go codebase. Everything else is for a specific occasion.

Key takeaways

- One program, `go`, replaces the whole C kit: compiler, linker, `make`, package manager, formatter, and static checker. There is no `Makefile`.
- `go version` verifies the install. `GOROOT` holds Go itself and the standard library; `GOPATH` (default `~/go`) holds the **module cache** (`pkg/mod`) and installed binaries (`bin`); `GOBIN` overrides where binaries go.
- Since modules, your code lives **anywhere** with a `go.mod` — not under `$GOPATH/src`. Ignore tutorials that say otherwise.
- `go run` runs, `go build` compiles, `go install` installs a binary; `go install pkg@version` installs tools. `go get` only edits dependencies now — it no longer installs binaries.
- `go mod tidy`, `gofmt`, and `go vet` are part of everyday work. `go env` reads/writes config; `go doc` and `pkg.go.dev` are your documentation.
- The build cache and module cache make builds incremental and fast with zero setup. Cross-compiling is just `GOOS/GOARCH`.

Watch out (gotchas for C programmers)

- `go get` does not install programs anymore (since Go 1.17). Use `go install pkg@version` for tools; use `go get` only to change `go.mod`.
- `$GOPATH/src` is **obsolete**. Modules freed your code from living in a fixed directory. Many old blog posts still assume the `GOPATH` layout.

- **go build ./... may produce no binaries.** When it matches several packages it only checks that they compile. Use `-o` on a single package, or `go install`, to actually get an executable.
- **Put `$(go env GOPATH)/bin` on your PATH,** or tools you `go install` will build successfully yet seem to “disappear.”
- **gofmt is not optional in practice.** Submitting unformatted code will fail most teams’ CI. Configure your editor to run it on save.

Interview questions

Q: What is the difference between `go build`, `go run`, and `go install`? A: `go run` compiles to a temporary binary, runs it, and discards it — good for quick iteration. `go build` compiles and writes a binary (or, for multiple packages, just checks that they compile) but does not run it. `go install` compiles a command and places the binary in `GOBIN/$GOPATH/bin` so it is on your PATH.

Q: What do `GOROOT` and `GOPATH` mean today, and how has `GOPATH`’s role changed? A: `GOROOT` is where the Go toolchain and standard library are installed. `GOPATH` (default `~/go`) used to be where *all* your source had to live, under `$GOPATH/src`. Since modules (Go 1.11), your code can live anywhere with a `go.mod`; `GOPATH`’s modern role is just to hold the module cache (`pkg/mod`) and binaries from `go install` (`bin`).

Q: Since Go 1.17, what does `go get` do, and how do you install a CLI tool? A: `go get` only adds, upgrades, downgrades, or removes module dependencies in `go.mod` (and downloads their source). It no longer builds or installs executables. To install a tool, run `go install path@version`, for example `go install golang.org/x/tools/cmd/stringer@latest`.

Q: How does Go build for another OS or architecture without a cross-toolchain? A: The Go toolchain includes code generators for every supported target and links statically by default, so you only set `GOOS` and `GOARCH`, e.g. `GOOS=linux GOARCH=arm64 go build`. With `CGO_ENABLED=0` the result has no libc dependency and runs on a bare system.

Q: Why are Go builds fast even without a Makefile? A: The `go` tool derives the dependency graph from the `import` statements, so it knows exactly what to compile and in what order. It caches compiled packages in the build cache (`GOCACHE`) and stores downloaded dependency source in the module cache, so rebuilds only recompile what actually changed.

Try it

1. Run `go version` and `go env GOROOT GOPATH GOMODCACHE GOCACHE`. Note where each path points on your machine.
2. Make a scratch project: `mkdir play && cd play && go mod init play`, write a tiny `main.go`, then run it with `go run .` and build it with `go build -o play ..`
3. Install a tool: `go install golang.org/x/tools/cmd/stringer@latest`, then check it landed with `ls $(go env GOPATH)/bin`. Add that directory to your PATH if `stringer` is not found.

Chapter 3 — Program Structure: Packages, Imports, and Visibility

What you’ll learn. How Go organizes code into *packages* instead of headers and translation units, how `import` works, how `package main` becomes a program, how cap-

italization decides what is public, and how `init` and package initialization order work — all compared directly to C.

In C, the unit of compilation is the **translation unit**: one `.c` file plus the headers it `#includes`, compiled to one `.o`. Files do not see each other’s functions unless you share a *declaration* through a header, and you fight a constant battle to keep that header in sync with the definition.

Go throws all of that out. The unit is the **package**, there are no headers, and the compiler reads a whole package at once. This chapter is about that new shape.

A directory is a package

The rule is simple: **one directory is exactly one package**. Every `.go` file in that directory starts with the same `package` clause, and all of them share a single scope. A function in one file can call a function, or use a type, defined in another file of the same directory — with no declaration, no header, no `extern`.

```
// file: shapes/area.go
package shapes

func Area(r float64) float64 {
    return pi * r * r // pi is defined in another file, below - this is fine
}

// file: shapes/constants.go
package shapes

const pi = 3.14159
```

Both files say `package shapes`. Together they *are* the package. `Area` uses `pi` even though `pi` is declared in a different file — and even though, textually, it appears “later.” That is allowed because:

Declaration order does not matter. The compiler first collects every top-level name in the whole package, then checks the bodies. You can call a function before it is defined, reference a type declared at the bottom of the file, or split related declarations across files in any order.

C vs Go. In C you must *forward-declare*: either order your definitions so each is declared before use, or put prototypes in a header. Get it wrong and you get “implicit declaration” warnings or errors. In Go there are no prototypes and no ordering rules for top-level declarations — the problem simply does not exist.

Mental model. Think of a package as one big source file that the compiler stitches together from all the `.go` files in the directory. The file boundaries are for *you*, to organize code; the compiler barely cares about them.

How a directory tree maps to packages

Each directory is one package; subdirectories are *separate* packages, not “sub-packages” in any special sense. Here is a small module and the package each file belongs to:

myapp/	module example.com/myapp	(named in go.mod)
go.mod		
main.go	package main	

```

server/
  server.go          package server
  routes.go          package server    (same dir => same package)
internal/
  store/
    store.go         package store
    cache.go         package store

```

`server.go` and `routes.go` share one scope because they are in the same directory. `store/` is a different package, reachable by the import path `example.com/myapp/internal/store`. (The special `internal/` directory limits who may import it; see Chapter 18 — Packages and Modules.)

Imports

After the `package` clause, a file lists the packages it uses. A single import:

```
import "fmt"
```

Or, idiomatically, a grouped block (which `gofmt` keeps sorted):

```

import (
    "fmt"
    "os"
    "strings"
)

```

Import path vs package name. The string in quotes is the *import path* — how Go locates the code. The identifier you actually type is the *package name*, declared by that package's own `package` clause. Usually the last element of the path is the package name ("`math/rand`" is used as `rand`), but not always, so do not assume.

Aliased imports rename a package locally — to resolve a clash or to shorten a name:

```

import (
    "math/rand"
    crand "crypto/rand" // both packages are named rand; alias one of them
)

```

Blank imports import a package *only for its side effects*. You write `_` as the name, do not reference the package, and rely on its `init` functions running (see below). A classic case is registering a driver or decoder:

```
import _ "github.com/lib/pq" // registers the Postgres driver with database/sql
```

Dot imports (`import . "fmt"`) dump a package's exported names into your file so you can write `Println` instead of `fmt.Println`. They are **discouraged**: they hide where each name comes from and invite clashes. You will mainly see them in some test files, rarely elsewhere.

Recall from Chapter 1 — Why Go for a C Programmer: an **unused import is a compile error**, not a warning. Dead imports never pile up.

C vs Go. `#include "foo.h"` textually pastes declarations into your file, which is why you need include guards and why order can matter. `import "foo"` references an

already-compiled package by path. There is no textual paste, no guard, no double-inclusion problem, and the order of imports is irrelevant.

package main and func main

One package name is special. A package named **main** that contains a function **func main()** (no parameters, no return value) builds into a runnable executable; **main** is the entry point.

```
package main

import "fmt"

func main() {
    fmt.Println(banner())
}
```

Any other package name is a **library**: it has no **main** and cannot be run directly, only imported.

A main package may be **split across several files**, just like any package. All of them say **package main** and share scope:

```
// file: banner.go
package main

func banner() string { return "myapp v1.0" }
```

`banner()` lives in `banner.go`; `main()` in `main.go`; both are `package main`, so `main` can call `banner` with no ceremony.

Exported vs unexported: capitalization is visibility

This is one of Go's most important rules, and it surprises every C programmer:

An identifier whose name starts with an uppercase letter is *exported* (visible to other packages). A lowercase first letter is *unexported* (visible only inside its own package). Spelling — not a keyword — controls visibility.

This applies to **everything** declared at package level *and* to struct fields and methods: functions, types, variables, constants, fields, methods.

```
package user

type User struct {
    Name string // exported: other packages can read and set it
    email string // unexported: only package user can touch it
}

func New(name string) *User { return &User{Name: name} } // exported
func validate(u *User) bool { return u.Name != "" }       // unexported helper
```

Concept	C	Go
Hidden from other files/packages	<code>static int x;</code>	<code>var x int</code> (lowercase)
Visible to other files/packages	<code>int x;</code> + <code>extern</code> in a header	<code>var X int</code> (uppercase)
Granularity of “private”	per file (translation unit)	per package (all its files)
How you choose	the keyword <code>static</code>	the case of the first letter

C vs Go. Lowercase is Go’s `static`, but with a key difference. C’s `static` hides a name from *other files*. Go’s lowercase hides a name from *other packages* — yet **every file in the same package can still see it**. There is no file-level privacy in Go; the package is the privacy boundary.

Watch out. Capitalization is *meaning*, not style. Renaming `Total` to `total` is not cosmetic — it makes the name disappear from every other package and will break their code. Also, libraries like `encoding/json` can only see **exported** fields, so an unexported `email` field will silently not be included when you marshal a `User` to JSON.

init and package initialization order

Before `main` runs, Go initializes packages in a precise, defined order. Within a single package it goes:

1. **Imported packages first.** Every package you import is fully initialized before yours — dependencies before dependents.
2. **Package-level variables next**, in *dependency* order. Go works out which variable’s initializer uses which other variable and runs them in the right order, not merely top to bottom.
3. **init functions last.** Each `init` runs after the variables are set, in the order the source files are presented to the compiler (alphabetical by filename), and top to bottom within a file.

A special function `func init()` takes no arguments and returns nothing. You **cannot call it yourself**, and a package (even a single file) may define **several**. Use it to register things, build lookup tables, or validate configuration.

```
package main

import "fmt"

var config = loadDefaults() // (2) package var: initialized before any init

func loadDefaults() map[string]string {
    return map[string]string{"mode": "fast"}
}

func init() { // (3) runs after config exists, before main
    config["mode"] = "safe"
}

func main() { // (4) runs last
```

```

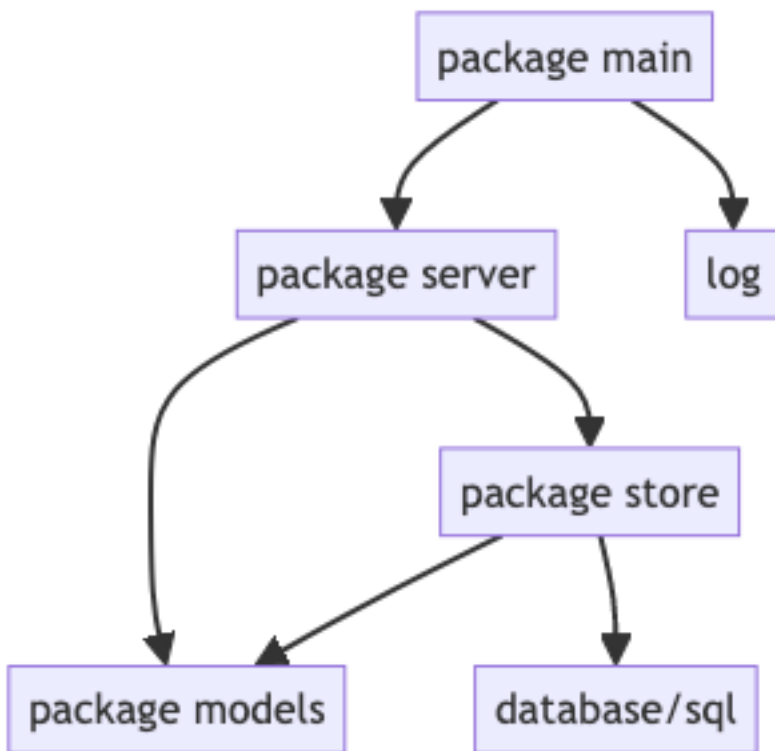
fmt.Println(config["mode"]) // prints: safe
}

```

C vs Go. C has no portable “run this before `main`.” Compilers offer `__attribute__((constructor))`, and C++ has static-object constructors, but the order *across* translation units is unspecified and a frequent source of bugs (the “static initialization order fiasco”). Go specifies the order exactly and portably, and a blank import (`import _ "pkg"`) is the standard way to *trigger* another package’s `init`.

Circular imports are forbidden

If package A imports B, then B (directly or through a chain) must not import A. Try it and the build stops with `import cycle not allowed`.



This graph is a **DAG** — a directed graph with no cycles — and Go *requires* it to be one. That is a feature, not a limitation:

- It keeps dependencies easy to reason about (arrows only point “downhill”).
- It lets the compiler build packages in dependency order, in parallel, and cache them — a big reason Go builds so fast.

C vs Go. C happily lets two headers include each other as long as include guards stop infinite recursion, so cyclic dependencies between modules are common and tolerated. Go forbids them outright. When you hit a cycle, fix the *design*: move the shared types into a third, lower-level package that both import, or define an interface in the consumer so the dependency points only one way (see Chapter 11 — Interfaces).

How Go finds a package (brief)

An import path such as `example.com/myapp/internal/store` is the **module path** from your `go.mod` plus the subdirectory inside the module. Standard-library paths like `fmt` and `net/http` resolve to the standard library in `GOROOT`; third-party paths resolve through the module cache. The full story — modules, versions, `go.sum`, and `internal/` — is in Chapter 18 — Packages and Modules.

Naming conventions

Because the package name prefixes everything you use from it, names are chosen to read well *at the call site*.

- **Package names are short, all-lowercase, single words** — no underscores, no camelCase: `http`, `strconv`, `io`, `bytes`, `time`.
- **Avoid stutter.** The package name is already part of the qualified name, so do not repeat it. Name the type `Server`, not `HTTPServer`, because callers write `http.Server`. Likewise `bytes.Buffer` (not `BytesBuffer`) and `errors.New` (not `errors.NewError`).
- **Avoid grab-bag names** like `util`, `common`, or `helpers`; they attract unrelated code. Name a package for what it provides (`tokenize`, `retry`, `store`).

Rule of thumb. Read the name as the caller will: `package user` with a type `User` reads `user.User`, which stutters — prefer `user.Account` or just put the type in a better-named package. Design the *qualified* name, not the bare one.

Key takeaways

- A package is **one directory**; all its `.go` files share one scope. There are no headers, no `#include`, and no forward declarations.
- **Declaration order does not matter** within a package; the compiler reads the whole package before checking bodies.
- `import` references a compiled package by its *path*; the *package name* is what you type. Imports can be aliased, blank (`_`, for side effects via `init`), or (rarely) dot-imported.
- `package main + func main()` builds a runnable program, and a `main` package can span several files.
- **Capitalization is visibility:** uppercase = exported (public), lowercase = unexported (package-private). It applies to funcs, types, vars, consts, fields, and methods.
- Package init order is defined: imports, then package variables (in dependency order), then `init` functions, then `main`.
- Circular imports are forbidden, which keeps the dependency graph a clean DAG.

Watch out (gotchas for C programmers)

- **There is no file-level privacy.** `static` hides a name from other *files*; Go's lowercase hides it from other *packages* only. All files in a package see each other's unexported names.
- **Renaming across the case boundary changes meaning.** Lowercasing an exported name removes it from every other package's view and breaks their code.
- **Unexported struct fields are invisible to reflection-based libraries** like `encoding/json`; they will not be marshaled. Export the fields you want serialized.

- **init cannot be called and its cross-file order is by filename.** Do not rely on subtle ordering between `init` functions; keep each one self-contained.
- **A cyclic import is a hard error, not a warning.** If two packages need each other, the design is wrong — extract a shared package or introduce an interface.

Interview questions

Q: How does Go organize code compared to C’s headers and translation units? A: Go has no headers and no preprocessor. Code is grouped into packages, where one directory is one package and all of its `.go` files share a single scope. Files in the same package see each other’s top-level declarations with no `extern` or prototypes, and declaration order does not matter because the compiler reads the whole package before checking it.

Q: How do you make a name public or private in Go? A: By capitalization. An identifier starting with an uppercase letter is exported and visible to other packages; a lowercase first letter is unexported and visible only within its own package. This applies to functions, types, variables, constants, struct fields, and methods. It is like C’s `static` versus external linkage, but decided by the name’s case and scoped to the package, not the file.

Q: What does a blank import (`import _ "pkg"`) do, and when would you use one? A: It imports a package solely for its side effects: the package’s `init` functions run, but you do not reference any of its names. It is used to register something with a central registry — for example a database driver registering itself with `database/sql`, or an image-format decoder registering with `image`.

Q: Explain Go’s package initialization order. A: First, all imported packages are fully initialized (dependencies before dependents). Then package-level variables are initialized in dependency order (Go orders them by what each initializer needs, not just textually). Then each `init` function runs, in filename order across files and top to bottom within a file. Finally, for package `main`, `main` runs.

Q: Why does Go forbid circular imports, and how do you resolve one? A: Forbidding cycles forces the package dependency graph to be a DAG, which makes dependencies easy to reason about and lets the compiler build and cache packages in order, quickly. To break a cycle, move the types both packages share into a third, lower-level package, or invert one direction of the dependency by defining an interface in the package that consumes the behavior.

Try it

1. Make a directory `shapes/` with two files, `area.go` and `constants.go`, both `package shapes`, where `area.go` uses a `const` defined in `constants.go`. Confirm it compiles even though the constant is in the “other” file.
2. Add an unexported field to a struct, marshal it with `encoding/json`, and watch it vanish from the output. Capitalize the field name and watch it reappear.

Chapter 4 — Types, Variables, and Constants

What you’ll learn. How Go declares variables and why the type comes *after* the name, the exact sizes of Go’s basic types, the all-important **zero value**, why Go refuses implicit

numeric conversions, and how constants and `iota` work. Everything is compared directly to `C`.

`C` and `Go` are both statically typed: every variable has a type fixed at compile time. But `Go` makes three choices that surprise a `C` programmer right away. It writes the type *after* the name. It gives every variable a defined starting value instead of garbage. And it refuses to convert between number types for you. This chapter covers all three, plus constants and named types.

Declaring variables

`Go` has one keyword for declaring a variable: `var`. There are a few forms.

```
var x int           // declare x of type int; it starts at 0 (the zero value)
var y int = 5       // declare and initialize
var z = 5           // declare and initialize; type is inferred as int
```

Inside a function you also get the **short variable declaration** `:=`, which declares and infers the type in one step:

```
w := 5              // same as: var w int = 5
name := "Ada"       // string
ok := true          // bool
```

Watch out. `:=` works **only inside a function**. At package level (outside any function) you must use `var`. Also, `:=` must declare at least one *new* variable on its left; otherwise use plain `=` to assign.

You can group several `var` declarations in one block. This is the common style for package-level variables, and it reads like a small table:

```
var (
    count    int           // 0
    rate     float64       // 0.0
    label     string       // ""
    enabled  bool          // false
)
```

`Go` also supports **multiple assignment**, which assigns several variables at once. The right-hand side is fully evaluated first, so swapping needs no temporary:

```
a, b := 1, 2
a, b = b, a           // swap; no temp variable, no XOR trick
fmt.Println(a, b)     // 2 1
```

This is also how you receive **multiple return values**, which are common in `Go` (see Chapter 6 — Functions):

```
v, err := strconv.Atoi("42") // v is int, err is error
```

Concept	C	Go
Declare, uninitialized	<code>int x;</code> (garbage value)	<code>var x int</code> (zero value 0)
Declare with value	<code>int x = 5;</code>	<code>var x = 5</code> or <code>x := 5</code>
Infer the type	not possible (must name it)	<code>x := 5</code> or <code>var x = 5</code>

Concept	C	Go
Declare many	<code>int a, b, c;</code>	<code>var (a int; b int; c int)</code>
Swap two values	needs a temp	<code>a, b = b, a</code>

Why the type comes after the name

In C, the type wraps around the name, and complex declarations become hard to read. The classic example:

```
int *p[10];           /* p is an array of 10 pointers to int... or is it? */
int (*q)[10];         /* q is a pointer to an array of 10 int */
char *(*f)(int);      /* f is a pointer to a function taking int, returning char* */
```

You must read these “inside out,” following C’s precedence rules. Go puts the name first and the type second, so every declaration reads left to right, like a sentence:

```
var p [10]*int        // p is an array of 10 pointers to int
var q *[10]int        // q is a pointer to an array of 10 ints
var f func(int) *byte // f is a function taking int, returning *byte
```

Mental model. Read a Go declaration left to right: “p is a [10] array of *int.” The type is a phrase that follows the name. There is no “spiral rule” like C’s.

C vs Go. In C the `*` binds to the variable, so `int* a, b;` declares one pointer and one plain `int` — a famous trap. In Go the type is written once and applies to the whole declaration, so this confusion cannot happen.

The basic types

Go’s numeric types have **exact, fixed sizes** that are the same on every platform. This is a real difference from C, where the standard only promises *minimum* sizes (`int` is at least 16 bits, `long` at least 32) and the real size depends on the compiler and platform.

Go type	Size	Meaning	Closest C type
<code>bool</code>	1 byte	true or false	<code>_Bool</code> / <code>bool</code>
<code>int8</code> / <code>uint8</code>	1 byte	8-bit signed / unsigned	<code>int8_t</code> / <code>uint8_t</code>
<code>int16</code> / <code>uint16</code>	2 bytes	16-bit	<code>int16_t</code> / <code>uint16_t</code>
<code>int32</code> / <code>uint32</code>	4 bytes	32-bit	<code>int32_t</code> / <code>uint32_t</code>
<code>int64</code> / <code>uint64</code>	8 bytes	64-bit	<code>int64_t</code> / <code>uint64_t</code>
<code>int</code> / <code>uint</code>	word size	32-bit on 32-bit CPUs, 64-bit on 64-bit CPUs	<code>int</code> / <code>size_t</code> (roughly)
<code>uintptr</code>	word size	unsigned int big enough to hold a pointer’s bits	<code>uintptr_t</code>
<code>byte</code>	1 byte	alias for uint8 ; raw data	<code>unsigned char</code>

Go type	Size	Meaning	Closest C type
<code>rune</code>	4 bytes	alias for <code>int32</code> ; one Unicode code point	<code>wchar_t</code> (loosely)
<code>float32</code>	4 bytes	IEEE-754 single precision	<code>float</code>
<code>float64</code>	8 bytes	IEEE-754 double precision	<code>double</code>
<code>complex64</code> / <code>complex128</code>	8 / 16 bytes	complex numbers (built in)	none
<code>string</code>	16-byte header	immutable UTF-8 bytes	<code>char * + length</code>

A few points a C programmer must internalize:

- **int is the machine word, not 32 bits.** On any modern 64-bit machine, Go's `int` is **64 bits**. Do not assume it is 32. When you need an exact width (file formats, wire protocols, hardware registers), use a sized type like `int32` or `uint64`, exactly as you would use `int32_t` in C.
- **byte and rune are aliases**, not new types. `byte` is literally `uint8` and `rune` is literally `int32`. Use `byte` for raw bytes and `rune` for a Unicode code point; the names document intent (see Chapter 8 — Arrays, Slices, and Strings).
- **string is its own type**, not a `char *`. It carries a length and its bytes are immutable. Strings get full treatment in Chapter 8.
- Complex numbers exist (`complex128`, the builtins `complex`, `real`, `imag`) but are rarely used outside scientific code; we will not dwell on them.

Here is the byte layout, the kind of picture a C programmer likes to see. Each box is one byte:

```

int8  / uint8   [#]                1 byte
int16 / uint16  [#] [#]            2 bytes
int32 / uint32  [#] [#] [#] [#]    4 bytes   (rune is int32)
int64 / uint64  [#] [#] [#] [#] [#] [#] [#] [#] 8 bytes
int   / uint    [#] [#] [#] [#] [#] [#] [#] [#] word size: 8 on 64-bit CPUs
                                           4 on 32-bit CPUs

```

The sized types are identical on every platform.

Only `int`, `uint`, and `uintptr` change with the machine word.

You can ask for a type's size at runtime with `unsafe.Sizeof`, the rough equal of C's `sizeof`. It returns a `uintptr` (a count of bytes):

```

import "unsafe"

var n int
fmt.Println(unsafe.Sizeof(n)) // 8 on a 64-bit machine
fmt.Println(unsafe.Sizeof(int32(0))) // always 4

```

Watch out. Despite the package name, `unsafe.Sizeof` is safe and computed at compile time. The `unsafe` package is only “unsafe” when you use it to bypass the type

system (pointer tricks); just asking for a size is fine.

Zero values

This is one of Go's most important rules, and it removes a whole category of C bugs.

Every variable is initialized to its type's *zero value* when declared, even if you do not assign one. There is no uninitialized garbage in Go.

Type	Zero value
numbers (int, float64, ...)	0 / 0.0
bool	false
string	"" (empty string)
pointers, slices, maps, channels, functions, interfaces	nil
struct	each field set to <i>its</i> zero value

```
var i int      // 0
var f float64  // 0.0
var s string   // "" (empty, not nil - a string is never nil)
var p *int     // nil
var b bool     // false
```

```
C:  int x;      // x holds whatever bytes were on the stack (garbage)
Go: var x int   // x is exactly 0, guaranteed
```

C vs Go. In C, a local `int x`; has an indeterminate value; reading it before assignment is undefined behavior, and forgetting to initialize is a classic bug. In Go, the same declaration always yields 0. Static and global variables in C *are* zero-initialized, but locals are not — Go makes the rule uniform.

Go's standard library is designed so the zero value is **useful**: a freshly declared variable is often ready to use without any setup.

```
var buf bytes.Buffer // ready to use; no New(), no init
buf.WriteString("hi") // works immediately

var mu sync.Mutex    // an unlocked mutex, ready to Lock()
mu.Lock()
```

Rule of thumb. When you design your own types, try to make the zero value do something sensible. A struct that works “empty” is easier to use than one that needs a constructor call first.

No implicit conversions

C converts between number types automatically, sometimes silently losing data:

```
int    i = 300;
char   c = i;      /* silently truncates to 44 on most systems */
double d = i;      /* int promoted to double, no warning */
double r = 7 / 2;   /* r is 3.0: integer division happened first! */
```

Go refuses all of this. **You must convert types explicitly**, with a conversion that looks like a function call: `T(value)`.

```
var i int = 300
var c byte = byte(i)           // explicit; you chose to truncate (c is 44)
var d float64 = float64(i)     // explicit widening
var r float64 = float64(7) / float64(2) // 3.5; convert before dividing
```

If you forget the conversion, it is a **compile error**, not a warning:

```
var i int = 10
var f float64 = i           // compile error:
// cannot use i (variable of type int) as float64 value in variable declaration
```

This even applies between `int` and `int64` on a 64-bit machine, *where they have the same size*. They are still different types, so you must write `int64(x)`. The compiler is checking *types*, not bit widths.

C vs Go. C's “usual arithmetic conversions” are convenient but cause silent truncation, signed/unsigned surprises, and accidental integer division. Go trades that convenience for safety: every conversion is visible in the code, so you can see exactly where a value changes type.

The one exception: untyped constants

There is a single, deliberate exception, and it is what makes Go pleasant to write despite the strictness. **Constant expressions are *untyped* until they are used.** An untyped constant adapts to whatever type the context needs, with no explicit conversion.

```
var x float64 = 3           // fine: the untyped constant 3 becomes a float64
var y int = 3               // fine: the same 3 becomes an int
const big = 1 << 40
var z int64 = big           // fine
f := 2.5
fmt.Println(f * 2)         // fine: 2 is untyped, becomes float64 here
```

Compare that to a *variable*, which has a fixed type and will not auto-convert:

```
i := 3                      // i is int (a typed variable now)
var w float64 = i           // ERROR: i is int, not an untyped constant
var w2 float64 = 3          // OK: 3 is an untyped constant
```

We cover constants next; this exception is the reason they are special.

Constants

A `const` is a value fixed at compile time. Unlike a C `#define` (which is just text substitution by the preprocessor) or a C `const` variable (which still lives in memory), a Go constant is a true compile-time value with no storage and no address.

```
const greeting = "hello"
const pi = 3.14159
const (
    maxRetries = 3
```

```

    timeoutSec = 30
)

```

Typed vs untyped constants

A constant can be **untyped** (the default) or **typed**.

```

const a = 100           // untyped: adapts to context, very flexible
const b int = 100       // typed: b is exactly int, behaves like a variable's type

```

Untyped constants have two powers:

1. **They adapt to the needed type** (the exception from the previous section).
2. **They are computed with very high precision** — far beyond `int64` or `float64` — and only rounded to a real type when assigned to one. This means constant math does not overflow during the calculation.

```

const Huge = 1 << 100    // fine as an untyped constant (arbitrary precision)
fmt.Println(Huge >> 97)  // 8 - the expression result fits in int, so it prints
// var n int = Huge      // ERROR: constant overflows int (caught at compile time)

```

Watch out. Assigning an out-of-range constant is a **compile error**, caught before your program ever runs. C might warn (or not) and then silently wrap or truncate at runtime. Go stops the build: `1 << 40 (untyped int constant ...) as int32 value ... (overflows)`.

A typed constant behaves like a variable of that type: it does *not* auto-convert.

```

const k int = 3
var f float64 = k    // ERROR: k is typed int; needs float64(k)

```

Rule of thumb. Leave constants untyped unless you have a specific reason to pin a type. Untyped constants compose more easily in expressions.

iota: Go's enum generator

Go has no `enum` keyword. Instead it uses `iota`, a counter that resets to 0 at the start of each `const` block and increases by 1 for each line.

```

type Weekday int

const (
    Sunday    Weekday = iota // 0
    Monday     // 1 (the expression "Weekday = iota" repeats)
    Tuesday    // 2
    Wednesday  // 3
    Thursday   // 4
    Friday     // 5
    Saturday   // 6
)

```

After the first line sets the pattern `Weekday = iota`, the following lines repeat it automatically, with `iota` increasing each time. This is the standard way to build an enumeration in Go (compare

to C's `enum { SUNDAY, MONDAY, ... }`).

Because `iota` can appear inside an expression, it is perfect for **bit flags** (`1 << iota`) and for sizes:

```
type Permission uint

const (
    Read    Permission = 1 << iota // 1  (0b001)
    Write   // 2  (0b010)
    Execute // 4  (0b100)
)

const (
    _ = iota // skip 0 with the blank identifier
    KB = 1 << (10 * iota) // 1 << 10 = 1024
    MB // 1 << 20
    GB // 1 << 30
)
```

You combine and test flags with the same bitwise operators you know from C:

```
p := Read | Write // turn two bits on
if p&Write != 0 { // test a bit
    fmt.Println("writable")
}
```

Concept	C	Go
Named constant	<code>#define MAX 3</code> or <code>const int MAX = 3;</code>	<code>const Max = 3</code>
Enumeration	<code>enum { RED, GREEN };</code>	<code>const (Red = iota; Green)</code>
Bit flags	<code>#define R (1<<0)</code>	<code>R = 1 << iota</code>
Computed at	preprocessor / runtime	compile time, high precision
Has an address	<code>const</code> variable does	no (pure value)

Named types vs type aliases

You can give a type a new name with **type**. There are two forms, and the difference matters.

A **named type** (also called a *defined type*) creates a brand-new, distinct type with the same underlying representation:

```
type Celsius float64
type Fahrenheit float64
```

`Celsius` and `Fahrenheit` are both `float64` underneath, but they are **different types**. You cannot mix them by accident, which prevents unit-confusion bugs:

```
var c Celsius = 100
var f Fahrenheit = 100
```

```
// c = f           // ERROR: cannot use f (Fahrenheit) as Celsius
c = Celsius(f)     // OK: explicit conversion when you really mean it
```

A **type alias** (note the =) creates a second *name for the exact same type*. There is no new type at all:

```
type Float = float64 // Float and float64 are interchangeable
var x Float = 3
var y float64 = x     // OK: same type, no conversion needed
```

This is precisely how `byte` and `rune` are defined in the language: `type byte = uint8` and `type rune = int32`.

	Named type <code>type T U</code>	Alias <code>type T = U</code>
Creates a new type?	Yes, distinct from U	No, same type as U
Needs conversion to mix with U?	Yes	No
Can have its own methods?	Yes	No (it is just U)
Main use	new domain types, enums	renaming during refactors, compatibility

The big payoff of a named type: **you can attach methods to it**. This is how Go gives behavior to a plain number or string without C's tricks. We cover methods in Chapter 10 — Structs and Methods, but here is a taste:

```
func (c Celsius) String() string {
    return fmt.Sprintf("%.1f°C", float64(c))
}
```

Printing and inspecting types

The `fmt` package has verbs that are invaluable while learning. Use them to see a value *and* its type.

```
type Point struct{ X, Y int }
p := Point{1, 2}
```

```
fmt.Printf("%v\n", p)           // {1 2}           default format
fmt.Printf("%+v\n", p)         // {X:1 Y:2}         adds field names
fmt.Printf("%#v\n", p)         // main.Point{X:1, Y:2} Go syntax
fmt.Printf("%T\n", p)          // main.Point       the type
fmt.Printf("%T\n", 3.0)         // float64           default type of an untyped float
fmt.Printf("%T\n", 'A')         // int32             a rune literal is int32
```

Verb	Shows	Like C's
<code>%v</code>	value in a default format	depends (<code>%d</code> , <code>%f</code> , ...)
<code>%+v</code>	value with struct field names	no equivalent
<code>%#v</code>	value as Go source code	no equivalent
<code>%T</code>	the dynamic type of the value	no equivalent

Rule of thumb. When something behaves oddly, print it with `%T` and `%#v` first. Nine times out of ten the surprise is “this value is not the type I thought it was.”

Key takeaways

- Declare with `var name type`, `var name = value`, or (inside functions) `name := value`. Group with `var ( ... )`; assign many at once with `a, b = b, a`.
- The **type comes after the name**, so declarations read left to right and avoid C’s confusing pointer/array syntax.
- Sized integer types (`int8..int64`, `uint8..uint64`) are **exact and the same everywhere**. `int/uint/uintptr` are the **machine word** — 64 bits on 64-bit machines, *not* always 32.
- Every variable starts at its **zero value** (0, 0.0, `false`, `""`, `nil`). There is no uninitialized garbage. Useful zero values are a design goal.
- Go has **no implicit numeric conversions**; write `float64(i)`, `int64(x)`, etc. The single exception is **untyped constants**, which adapt to context.
- Constants are compile-time values. Untyped ones have high precision and a default type; out-of-range assignments fail at **compile time**. `iota` builds enums and bit flags.
- `type T U` makes a **distinct** type (you can add methods); `type T = U` is a mere **alias**. `byte` and `rune` are aliases for `uint8` and `int32`.

Watch out (gotchas for C programmers)

- **int is not 32 bits.** It is the word size (64 bits on 64-bit CPUs). Use a sized type when the width must be exact.
- **No implicit conversion**, even between same-size types like `int` and `int64`. The compiler checks types, not bit widths.
- **:= opens a new scope and can shadow.** Inside an `if` or `for`, `x := ...` may create a *new* `x` that hides an outer one. We cover this trap in Chapter 5 — Control Flow.
- **Unused local variables are a compile error**, not a warning (so are unused imports). Declare it, use it, or remove it.
- **Integer division truncates toward zero**, just like C: `7 / 2 == 3`. Convert to `float64` first if you want `3.5`.
- **Signed overflow is defined, not undefined.** Go uses two’s-complement wraparound for integers: `int8(127) + 1 == -128`. In C, signed overflow is *undefined behavior*; in Go it is specified and predictable. (Constant overflow, by contrast, is caught at compile time.)

Interview questions

Q: Why does Go put the type after the variable name? A: So declarations read left to right like a sentence and avoid C’s “inside-out” pointer and array syntax. `var p [10]*int` reads “p is an array of 10 pointers to int,” with no precedence puzzles. It also means the type is written once per declaration, so the `int* a, b`; trap (only `a` is a pointer) cannot happen.

Q: What is a zero value, and how does it differ from C? A: Every Go variable is automatically set to its type’s zero value when declared: 0 for numbers, `false` for `bool`, `""` for strings, and `nil` for pointers, slices, maps, channels, functions, and interfaces. In C, local variables are uninitialized (garbage) unless you assign them; only statics and globals are zeroed. Go makes the rule uniform and reliable.

Q: How big is a Go int? How is that different from C? A: Go's `int` is the machine word size — 32 bits on 32-bit platforms and 64 bits on 64-bit platforms — and it is its own type. The sized types (`int8..int64`) are exact on every platform. In C, `int`, `long`, etc. have only minimum sizes guaranteed and vary by compiler/platform; you reach for `<stdint.h>` types like `int32_t` for exact widths, just as you use `int32` in Go.

Q: What are untyped constants and why are they useful? A: Constant expressions in Go are untyped by default. They have arbitrary precision and adopt whatever type the surrounding context requires, so `var x float64 = 3` and `var y int = 3` both work without a conversion. This is the one exception to Go's “no implicit conversion” rule, and it lets constant math stay exact until it is assigned to a concrete type.

Q: What is the difference between `type Celsius float64` and `type Alias = float64`? A: The first defines a new, distinct type whose underlying type is `float64`; it is not interchangeable with `float64` (you must convert), and you can attach methods to it. The second is a type alias: `Alias` is just another name for `float64`, fully interchangeable, and cannot have its own methods. `byte` and `rune` are aliases for `uint8` and `int32`.

Try it

1. Declare `var x int8 = 127`, then print `x + 1`. Confirm it wraps to `-128` (defined behavior), unlike C's undefined signed overflow.
2. Define `type Meters float64` and `type Feet float64`, declare one of each, and try to add them. Read the compile error, then fix it with an explicit conversion.
3. Build a `const` block of permission flags using `1 << iota`, combine two with `|`, and test one with `&`. Print the result with `%b` to see the bits.

Chapter 5 — Control Flow

What you'll learn. Go's control flow keywords: `if` without parentheses, the single `for` loop that replaces `while` and `do`, `range` over every kind of collection, and `switch` with **no automatic fallthrough**. Each one is compared to its C counterpart, including the traps.

Go keeps the control-flow keywords you know from C — `if`, `for`, `switch`, `break`, `continue`, `goto` — but trims the set and changes a few rules. There is **one loop keyword** (`for`), **no while** or `do-while`, **no `?:` ternary**, and `switch` cases **do not fall through** by default. The result is less syntax to learn and fewer ways to write the same thing.

if

An `if` in Go looks like C with two differences: **the condition has no parentheses**, and **the braces are mandatory** (even for a single statement).

```
if x > 0 {
    fmt.Println("positive")
} else if x == 0 {
    fmt.Println("zero")
} else {
```

```

    fmt.Println("negative")
}

/* C: parentheses required, braces optional (a frequent bug source) */
if (x > 0)
    printf("positive\n");

```

C vs Go. Optional braces in C lead to real bugs (the “goto fail” SSL bug, and the dangling-else trap). Go removes the choice: braces are always required, and the opening brace must be on the same line as the `if`. There is no debate about style.

The `if` with an `init` statement

`if` can run a short statement *before* the condition, separated by a semicolon. The variables it declares are **scoped to the `if/else` block** and disappear afterward. This is the idiomatic way to handle a function that returns a value and an error:

```

if v, err := strconv.Atoi(s); err != nil {
    fmt.Println("bad number:", err)
} else {
    fmt.Println("got", v) // v and err are visible here too
}
// v and err do NOT exist here

```

This keeps the variable’s scope tight — it exists only where it is relevant. In C you would declare `v` and `err` in the surrounding scope, where they linger after you are done with them.

Watch out. The `:=` in an `if` `init` opens a **new scope**. If an outer variable has the same name, the inner one *shadows* it. Assigning to the inner `err` does not change the outer `err`. This is the most common shadowing bug in Go; see the end-of-chapter gotchas.

No ternary operator

Go has no `?:` operator. The designers considered it and left it out because nested ternaries become unreadable. Use a short `if`, or — for numbers — the `min/max` builtins:

```

// C: int larger = (a > b) ? a : b;
larger := b
if a > b {
    larger = a
}

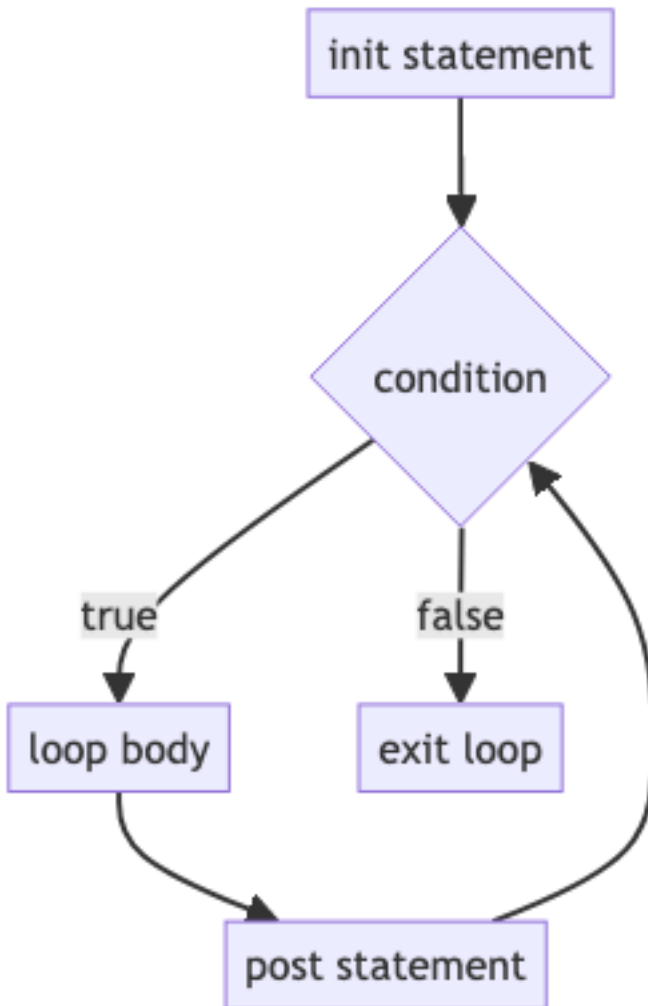
m := max(a, b) // builtin since Go 1.21; also min(a, b)

```

Rule of thumb. If you miss `?:`, you usually want a small helper function or the `min/max` builtins. A three-line `if` is considered clearer than a clever one-liner in Go.

`for`: the only loop

Go has exactly one loop keyword: `for`. It covers every looping pattern that C splits across `for`, `while`, and `do-while`.



1. **The C-style three-part loop.** Same three parts as C — init, condition, post — but no parentheses:

```
for i := 0; i < 10; i++ {  
    fmt.Println(i)  
}
```

2. **The while-style loop.** Drop the init and post, keep only the condition. This *is* Go's `while`:

```
n := 1  
for n < 100 { // C: while (n < 100)  
    n *= 2  
}
```

3. **The infinite loop.** Drop everything. This is Go's `for (;;) or while (1):`

```
for { // loops forever; leave with break or return  
    line, err := readLine()  
    if err != nil {  
        break  
    }  
}
```

```

    }
    process(line)
}

```

4. The integer range loop. Since Go 1.22 you can range over an integer to count from 0 to $n-1$. It is the cleanest way to “do something n times”:

```

for i := range 5 { // i goes 0, 1, 2, 3, 4
    fmt.Println(i)
}

```

Loop	C	Go
Counted	for (int i=0; i<n; i++)	for i := 0; i < n; i++
While	while (cond)	for cond
Do-while	do { } while (cond);	none; use for { ...; if !cond { break } }
Forever	for(;;) / while(1)	for {}
Count n times	for (int i=0; i<n; i++)	for i := range n

Watch out. There is **no do-while**. When you need “run the body at least once,” use an infinite for with a **break** at the bottom. There are also **no while or do keywords** at all — only **for**.

break and **continue** work exactly as in C: **break** leaves the loop, **continue** jumps to the next iteration (the post statement runs first in a three-part loop).

Labeled break and continue

To break or continue an *outer* loop from inside a nested loop, label the outer loop and name it. In C this is the one place people reach for **goto**; Go gives you a structured tool instead.

```

outer:
    for i := range 3 {
        for j := range 3 {
            if i*j > 2 {
                break outer // breaks BOTH loops, not just the inner one
            }
            fmt.Println(i, j)
        }
    }
}

```

continue label works the same way: it continues the labeled loop. A label is a name followed by a colon, placed directly before the loop.

C vs Go. In C you escape nested loops with a **goto done**; and a label at the bottom, or a flag variable. Go’s labeled **break/continue** does this cleanly and keeps the structure obvious — the label names a loop, not an arbitrary point.

range

`range` iterates over a built-in collection and hands you each element. What it yields depends on what you range over.

You range over	First value	Second value
array or slice	index	a copy of the element
string	byte index	the rune (code point) at that index
map	key	a copy of the value
channel	the received value	(none)
integer <code>n</code>	0 to <code>n-1</code>	(none)

```
nums := []int{10, 20, 30}
for i, v := range nums { // index and value
    fmt.Println(i, v)
}

for i := range nums { // index only (drop the value)
    fmt.Println(i)
}

for _, v := range nums { // value only (drop the index with _)
    fmt.Println(v)
}
```

Maps yield key and value, but in a **random order** that changes each run (this is deliberate; see Chapter 9 — Maps):

```
for k, v := range m {
    fmt.Println(k, v) // order is intentionally not stable
}
```

Strings range over *runes*, not bytes. The index is the byte offset where each rune starts, so it can jump by more than one for multi-byte UTF-8 characters (see Chapter 8 — Arrays, Slices, and Strings):

```
for i, r := range "hello" {
    fmt.Printf("%d:%c ", i, r) // 0:h 1:é 3:l 4:l 5:o (é is 2 bytes)
}
```

Channels yield values until the channel is closed (more in Chapter 14 — Channels and Select):

```
for v := range ch { // receives until ch is closed
    fmt.Println(v)
}
```

Two traps: the value is a copy, and the expression is evaluated once

These two rules surprise C programmers, so learn them now.

The range value is a copy. Modifying the loop variable does **not** change the element in the collection. To mutate the element, index into it:


```

type Item struct{ price int }
items := []Item{{1}, {2}, {3}}

for _, it := range items {
    it.price *= 10 // changes the COPY only
}
fmt.Println(items) // [{1} {2} {3}] - unchanged!

for i := range items {
    items[i].price *= 10 // index in to mutate the real element
}
fmt.Println(items) // [{10} {20} {30}]

```

The range expression is evaluated once. For a slice, the length is read at the start, so appending inside the loop does not loop forever:

```

s := []int{1, 2, 3}
for range s { // bound is fixed at 3 up front
    s = append(s, 0)
}
fmt.Println(len(s)) // 6, not infinite

```

Mental model. for i, v := range xs is roughly n := len(xs); for i := 0; i < n; i++ { v := xs[i]; ... }. The v := xs[i] is a copy, and n is computed once. Hold that picture and both traps make sense.

Watch out (historical). Before Go 1.22, the loop variable was **shared** across iterations, so launching goroutines that captured v inside a loop was a famous bug (every goroutine saw the last value). **Since Go 1.22 each iteration gets a fresh copy of the loop variables**, and this bug is gone. You may still see warnings about it in old articles; on Go 1.26 it does not apply to for loops.

Range over functions (iterators)

Since Go 1.23, range can also iterate over a function — an *iterator*. This lets your own types provide a clean range loop. You will mostly use these (for example, maps.Keys or strings.Lines) rather than write them, but here is the shape:

```

func CountTo(n int) func(yield func(int) bool) {
    return func(yield func(int) bool) {
        for i := range n {
            if !yield(i) { // yield returns false if the caller breaks
                return
            }
        }
    }
}

for x := range CountTo(3) { // 0 1 2
    fmt.Println(x)
}

```

```
}
```

We return to iterators in Chapter 8 — Arrays, Slices, and Strings.

switch

Go's `switch` is more powerful than C's and has one rule that is the **opposite of C**: a case **does not fall through** to the next one. There is an implicit `break` at the end of every case. You almost never write `break` in a Go `switch`.

```
switch n {
case 1:
    fmt.Println("one") // stops here; no fall-through to case 2
case 2:
    fmt.Println("two")
default:
    fmt.Println("many")
}
```

C switch: falls through unless you break	Go switch: each case breaks for you
case 1: A; // also runs B, C!	case 1: A // stops
case 2: B; break;	case 2: B // stops
case 3: C;	case 3: C
	fallthrough // opt IN

C vs Go. In C, forgetting `break` is a classic bug — execution “falls through” into the next case. Go flips the default: cases break automatically, and you use the keyword `fallthrough` to *opt in* when you actually want it.

Opt in with `fallthrough`. It transfers control to the next case unconditionally (it does not re-test that case's value):

```
switch n {
case 2:
    fmt.Println("two")
    fallthrough // continue into the next case on purpose
case 3:
    fmt.Println("three") // printed too when n == 2
}
```

Multiple values per case. List them comma-separated — no need for stacked empty cases like in C:

```
switch c {
case 'a', 'e', 'i', 'o', 'u':
    fmt.Println("vowel")
default:
    fmt.Println("consonant")
}
```

Conditionless switch (a clean `if/else if` chain). Omit the value after `switch`; each case is then a boolean test. This reads better than a long `else if` ladder:

```

switch { // like: switch true
case score >= 90:
    grade = "A"
case score >= 80:
    grade = "B"
default:
    grade = "C"
}

```

Switch with an init statement, just like if:

```

switch day := time.Now().Weekday(); day {
case time.Saturday, time.Sunday:
    fmt.Println("weekend")
default:
    fmt.Println("weekday")
}

```

Type switch (a brief preview). A special form switches on the *dynamic type* of an interface value. It is essential for working with `any`; we cover it fully in Chapter 11 — Interfaces:

```

func describe(x any) string {
    switch v := x.(type) {
    case int:
        return fmt.Sprintf("int %d", v)
    case string:
        return fmt.Sprintf("string %q", v)
    default:
        return "other"
    }
}

```

Feature	C switch	Go switch
Default behavior	falls through	breaks after each case
<code>break</code> needed	yes, every case	no (implicit)
Continue to next case	automatic	explicit <code>fallthrough</code>
Case type	integer constants only	any comparable values, multiple per case
No value after <code>switch</code>	not allowed	allowed (boolean cases)
Switch on a type	not possible	<code>switch v := x.(type)</code>

goto and labels

Go *does* have `goto` and labels, but they are rare. `goto` can only jump within the same function and cannot jump over a variable declaration into its scope.

```

i := 0
loop:
    if i < 3 {
        fmt.Println(i)
    }
}

```

```

    i++
    goto loop
}

```

In C, `goto` is most often used for cleanup: jump to a label at the end of a function to free resources on an error path. **Go replaces that pattern with `defer`**, which schedules cleanup to run automatically when the function returns — no labels, no jumps. We cover `defer` in Chapter 6 — Functions.

Rule of thumb. Reach for labeled `break/continue` to escape loops, and `defer` for cleanup. Genuine `goto` is almost never needed in Go; some code generators emit it, but hand-written Go rarely does.

A note on select

There is one more switch-like statement: `select`. It looks like a `switch` but its cases are **channel operations**, and it waits until one of them can proceed. Think of it as “switch for channels.” It is the heart of Go concurrency and gets a full treatment in Chapter 14 — Channels and Select:

```

select {
case msg := <-ch1:
    fmt.Println("received", msg)
case ch2 <- 42:
    fmt.Println("sent")
default:
    fmt.Println("nothing ready right now")
}

```

Key takeaways

- `if` has **no parentheses** and **mandatory braces**. The `if init; cond` form scopes a short statement (like `v, err := f()`) to the `if/else`.
- There is **no ternary operator**; use a short `if` or `min/max`.
- `for` is the **only loop**. It does counted loops, `while (for cond)`, infinite (`for {}`), and counting (`for i := range n`, Go 1.22+).
- Use **labeled `break/continue`** to control an outer loop from a nested one.
- `range` walks slices/arrays (index, value), maps (key, value, **random order**), strings (byte index, **rune**), channels, and integers. The value is a **copy**, and the range expression is **evaluated once**.
- Loop variables are **per-iteration since Go 1.22**, so the old goroutine-capture bug is gone.
- **`switch` does not fall through**; cases break automatically. Use `fallthrough` to opt in. Cases can list multiple values, a `switch` can have no condition (boolean cases) or an init statement, and a type switch inspects dynamic types.
- `goto` exists but is rare; use `defer` for C’s goto-to-cleanup pattern.

Watch out (gotchas for C programmers)

- **`switch` does not fall through by default**. This is the reverse of C. You will *not* see `break` in most Go switches; use `fallthrough` only when you mean it.

- **The range value is a copy.** `for _, v := range s { v.field = ... }` changes nothing in `s`. Index with `s[i]` to mutate the real element.
- **No while or do-while keywords.** Use `for cond` and `for { ...; if !cond { break } }`.
- **You cannot take the address of a map element.** `&m[k]` is a compile error, because a map may move its entries in memory. Take a copy, modify it, and store it back, or use a `map[K]*V` of pointers (see Chapter 9 — Maps).
- **:= inside if, for, or switch opens a new scope and can shadow.** A new `err` declared in an inner block hides the outer `err`; assigning to it does not affect the outer one. Linters such as `staticcheck` and the `shadow` analyzer catch this (see Chapter 22 — Tooling).

Interview questions

Q: How does Go’s switch differ from C’s? A: Go cases do not fall through; each case ends with an implicit `break`, so you rarely write `break`. You opt in to fall-through with the `fallthrough` keyword. Go also allows multiple values per case, a conditionless `switch` (boolean cases as a clean if-else chain), an init statement, and a type switch on an interface’s dynamic type. C falls through by default and only switches on integer constants.

Q: Go has only one loop keyword. How do you write a while loop? A: Use `for` with just a condition: `for cond { ... }`. An infinite loop is `for {}`, a counted loop is `for i := 0; i < n; i++`, and `for i := range n` counts from 0 to `n-1`. There is no `while` or `do-while` keyword; for “run at least once” use an infinite `for` with a `break` at the bottom.

Q: When you range over a slice and modify the value variable, why doesn’t the slice change? A: Because `range` copies each element into the loop variable. You are modifying the copy, not the element in the slice. To change the element, index into the slice: `s[i].field = ....` Likewise, the range expression (and a slice’s length) is evaluated once at the start of the loop.

Q: What is the loop-variable capture bug, and does it still exist? A: Before Go 1.22, a `for` loop reused one variable across all iterations, so closures or goroutines that captured it all saw the final value. Since Go 1.22 each iteration gets its own copy of the loop variables, so the bug is fixed. On Go 1.26 you do not need the old `v := v` workaround inside `for` loops.

Q: How do you break out of nested loops in Go without goto? A: Label the outer loop and use `break label` (or `continue label`). The label is a name with a colon placed right before the loop. This is the structured replacement for C’s “goto a label after the loops.” For cleanup on the way out of a function, use `defer` instead of a goto-to-cleanup label.

Try it

1. Write a loop that prints the multiplication pairs `i, j` for `i, j` in `0..2` but stops *all* looping the moment `i*j > 2`, using a labeled `break`.
2. Range over the string “naïve” printing each byte index and rune. Notice the index skips a number where the multi-byte character sits.
3. Write a `switch` with no condition that maps a score to a letter grade, then add a `fallthrough` somewhere and observe how the output changes.

Chapter 6 — Functions

What you’ll learn. How to declare Go functions, return *multiple* values (the heart of Go’s error handling), write type-safe variadic functions, pass functions around as values, build closures, and use `defer` to clean up reliably. Every idea is compared to the C function you already know.

A function in C and a function in Go do the same job: take inputs, run code, return a result. The shape is different, and Go adds a few abilities C does not have built in. None of them are hard. This chapter walks through each one.

Declaring a function

In C the return type comes first, then the name, then the parameters. In Go the keyword `func` comes first, the **type comes after each name**, and the return type comes after the parameter list.

```
int add(int x, int y) {  
    return x + y;  
}  
  
func add(x int, y int) int {  
    return x + y  
}
```

When several parameters share a type, you can write the type once, after the last of them. This is just shorthand.

```
func add(x, y int) int { // x and y are both int  
    return x + y  
}
```

C vs Go. The “type after the name” order feels backward for a day. The reason is that it reads left to right: “func add takes x int and y int, returns int.” Declarations and uses then look alike, which helps with the harder types later (function values, slices, maps).

Concept	C	Go
Declaration	<code>int add(int x, int y)</code>	<code>func add(x, y int) int</code>
Type position	before the name	after the name
Multiple results	out-pointers or a struct	native: <code>(int, error)</code>
Variadic args	<code>stdarg.h</code> , untyped	<code>...T</code> , type-safe
Overloading	C: no. C++: yes	no
Default arguments	C: no. C++: yes	no
Function pointer	<code>int (*f)(int)</code>	<code>f func(int) int</code>
Cleanup before return	<code>goto cleanup</code>	<code>defer</code>

There are no forward declarations and no prototypes. A function can call another function defined later in the file or in another file of the same package. We covered why in Chapter 3 — Program Structure: Packages, Imports, and Visibility.

Multiple return values

This is one of the first things you will love. A C function returns exactly one value. To return more, you pass *output pointers* or wrap the results in a struct. Go functions can return **several values** directly.

```
/* C: caller passes a pointer for the second result */
int divmod(int a, int b, int *rem) {
    *rem = a % b;
    return a / b;
}

func divmod(a, b int) (int, int) {
    return a / b, a % b
}

func main() {
    q, r := divmod(17, 5) // q = 3, r = 2
    fmt.Println(q, r)      // 3 2
}
```

The caller receives both values at once with `q, r := divmod(17, 5)`.

The (T, error) pattern

The most important use of multiple returns is **returning a result and an error** together. Go has no exceptions. A function that can fail returns its normal value plus an **error** value. The caller checks the error right away. (Errors get a full chapter: Chapter 12 — Errors.)

```
func parsePort(s string) (int, error) {
    n, err := strconv.Atoi(s) // Atoi returns (int, error)
    if err != nil {
        return 0, fmt.Errorf("invalid port %q: %w", s, err)
    }
    if n < 1 || n > 65535 {
        return 0, fmt.Errorf("port %d out of range", n)
    }
    return n, nil // nil means "no error"
}
```

The caller handles the error before using the result:

```
port, err := parsePort(os.Args[1])
if err != nil {
    log.Fatal(err) // print the error and exit
}
fmt.Println("listening on port", port)
```

Rule of thumb. Put the **error** last in the return list. Return it as `nil` on success. On failure return a zero value for the other results plus a real error. This order is a strong convention; follow it so your code reads like everyone else's.

Mental model. Multiple returns are not a tuple type you can store. They are just “this call produces N values.” You consume them immediately, usually with `:=`.

Named return values and the naked return

You may give the return values **names**. Named results act like local variables that already exist when the function starts, set to their zero value. A `return` with no values then returns whatever those names currently hold. That is called a **naked return**.

```
func divmod(a, b int) (q, r int) {
    q = a / b
    r = a % b
    return // naked: returns the current q and r
}
```

Named results are useful for two things: documenting what each result means, and letting a deferred function change the result (we use that with `defer` below).

Watch out. Naked returns hurt readability in anything longer than a few lines. The reader must scroll up to learn what is being returned. Use named results for documentation, but prefer an explicit `return q, r` in all but the shortest functions.

Variadic functions

A **variadic function** takes any number of trailing arguments of one type. You write `...T` for the last parameter. Inside the function that parameter is a slice (`[]T`); see Chapter 8 — Arrays, Slices, and Strings.

```
func sum(nums ...int) int {
    total := 0
    for _, n := range nums { // nums is a []int here
        total += n
    }
    return total
}
```

You can call it with zero, one, or many arguments. To pass an existing slice, add `...` after it to **spread** its elements.

```
sum(1, 2, 3) // 6
sum()       // 0 (nums is an empty slice)

xs := []int{4, 5, 6}
sum(xs...) // 15 - spread the slice into the variadic parameter
```

C has variadic functions too, through `<stdarg.h>` — but they are **untyped and unsafe**. The callee must know the count and type of every argument by some other means (a format string, a sentinel value, an explicit count). Pass the wrong type and you get undefined behavior, not a compile error.

```
#include <stdarg.h>
```



```

/* The caller MUST pass the correct count; nothing checks the types. */
int sum(int count, ...) {
    va_list args;
    va_start(args, count);
    int total = 0;
    for (int i = 0; i < count; i++)
        total += va_arg(args, int); /* wrong type here = silent corruption */
    va_end(args);
    return total;
}

```

C vs Go. Go’s variadics are **type-safe**. `sum(nums ...int)` accepts only `int` values; the compiler rejects anything else. There is no separate count to pass and keep in sync, because the slice knows its own length. The closest C cousin, `printf`, takes ...any in Go: `func Printf(format string, a ...any)`.

No overloading, no default arguments

Go has **no function overloading** (two functions with the same name but different parameters) and **no default argument values**. C also lacks both; C++ has both. Coming from C++ this is the surprise.

The idiomatic replacements:

- **Different names** instead of overloads. The standard library does this: `strings.Index`, `strings.IndexByte`, `strings.IndexRune`; `strings.Split` and `strings.SplitN`.
- **Variadics** when “zero or more” is the natural shape.
- **Type parameters (generics)** when one function should work for many types (Chapter 19 — Generics). The builtins `min` and `max` already work this way:

```

max(3, 5)      // 5    - works for ints
max(2.5, 1.5) // 2.5  - and floats

```

- **An options struct** instead of default arguments. The caller sets only the fields it cares about; the zero value supplies the defaults.

```

type ServerOpts struct {
    Port      int
    Timeout time.Duration
}

func NewServer(opts ServerOpts) *Server {
    if opts.Port == 0 {
        opts.Port = 8080 // default when the caller left it zero
    }
    // ...
    return &Server{ /* ... */ }
}

```

For libraries, this grows into the **functional options** pattern, covered in Chapter 25 — Idioms and Style.

Functions are values

In Go a function is a **first-class value**, like an `int` or a pointer. You can store it in a variable, pass it to another function, and return it from a function. C can do this too, with **function pointers**, but Go's syntax is much simpler.

The **type** of a function is its signature: `func(int, int) int` is “a function taking two ints and returning an int.”

```
/* C: a function pointer parameter, with the famous spiral syntax */
int apply(int (*f)(int), int x) {
    return f(x);
}

// Go: the parameter type is just "func(int) int"
func apply(f func(int) int, x int) int {
    return f(x)
}

func double(x int) int { return x * 2 }

func main() {
    fmt.Println(apply(double, 21)) // 42
}
```

You can also keep a function in a variable and call it later:

```
var op func(int, int) int // zero value is nil; calling a nil func panics
op = add
fmt.Println(op(2, 3)) // 5
```

C vs Go. A C function-pointer type like `int (*f)(int)` reads inside-out. The Go form `func(int) int` reads left to right and nests cleanly: `func(func(int) int) func(int) int` is a function that takes one int-to-int function and returns another. Try writing that as a C typedef.

Closures

An **anonymous function** is a function with no name, written inline. When an anonymous function uses variables from the function around it, it **captures** them. The combination is a **closure**: the function plus the captured variables.

Key point: a closure captures variables **by reference**, not by copy. It shares the actual variable with the enclosing function, so it sees and can change the latest value.

```
// counter returns a function. Each call to that function returns the next int.
func counter() func() int {
    count := 0 // captured by the returned closure
    return func() int {
```

```

        count++
        return count
    }
}

func main() {
    next := counter()
    fmt.Println(next()) // 1
    fmt.Println(next()) // 2
    fmt.Println(next()) // 3
}

```

`count` lives on after `counter` returns, because the closure still refers to it. The garbage collector keeps it alive (Chapter 17 — Memory and the GC). In C you would fake this with a `static` variable or a heap struct you pass around; Go handles the lifetime for you.

Watch out (historical). Before Go 1.22, a `for` loop reused **one** loop variable for every iteration. A closure that captured it saw the final value, so launching goroutines in a loop printed the last index many times. Since **Go 1.22 each iteration gets its own copy** of the loop variable, and this book targets Go 1.26, so the classic bug is gone:

```

funcs := make([]func(), 0, 3)
for i := range 3 { // Go 1.22+: each iteration has its own i
    funcs = append(funcs, func() { fmt.Print(i, " ") })
}
for _, f := range funcs {
    f() // prints: 0 1 2 (before Go 1.22 this printed: 3 3 3)
}

```

Watch out. Per-iteration loop variables fix the *loop* case. You can still create a sharing bug by capturing the same variable from outside a loop into several goroutines and mutating it. Concurrency safety is Chapter 13 — Goroutines and the Scheduler.

defer: cleanup that always runs

`defer` schedules a function call to run **when the surrounding function returns** — on any return path, and even while a panic is unwinding the stack. It is Go’s answer to “always close this, always unlock that.”

```

func readConfig(name string) error {
    f, err := os.Open(name)
    if err != nil {
        return err
    }
    defer f.Close() // runs no matter how readConfig returns below

    // ... many return statements can follow; the file still gets closed ...
    return nil
}

```

Compare the C idiom. Without `defer`, C programmers funnel every exit through a single cleanup label with `goto`:

```
int read_config(const char *name) {
    FILE *f = fopen(name, "r");
    if (!f) return -1;
    int rc = -1;
    /* ... work; on error: goto done ... */
    rc = 0;
done:
    fclose(f);    /* one cleanup point, reached by every path */
    return rc;
}
```

C vs Go. `defer` replaces the `goto` cleanup pattern and GCC/Clang's `__attribute__((cleanup(...)))`. The cleanup sits right next to the resource it frees — you **Open** then immediately **defer Close** — so you cannot forget it on a new early return. This is clearer and harder to get wrong.

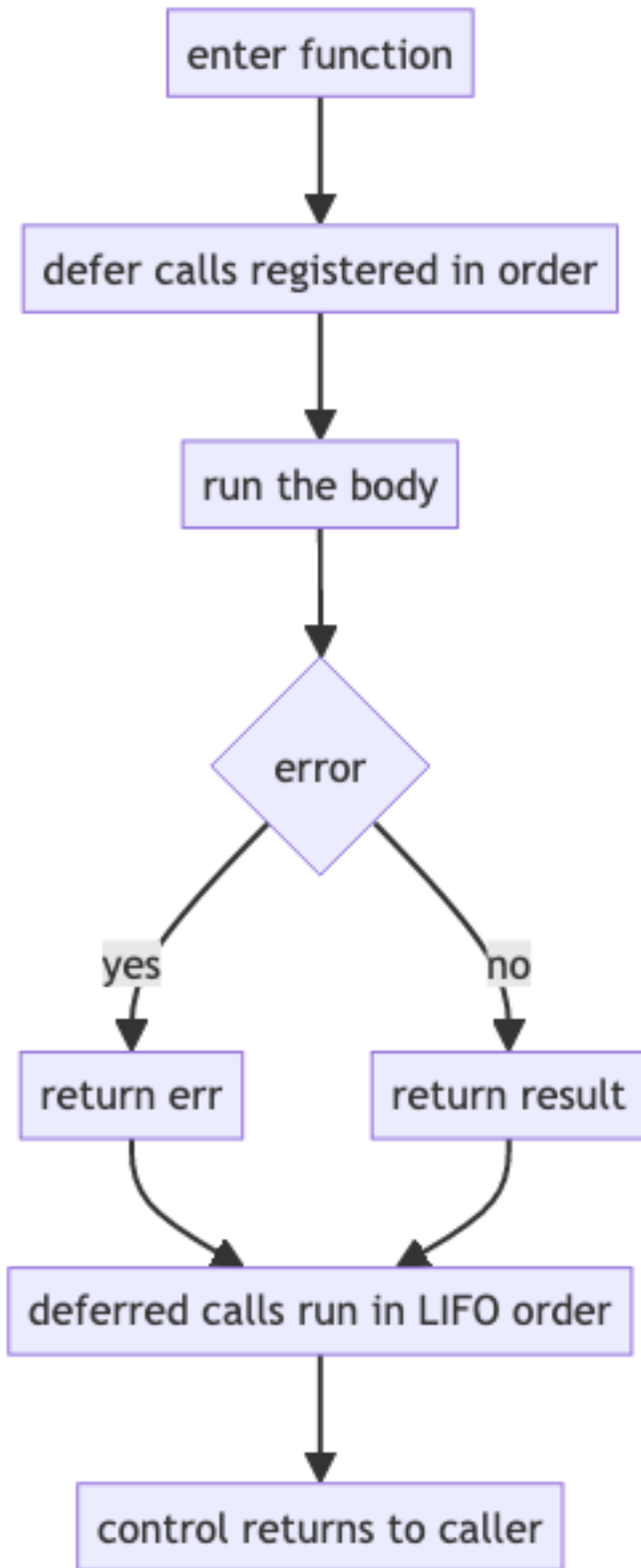
The other classic use is releasing a lock. A **mutex** is a lock that protects shared data (covered in the concurrency chapters). `defer` guarantees the unlock runs on every path:

```
mu.Lock()
defer mu.Unlock() // unlocked when the function returns, even on an early return
// ... touch the shared data safely ...
```

Rule 1: deferred calls run in LIFO order

If a function defers several calls, they run in **last-in, first-out** order: the most recently deferred call runs first. Think of a stack of plates.

```
func main() {
    for i := range 3 {
        defer fmt.Print(i, " ")
    }
    fmt.Println("body done")
}
// Output:
// body done
// 2 1 0
```



The defer stack, as a picture. Each `defer` pushes; the return pops from the top:

```
defer A    (pushed 1st)           run order at return
defer B    (pushed 2nd)
defer C    (pushed 3rd)
```

```

push ↓                pop / run ↑

    C    ← runs 1st    (last in)

    B    ← runs 2nd

    A    ← runs 3rd    (first in)
```

Rule 2: arguments are evaluated at the defer line

This is the subtle one. When you write `defer f(x)`, Go evaluates `f` and the argument `x` **right then**, at the `defer` statement, and remembers those values. The *call* happens later, but the arguments are already fixed.

```
package main

import "fmt"

func main() {
    i := 0
    defer fmt.Println("deferred i =", i) // i is read NOW: 0
    i = 99
    fmt.Println("current i =", i) // 99
}

// Output:
// current i = 99
// deferred i = 0
```

If you want the deferred code to see the *final* value, wrap it in a closure. A closure captures the variable by reference, so it reads the latest value when it runs:

```
i := 0
defer func() { fmt.Println("deferred i =", i) }() // reads i at return time
i = 99
// prints: deferred i = 99
```

Rule 3: a deferred closure can change named results

A deferred closure runs after the `return` statement has set the result variables but before the caller receives them. If the results are **named**, the closure can modify them. This is the standard way to add or transform an error on the way out.

```
package main

import "fmt"
```

```

// doubleViaDefer returns 2*x to show the order: return sets result,
// then the deferred closure modifies it before the caller sees it.
func doubleViaDefer(x int) (result int) {
    defer func() { result *= 2 }()
    result = x
    return // returns x, then defer turns it into 2*x
}

func main() {
    fmt.Println(doubleViaDefer(21)) // 42
}

```

A realistic use is catching a panic and turning it into an error (the full story is in Chapter 12 — Errors):

```

func safeRun() (err error) {
    defer func() {
        if r := recover(); r != nil {
            err = fmt.Errorf("recovered from panic: %v", r)
        }
    }()
    mightPanic()
    return nil
}

```

Watch out: defer inside a loop

A `defer` is tied to the **function**, not to the loop body or any inner block. Deferred calls pile up and run only when the whole function returns. So deferring inside a long loop can hold thousands of resources open at once.

```

// BAD: every file stays open until processAll returns.
func processAll(names []string) error {
    for _, name := range names {
        f, err := os.Open(name)
        if err != nil {
            return err
        }
        defer f.Close() // accumulates; nothing closes until the end
        // ... use f ...
    }
    return nil
}

```

Fix it by giving each iteration its own function, so `defer` fires every loop:

```

// GOOD: the deferred Close runs at the end of each call to processOne.
func processAll(names []string) error {
    for _, name := range names {

```

```

        if err := processOne(name); err != nil {
            return err
        }
    }
    return nil
}

func processOne(name string) error {
    f, err := os.Open(name)
    if err != nil {
        return err
    }
    defer f.Close() // runs when processOne returns, once per file
    // ... use f ...
    return nil
}

```

Recursion

Go functions may call themselves. Recursion works exactly as in C.

```

func factorial(n int) int {
    if n <= 1 {
        return 1
    }
    return n * factorial(n-1)
}

```

Watch out. Go does **not** guarantee tail-call optimization. A function that recurses very deeply can exhaust its stack. Goroutine stacks start small and grow automatically (Chapter 13 — Goroutines and the Scheduler), so the limit is large but not infinite. For deep or unbounded work, prefer a loop with an explicit stack (a slice).

Key takeaways

- A function is `func name(params) results { ... }`; types come **after** names, and shared types can be written once: `func add(x, y int) int`.
- Go functions return **multiple values**. The dominant pattern is `(result, error)`, with `error` last and `nil` meaning success.
- **Named results** document the outputs and let a deferred closure modify them; the **naked return** is fine for tiny functions but hurts readability otherwise.
- **Variadic** functions use `...T` and are type-safe, unlike C's `stdarg.h`. Spread a slice into one with `slice...`.
- Go has **no overloading and no default arguments**. Use distinct names, variadics, generics, or an options struct instead.
- Functions are **values**: store, pass, and return them. The function-type syntax is simpler than C function pointers.
- **Closures** capture variables by reference and keep them alive.

- **defer** runs at function return in **LIFO** order, evaluates its arguments at the **defer** line, runs on every path including panics, and (with a closure) can modify named results.

Watch out (gotchas for C programmers)

- **defer arguments are evaluated immediately**, not at return time. Wrap code in a closure (`defer func() { ... }()`) if it must see later values.
- **defer is per function, not per loop iteration**. Deferring inside a loop accumulates calls until the function returns; factor the body into its own function instead.
- **Naked returns reduce clarity**. Outside very short functions, write the values explicitly: `return q, r`.
- **No overloading or default arguments**. Do not look for them; reach for the idiomatic alternatives.
- **Shadowing err with :=**. Inside an `if` or `for` block, `:=` creates a *new* variable that hides the outer one. You can accidentally write to a new local `err` and return the outer (still `nil`) one. `go vet` catches some cases.

```
x := 1
if true {
    x := 2           // a NEW x, only inside this block (shadows the outer x)
    fmt.Println(x) // 2
}
fmt.Println(x) // 1 - the outer x never changed
```

- **Calling a nil function value panics**. A `func` variable's zero value is `nil`; assign a real function before calling it.

Interview questions

Q: Why does Go return multiple values instead of using output pointers like C? A: It makes the common “result plus status” case direct and readable, and it is the foundation of Go’s error handling: a function returns (**value**, **error**) and the caller checks the error immediately. There is no need for out-pointers or a wrapper struct, and no exceptions.

Q: When are the arguments to a deferred call evaluated? A: At the moment the **defer** statement runs, not when the deferred call actually executes at return time. The argument values are captured then. To observe a variable’s final value instead, defer a closure that reads the variable, because a closure captures it by reference.

Q: In what order do deferred calls run, and when? A: In last-in, first-out (LIFO) order, when the surrounding function returns — through a normal **return**, the end of the body, or a panic unwinding the stack. The most recently deferred call runs first.

Q: How can a deferred function change what a function returns? A: Only if the function uses named return values. The **return** statement sets the named result variables first, then deferred calls run before control goes back to the caller. A deferred closure can read and modify those named variables, which is how code recovers from a panic and converts it into a returned error.

Q: What is a closure, and how does Go decide the lifetime of captured variables? A: A closure is an anonymous function together with the variables it captures from its surrounding scope. Capture is by reference, so the closure sees the latest values. Captured variables stay alive as long as the closure is reachable; the garbage collector frees them afterward, so returning a closure that captures a local is safe.

Q: Does Go support function overloading or default arguments? A: No to both. Use different function names, variadic parameters, generics, or an options struct (and, for libraries, the functional options pattern) to get the same flexibility.

Try it

1. Write `func stats(nums ...int) (lo, hi, sum int)` using the named results. Call it with both a list of literals and a spread slice (`stats(xs...)`).
2. Add `defer fmt.Println("done")` and a few `defer fmt.Println(i)` calls in a loop, and predict the output order before you run it.
3. Inside `stats`, replace your manual `lo/hi` updates with the builtins `min` and `max`, and confirm the results match. (Note: do not name the results `min` or `max`, or you would shadow the builtins.)

Chapter 7 — Pointers

What you'll learn. How Go pointers work, why they are *safer* and *simpler* than C pointers, and the rules that surprise C programmers: no pointer arithmetic, dereferencing `nil` panics instead of corrupting memory, and returning the address of a local variable is perfectly safe.

Good news: you already understand pointers. A pointer is a value that holds the address of another value. Go uses the same `&` and `*` operators as C, and the same idea of “point at it, then read or write through the pointer.” This chapter is mostly about what Go **removes** to make pointers safe, plus a few habits that differ from C.

The basics: `&`, `*`, and `*T`

`&x` takes the **address** of `x`. `*p` **dereferences** `p` — it reads or writes the value `p` points at. The type `*T` means “pointer to a value of type `T`.” This is identical to C.

```
int x = 42;
int *p = &x;      /* p points at x */
printf("%d\n", *p); /* 42 */
*p = 7;           /* write through p */
printf("%d\n", x); /* 7 */

x := 42
p := &x           // p has type *int and points at x
fmt.Println(*p)   // 42 - read through p
*p = 7           // write through p
fmt.Println(x)    // 7
```

Here is the picture every C programmer carries in their head. It is the same in Go:

```

p                x

42

*int            int

p == &x        p holds the address of x
*p == 42        dereference p to reach x

```

A pointer variable that points at nothing holds `nil`, Go's version of `NULL`. The **zero value** of any pointer type is `nil`.

```

var p *int      // declared, not pointing anywhere yet: p == nil
fmt.Println(p == nil) // true

```

Concept	C	Go
Address-of	<code>&x</code>	<code>&x</code>
Dereference	<code>*p</code>	<code>*p</code>
Pointer type	<code>int *p</code>	<code>var p *int</code>
Pointer arithmetic	<code>p++, p + 1</code>	not allowed
Allocate one value	<code>malloc(sizeof(T))</code> (garbage)	<code>new(T)</code> (zeroed) or <code>&T{}</code>
Free	<code>free(p)</code>	automatic (garbage collector)
Null literal	<code>NULL</code>	<code>nil</code>
Dereference null	undefined behavior / segfault	defined panic
Field through pointer	<code>p->field</code>	<code>p.field</code>
Return address of local	dangling-pointer bug	safe

The big difference: no pointer arithmetic

In C a pointer is a number you can do math on. `p++` moves it to the next element; `p + 3` skips ahead three elements. This is powerful and is also the source of buffer overruns and a large share of security bugs.

Go **forbids pointer arithmetic**. You cannot add to a pointer, subtract from it, or increment it. A Go pointer only points; it does not walk.

```

int a[5] = {10, 20, 30, 40, 50};
int *p = a;
p++;          /* legal in C: now points at a[1] */
printf("%d\n", *p); /* 20 */

a := []int{10, 20, 30, 40, 50}
p := &a[0]    // taking the address of an element is fine
fmt.Println(*p) // 10
// But you cannot do arithmetic on the pointer:
// p++          // COMPILER ERROR: invalid operation
// p = p + 1    // COMPILER ERROR
fmt.Println(a[1]) // 20 - to "move", index the slice instead

```

To walk through memory in Go, you index a **slice**, which knows its own length and is bounds-checked at run time. That is the subject of Chapter 8 — Arrays, Slices, and Strings. The result: the classic off-by-one that scribbles past the end of an array cannot silently corrupt memory in Go. An out-of-range index panics with a clear message.

C vs Go. In C, a pointer and an array are almost interchangeable, and you walk data with pointer math. In Go those jobs split cleanly: **pointers point at one thing; slices represent a run of things**. If you find yourself wanting `p + 1`, you want a slice and an index.

new(T): allocate a zeroed value

`new(T)` allocates storage for one value of type `T`, sets it to the **zero value**, and returns a `*T` pointing at it. It is roughly C's `malloc`, with two big differences: the memory is **zeroed**, not garbage, and you never **free** it — the garbage collector reclaims it when nothing points at it (Chapter 17 — Memory and the GC).

```
int *p = malloc(sizeof(int)); /* contents are garbage */
*p = 5;
/* ... */
free(p);                      /* you must free, exactly once */

p := new(int)    // p is *int, *p starts at 0 (zeroed, not garbage)
fmt.Println(*p) // 0
*p = 5
// no free: the GC handles it
```

In practice `new` is **rarely used**. For structs, the idiomatic way to get a pointer is a composite literal with `&`, which also lets you set fields at the same time:

```
type Point struct{ X, Y int }

q := new(Point)          // &Point{} with all fields zero
p := &Point{X: 1, Y: 2} // idiomatic: allocate AND initialize
```

Rule of thumb. Reach for `&T{...}` to make a pointer to a struct. Keep `new` for the occasional pointer to a basic type like `*int`, and even then a local variable plus `&` usually reads better.

nil and the defined panic

Dereferencing a `nil` pointer in C is **undefined behavior**: usually a segfault, sometimes silent corruption, occasionally “it worked on my machine.” In Go, dereferencing `nil` is a **defined runtime error**: the program panics with a clear message at the exact line.

```
var p *int
fmt.Println(*p)
// panics: runtime error: invalid memory address or nil pointer dereference
```

A panic is recoverable and prints a stack trace, so the failure is visible and debuggable rather than mysterious. (Panic and recover are covered in Chapter 12 — Errors.) The lesson is the same as in C — check for `nil` before dereferencing — but the failure mode is far kinder.

Pass-by-value, always

This is the rule that explains when you need pointers at all. **Go passes every argument by value.** The function gets a *copy*. Writing to a parameter changes only the copy, never the caller's variable. C works the same way, which is why C also uses pointers to let a function modify the caller's data.

The classic example is `swap`. Without pointers it cannot work, in either language, because each side swaps its own copies:

```
// Does NOT swap the caller's variables: a and b are copies.
func swap(a, b int) {
    a, b = b, a
}
```

Pass **pointers** so the function can reach the originals:

```
void swap(int *a, int *b) {
    int tmp = *a;
    *a = *b;
    *b = tmp;
}

/* call: swap(&x, &y); */

func swap(a, b *int) {
    *a, *b = *b, *a // write through the pointers
}

func main() {
    x, y := 1, 2
    swap(&x, &y)
    fmt.Println(x, y) // 2 1
}
```

Mental model. “Pass by value” means “pass a copy of the bits.” For an `int`, that copies the number. For a pointer, that copies the *address* — both the original and the copy then point at the same value, so the callee can change it. This is exactly how C behaves.

Watch out. Slices, maps, and channels are special. They are small *headers* that already contain a pointer to shared backing data. Passing one by value copies the header but not the underlying data, so the callee can change the elements a slice points at. We unpack this in Chapter 8 — Arrays, Slices, and Strings, and Chapter 9 — Maps.

Struct pointers: no `->`, just `.`

In C you reach a field through a pointer with the arrow operator: `p->field`. In Go there is **no** `->`. You always use a dot, and Go **automatically dereferences** a pointer to a struct when you access a field. So `p.field` works whether `p` is a struct or a pointer to a struct.

```
struct Point { int x, y; };
struct Point *p = &pt;
```

```
printf("%d\n", p->x); /* arrow for pointers, dot for values */

type Point struct{ X, Y int }

p := &Point{X: 1, Y: 2}
fmt.Println(p.X) // 1 - same as (*p).X, but you never write that
p.Y = 9          // auto-dereference on assignment too
fmt.Println(*p)  // {1 9}
```

C vs Go. One operator (.) for both values and pointers means less to think about. The compiler inserts the dereference for you. The explicit (*p).X form is legal but no one writes it. This same auto-dereference makes calling methods on pointers seamless (Chapter 10 — Structs and Methods).

Returning the address of a local is safe

Here is the rule that feels *wrong* to a C programmer, and it is one of Go's best features. In C, returning the address of a local variable is a bug: the variable lives on the stack frame, which disappears when the function returns, leaving a **dangling pointer**.

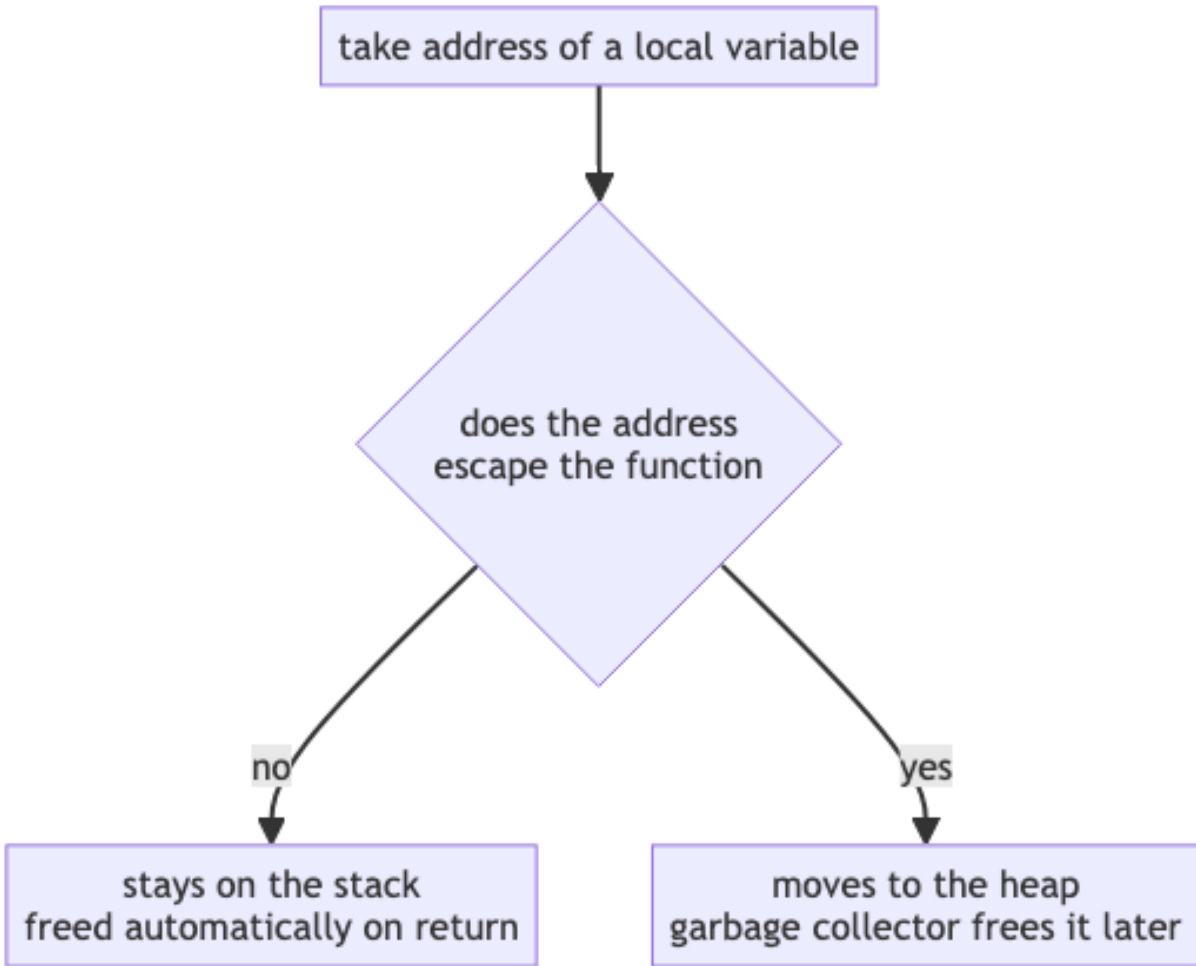
```
int *new_counter(void) {
    int c = 0;
    return &c; /* BUG: c is gone after return; dangling pointer */
}
```

In Go the same code is **correct and common**. The compiler performs **escape analysis**: if it sees that a local's address outlives the function, it places that variable on the **heap** instead of the stack. The garbage collector frees it later, when nothing points at it.

```
func newCounter() *int {
    c := 0 // local variable
    return &c // SAFE: c "escapes", so Go allocates it on the heap
}

func main() {
    p := newCounter()
    *p += 1 // write through the pointer; in Go (*p)++ also works
    fmt.Println(*p) // 1
}
```

You do not annotate anything; the compiler decides stack vs heap for you. This is why constructor-style functions that return &T{...} are everywhere in Go and are completely safe. The full mechanics — escape analysis, the heap, and the GC — are in Chapter 17 — Memory and the GC.



C vs Go. Memorize this inversion: in C, `return &local;` is a classic bug; in Go, `return &local` is a classic *pattern*. Escape analysis plus garbage collection make it safe.

When to use a pointer (and when not to)

Pointers in Go are for **sharing** and **mutation**, not for “walking memory.” Use them deliberately.

Situation	Use a pointer?	Why
Let a function modify the caller’s value	Yes	The only way to write back.
A large struct you pass around a lot	Yes	Avoid copying many bytes each call.
An optional / nullable field	Yes	<code>nil</code> cleanly means “absent.”
A small value: <code>int</code> , <code>bool</code> , small struct	No	Copying is cheap; a value is simpler and safer.
A slice, map, or channel	No	They are already reference-like; passing the value shares the data.

Rule of thumb. Default to passing values. Switch to a pointer when you must mutate the caller’s data, when profiling shows copying a big struct is costly, or when you genuinely need “no value here” (`nil`). Do not reach for a pointer just out of C habit.

`unsafe.Pointer` (briefly)

Go does provide an escape hatch. The `unsafe` package has `unsafe.Pointer`, which can convert between unrelated pointer types and to/from `uintptr` (an integer big enough to hold an address). With it you *can* do C-style pointer tricks, including a form of pointer arithmetic, and it is how some `cgo` interop and high-performance code works.

```
import "unsafe"
// unsafe.Pointer bypasses Go's type and memory safety.
// It is the tool behind some C interop and a few performance hacks.
```

It is **discouraged for everyday code**. The name is a warning: you give up the safety guarantees this chapter is about, and the rules for using it correctly are strict and easy to break. Reach for it only for low-level interop or measured, proven hot paths — and document why.

Key takeaways

- Go pointers use the same `&`, `*`, and `*T` as C, and a pointer’s zero value is `nil`.
- **No pointer arithmetic.** To traverse data, index a bounds-checked slice (Chapter 8 — Arrays, Slices, and Strings).
- `new(T)` returns a `*T` to a **zeroed** value and is rarely used; `&T{...}` is the idiomatic way to allocate and initialize a struct. The GC frees memory; you never call `free`.
- Dereferencing `nil` is a **defined panic**, not C’s undefined behavior.
- Go is **pass-by-value always**. Pass a pointer when a function must modify the caller’s data (see the `swap` example).
- Field access through a pointer uses `.` and auto-dereferences; there is no `->`.
- **Returning `&local` is safe:** escape analysis moves the variable to the heap. This is the opposite of the C rule (Chapter 17 — Memory and the GC).
- Use pointers for mutation, large structs, and optional values; not for small values or the already-reference-like slices, maps, and channels.

Watch out (gotchas for C programmers)

- **return `&local` is SAFE here** — the inverse of C. Do not add workarounds for a dangling-pointer bug that cannot happen.
- **Dereferencing `nil` panics** with a defined runtime error; it does not segfault or corrupt memory. Still check for `nil` before dereferencing.
- **No pointer arithmetic.** `p++` and `p + 1` do not compile. Use a slice index.
- **You cannot take the address of a map element.** `&m[k]` is a compile error, because a map may move its entries when it grows, which would invalidate the address. To update in place, reassign (`m[k] = ...` or `m[k]++`) or store pointers (`map[K]*V`).

```
m := map[string]int{"a": 1}
// p := &m["a"] // COMPILER ERROR: cannot take address of m["a"]
m["a"]++        // fine: read-modify-write by key
```


- **Comparing pointers compares addresses, not the pointed-at values.** Two pointers are `==` only when they hold the same address (or are both `nil`).

```
a, b := 1, 1
p, q := &a, &b
fmt.Println(p == q) // false - different addresses, even though *p == *q
fmt.Println(p == &a) // true - same address
```

- **A pointer does not let you “walk” a struct’s fields** the way C lets you walk bytes. Field layout is the compiler’s business; use the field names.

Interview questions

Q: Does Go have pointer arithmetic? How do you traverse a buffer without it? A: No. You cannot add to or increment a pointer. To traverse data you use a slice and index it (`s[i]`), which is bounds-checked at run time. This removes a whole class of memory-safety bugs while still letting you process sequences efficiently.

Q: Why is returning the address of a local variable safe in Go but a bug in C? A: The Go compiler does escape analysis. If a local’s address escapes the function, the compiler allocates that variable on the heap instead of the stack, and the garbage collector frees it once nothing references it. In C a local lives on the stack frame, which is reclaimed on return, so its address dangles.

Q: What happens when you dereference a nil pointer in Go versus C? A: In Go it is a defined runtime panic with a clear message and a stack trace, and it can be recovered. In C it is undefined behavior — typically a segfault, possibly silent memory corruption.

Q: What is the difference between `new(T)` and `&T{}`? A: Both return a `*T` to a freshly allocated, zeroed value. `&T{...}` is a composite literal, so you can initialize fields at the same time, which is why it is the idiomatic choice for structs. `new(T)` only gives you the zero value and is mostly used for basic types like `*int`. Neither requires freeing; the GC handles it.

Q: When should you pass a pointer to a function instead of a value? A: When the function must modify the caller’s data, when copying a large struct on every call is measurably expensive, or when you need a nullable/optional value where `nil` means “absent.” For small values, prefer passing by value; slices, maps, and channels are already reference-like, so you rarely point at them.

Q: Why can’t you take the address of a map element? A: Because a map may reorganize its internal storage when it grows, moving entries to new memory. A pointer into the map would then dangle. Go forbids `&m[k]` to keep that impossible. Update entries by key, or store pointers as the map’s values.

Try it

1. Write `func incr(p *int)` that does `*p += 1`. Call it on a local, print the result, and confirm the original changed.
2. Write `func makePoint(x, y int) *Point` that returns `&Point{x, y}`. Note that returning the address of a value created inside the function is safe.
3. Try to compile `p := &m["a"]` for a map `m`, read the error, then rewrite it as `m["a"]++`.

Chapter 8 — Arrays, Slices, and Strings

What you'll learn. The single biggest difference between C and Go: the *slice*, a small three-field header that replaces “a pointer plus a length.” You will learn how arrays, slices, and strings are laid out in memory, why sub-slices can secretly share storage, how `append` can quietly move your data, and why a Go string is UTF-8 bytes — not a NUL-terminated `char *`.

This is the chapter to read slowly. Almost every Go bug a C programmer hits in their first month lives here. The good news: once the slice model clicks, it stays clicked.

Arrays: fixed size, and a value type

A Go **array** is written `[N]T`: `N` elements of type `T`. The length `N` is a constant and is **part of the type**.

```
var a [3]int           // three ints, all zero: [0 0 0]
b := [3]int{1, 2, 3}
c := [...]int{1, 2, 3, 4} // [...] = "count the elements for me" -> [4]int
```

This looks like C, but two rules are different and both surprise C programmers.

1. The size is part of the type. `[3]int` and `[4]int` are *different types*. You cannot assign one to the other, and a function that takes `[3]int` will not accept a `[4]int`. In C, `int[3]` and `int[4]` are loosely interchangeable through a pointer; in Go they are as distinct as `int` and `float64`.

2. An array is a value, not a pointer. This is the opposite of C. In C, an array name *decays* into a pointer to its first element, so passing an array to a function passes a pointer — the function edits your data. In Go, an array is a plain value. **Assigning it copies every element. Passing it to a function copies every element.**

```
func zero(arr [3]int) { arr[0] = 999 } // arr is a COPY of the caller's array

func main() {
    a := [3]int{1, 2, 3}
    b := a           // a FULL copy; b and a are now independent
    b[0] = 100       // a is still [1 2 3]
    zero(a)          // a is STILL [1 2 3] - zero edited only its own copy
}
```

C vs Go. In C, `void zero(int arr[3])` is really `void zero(int *arr)`, and it edits the caller's array. In Go, `func zero(arr [3]int)` gets a copy and edits nothing the caller can see. The C array decays to a pointer; the Go array does not. This is the reverse of your instinct, so tattoo it on your brain.

Because copying whole arrays is rarely what you want, **arrays are used directly quite rarely** in Go. They show up as fixed-size buffers (`[16]byte` for a hash), as the element type of something else, or inside a struct. For everything “array-like” and growable, you use a **slice**.

Slices: the workhorse

A **slice** is Go's dynamic array — the type you reach for constantly: lists, buffers, stacks, queues, function arguments. A slice is **not** an array. It is a small, fixed-size *header* that *points into* a

backing array, with three fields:

```
type slice struct {
    ptr *T    // pointer to the first element in a backing array
    len int   // number of elements you can index right now
    cap int   // elements available from ptr to the end of the backing array
}
```

Mental model. A slice is a “fat pointer”: the pointer you would pass in C, bundled together with the length *and* the capacity, in one value. The backing array lives on the heap and is freed by the garbage collector when no slice refers to it. You never `malloc` or `free` it.

Here is the layout. `s := make([]int, 3, 5)` makes a slice of length 3 and capacity 5:

```
    slice header (a value: 3 machine words)

s =      ptr      len = 3    cap = 5

           backing array on the heap (GC-managed)

      0      0      0      -      -

      0      1      2      3      4
      len = 3
      cap = 5
```

- `len(s)` is how many elements you can index. `s[0]` to `s[len-1]` are valid; `s[len]` panics with “index out of range.”
- `cap(s)` is how many elements fit before the backing array runs out. The gap between `len` and `cap` is room to grow *without reallocating* (more on this under `append`).

Creating slices

// 1. Slice literal: declare and fill in one step. No size, no malloc.

```
nums := []int{3, 1, 4, 1, 5} // len 5, cap 5
```

// 2. make(): allocate with a length (and optional capacity).

```
buf := make([]byte, 1024) // len 1024, cap 1024, all zero
```

```
grow := make([]int, 0, 100) // len 0, cap 100: empty but pre-sized
```

// 3. Slicing an existing array or slice (next section).

C vs Go. `[]int{...}` has no number between the brackets. With a number, `[5]int{...}`, it is an **array** (fixed). Without one, `[]int{...}`, it is a **slice** (dynamic). That tiny difference — empty brackets or not — decides which world you are in.

nil slice vs empty slice

A slice’s zero value is `nil`: a header with `ptr == nil`, `len == 0`, `cap == 0`.

```
var s []int           // nil slice: len 0, cap 0, no backing array
e := []int{}          // empty (non-nil) slice: len 0, cap 0, but not nil
```

Here is the kind thing about Go: **a nil slice is safe to use**. You can call `len(s)` on it (returns 0), `range` over it (zero iterations), and — most importantly — `append` to it. There is no need to check for nil first.

```
var s []int           // nil
for range s { }        // fine: zero iterations
s = append(s, 1)        // fine: append allocates a backing array
fmt.Println(s)          // [1]
```

Rule of thumb. Prefer the nil slice (`var s []T`) as your “empty list” starting point. It needs no allocation and behaves the same as an empty slice in every operation you care about. The one place the difference shows is JSON: a nil slice marshals to `null`, an empty slice to `[]`.

Slicing: sub-slices share memory

You can take a slice *of* a slice (or of an array) with `s[low:high]`. The result covers elements `low` up to **but not including** `high`. Both indices are optional: `s[:n]`, `s[n:]`, and `s[:]` all work.

```
a := []int{10, 20, 30, 40, 50}
b := a[1:3] // b = [20 30]: elements at index 1 and 2
```

The new length is `high - low`. But here is the part that bites every C programmer:

A sub-slice does not copy. It points into the *same backing array*. `b` and `a` share storage. Writing through one is visible through the other. This is called **aliasing**.

```
a := []int{10, 20, 30, 40, 50}
b := a[1:3] // b shares a's backing array

b[0] = 99    // this writes to the SAME memory as a[1]
fmt.Println(a) // [10 99 30 40 50]  <-- a changed!
fmt.Println(b) // [99 30]
```

```
a
    10    99    30    40    50    backing array (shared)
```

```
b      b[0] is the same cell as a[1]
```

C vs Go. This is exactly like two C pointers into the same buffer: `int *b = &a[1]`; then `b[0] = 99`; also changes `a[1]`. The difference is that in Go the slicing syntax makes it *easy and quiet* to create such an alias, so you must stay aware of who shares what.

Capacity, and the three-index slice

When you slice, the new capacity runs from `low` to the **end of the original backing array**, not to `high`. So a sub-slice often keeps spare room reaching into its parent — the trap behind the next section:

```

a := []int{10, 20, 30, 40, 50} // len 5, cap 5
b := a[1:3]                     // len 3-1 = 2, cap 5-1 = 4 (spare cells a[3], a[4])
fmt.Println(len(b), cap(b))    // 2 4

```

A third index, `s[low:high:max]`, sets capacity to `max-low`, cutting off that spare room so the sub-slice cannot reach past its end:

```

b := a[1:3:3]                  // len 2, cap 3-1 = 2: no spare capacity
fmt.Println(len(b), cap(b))    // 2 2

```

append: grow a slice (carefully)

`append` adds elements to the end of a slice and returns the (possibly new) slice. You must always assign the result back:

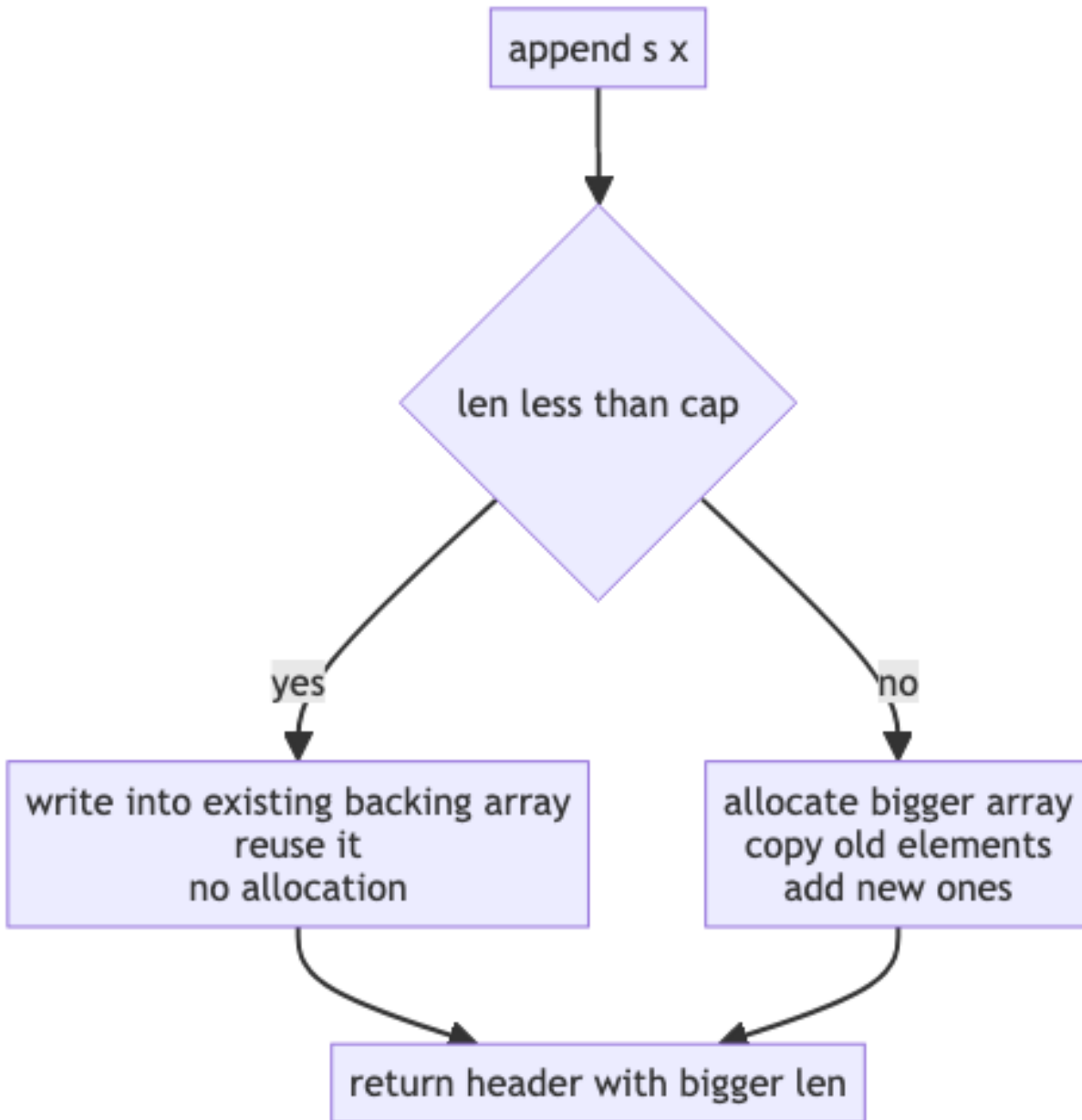
```

s := []int{1, 2, 3}
s = append(s, 4)              // [1 2 3 4]
s = append(s, 5, 6)           // [1 2 3 4 5 6]: append takes any number of values
s = append(s, other...)       // the ... spreads another slice's elements

```

What `append` does depends on capacity:

- If `len < cap` (there is spare room), `append` writes into the *existing* backing array and returns a header with a bigger `len`. No allocation. The backing array is reused.
- If `len == cap` (no room), `append` **allocates a new, bigger backing array**, copies the old elements into it, adds the new ones, and returns a header pointing at the *new* array. The old array is left behind (and collected later if nothing else uses it).



Amortized growth

When `append` must grow, it does not add one cell — it grows the capacity by a factor (roughly doubling for small slices, slower for large ones). So n appends cost $O(n)$ total, not $O(n^2)$. This is the same trick as C++’s `std::vector` or a hand-rolled “grow by doubling” buffer in C.

```
s := make([]int, 0) // len 0, cap 0
for i := range 10 {
    s = append(s, i)
    fmt.Println(len(s), cap(s)) // len climbs 1..10; cap grows in big jumps,
                                // each one roughly double the previous
}
```

The exact capacities are an **implementation detail** (they vary by Go version, element size, and platform — never hard-code them). On Go 1.26 the run prints caps 4, 4, 4, 4, 8, 8, 8, 8, 16, 16: the jumps keep doubling as the slice fills. Learn the *pattern* (geometric growth), not the numbers.

Rule of thumb. If you know roughly how many elements you will add, give the capacity up front: `make([]T, 0, n)`. This avoids repeated reallocation and copying. It is the Go version of reserving buffer space before a loop.

The classic append-aliasing bug

Combine “sub-slices share memory” with “append may reuse the backing array” and you get Go’s most famous footgun.

```
a := []int{10, 20, 30, 40, 50}
b := a[1:3]           // len 2, cap 4 - shares a's array, with spare room

b = append(b, 777)    // len 2 < cap 4, so NO new array:
                      // append writes 777 into the spare cell, which is a[3]!

fmt.Println(a)        // [10 20 30 777 50]  <-- append silently clobbered a[3]
fmt.Println(b)        // [20 30 777]
```

The `append` did not allocate, because `b` had spare capacity reaching into `a`. So it overwrote `a[3]`. If you expected `append` to leave `a` alone, you have a bug.

The fix is the three-index slice, which removes the spare capacity so `append` is forced to allocate a fresh array:

```
b := a[1:3:3]         // cap 2, no spare room
b = append(b, 777)    // len 2 == cap 2, so append ALLOCATES a new array
fmt.Println(a)        // [10 20 30 40 50]  <-- a is safe now
fmt.Println(b)        // [20 30 777]
```

Watch out. Any time you hand out a sub-slice that someone else may `append` to, decide who owns the backing array. Either cap it with the three-index form (`a[low:high:high]`) or hand them a copy (see `copy` below).

Always write `s = append(s, x)`

Because `append` may return a *different* header, ignoring the return value is almost always a bug:

```
append(s, x)          // WRONG: the grown slice is thrown away; go vet warns
s = append(s, x)      // RIGHT: keep the new header
```

copy: duplicate elements

`copy(dst, src)` copies `min(len(dst), len(src))` elements and returns that count. It does **not** allocate or grow `dst`; you must size `dst` yourself.

```
src := []int{1, 2, 3, 4, 5}
dst := make([]int, 3)
```

```
n := copy(dst, src)      // copies 3 elements (dst is the limit)
fmt.Println(n, dst)      // 3 [1 2 3]
```

To make an independent copy of a whole slice (breaking all aliasing):

```
clone := make([]int, len(src))
copy(clone, src)
// or, since Go 1.21:
clone := slices.Clone(src) // same effect, one line
```

C vs Go. `copy` is like `memmove`: it handles overlapping ranges correctly, and it never runs past the end because it stops at the shorter length. There is no separate length argument to get wrong — the slices carry their own lengths.

Common slice operations

The `slices` package (Go 1.21+) has clean helpers; it is still worth knowing the moves underneath.

Delete and insert at index `i` (order preserved):

```
s = slices.Delete(s, i, i+1) // remove element i; classic form: append(s[:i], s[i+1:]...)
s = slices.Insert(s, i, x)    // insert x at index i, shifting the rest right
```

Stack and queue — a slice is all you need:

```
stack = append(stack, x)           // push
top := stack[len(stack)-1]         // peek (read the last element)
stack = stack[:len(stack)-1]       // pop (drop the last element)

queue = append(queue, x)           // enqueue
front := queue[0]                  // peek
queue = queue[1:]                  // dequeue (see the Watch out below)
```

Watch out. The `queue = queue[1:]` trick never reclaims the front cells: the backing array keeps growing as the head pointer walks forward, even though `len` stays small. For a long-lived, high-throughput queue, use a ring buffer or `container/list` instead.

Iterating copies values. `range` gives you a *copy* of each element. Writing to the loop variable does nothing to the slice:

```
for _, v := range s {
    v = 0 // changes only the copy; s is untouched
}
for i := range s {
    s[i] = 0 // THIS edits the slice (write through the index)
}
```

Passing slices to functions

A slice is passed **by value**, like everything in Go (Chapter 7 — Pointers), but the value *is the three-field header*. So the function gets its **own** copy of `ptr/len/cap`, and that copy points at the **same backing array** as the caller. The consequence is subtle but important:


```

func fill(s []int) {
    for i := range s {
        s[i] = i * i    // writes go through the shared backing array
    }
}

func tryAppend(s []int) {
    s = append(s, 999)    // reassigns the LOCAL header copy only
}

func main() {
    nums := make([]int, 3)
    fill(nums)
    fmt.Println(nums)    // [0 1 4] -- element writes ARE visible

    tryAppend(nums)
    fmt.Println(nums)    // [0 1 4] -- the append is NOT visible
}

```

- **Element writes are visible** to the caller, because both headers point at the same array. (This is like passing `int *` in C.)
- **append inside the function may or may not be visible.** It reassigns the callee’s local header, which the caller never sees. If `append` also reallocated, the callee is now writing to a *different* array entirely. Either way, the caller’s `len` does not change.

Rule of thumb. If a function needs to grow a slice and the caller must see the result, **return the slice**: `s = grow(s)`. That is why the standard library’s own `append` returns the slice instead of mutating in place.

Multidimensional data: slices of slices

Go has no built-in 2-D slice. You build one as a slice of slices, where each row is its own slice (and rows can even have different lengths — a “jagged” array).

```

rows, cols := 3, 4
grid := make([][]int, rows)    // a slice of 3 (nil) int-slices
for i := range grid {
    grid[i] = make([]int, cols) // give each row its own backing array
}
grid[1][2] = 7

```

C vs Go. A C `int grid[3][4]` is one contiguous block of 12 ints. A Go `[] []int` is 3 separate row arrays plus a header array that points at them — *not* contiguous. If you need one flat block for performance, allocate `make([]int, rows*cols)` and index it as `flat[r*cols+c]` yourself.

Strings: immutable UTF-8 bytes

A Go **string** is a read-only sequence of bytes. Like a slice, it is a small header — but with only **two** fields: a pointer and a length.

string header (2 machine words)

s = ptr len = 6 read-only bytes (UTF-8), no NUL needed

Three facts change everything for a C programmer:

1. **A string is immutable.** You cannot assign to `s[i]`. To “change” a string you build a new one.
2. **A string carries its length.** It is *not* NUL-terminated. A string may contain a 0 byte with no trouble, and finding its length is $O(1)$, not a scan.
3. **A string is UTF-8.** Source code is UTF-8, and string literals are stored as UTF-8 bytes.

```
s := "héllö"
// s[0] = 'H'           // compile error: cannot assign to s[0]
s = "H" + s[1:]         // build a NEW string instead
```

C vs Go. A C string is `char *`: a pointer to bytes ending in `'\0'`, with no length and no character encoding (it is just bytes). A Go string knows its byte length, is immutable, and is understood as UTF-8 text. `len(s)` is instant; in C you call `strlen`, which walks to the NUL.

len and indexing are about *bytes*, not characters

This is the number-one string surprise. `len(s)` is the number of **bytes**, and `s[i]` is a single **byte** (type `byte`, an alias for `uint8`) — not a character.

```
s := "héllö"
fmt.Println(len(s))           // 6, NOT 5 - 'é' takes two bytes in UTF-8
fmt.Println(s[0])             // 104 - a byte value (the code for 'h'), printed as a number
fmt.Printf("%c\n", s[1])      // Ã - s[1] is just the FIRST byte of 'é', not 'é'
```

Here is the byte layout of `"héllö"`. The character `é` (Unicode code point `U+00E9`) needs two bytes (`0xC3 0xA9`); the ASCII letters need one each:

string `"héllö"` -> `len(s) == 6 bytes`, but 5 characters (runes)

byte index:	0	1	2	3	4	5
-------------	---	---	---	---	---	---

bytes:	0x68	0xC3	0xA9	0x6C	0x6C	0x6F
--------	------	------	------	------	------	------

	'h'	'é'	'l'	'l'	'ö'
--	-----	-----	-----	-----	-----

code point:	U+0068	U+00E9	U+006C	U+006C	U+006F
-------------	--------	--------	--------	--------	--------

A non-Latin character takes even more bytes. The CJK character (`U+4E16`) is three UTF-8 bytes (`0xE4 0xB8 0x96`), so `len(" ")` is 3.

range over a string yields *runes*

A **rune** is a Unicode *code point*: one character’s numeric value. The type `rune` is an alias for `int32`. When you **range** over a string, Go decodes the UTF-8 for you and gives you (`byteIndex`, `rune`) for each character.

```

for i, r := range "héllo" {
    fmt.Printf("%d: %c (%U)\n", i, r, r)
}
// 0: h (U+0068)
// 1: é (U+00E9)
// 3: l (U+006C)  <- index jumps from 1 to 3: 'é' used bytes 1 and 2
// 4: l (U+006C)
// 5: o (U+006F)

```

Notice the index skips from 1 to 3, because `é` occupied two bytes. The loop variable `r` is the whole character, correctly decoded.

Watch out. To count characters, do **not** use `len(s)`. Use `utf8.RuneCountInString(s)` (from `unicode/utf8`), which returns 5 for "héllo". `len(s)` counts bytes.

Converting between strings, bytes, and runes

You convert with `[]byte(s)`, `[]rune(s)`, and `string(...)`. These conversions allocate and copy — a string is immutable, so Go cannot share its storage with a mutable `[]byte`.

```

s := "héllo"

bs := []byte(s) // a fresh, mutable copy of the 6 bytes (allocates)
bs[0] = 'H'     // editing bs does NOT change s
fmt.Println(string(bs)) // "Héllo"

rs := []rune(s) // decode UTF-8 into 5 code points (allocates)
fmt.Println(len(rs)) // 5
fmt.Println(string(rs[1])) // "é" - index a rune slice to get whole characters

```

Watch out. `string(i)` where `i` is an integer does **not** turn 65 into "65". It returns the *character* for that code point: `string(65)` is "A". To format a number as text, use `strconv.Itoa(i)` or `fmt.Sprint(i)`. (go vet flags the likely-wrong `string(int)` conversion.)

Building strings efficiently

Because strings are immutable, `+=` in a loop is $O(n^2)$: each `+=` allocates a brand-new string and copies everything so far. For building up text, use `strings.Builder`, which writes into a growing byte buffer and hands you the string once at the end (no extra copy).

```

var b strings.Builder
for i := range 3 {
    fmt.Fprintf(&b, "row %d\n", i) // Builder is an io.Writer
}
result := b.String()
fmt.Print(result)
// row 0
// row 1
// row 2

```

C vs Go. This is the same idea as growing a `char` buffer with a length and `realloc` instead of calling `strcat` in a loop (which re-scans for the NUL each time). `strings.Builder` does the buffer management for you, safely.

Key takeaways

- An **array** `[N]T` has a fixed size that is part of its type, and it is a **value**: assigning or passing it **copies** every element. (The opposite of C, where arrays decay to pointers.) You use arrays directly only rarely.
- A **slice** `[]T` is a three-field header `{ptr, len, cap}` — a fat pointer into a heap backing array that the GC frees. This is the workhorse type.
- `len` is what you can index; `cap` is room before reallocation. Make slices with literals `[]T{...}`, `make([]T, len, cap)`, or by slicing.
- A **nil slice** is a safe, ready-to-use empty slice: `len`, `range`, and `append` all work on it.
- **Slicing shares the backing array** (aliasing). Writes through one slice show up in the others. The three-index slice `s[low:high:max]` caps capacity to stop `append` from reaching into shared cells.
- `append` reuses the backing array if `cap` allows, otherwise allocates a new one and copies. Always write `s = append(s, x)`.
- Passing a slice copies the *header* but shares the *array*: element writes are visible to the caller; an `append` inside the function is not. Return the slice if the caller must see growth.
- A **string** is immutable UTF-8 bytes with a length (not NUL-terminated). `len(s)` is bytes, `s[i]` is a byte, and `range s` yields runes (code points). `[]byte(s)` and `[]rune(s)` allocate. Build strings with `strings.Builder`.

Watch out (gotchas for C programmers)

- **An array is a value — it copies.** Passing `[1024]byte` to a function copies 1 KB. If you meant to share it, pass a slice (`buf[:]`) or a pointer.
- **Sub-slices alias the same memory.** `b := a[1:3]; b[0] = 9` changes `a` too.
- **append can reallocate (or not).** When it reuses a shared backing array it can overwrite a neighbor's data; when it reallocates, writes through the old slice stop being visible. Know the capacity.
- **len vs cap confusion.** `make([]int, 5)` has `len` 5 (five zeros), not an empty slice with room for 5. For “empty with room,” use `make([]int, 0, 5)`.
- **s[i] on a string is a byte, not a character, and len(string) counts bytes, not characters.** Use `range` or `[]rune` for characters.
- **Strings are immutable.** You cannot assign `s[i] = c`. Build a new string.
- **[]byte(s)/[]rune(s) allocate.** They are not free casts; they copy.
- **string(65) is "A", not "65".** Use `strconv.Itoa` for numbers.
- **A tiny sub-slice can pin a huge backing array in memory.** Returning `big[:10]` keeps all of `big` alive. To release it, copy out what you need (`out := make([]byte, 10); copy(out, big) or slices.Clone(big[:10])`).

Interview questions

Q: What is a Go slice, exactly, and how does it differ from a C “pointer plus length”?

A: A slice is a small header with three fields: a pointer to a backing array, a length, and a capacity. It is the same information you would pass in C as a pointer and a separate length, but bundled

into one value — and it also tracks capacity so `append` knows when it must reallocate. The backing array is on the heap and freed by the garbage collector, so there is no manual `free`.

Q: When does `append` allocate a new backing array, and why must you write `s = append(s, x)`? A: If the slice's length is less than its capacity, `append` writes into the existing backing array and returns a header with a larger length — no allocation. If length equals capacity, `append` allocates a larger array, copies the old elements, and returns a header pointing at the new array. Because the returned header can differ from the input (new pointer, length, or capacity), you must assign it back; ignoring the return value loses the growth and is a bug.

Q: Two slices were made by slicing the same array. One `append` to the first slice changed the second. Why? A: Slicing does not copy; both slices point into the same backing array. The first slice had spare capacity that overlapped the second slice's elements, so `append` reused that shared storage instead of reallocating, overwriting the other slice's data. The fix is to cap the first slice's capacity with a three-index slice (`a[low:high:high]`) or to copy the data so the slices no longer share an array.

Q: Why is `len(s)` for a string sometimes larger than the number of characters you see? A: Go strings are UTF-8, and `len` counts bytes. Characters outside ASCII take more than one byte (é is 2 bytes, many CJK characters are 3), so the byte count exceeds the character count. To count characters, use `utf8.RuneCountInString(s)`; to iterate characters, `range` over the string, which yields runes (code points).

Q: How can a small slice cause a memory leak, and how do you prevent it? A: A slice keeps its entire backing array alive as long as the slice exists. If you slice a small window out of a very large array and keep only that window, the whole large array cannot be collected. To release it, copy the needed elements into a fresh, small slice (`out := make([]T, n); copy(out, window)` or `slices.Clone(window)`) and drop the reference to the large one.

Try it

1. Make `a := []int{1, 2, 3, 4, 5}`, then `b := a[1:4]`. Print `len(b)` and `cap(b)`. Do `b = append(b, 99)` and print `a`. Explain what happened. Then redo it with `b := a[1:4:4]` and see `a` stay unchanged.
2. Loop `s := ""` and `s += "x"` ten thousand times, then do the same with a `strings.Builder`. Time both with `time` (or a quick benchmark from Chapter 21 — Testing). Notice how much the immutable-string version copies.

Chapter 9 — Maps

What you'll learn. Go's built-in hash table: how to create, read, write, and delete entries; the comma-ok idiom; why iteration order is random on purpose; what may be used as a key; and the traps — a `nil-map` write panics, values are not addressable, and maps are not safe for concurrent use.

In C there is no built-in hash table. You either hand-roll one (an array of buckets, a hash function, linked lists for collisions) or pull in a library. Go builds the hash table into the language as the `map`, written `map[K]V`: a collection of key-value pairs with average $O(1)$ lookup, insert, and delete.

Creating a map

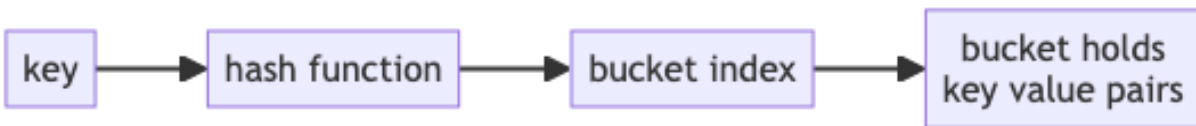
A map type is `map[K]V`: `K` is the key type, `V` is the value type. You create one with `make` or a literal.

```
// make: an empty, ready-to-use map
ages := make(map[string]int)

// literal: create and fill in one step
caps := map[string]string{
    "France": "Paris",
    "Japan":  "Tokyo", // the trailing comma is required by gofmt
}

// empty literal: same as make(map[string]int)
empty := map[string]int{}
```

C vs Go. Think of `map[string]int` as a generic, type-safe version of a C hash table where the keys are strings and the values are ints — but you write no hashing, no buckets, no collision lists, and no resize logic. It is all in the runtime.



Internally a map is an array of **buckets**. The runtime hashes the key to pick a bucket, then checks the keys in that bucket (collisions land in the same bucket). As the map fills, it grows and rehashes. You never see any of this; the diagram is just so you know what is under the hood:

```
ages := map[string]int{"alice":30, "bob":25, "carol":40}

hash("alice") -> bucket 0      hash("bob")   -> bucket 2
hash("carol") -> bucket 0      (collision: shares bucket 0 with alice)

buckets

0      alice=30 , carol=40      <- two keys, same bucket

1      (empty)

2      bob=25
```

Basic operations

```
m := make(map[string]int)

m["alice"] = 30      // set (insert or overwrite)
m["alice"] = 31      // overwrite: now 31
```

```
age := m["alice"]           // get: 31
n := len(m)                 // number of entries: 1

delete(m, "alice")         // remove the entry; no-op if the key is absent
clear(m)                    // remove ALL entries (Go 1.21+)
```

- **Set:** `m[k] = v` inserts the pair, or replaces the value if `k` already exists.
- **Get:** `v := m[k]` reads the value.
- `delete(m, k)` removes the entry. Deleting a missing key does nothing (no error, no panic). `delete` on a nil map is also a safe no-op.
- `len(m)` is the number of entries. There is no `cap` for maps.
- `clear(m)` empties the map in one call (Go 1.21+). Before 1.21 you looped with `delete`.

A missing key returns the zero value

Reading a key that is **not** in the map is not an error. Go returns the **zero value** of the value type (Chapter 4 — Types, Variables, and Constants): 0 for `int`, "" for `string`, `false` for `bool`, `nil` for pointers and slices.

```
m := map[string]int{"alice": 31}
fmt.Println(m["alice"]) // 31
fmt.Println(m["bob"])   // 0 -- "bob" is absent, so you get the zero int
```

This is convenient but ambiguous: if `m["bob"]` is 0, you cannot tell whether "bob" is **absent** or **present with the value 0**. That is what the comma-ok idiom solves.

The comma-ok idiom

A map read can return a **second** boolean value: `true` if the key was present, `false` if it was absent.

```
v, ok := m["bob"]
if ok {
    fmt.Println("bob is present, value =", v)
} else {
    fmt.Println("bob is absent") // v is the zero value here
}
```

Rule of thumb. Use the one-value form `v := m[k]` when the zero value is a fine answer for “missing.” Use the two-value form `v, ok := m[k]` whenever you must tell “absent” apart from “present but zero” — for example a map of counts, scores, or flags where 0/`false` is a real value.

A common pattern is to act only when a key is missing:

```
if _, ok := m[key]; !ok {
    m[key] = compute(key) // fill it in the first time we see it
}
```

The zero value of a map is nil — and writing to it panics

A declared-but-not-made map is `nil`. Reading a nil map is safe (every key is “absent,” so you get zero values). **Writing to a nil map panics at runtime.**

```

var m map[string]int    // nil map (no hash table allocated yet)

fmt.Println(m["x"])      // 0    -- reading nil is fine
fmt.Println(len(m))     // 0    -- fine
_, ok := m["x"]         // ok == false, fine

m["x"] = 1              // PANIC: assignment to entry in nil map

```

The fix is to allocate the map with `make` (or a literal) before writing:

```

m = make(map[string]int) // now it is a real, writable map
m["x"] = 1               // fine

```

C vs Go. A nil map is like a NULL pointer to your hash table: you can ask “is anything there?” and get “no,” but you cannot store into it until you allocate the structure. Unlike a nil **slice**, which you *can* **append** to, a nil **map** must be **make**-d before its first write.

Watch out. This bites most often with struct fields and maps inside maps. A freshly zero-valued struct has its map fields set to nil; you must **make** them before writing. Same for `m[k]` when the value type is itself a map.

Iteration order is randomized on purpose

You loop over a map with `range`, which yields (key, value) pairs:

```

for k, v := range m {
    fmt.Println(k, v)
}

```

The **order is not defined, and Go deliberately randomizes it** on every run. This is intentional: it stops programs from accidentally depending on an order the map does not promise. If you need a stable order, **collect the keys and sort them**.

```

import (
    "fmt"
    "slices"
)

func printSorted(m map[string]int) {
    keys := make([]string, 0, len(m)) // pre-size to avoid reallocation
    for k := range m {                // range with one variable = keys only
        keys = append(keys, k)
    }
    slices.Sort(keys)                 // sort the keys (Go 1.21+)
    for _, k := range keys {
        fmt.Println(k, m[k])
    }
}

```

Since Go 1.23 you can do the same in one line with iterators:


```
import (
    "maps"
    "slices"
)

for _, k := range slices.Sorted(maps.Keys(m)) {
    fmt.Println(k, m[k])
}
```

C vs Go. A C hash table you wrote yourself probably iterated buckets in some fixed internal order, which felt stable but was an accident of the implementation. Go makes the randomness explicit so you never lean on it.

Keys must be comparable

A key type must support `==` and `!=`. The runtime uses equality to find a key within its bucket. So:

- **Allowed keys:** all numbers, `string`, `bool`, pointers, channels, interfaces, and **structs or arrays whose fields/elements are all comparable**.
- **Forbidden keys:** `slice`, `map`, and `func` — these are not comparable, and using one as a key is a **compile error**.

```
type point struct{ X, Y int } // all fields comparable -> usable as a key

grid := map[point]string{
    {0, 0}: "origin",
    {1, 2}: "somewhere",
}

fmt.Println(grid[point{0, 0}]) // "origin"

// bad := map[[]int]string{} // compile error: invalid map key type []int
```

Watch out. A struct used as a key is compared **field by field**. Two keys are the “same” only if every field is equal. If the struct contains a non-comparable field (like a slice), the whole struct becomes unusable as a key and you get a compile error.

Maps are reference-like

A map value is a small header that points at the underlying hash table (much like a slice points at a backing array). When you pass a map to a function or assign it to another variable, you copy the *header*, not the table — both names refer to the **same** map. Changes through one are visible through the other.

```
func addOne(m map[string]int, key string) {
    m[key]++ // edits the caller's map; no pointer needed
}

func main() {
    counts := map[string]int{}
    addOne(counts, "hits")
    addOne(counts, "hits")
}
```

```
    fmt.Println(counts["hits"]) // 2
}
```

C vs Go. You do not pass `*map[K]V` to let a function mutate a map; passing the map by value already shares the underlying table. (You would only take a pointer to a map in the rare case you need to *replace the whole map* and have the caller see the new one.)

Maps are not safe for concurrent use

A map is **not** protected against concurrent access. If one goroutine writes to a map while another reads or writes the **same** map, the runtime may detect it and crash the whole program with a fatal error:

```
fatal error: concurrent map read and map write
```

This is not a recoverable panic — it stops the process. To share a map across goroutines, guard it with a `sync.Mutex`, or use `sync.Map` for some workloads. Concurrency, mutexes, and `sync.Map` are covered in Chapter 15 — Synchronization and context.

Watch out. Concurrent-only-reads are fine; the danger is a write happening at the same time as any other access. The race detector (`go test -race`, Chapter 22 — Tooling) finds these bugs quickly.

The set idiom (Go has no built-in set)

Go has no built-in set type, but a map gives you one. Two common spellings:

```
// Option A: map to bool - simple and readable
seen := map[string]bool{}
seen["a"] = true
if seen["a"] {           // missing keys read as false, which is what you want
    fmt.Println("a is in the set")
}
delete(seen, "a")

// Option B: map to empty struct - uses zero bytes per value
set := map[string]struct{}{}
set["a"] = struct{}{}
_, exists := set["a"]    // membership via comma-ok
```

`struct{}` is the **empty struct**: a type that occupies zero bytes. `map[K]struct{}` stores only the keys, so it is the most memory-efficient set. `map[K]bool` is easier to read because membership is just `if set[k]`. Either is idiomatic.

Map values are not addressable

You cannot take the address of a map value: `&m[k]` is a **compile error**. The reason is practical — when the map grows, it moves its entries to new memory, so any pointer into it would dangle.

A direct consequence bites C programmers immediately: if the value type is a **struct**, you cannot assign to one of its fields in place.

```

type counter struct{ n int }

m := map[string]counter{"x": {}}

// m["x"].n++           // COMPILER ERROR: cannot assign to struct field m["x"].n
// p := &m["x"]         // COMPILER ERROR: cannot take the address of m["x"]

fmt.Println(m["x"].n) // reading a field is fine

You have two clean fixes:

// Fix 1: read the whole value, modify the copy, write it back.
v := m["x"]
v.n++
m["x"] = v

// Fix 2: store POINTERS as the values. Then the value IS a pointer, and the
// thing it points at is freely mutable.
m2 := map[string]*counter{"x": {}}
m2["x"].n++           // OK: m2["x"] is a *counter; this edits the pointed-to struct
fmt.Println(m2["x"].n) // 1

```

Rule of thumb. If you will frequently mutate the values in place, make the value type a pointer (`map[K]*V`). If the values are small and you mostly read or replace them wholesale, `map[K]V` with the read-modify-write pattern is simpler and avoids extra allocations.

Key takeaways

- A map `map[K]V` is Go's built-in hash table: average $O(1)$ get/set/delete, with no hashing or bucket code to write yourself. Create it with `make` or a literal.
- Operations: `m[k] = v` (set), `v := m[k]` (get), `v, ok := m[k]` (comma-ok), `delete(m, k)`, `len(m)`, and `clear(m)` (Go 1.21+).
- A **missing key returns the zero value**, not an error. Use comma-ok to tell “absent” from “present but zero.”
- A map's zero value is `nil`. Reading `nil` is safe; **writing to `nil` panics**. Always make (or use a literal) before the first write.
- **Iteration order is randomized** deliberately. To get order, collect keys and `slices.Sort` them (or `slices.Sort(maps.Keys(m))` on Go 1.23+).
- **Keys must be comparable** (`==`). Numbers, strings, bools, pointers, and comparable structs/arrays are fine; slices, maps, and funcs are not.
- Maps are **reference-like**: passing one to a function shares the same underlying table, so the function can mutate it without a pointer.
- Maps are **not safe for concurrent use**; a concurrent read+write triggers a fatal runtime error. Use a mutex or `sync.Map` (Chapter 15 — Synchronization and context).
- A **set** is a map: `map[T]bool` (readable) or `map[T]struct{}` (zero-size values).
- **Map values are not addressable**. `m[k].field = x` and `&m[k]` do not compile. Replace the whole value, or use `map[K]*V`.

Watch out (gotchas for C programmers)

- **Writing to a nil map panics** (assignment to entry in nil map). Unlike a nil slice — which you can **append** to — a nil map must be **make**-d first.
- **m[k] never fails for a missing key**. It returns the zero value, so 0 or "" may mean “absent” or “really zero.” Use `v, ok := m[k]` when it matters.
- **Iteration order changes every run**. Never rely on it; sort keys when you need order, especially in tests and printed output.
- **Map values are not addressable**. You cannot do `&m[k]` or assign to `m[k].field`. Use read-modify-write or `map[K]*V`.
- **Maps are not concurrency-safe**. A simultaneous read and write is a fatal error that kills the process, not a recoverable panic. Protect shared maps.
- **Bad key types fail at compile time**. Slices, maps, and functions cannot be keys; a struct key with any non-comparable field is rejected too.
- **len(m) exists but there is no cap(m)**, and you cannot pre-size a map’s contents the way you size a slice (though `make(map[K]V, hint)` gives a capacity *hint* to reduce rehashing).

Interview questions

Q: What does reading a key that is not in the map return, and how do you detect a missing key? A: It returns the zero value of the value type (for example 0, "", false, or nil) and does not error. To distinguish “absent” from “present with a zero value,” use the comma-ok form `v, ok := m[k]`, where `ok` is true only if the key exists.

Q: What happens if you write to a nil map, and how is that different from a nil slice?

A: Writing to a nil map panics at runtime with “assignment to entry in nil map”; you must **make** the map (or use a literal) before the first write. Reading a nil map is safe and returns zero values. A nil slice is friendlier: you can **append** to it directly, because **append** allocates a backing array as needed.

Q: Why is map iteration order randomized, and how do you iterate in a defined order?

A: The Go runtime randomizes iteration order on purpose so programs cannot accidentally depend on an order the map never guarantees. To iterate in order, copy the keys into a slice, sort the slice (`slices.Sort`, or `slices.Sorteds(maps.Keys(m))` on Go 1.23+), and then range over the sorted keys.

Q: Why can’t you write `m[k].field = x` when the value is a struct? A: Map values are not addressable, because the runtime may relocate entries when the map grows, which would invalidate any pointer into it. Since you cannot take the address of `m[k]`, you cannot assign to a field of it in place. Either read the value into a variable, modify it, and assign it back (`v := m[k]; v.field = x; m[k] = v`), or store pointers as the values (`map[K]*V`) so the value itself is an addressable pointer.

Q: Are Go maps safe to use from multiple goroutines? A: No. Concurrent reads alone are fine, but a write happening at the same time as any other read or write is a data race; the runtime often detects it and aborts the program with a fatal “concurrent map read and map write” error. Protect a shared map with a `sync.Mutex`/`sync.RWMutex`, or use `sync.Map` for workloads it suits (see Chapter 15 — Synchronization and context).

Q: What types can be used as map keys? A: Any comparable type — one that supports `==`. That includes all numeric types, strings, booleans, pointers, channels, interfaces, and structs or

arrays composed entirely of comparable types. Slices, maps, and functions are not comparable and cannot be keys; a struct containing such a field cannot be a key either, and using one is a compile error.

Try it

1. Build a word-frequency counter: read a string, split it on spaces, and count each word in a `map[string]int` with `counts[w]++`. Then print the words in sorted order. Notice that `counts[w]++` works even the first time, because the missing key reads as 0.
2. Declare `var m map[string]int` and try `m["x"] = 1`. Watch it panic. Add `m = make(map[string]int)` before the write and watch it succeed.

Chapter 10 — Structs and Methods

What you'll learn. How Go structs compare to C structs (layout, padding, copying, comparison), how to attach *methods* to a type, when to use a value versus a pointer receiver, and how *embedding* gives you composition — which is not the same as inheritance.

A Go **struct** is the same idea as a C **struct**: a fixed group of named fields laid out in memory in declared order. If you understand C structs, you already understand most of this. Go then adds three things C does not have: **methods** (functions attached to a type), **visibility by capitalization** (see Chapter 3 — Program Structure: Packages, Imports, and Visibility), and **embedding** (one struct reusing another). This chapter walks each one, always against the C you know.

Declaring a struct

```
/* C */
struct circle {
    double x, y;    /* center */
    double radius;
};

// Go
type Circle struct {
    X, Y    float64 // center
    Radius float64
}
```

The shape is the same. The differences are small but real:

- The type keyword is `type Name struct { ... }`, and the **field type comes after the field name** (`Radius float64`, not `float64 Radius`).
- Fields starting with an **uppercase** letter are *exported* (visible to other packages); lowercase fields are package-private. This is Go's **static**, decided by spelling, not a keyword (Chapter 3).
- There is no trailing semicolon and no separate **typedef** — `type Circle struct` both declares the layout and names the type.

Creating struct values

A **struct literal** builds a value. Go has two forms.

```
c1 := Circle{0, 0, 5}           // positional: order must match the fields
c2 := Circle{X: 0, Y: 0, Radius: 5} // named: clear, order-free, robust
c3 := Circle{Radius: 5}         // omitted fields take the zero value
```

Rule of thumb. Prefer the **named** form (`Circle{Radius: 5}`). The positional form breaks silently if someone reorders or adds a field, and it is unreadable for anything past two fields. Go even lets you *omit* fields you do not set; they get the zero value.

The zero-value struct

In C, a local struct `circle c;` contains garbage until you set it. In Go a declared struct is **fully zeroed**: every numeric field is 0, every string "", every pointer, slice, and map nil, every bool false (see Chapter 4 — Types, Variables, and Constants).

```
var c4 Circle // {0, 0, 0} - ready to use, never garbage
```

C vs Go. C's struct `circle c = {0};` is the closest analog, but you must *remember* to write it. In Go the zero value is automatic and guaranteed. Idiomatic Go leans on this: design a struct so its zero value is already useful (a `bytes.Buffer{}` is an empty, ready buffer; a `sync.Mutex{}` is unlocked).

new, anonymous structs, and nesting

`new(T)` allocates a zeroed `T` and returns a `*T` (a pointer). `&Circle{...}` does the same and lets you set fields at once; the second form is far more common.

```
p := new(Circle)           // *Circle, all fields zero
q := &Circle{Radius: 5}    // *Circle, Radius set - the usual way
```

An **anonymous struct** has no named type. It is handy for a one-off grouping, for example a table-driven test row or a small JSON shape.

```
point := struct{ X, Y int }{X: 1, Y: 2}
```

Structs **nest** by value, exactly like C: the inner struct's bytes live *inside* the outer struct, not behind a pointer.

```
type Point struct{ X, Y float64 }
type Segment struct {
    Start Point
    End   Point
}
s := Segment{Start: Point{0, 0}, End: Point{3, 4}}
```

Field tags (a preview)

A **field tag** is a string literal after a field. The compiler ignores it; libraries read it by reflection. The most common use is `encoding/json`:

```

type User struct {
    Name  string `json:"name"`
    Email string `json:"email,omitempty"` // skip this key if the field is empty
}

```

You will see this in detail in Chapter 20 — Standard Library Tour. For now just note: tags are metadata, and only **exported** fields are visible to `json` and similar packages.

Memory layout: just like C

A Go struct is laid out in memory in **declared field order**, with **padding** inserted so each field meets its alignment requirement — the same rules a C compiler follows. You can inspect it with the `unsafe` package.

```

import "unsafe"

type Bad struct {
    a bool // 1 byte
    b int64 // 8 bytes (wants 8-byte alignment)
    c bool // 1 byte
}

type Good struct {
    b int64 // 8 bytes
    a bool // 1 byte
    c bool // 1 byte
}

```

```

unsafe.Sizeof(Bad{}) // 24
unsafe.Sizeof(Good{}) // 16

```

Bad wastes 8 bytes purely on padding. Here is why, byte by byte:

Bad{a bool; b int64; c bool} -> 24 bytes

```

byte:  0    1 2 3 4 5 6 7      8 ..... 15      16    17 ..... 23
      +---+-----+ +---+-----+ +---+ +---+-----+
      | a | padding (7) | | b (int64) | | c | | padding (7) |
      +---+-----+ +---+-----+ +---+ +---+-----+
      ^   b must start on an 8-byte boundary   ^   tail padding so the
      offset 0                               offset 16   size is a multiple of 8

```

Good{b int64; a bool; c bool} -> 16 bytes

```

byte:  0 ..... 7      8    9    10 ..... 15
      +-----+ +---+ +---+ +-----+
      | b (int64) | | a | | c | | padding (6) |
      +-----+ +---+ +---+ +-----+

```

By moving the big field first you remove a hole. **You can reorder fields to shrink a struct** — exactly the trick you would use in C. Useful tools:

- `unsafe.Sizeof(x)` — total bytes, including padding.
- `unsafe.Alignof(x)` — required alignment (the largest field's alignment).
- `unsafe.Offsetof(s.field)` — byte offset of a field inside the struct.

Rule of thumb. Do not obsess over field order. Reorder only for structs you allocate in huge numbers (millions), where the savings matter. Otherwise, order fields for readability. Reading sizes with `unsafe.Sizeof` is harmless and is a compile-time constant; only *unsafe pointer conversions* are risky, and you will rarely need them.

Comparing and copying structs

Two struct values are equal with `==` if **all their fields are equal** — a field-by-field comparison, no `memcmp` needed. This works only when every field is itself comparable.

```
a := Point{1, 2}
b := Point{1, 2}
a == b // true
```

If a struct contains a non-comparable field (a slice, a map, or a function), the compiler **rejects** `==` outright:

```
type Bag struct{ items []int }
// Bag{} == Bag{}
// compile error: struct containing []int cannot be compared
```

Assignment and argument passing **copy the whole struct**, every byte, just like C. There are no hidden references.

```
c := a // c is an independent copy
c.X = 99
// a.X is still 1
```

C vs Go. This matches C's value semantics. `f(c)` passes a *copy*; to let a function modify the original (or to avoid copying a large struct), pass a pointer `f(&c)` — the same choice you make in C. See Chapter 7 — Pointers.

Methods

A **method** is a function with a *receiver*: an extra parameter written *before* the function name. Think of the receiver as C's explicit `self` pointer, but the language wires it up for you and lets you call with dot syntax.

```
/* C: a free function takes the struct explicitly */
double circle_area(const struct circle *c) {
    return 3.14159 * c->radius * c->radius;
}
/* call: circle_area(&c); */

// Go: the receiver (c Circle) is bound before the name
func (c Circle) Area() float64 {
    return math.Pi * c.Radius * c.Radius
}
// call: c.Area()
```


Value receiver vs pointer receiver

The receiver can be a **value** (`c Circle`) or a **pointer** (`c *Circle`). The difference is the same as passing a struct by value versus by pointer in C.

```
func (c Circle) Area() float64 { // value receiver: gets a COPY
    return math.Pi * c.Radius * c.Radius
}

func (c *Circle) Scale(f float64) { // pointer receiver: can mutate the original
    c.Radius *= f
}
```

Question	Value receiver (<code>c T</code>)	Pointer receiver (<code>c *T</code>)
Sees a copy or the original?	a copy	the original
Can it mutate the receiver?	no (changes are lost)	yes
Cost for a big struct	copies every byte	copies one pointer
C analog	<code>f(struct T x)</code>	<code>f(struct T *x)</code>

Use a **pointer receiver** when any of these is true:

1. The method must **modify** the receiver.
2. The struct is **large** and you want to avoid copying it on every call.
3. The type contains something that must not be copied (for example a `sync.Mutex`; see Chapter 15 — Synchronization and Context).

Rule of thumb. If any method of a type needs a pointer receiver, give *all* its methods pointer receivers. Mixing the two is a common source of confusion and method-set surprises (Chapter 11 — Interfaces). Consistency wins.

Go takes the address for you

In C you must write `circle_area(&c)` or `c->scale(...)` yourself. In Go, if a value is **addressable** (a variable, a struct field, a slice element), the compiler inserts the `&` or `*` automatically:

```
c := Circle{Radius: 2} // c is a value, not a pointer
c.Scale(3)              // Go rewrites this as (&c).Scale(3)

p := &Circle{Radius: 1} // p is a pointer
p.Area()                // Go rewrites this as (*p).Area()
```

So `c.Method()` works whether `c` is a `Circle` or a `*Circle`. The one case where it fails: a value that has no address (such as a map element, or a literal) cannot have a pointer-receiver method called on it, because there is nothing to take the address of.

Methods on any named type

Methods are **not** limited to structs. You can attach a method to *any named type* you declare — an integer, a float, a slice, a function type. This is something C cannot express at all.

```

type Celsius float64

func (c Celsius) String() string {
    return fmt.Sprintf("%.1f deg C", float64(c))
}

```

```

fmt.Println(Celsius(100)) // prints: 100.0 deg C

```

A method named `String()` `string` is special by convention: it satisfies the `fmt.Stringer` interface, so `fmt` uses it when printing (Chapter 11). The one rule: you may only define methods on types declared **in your own package** — you cannot bolt a method onto `int` or onto a type from another package.

Constructors: a convention, not a keyword

Go has **no constructors** and no `new`-with-arguments like C++. There is nothing to override and nothing that runs “automatically.” When a type needs setup beyond its zero value, the convention is a plain function named `NewType` that returns the value (usually a pointer):

```

type Buffer struct {
    data []byte
    max  int
}

func NewBuffer(max int) *Buffer {
    return &Buffer{
        data: make([]byte, 0, max),
        max:  max,
    }
}

```

```

buf := NewBuffer(4096)

```

Rule of thumb. If a type’s zero value already works, do **not** write a constructor — let callers say `var b Buffer`. Add `NewBuffer` only when you must allocate, validate, or set non-zero defaults.

Embedding: composition, not inheritance

You can declare a field with **no name** — just a type. That is **embedding**. The embedded type’s fields and methods are *promoted*: you can reach them on the outer struct as if they were declared there.

```

type Animal struct {
    Name string
}

func (a Animal) Describe() string { return a.Name + " is an animal" }
func (a Animal) Speak() string   { return "... " }

```

```

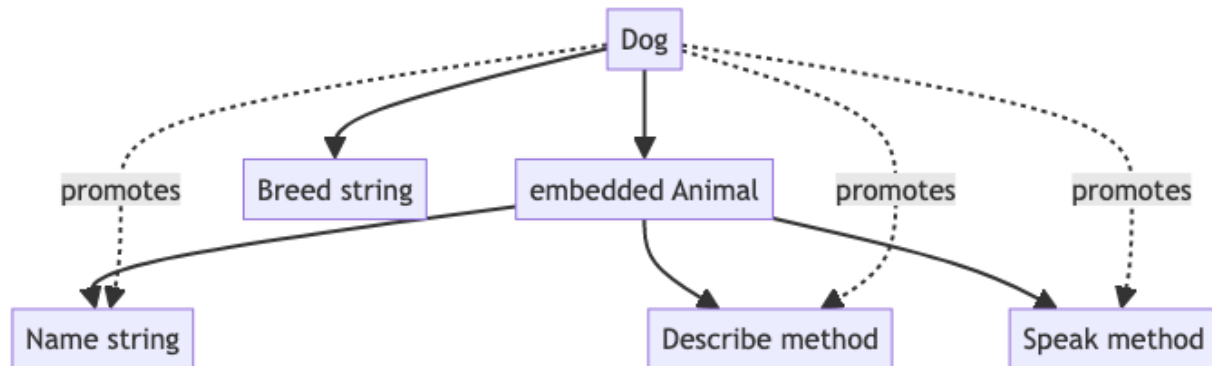
type Dog struct {
    Animal // embedded: no field name, just the type
    Breed  string
}

```

```

d := Dog{Animal: Animal{Name: "Rex"}, Breed: "Labrador"}
d.Name           // "Rex"           promoted FIELD
d.Describe()     // "Rex is an..." promoted METHOD

```



Under the hood, `Dog` simply *contains* an `Animal` value (its bytes are inside the `Dog`). Promotion is a compiler convenience: `d.Name` is shorthand for `d.Animal.Name`. Nothing is virtual, and there is no hidden base-class pointer.

Shadowing (the closest thing to “override”)

If the outer type defines a method with the **same name**, that method *shadows* the promoted one. The embedded method is still reachable by naming the field.

```

func (d Dog) Speak() string { return d.Name + " says woof" }

```

```

d.Speak()           // "Rex says woof" -> Dog's method wins
d.Animal.Speak()    // "..."         -> the embedded one, explicitly

```

C vs Go. This *looks* like inheritance, but it is not. Embedding is **composition**: a `Dog` *has an* `Animal`, it is not a *kind of* `Animal`. Crucially, there is **no subtype polymorphism**: a function that takes an `Animal` will **not** accept a `Dog`. Promotion is resolved at compile time by name, not by a runtime vtable. The only polymorphism Go offers is **interface satisfaction** (Chapter 11). When `Dog` shadows `Speak`, code that holds a plain `Animal` keeps calling `Animal.Speak` — there is no virtual dispatch that would reach `Dog.Speak`.

Concept	C++ inheritance	Go embedding
Relationship	<code>Dog is a Animal</code>	<code>Dog has an Animal</code>
Method dispatch	virtual (runtime vtable)	static (compile-time name lookup)
Pass <code>Dog</code> where base wanted	yes (slicing risks)	no (use an interface instead)

Concept	C++ inheritance	Go embedding
“Override”	virtual override	shadow by same-named method

You can also **embed an interface** (in a struct or in another interface) to require or forward a set of methods; we cover that in Chapter 11 — Interfaces.

C vs Go. C has neither methods nor inheritance: for “polymorphism” you build a struct of function pointers plus a `void *self` (think `struct file_operations`). Go’s *methods* replace the function pointers; Go’s *interfaces* (Chapter 11) replace the `void *self`, type-safely; and embedding assembles behavior by *including* other types instead of forwarding calls by hand.

Key takeaways

- A Go struct is a C struct plus methods, capitalization-based visibility, and embedding. Layout, padding, and value-copy semantics match C.
- Prefer **named** struct literals; rely on the **zero value** instead of writing needless constructors. Use `NewType` only when setup is required.
- Fields are laid out in declared order with padding; reorder big-volume structs (large field first) to shrink them. Inspect with `unsafe.Sizeof/Alignof/Offsetof`.
- `==` compares all fields and works only if every field is comparable; assignment copies the whole struct.
- A **method** has a receiver. Use a **pointer receiver** to mutate, to avoid copying a large struct, or for consistency — and if one method needs a pointer receiver, make them all pointer receivers.
- Go auto-takes the address of an addressable value, so `c.Method()` works for both values and pointers.
- Methods can be defined on **any named type**, not just structs.
- **Embedding is composition, not inheritance.** Fields and methods are promoted at compile time; there is no subtype polymorphism except interface satisfaction.

Watch out (gotchas for C programmers)

- **Value vs pointer receiver confusion.** A value-receiver method gets a *copy*; mutating the receiver does nothing to the caller’s value. If a method must change the struct, it needs a pointer receiver.
- **Do not copy a struct that contains a `sync.Mutex`** (or any field that embeds a lock). Copying the lock copies a bug. `go vet` flags this: “*passes lock by value ... contains sync.Mutex.*” Pass such structs by pointer.
- **Embedding is not inheritance.** A `Dog` is not an `Animal`; you cannot pass it where an `Animal` is required, and shadowing a method does not give you virtual dispatch.
- **Comparing structs with non-comparable fields fails to compile**, not at runtime. A struct holding a slice, map, or function cannot use `==`.
- **Large-struct copies are silent.** Passing a big struct by value copies every byte on each call. Pass a pointer when the struct is large or hot.
- **Positional literals are fragile.** Adding or reordering a field silently changes what `T{a, b, c}` means. Use named fields.

Interview questions

Q: When should a method use a pointer receiver instead of a value receiver? A: Use a pointer receiver when the method must modify the receiver, when the struct is large enough that copying it on every call is wasteful, or when the type contains something that must not be copied (such as a `sync.Mutex`). A common rule is: if any method of the type needs a pointer receiver, give all of them pointer receivers for consistency and to keep the method set uniform.

Q: What does it mean that “assignment copies a struct,” and how is that different from C++ references or Java objects? A: Assigning or passing a struct value copies every field, byte for byte, like C. There is no implicit reference or shared object: changing the copy does not affect the original. To share or mutate, you pass a pointer explicitly. Java object variables are references by default; Go struct variables are values by default.

Q: Is Go embedding the same as inheritance? A: No. Embedding is composition: the outer type *contains* the embedded type, and its fields and methods are *promoted* (resolved at compile time by name). There is no base-class pointer and no virtual dispatch. A type that embeds `Animal` is not a subtype of `Animal` and cannot be used where an `Animal` is expected. The only polymorphism in Go is interface satisfaction.

Q: Why might two structs with the same fields and values fail to compile when compared with ==? A: Because `==` on a struct requires every field to be comparable. If any field is a slice, a map, or a function, the type is not comparable and the compiler rejects `==`. You must compare such structs field by field with custom logic (for example `reflect.DeepEqual` or a hand-written `Equal` method).

Q: How do you shrink a struct’s memory footprint, and how do you measure it? A: Reorder fields so larger-aligned fields come first, which removes padding holes the compiler would otherwise insert to satisfy alignment. Measure with `unsafe.Sizeof` for total size, `unsafe.Alignof` for alignment, and `unsafe.Offsetof` for individual field offsets. This matters mainly for structs allocated in very large quantities.

Try it

1. Declare type `Pixel struct { R, G, B uint8; A uint8 }` and print `unsafe.Sizeof(Pixel{})`. Now insert a `float64` field between R and G and print the size again. Explain the jump using the alignment rules.
2. Give `Pixel` a value-receiver method `Lighten()` that adds to each channel, call it, and observe that the original is unchanged. Switch it to a pointer receiver and watch the change stick.

Chapter 11 — Interfaces

What you’ll learn. What a Go interface is, why a type satisfies it *without any implements keyword*, how an interface value is a (type, value) fat pointer, how to use `any`, type assertions, and type switches, and the two traps that bite everyone: method sets and the nil-interface bug.

In C, when you want one piece of code to work with many concrete types, you build a **struct of function pointers** plus a `void *self` to carry the data — think `struct file_operations` in the Linux kernel. You fill in the function pointers for each implementation, then call through them.

It works, but it is manual and not type-safe: nothing stops you from pairing the wrong `self` with the wrong functions.

A Go **interface** is exactly that pattern, made first-class and type-safe. This is one of the most important ideas in the language, and the chapter where Go stops looking like “C with garbage collection” and starts paying off.

What an interface is

An interface is a **set of method signatures** — a list of methods a type must have, with no implementation. The classic small example:

```
type Stringer interface {
    String() string
}
```

This says: “a `Stringer` is anything that has a method `String()` returning a `string`.” It does not say *which* types; it describes a *capability*.

```
/* The C version: a struct of function pointers + a self pointer */
struct stringer {
    void *self;
    const char *(*to_string)(void *self);
};
```

Concept	C (function-pointer struct)	Go interface
Declares required behavior	by hand, field by field	<code>type I interface { ... }</code>
Carries the data	a separate <code>void *self</code>	built in (the value half of the pair)
Type safety	none (<code>void *</code>)	full, checked by the compiler
First-class value	awkward	yes — pass it, store it, return it

Implicit satisfaction: no `implements` keyword

Here is the rule that surprises every newcomer: **a type satisfies an interface just by having the required methods**. There is no `implements`, no `: public`, no declaration of intent anywhere. If `Celsius` has a `String() string` method, then `Celsius` *is* a `Stringer` — automatically.

```
type Celsius float64

func (c Celsius) String() string {
    return fmt.Sprintf("%.1f deg C", float64(c))
}
```

```
var s Stringer = Celsius(100) // works: Celsius has String(), so it satisfies Stringer
```

The satisfaction is checked by the **compiler**, at the **point of use** — the line where you assign or pass the concrete type as the interface. If a method is missing or has the wrong signature, the build fails right there, naming the problem.

C vs Go. In C you wire the function pointers together by hand and hope you got it right. In Go the compiler does the wiring and verifies it. An interface is the **type-safe, first-class version of a C vtable**. Because no type has to *announce* which interfaces it implements, you can define a new interface and have existing types (even ones from the standard library) satisfy it without touching their code.

Mental model. An interface describes a *shape of behavior*. Any type with that shape fits the hole, the way any object with the right outline fits a cut-out — no label required.

An interface value is a (type, value) pair

This is the part C programmers must internalize. An interface value is **not** just a pointer to data. It is **two words**: a pointer to type-and-method information, and a pointer to (or copy of) the concrete value. People call it a *fat pointer* or a (type, value) pair.

```
var w io.Writer = os.Stdout
```

w (two machine words)

	type		data	
--	------	--	------	--

|

| +---> the *os.File value for stdout (the concrete data)

|

+---> itable: the concrete type (*os.File) PLUS a method table
 { Write -> (*os.File).Write, ... }

The first word points to an **itable** (“interface table”): it records the concrete type and holds the addresses of the methods that satisfy the interface — exactly a vtable. The second word points to the data. Calling `w.Write(p)` follows the itable to the right function and runs it. That is **dynamic dispatch**, the same mechanism as a C++ virtual call or a C function-pointer call, but safe and automatic.

The **zero value of any interface is nil**: both words are zero.

```
var w io.Writer // nil: (type=nil, data=nil)
w == nil        // true
```

Remember this two-word picture. The infamous nil-interface bug at the end of the chapter is a direct consequence of it.

The empty interface: any

An interface with **no methods** is satisfied by *every* type (every type has at least zero methods). Go calls it **any**, which is an alias for `interface{}`.

```
var x any           // can hold anything
x = 42
x = "hello"
x = Celsius(20)
```

C vs Go. `any` is Go's void `*`, but it remembers the real type inside the pair, so it stays type-safe: you cannot misuse the value without first asking what it is. Use **any sparingly** — it throws away the static checking that makes Go pleasant. Reach for a real interface, or generics (Chapter 19 — Generics), before falling back to `any`.

To get the concrete value back out, you **assert** its type.

Type assertions

```
var i any = "hello"

s := i.(string) // single value: panics if i is not a string
fmt.Println(s)  // hello

n, ok := i.(int) // comma-ok: never panics
fmt.Println(n, ok) // 0 false -> wrong type, n is the zero value, ok is false
```

Watch out. The single-value form `i.(int)` **panics** at runtime if the type does not match: `panic: interface conversion: interface {} is string, not int`. Use the **comma-ok** form `v, ok := i.(T)` whenever you are not certain. The compiler does not catch a bad assertion; it fails at run time.

Type switches

To branch on the concrete type, use a **type switch** — a `switch` whose cases are types. The variable `v` takes the matched type in each case.

```
func classify(i any) string {
    switch v := i.(type) {
    case nil:
        return "nil"
    case int:
        return fmt.Sprintf("int %d", v) // v is an int here
    case string:
        return fmt.Sprintf("string len %d", len(v)) // v is a string here
    case Stringer:
        return "stringer: " + v.String() // matches anything with String()
    default:
        return fmt.Sprintf("other %T", v)
    }
}
```

`%T` in `fmt` prints the dynamic type — your best friend when debugging interface values.

Interfaces you will meet everywhere

Three small interfaces from the standard library show up constantly.

error — the whole basis of Go error handling (Chapter 12 — Errors):

```
type error interface {
    Error() string
}
```



```
}
```

fmt.Stringer — how a type controls its own printed form:

```
type Stringer interface {  
    String() string  
}
```

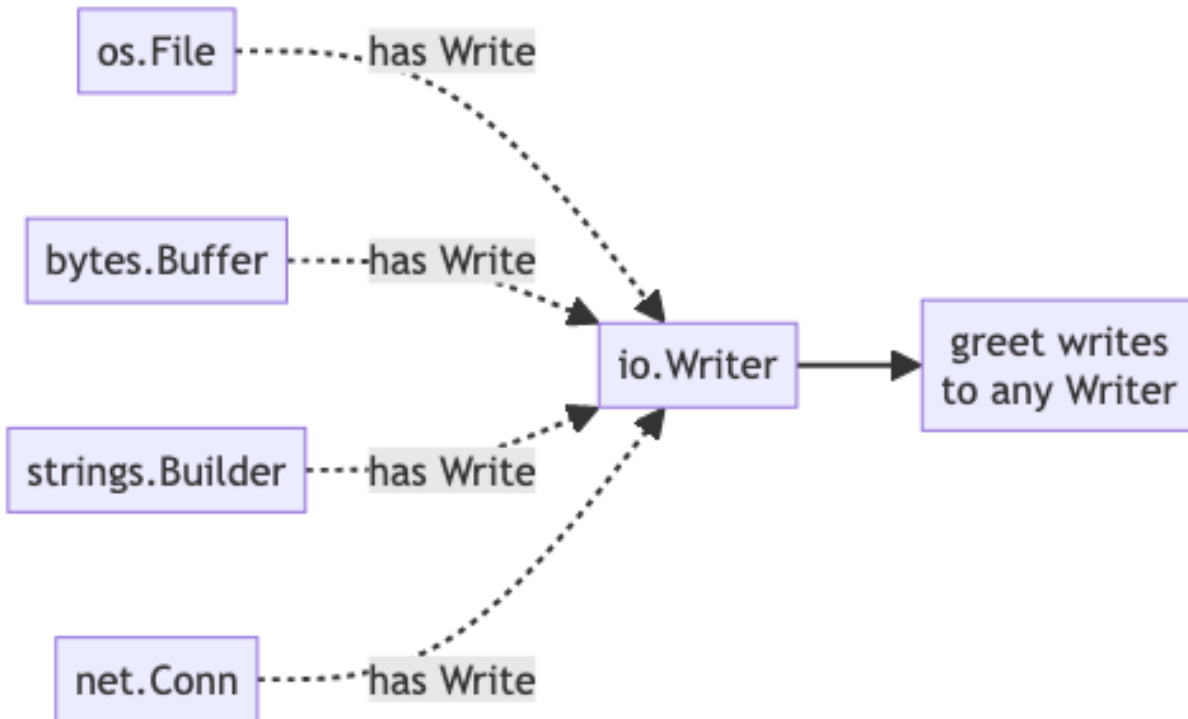
io.Writer and **io.Reader** — the universal “stream of bytes” plugs:

```
type Writer interface {  
    Write(p []byte) (n int, err error)  
}  
type Reader interface {  
    Read(p []byte) (n int, err error)  
}
```

Because satisfaction is implicit, *one* function written against **io.Writer** works with a file, an in-memory buffer, a network socket, a gzip stream, an HTTP response — anything with a **Write** method:

```
func greet(w io.Writer, name string) {  
    fmt.Fprintf(w, "Hello, %s\n", name)  
}
```

```
greet(os.Stdout, "stdout")           // *os.File  
var buf bytes.Buffer  
greet(&buf, "buffer")                // *bytes.Buffer  
var sb strings.Builder  
greet(&sb, "builder")                // *strings.Builder
```



Two pieces of hard-won Go wisdom about interfaces:

- **Keep interfaces small.** One to three methods is ideal; `io.Writer` has one. The smaller the interface, the more types satisfy it and the more reusable your code is. (“The bigger the interface, the weaker the abstraction.”)
- **Define an interface in the package that *consumes* it**, not next to the types that implement it. The consumer states the behavior it needs; any provider that has those methods fits, with no import dependency pointing the wrong way. (This is also how you break import cycles — see Chapter 3 — Program Structure: Packages, Imports, and Visibility.)

Rule of thumb. A common Go motto is “**accept interfaces, return structs.**” Take the smallest interface you need as a parameter (so callers can pass anything), but return concrete types (so callers get the full, documented type).

Method sets: which types satisfy an interface

This is the most common interface gotcha, and it ties back to value vs pointer receivers from Chapter 10 — Structs and Methods.

A type’s **method set** is the set of methods you can call on it through an interface. The rule:

- The method set of `T` includes only its **value-receiver** methods.
- The method set of `*T` includes **both** value- and pointer-receiver methods.

So if a method is declared with a **pointer receiver**, only `*T` has it in its method set — and therefore **only `*T` satisfies an interface that requires it.**

```
type Greeter interface{ Greet() string }
```

```

type Person struct{ Name string }

func (p *Person) Greet() string { return "Hi, " + p.Name } // POINTER receiver

var g Greeter
g = &Person{Name: "Sam"} // OK: *Person has Greet
// g = Person{Name: "Sam"} // COMPILE ERROR:
//   Person does not implement Greeter (method Greet has pointer receiver)

```

The error message even tells you why: “method Greet has pointer receiver.” The fix is to use a pointer (&Person{...}), or — if it is safe — to switch the method to a value receiver.

Watch out. Why can T *not* borrow the pointer methods? Because to call a pointer-receiver method, Go needs the value’s **address**, and an arbitrary interface value may hold an unaddressable copy. So the language simply leaves pointer methods out of T’s method set. This is the deep reason behind the Chapter 10 advice: **pick one receiver kind per type**.

The nil-interface trap (the #1 interface bug)

Recall that an interface is a (type, value) pair, and it is nil only when **both** halves are nil. An interface that holds a **nil pointer of a concrete type** is **not** nil — its type half is set.

```

type MyError struct{ Msg string }

func (e *MyError) Error() string { return e.Msg }

// BUG: this returns a non-nil error even when nothing failed.
func doThing() error {
    var p *MyError // p is a nil *MyError
    // ... no failure, p stays nil ...
    return p // returns interface (type=*MyError, value=nil) - NOT nil!
}

func main() {
    err := doThing()
    fmt.Println(err == nil) // prints: false  <-- surprise and bug
}

```

p is a nil pointer, but the moment it is stored in the **error** interface, the interface’s *type* word becomes ***MyError**. The pair is (***MyError**, nil), which is **not** equal to (nil, nil). So **err != nil**, and every if **err != nil** check upstream fires on a “success.”

```

return nil (literal):      (type=nil,      value=nil)    -> == nil is TRUE
return p   (nil *MyError): (type=*MyError, value=nil)    -> == nil is FALSE

```

The fix is to **return the untyped nil literal** on success, never a typed nil pointer:

```

func doThing() error {
    if somethingFailed() {
        return &MyError{Msg: "boom"}
    }
    return nil
}

```

```

}
return nil // correct: the literal nil, type half stays empty
}

```

Rule of thumb. Functions that return error should `return nil` directly on success. Never declare `var err *MyConcreteError` and return it; assign and return only when you actually have an error. This single rule prevents the most confusing bug in Go.

The compile-time satisfaction check

Because satisfaction is implicit, nothing forces a type to *prove* it implements an interface until someone uses it that way. To assert it on purpose — and get a clear compile error if you break it — use this zero-cost idiom:

```

var _ io.Writer = (*bytes.Buffer)(nil)
var _ fmt.Stringer = Celsius(0)

```

Each line declares a throwaway variable (`_` discards it) of the interface type and assigns a nil concrete value. It emits **no runtime code**; it just makes the compiler verify, right there, that `*bytes.Buffer` is an `io.Writer` and `Celsius` is a `fmt.Stringer`. Put it next to a type to document intent and to fail fast if a method signature ever drifts.

Embedding interfaces

Interfaces can **embed** other interfaces to compose a larger set of methods. The standard library does this constantly:

```

type Reader interface{ Read(p []byte) (int, error) }
type Writer interface{ Write(p []byte) (int, error) }

// ReadWriter requires BOTH Read and Write.
type ReadWriter interface {
    Reader
    Writer
}

```

A type satisfies `ReadWriter` if it has both `Read` and `Write`. This is the same composition idea as struct embedding (Chapter 10), applied to method sets. You can also embed an interface in a *struct* to forward its methods — useful for wrappers that override one method and pass the rest through.

Key takeaways

- An interface is a **set of method signatures** — a capability, not a concrete type. It is the type-safe, first-class version of a C struct of function pointers.
- **Satisfaction is implicit:** a type implements an interface by having the methods. There is no `implements` keyword; the compiler checks at the point of use.
- An interface value is a **(type, value) pair** (a fat pointer: itable + data). Its zero value is `nil`; method calls dispatch dynamically through the itable.
- **any** (alias for `interface{}`) holds any value. Use it sparingly. Recover the concrete value with a **type assertion** `v, ok := i.(T)` or a **type switch**.

- The single-value assertion `i.(T)` **panics** on mismatch; the comma-ok form does not.
- Favor **small interfaces** (1–3 methods) and **define them in the consumer** package. “Accept interfaces, return structs.”
- **Method sets:** `*T` has value- and pointer-receiver methods; `T` has only value-receiver methods. A pointer-receiver method means only `*T` satisfies the interface.
- The **nil-interface trap:** an interface holding a typed nil pointer is **not** nil. Return the literal nil for success.

Watch out (gotchas for C programmers)

- **A nil interface is not the same as an interface holding a nil pointer.** The pair (`type=*T`, `value=nil`) is not equal to nil. This is the #1 interface bug; always **return** nil literally on success.
- **Value vs pointer method sets.** If a method has a pointer receiver, a *value* of the type does not satisfy the interface — only a pointer does. Pass `&x`, not `x`.
- **Overusing any.** It is `void *` with a memory of the type, but it discards Go’s static checking and pushes errors to run time. Prefer a real interface or generics.
- **Type assertion without comma-ok panics.** `i.(T)` crashes at run time on a wrong type; use `v, ok := i.(T)` unless a mismatch truly should abort.
- **Interfaces add a small indirection.** A method call through an interface is a dynamic dispatch (a pointer hop), slightly slower than a direct call. It is cheap, but do not wrap a hot inner loop in an interface out of habit.

Interview questions

Q: How does a type implement an interface in Go, and how is that different from Java or C++? A: Implicitly. A type satisfies an interface simply by having all the required methods; there is no `implements` or inheritance declaration. Satisfaction is verified by the compiler at the point where the concrete type is used as the interface. In Java/C++ a class must explicitly declare the interfaces or base classes it implements; in Go the relationship is structural and requires no coordination between the interface and the type.

Q: What is an interface value made of? A: Two words: a pointer to an itable (which identifies the concrete type and holds the method addresses that satisfy the interface) and a pointer to the concrete data. It is a fat pointer, a (`type`, `value`) pair. The zero value is nil, meaning both words are zero. Calling a method dispatches through the itable, like a vtable.

Q: Why can an interface that “contains nil” still be non-nil? A: Because an interface is nil only when both its type and value words are nil. If you store a nil pointer of a concrete type (say a `*MyError`) in an `error`, the type word is set to `*MyError`, so the interface is `(*MyError, nil)`, which is not equal to nil. The fix is to return the untyped nil literal on success rather than a typed nil pointer.

Q: A value of type T won’t satisfy an interface, but *T does. Why? A: Because the method that satisfies the interface uses a pointer receiver, and pointer-receiver methods are only in the method set of `*T`, not `T`. To call a pointer-receiver method, Go needs the value’s address; an interface may hold an unaddressable copy, so the language excludes pointer methods from `T`’s method set. Pass a pointer (`&x`) to satisfy the interface.

Q: When should you use any versus a specific interface or generics? A: Use a specific

small interface when you need a known behavior (a few methods). Use generics when you need the same logic across many types while keeping full type safety (Chapter 19). Reserve **any** for genuinely heterogeneous data where the type is unknown until run time (for example decoding arbitrary JSON), and pair it with type assertions or a type switch. Overusing **any** discards Go's compile-time checking.

Try it

1. Write `Shape` interface `{ Area() float64 }`. Make `Rectangle` and `Circle` satisfy it (value receivers). Put both in a `[]Shape` and sum their areas in a loop — one loop, two concrete types, no type switch needed.
2. Now add a pointer-receiver method to `Rectangle` and try to store a `Rectangle` value (not a pointer) in a `Shape`. Read the compiler error, then fix it with `&`.
3. Write a function returning `error` that declares `var e *MyErr` and returns `e`. Print `err == nil` and watch it say `false`. Fix it by returning the literal `nil`.

Chapter 12 — Errors

What you'll learn. Why Go has no exceptions, and treats errors as ordinary values you check by hand. How to create, wrap, and inspect errors, and how `panic/recover/defer` work for the rare cases where a value will not do.

In C, a function that can fail tells you in one of two ways: it returns a special value (a negative number, a NULL pointer, a non-zero status), and it may also set the global `errno`. You then check the return value and decide what to do.

Go keeps the spirit of that approach and cleans it up. A function that can fail returns an extra value of type `error`. You check it right away. There is **no try/catch**, no stack-unwinding exception for ordinary failures. This chapter is about that model and the small standard library that supports it.

An error is just an interface

`error` is not a special language feature. It is a normal interface, declared in the language itself:

```
type error interface {  
    Error() string  
}
```

That is the whole definition. Any type with a method `Error() string` *is* an `error`. (Interfaces are covered in Chapter 11 — Interfaces; the short version is: a type satisfies an interface just by having the right methods, with no `implements` keyword.)

Because `error` is an interface, an error is a **value**. You can store it in a variable, put it in a struct, pass it to a function, compare it, and return it. The zero value of any interface is `nil`, and `nil` means “no error.”

Mental model. An `error` is like a C `int` status code that also knows how to describe itself. Instead of returning `-1` and making the caller look up what `-1` means, the function returns a value whose `Error()` method spells out what went wrong.

Why values, not exceptions

C++ and Java use *exceptions*: a failure deep in the call stack **throws**, the stack unwinds automatically, and some **catch** far away handles it. The control flow is invisible at the call site — you cannot tell by reading `x = parse(s)` whether it might jump somewhere else.

Go’s designers chose explicit values on purpose:

Topic	C	C++/Java	Go
Signal failure	return code + errno	throw an exception	return an error value
Control flow	explicit, local	hidden jumps up the stack	explicit, local
Can you ignore it	yes, silently	no (it propagates)	yes, but you must do it on purpose
Carries a message	no (just a number)	yes	yes
Carries more data	no	yes (exception object)	yes (custom error type)

The trade-off: Go code has many `if err != nil` blocks. In exchange, the control flow is always visible. You can see exactly where each failure is handled, and the compiler does not let a failure path silently skip your code.

C vs Go. **errno** is one global, shared by the whole thread; you must read it *immediately*, before the next library call overwrites it. A Go **error** is a return value tied to that one call. There is no global to clobber and no race to read it in time.

The core idiom: check **err** immediately

Almost every Go function that can fail returns the error **last**:

```
f, err := os.Open("config.txt")
if err != nil {
    return err // cannot continue; hand the error back to the caller
}
defer f.Close() // runs when this function returns (see Chapter 6 - Functions)
// ... use f ...
```

The pattern is always the same: call the function, then immediately test `if err != nil`. You check right away because the other returned values are not trustworthy when **err** is non-**nil**. Above, **f** is only valid if **err** is **nil**.

Rule of thumb. Handle the error where you get it: return it, wrap it (below), log it, or recover from it. Do not stash it in a variable and “deal with it later.” Later never comes, and the next call may overwrite the good value.

The result is Go’s typical shape: a short happy path down the left margin, with small `if err != nil { return ... }` blocks branching off it. That is normal and idiomatic — not a code smell.

Creating errors

`errors.New` and `fmt.Errorf`

The two basic constructors live in the `errors` and `fmt` packages:

```
import (
    "errors"
    "fmt"
)

err1 := errors.New("disk is full")           // a fixed message
err2 := fmt.Errorf("cannot open %s", filename) // a formatted message
```

`errors.New` makes an error from a fixed string. `fmt.Errorf` works like `printf`: it formats a message and returns it as an error. Use `fmt.Errorf` when the message needs runtime data (a filename, an index, a count).

Sentinel errors

A **sentinel error** is a known error value, exported so callers can test for it by identity. It is Go's version of a named status code like EOF or ENOENT:

```
package store

import "errors"

// ErrNotFound is returned when a key does not exist.
var ErrNotFound = errors.New("not found")

func Get(key string) (string, error) {
    v, ok := lookup(key)
    if !ok {
        return "", ErrNotFound
    }
    return v, nil
}
```

The caller compares against the sentinel (with `errors.Is`, shown below) to react to that specific case. The standard library does this everywhere: `io.EOF`, `sql.ErrNoRows`, `os.ErrNotExist`.

Custom error types that carry data

Sometimes a message is not enough — you want to attach structured data. Because `error` is just an interface, you make your own type and give it an `Error()` method:

```
type HTTPError struct {
    Code int
    URL  string
}

func (e *HTTPError) Error() string {
```



```

    return fmt.Sprintf("request to %s failed with status %d", e.URL, e.Code)
}

func fetch(url string) error {
    // ...
    return &HTTPError{Code: 404, URL: url}
}

```

Now a caller can extract the `Code` field and branch on it (with `errors.As`, below). This is like defining a C struct for richer error context, except the value also formats itself for printing.

Watch out. `Error()` is usually defined on the **pointer** receiver (`*HTTPError`), so the value that satisfies `error` is `*HTTPError`, not `HTTPError`. Keep this consistent or callers will be confused about which one to use.

Wrapping errors: adding context

A bare error like `not found` is useless if you cannot tell *what* was not found. As an error travels up through many functions, each layer can add context by **wrapping** it. Wrapping keeps the original error inside the new one, like nested boxes.

The `%w` verb in `fmt.Errorf` wraps:

```

func loadConfig(path string) error {
    data, err := os.ReadFile(path)
    if err != nil {
        return fmt.Errorf("loadConfig %s: %w", path, err) // %w wraps err
    }
    // ...
    return nil
}

```

`%w` (“wrap”) stores the inner error inside the new one and adds its message to the front. `%v` would only copy the text; `%w` keeps the actual value reachable. The result is a **chain** of errors, much like a singly linked list:

error value (a chain, like a linked list):

<code>*fmt.wrapError</code>	<code>*fmt.wrapError</code>	<code>*errors.errorString</code>
+-----+	+-----+	+-----+
msg: read config:..	msg: open file:..	s: not found
err -----+--->	err -----+--->	(nothing wrapped)
+-----+	+-----+	+-----+
Unwrap ----->	Unwrap ----->	nil

`errors.Unwrap` follows one link; `errors.Is` and `errors.As` (below) walk the whole chain to the end. Printing the outer error prints the whole chain joined by `:` — `read config: open settings.json: not found`.

Walking the chain: Unwrap, Is, As

Three helpers in the `errors` package work on the chain:

- `errors.Unwrap(err)` returns the next error inside, or `nil` at the end. You rarely call it directly; the next two are what you use day to day.
- `errors.Is(err, target)` reports whether `target` appears *anywhere* in the chain. Use it to match a sentinel.
- `errors.As(err, &target)` searches the chain for an error of a given *type*, and if found, copies it into `target`. Use it to extract a custom error type.

```
err := loadConfig("settings.json")

// Match a sentinel anywhere in the chain:
if errors.Is(err, os.ErrNotExist) {
    fmt.Println("config file is missing; using defaults")
}

// Extract a typed error from anywhere in the chain:
var httpErr *HTTPError
if errors.As(err, &httpErr) {
    fmt.Println("HTTP status was", httpErr.Code)
}
```

Here is a complete, runnable example of a chain and matching through it:

```
package main

import (
    "errors"
    "fmt"
)

var ErrNotFound = errors.New("not found")

func findFile() error {
    return fmt.Errorf("open settings.json: %w", ErrNotFound)
}

func readConfig() error {
    if err := findFile(); err != nil {
        return fmt.Errorf("read config: %w", err)
    }
    return nil
}

func main() {
    err := readConfig()
    fmt.Println(err) // read config: open settings.json: not found

    if errors.Is(err, ErrNotFound) {
        fmt.Println("the file was not found") // this prints
    }
}
```

```
}
```

Watch out. When errors may be wrapped, compare with `errors.Is(err, ErrX)`, **not** `err == ErrX`. The `==` test only checks the outermost error; if the sentinel was wrapped, `==` returns `false` and you miss the case. `errors.Is` walks the whole chain.

When to add context, and when not to

Wrap to add information the caller does not already have: which file, which key, which step failed. Do **not** wrap just to repeat what the inner error already says, and do not start the message with “error” or “failed to” — that text piles up into noise like `error: failed to: error: ....`

```
// Good: adds the path, which the inner error did not know.  
return fmt.Errorf("read config %s: %w", path, err)
```

```
// Noisy: adds nothing and stutters when printed.  
return fmt.Errorf("error: failed: %w", err)
```

Rule of thumb. Each layer adds one short noun phrase of context, lowercase, no trailing punctuation: “`parse header: %w`”. Read the final chain out loud; it should be a clear breadcrumb trail from outer to inner.

Joining multiple errors: `errors.Join`

Sometimes you collect several independent failures and want to report all of them at once — for example, validating many fields. `errors.Join` (added in Go 1.20) combines them into one error:

```
func validate(name, email string) error {  
    var errs []error  
    if name == "" {  
        errs = append(errs, errors.New("name is empty"))  
    }  
    if email == "" {  
        errs = append(errs, errors.New("email is empty"))  
    }  
    return errors.Join(errs...) // returns nil if errs is empty  
}
```

The joined error prints each message on its own line, and `errors.Is/errors.As` look inside every joined error, not just one. `errors.Join` returns `nil` when all its arguments are `nil`, so the happy path needs no special case.

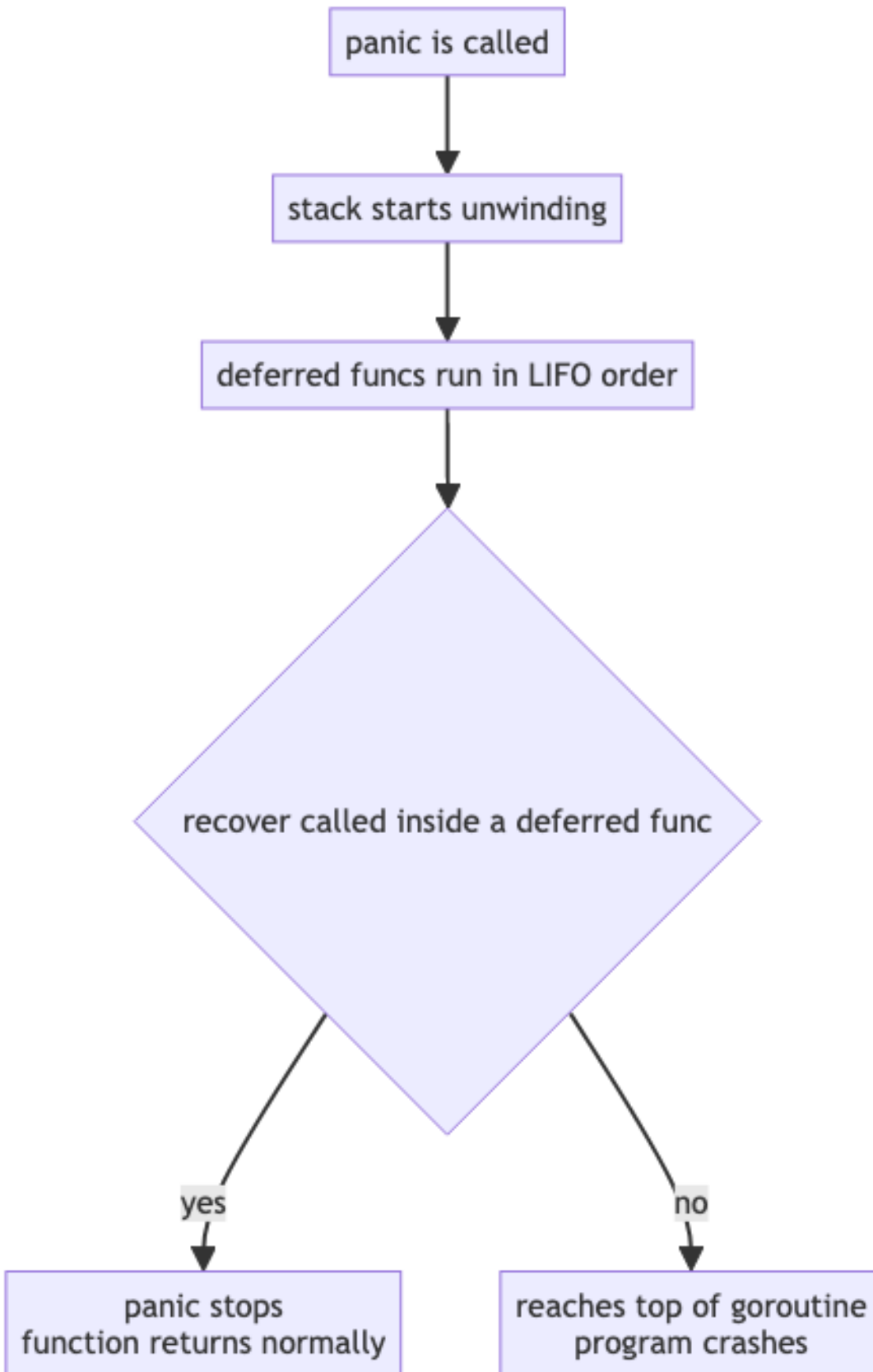
panic, recover, and defer

For ordinary, expected failures you return an `error`. For the rare, *truly broken* situations Go has `panic`. A `panic` is closer to `abort()` in C than to a C++ exception: it means “this program has reached a state that should be impossible.”

What `panic` does

When you call `panic(value)`:

1. The current function stops.
2. The stack **unwinds**: Go runs every **deferred** function in the current goroutine, in last-in-first-out order, as each frame is torn down.
3. If nothing stops it, the program prints the panic value and a stack trace, and **exits with a non-zero status**.



You also get a panic from the runtime for programmer bugs: a `nil` pointer dereference, an out-of-range slice index, or an integer divide by zero. These are the Go equivalent of a segfault, except Go tells you exactly what happened and where.

recover stops a panic

`recover` is a builtin that stops a panic in progress. It is **only meaningful inside a deferred function**. Called there during a panic, it returns the panic value and lets the function return normally. Called anywhere else, it returns `nil` and does nothing.

```
package main

import "fmt"

// safeDivide turns a divide-by-zero panic into a normal error value.
func safeDivide(a, b int) (result int, err error) {
    defer func() {
        if r := recover(); r != nil {
            err = fmt.Errorf("recovered: %v", r) // sets the named return
        }
    }()
    result = a / b // panics if b == 0
    return result, nil
}

func main() {
    q, err := safeDivide(10, 0)
    fmt.Println(q, err) // 0 recovered: runtime error: integer divide by zero
}
```

This works because the deferred closure can assign to the **named return value** `err` (named returns are covered in Chapter 6 — Functions). The panic unwinds into the deferred function, `recover` catches it, and the function returns cleanly.

Recover only at a boundary

Do not sprinkle `recover` everywhere. Use it at a **boundary** where one failing task must not take down the whole process: an HTTP request handler, or a worker goroutine in a pool. The pattern is the same deferred closure:

```
func handle(w http.ResponseWriter, r *http.Request) {
    defer func() {
        if rec := recover(); rec != nil {
            log.Printf("handler panic: %v", rec)
            http.Error(w, "internal server error", http.StatusInternalServerError)
        }
    }()
    doWork(w, r) // if this panics, one request fails, not the server
}
```

The standard library's HTTP server already does this for you, but you must add the guard yourself for your own worker goroutines.

When to panic, and when not to

Rule of thumb. Return an **error** for anything a caller might reasonably want to handle — a missing file, bad input, a failed network call. Reserve **panic** for *impossible* states and programmer bugs that mean the program is unsafe to continue.

A common, accepted use is a **Must**-style helper for setup that should never fail, such as compiling a constant regular expression at startup:

```
// MustCompile panics if the pattern is invalid. Used for compile-time-constant
// patterns, where a failure means the program itself is wrong.
var emailRE = regexp.MustCompile(`^[^@]+@[^@]+$`)
```

If the pattern is wrong, the program cannot work at all, so failing loudly at startup is correct.

Two things that will surprise a C programmer

Watch out. `os.Exit` does **not** run deferred functions. It stops the program immediately, skipping every `defer`. Use it only when you truly want an abrupt exit (and have already flushed/closed what matters). Returning from `main`, or panicking, *does* run defers.

Watch out. An **unrecovered panic in any goroutine crashes the whole program** — even if `main` and every other goroutine are fine. You cannot recover a panic from *outside* the goroutine that panicked; each goroutine must guard itself with its own deferred `recover`. (Goroutines are covered in Chapter 13 — Goroutines and the Scheduler.)

Don't ignore errors

A returned error you never look at is a silent bug. Beware: unlike an unused *variable*, an unused *return value* is **not** a compile error. You can call a function and drop its error with no warning. So be deliberate — when you really do mean to ignore an error, say so with the blank identifier `_`:

```
_ = os.Remove(tmpfile) // intentionally ignore: best-effort cleanup
```

Writing `_ =` documents that ignoring was a choice, not an oversight. The `go vet` tool and linters such as `errcheck` flag the errors you forgot. Silently dropping an error is the Go equivalent of ignoring a `-1` return in C — and just as likely to bite you later.

Key takeaways

- **error** is an ordinary interface: `type error interface { Error() string }`. Errors are **values**, returned last, and checked with `if err != nil`.
- Go has no exceptions for normal failures. Control flow stays explicit and local, unlike C++/Java `throw/catch`.
- Create errors with `errors.New` and `fmt.Errorf`. Use **sentinel** errors for known cases and **custom types** to carry structured data.

- Wrap with `fmt.Errorf("context: %w", err)` to build a chain. Match a sentinel with `errors.Is`, extract a type with `errors.As`, and combine many with `errors.Join`.
- `panic` unwinds the stack running `defers`; `recover` (only inside a deferred function) stops it. Use `panic` only for unrecoverable states and programmer bugs, never for ordinary errors.
- Recover at boundaries (handlers, worker goroutines). A panic in *any* goroutine with no `recover` crashes the whole program.

Watch out (gotchas for C programmers)

- **Ignoring an error is silent.** There is no warning unless a linter is run. Write `_ = f()` to ignore on purpose; otherwise handle it.
- **Shadowing `err` with `:=`.** Inside a nested block, `x, err := f()` may create a *new* `err` that hides the outer one, so the outer check sees a stale value. Use `=` (not `:=`) when `err` already exists, and heed `go vet`'s shadow checks.
- **Typed-`nil` error trap.** A non-`nil` interface can hold a `nil` pointer. If a function returns `error` but you give it a `*MyError` variable that is `nil`, the returned `error` is **not** `nil` (its type is set), so `if err != nil` is true. Return a literal `nil`, not a `nil` typed pointer. (See Chapter 11 — Interfaces.)
- **Use `errors.Is`, not `==`,** when an error might be wrapped. `==` checks only the outermost value and will miss a wrapped sentinel.
- **panic in a goroutine crashes everything.** Each goroutine needs its own deferred `recover`; you cannot catch another goroutine's panic.
- **recover only works inside a deferred function.** Calling it directly in the body returns `nil` and does nothing.
- **os.Exit skips defer.** Anything you deferred (closing files, flushing buffers) will not run.

Interview questions

Q: Why does Go use error values instead of exceptions? A: To keep control flow explicit and local. A function returns an `error` as its last result, and the caller checks it right there with `if err != nil`. There are no hidden jumps up the stack, so you can always see where a failure is handled. The cost is more `if err != nil` blocks; the benefit is readable, predictable control flow.

Q: What is the difference between `errors.Is` and `errors.As`? A: `errors.Is(err, target)` tests whether a specific error *value* (usually a sentinel) appears anywhere in the wrapped chain. `errors.As(err, &target)` searches the chain for an error of a specific *type* and, if found, assigns it to `target` so you can read its fields. Use `Is` to match identity, `As` to extract a typed error.

Q: What does `%w` do in `fmt.Errorf`, and why prefer it over `%v`? A: `%w` wraps: it stores the inner error inside the new one and keeps the original value reachable, building a chain. `%v` only copies the inner error's text, losing the value. Use `%w` so callers can still match the cause with `errors.Is` or `errors.As`; use `%v` only when you deliberately want to hide the cause.

Q: When should you use `panic` instead of returning an error? A: Almost never for expected failures. Return an `error` for anything a caller might handle (bad input, missing file, network failure). Use `panic` only for truly unrecoverable states or programmer bugs — impossible conditions, failed startup invariants — where continuing would be unsafe. *Must*-style setup helpers that fail loudly at startup are an accepted use.

Q: How does `recover` work, and what are its limits? A: `recover` stops a panic, but only

when called inside a deferred function during the panic; elsewhere it returns `nil`. It catches a panic only in its **own** goroutine — an unrecovered panic in any other goroutine crashes the whole program. It is typically used at a boundary (an HTTP handler or worker goroutine) to keep one failing task from taking down the process.

Q: What is the typed-`nil` error trap? A: An interface value is `nil` only when both its type and value are `nil`. If a function declares a return type of `error` but returns a typed pointer such as a `*MyError` that happens to be `nil`, the interface's type field is set, so the returned `error` is non-`nil`. The caller's `if err != nil` is then true even though “there was no error.” Fix it by returning a literal `nil`.

Try it

1. Write `find(key string) (int, error)` that returns a sentinel `ErrMissing` when the key is absent. Wrap it in a caller with `fmt.Errorf("loading %s: %w", key, err)`, then confirm `errors.Is(err, ErrMissing)` is still true through the wrap.
2. Change the wrap from `%w` to `%v` and watch `errors.Is` start returning `false`. That one-character difference is the whole point of wrapping.

Chapter 13 — Goroutines and the Scheduler

What you'll learn. What a goroutine is, why it is far cheaper than an OS thread, and how Go's runtime schedules millions of them onto a few threads with the G-M-P model. You will also see how goroutines start, stop, grow their stacks, and leak — and why sharing memory without synchronization is a bug.

In C, concurrency is a *library*. You call `pthread_create` to start a thread, and the operating system (OS) schedules it. Threads are heavy: each one reserves one or two megabytes of stack, so you can realistically run a few thousand of them.

Go builds concurrency into the *language*. The keyword `go` starts a **goroutine**: a function that runs concurrently with the rest of your program. Goroutines are so cheap that running hundreds of thousands of them is normal. This chapter explains what they are and how the Go runtime makes them so cheap.

A goroutine is a cheap thread

A **goroutine** is a function running independently, managed by the Go *runtime* (the support code linked into every Go program). You start one by writing `go` in front of a function call:

```
go f(x, y) // start f(x, y) as a goroutine; do not wait for it
```

The `go` statement returns immediately. The call to `f` runs concurrently — maybe on another CPU core, maybe interleaved with other goroutines on the same core. Your code does not wait for it to finish.

C vs Go. `go f(x)` is the rough equivalent of packing arguments into a struct and calling `pthread_create` — but in one keyword, with no thread handle to manage and no stack size to choose.

Concept	C (pthreads)	Go
Start a unit of work	<code>pthread_create(&t, NULL, f, arg)</code>	<code>go f(arg)</code>
Arguments	one <code>void *</code> you pack and cast	normal, typed arguments
Handle / id	<code>pthread_t t</code>	none
Wait for it to finish	<code>pthread_join(t, NULL)</code>	<code>sync.WaitGroup</code> or a channel
Initial stack	fixed, ~1–2 MB	~2 KB, grows and shrinks
Practical count	thousands	hundreds of thousands and up
Scheduled by	the OS kernel	the Go runtime (user space)

Arguments are evaluated now; the body runs later

This detail trips up newcomers. The arguments to the function are evaluated **when the go statement runs**, not when the goroutine actually starts executing. The values are copied right away.

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    var wg sync.WaitGroup
    i := 1
    wg.Add(1)
    go func(n int) {
        defer wg.Done()
        fmt.Println("captured:", n) // n is a copy taken at the go statement
    }(i) // i is read and copied HERE, while it is 1
    i = 2
    wg.Wait()
    fmt.Println("i is now:", i)
}
```

This prints `captured: 1` then `i is now: 2`. The argument `i` was read and copied at the `go` statement, before `i` changed.

Waiting for a goroutine (a WaitGroup preview)

We need a way to wait for goroutines, or our examples cannot print anything reliably (the next section explains why). The simplest tool is `sync.WaitGroup`, a counter from the standard library:

```
package main

import (
    "fmt"
    "sync"
)
```

```

)

func main() {
    var wg sync.WaitGroup
    wg.Add(1) // we will wait for one goroutine
    go func() {
        defer wg.Done() // signal "I am finished"
        fmt.Println("from the goroutine")
    }()
    fmt.Println("from main")
    wg.Wait() // block here until the counter reaches zero
}

```

A `sync.WaitGroup` counts running goroutines. `Add(n)` raises the counter, `Done` lowers it by one (we call it with `defer` so it always runs), and `Wait` blocks until the counter is zero. We use it here only as a way to wait; Chapter 15 — `sync` and `context` covers it fully.

Goroutines vs OS threads

A goroutine is **not** an OS thread. It is a much lighter object that the runtime multiplexes onto a small pool of OS threads. This is **M:N scheduling**: M goroutines run on N OS threads, where M can be huge and N is small.

Property	OS thread (pthread)	Goroutine
Created by	the OS kernel	the Go runtime
Initial stack	~1–2 MB, fixed	~2 KB, grows and shrinks
Switching cost	high (a kernel trap)	low (user-space bookkeeping)
Scheduled by	the kernel	the Go runtime, onto threads
Practical count	thousands	hundreds of thousands to millions
Identity	thread id / handle	none

The key number is the stack. A new goroutine starts with about 2 KB that can grow and shrink (we explain how below). A C thread reserves its whole stack up front. That one difference is why you can have a million goroutines but only a few thousand threads.

Mental model. Threads are like full-time employees: powerful but expensive, so you hire few. Goroutines are like tasks on a to-do list: you can write down a million, and a few workers (the threads) work through them.

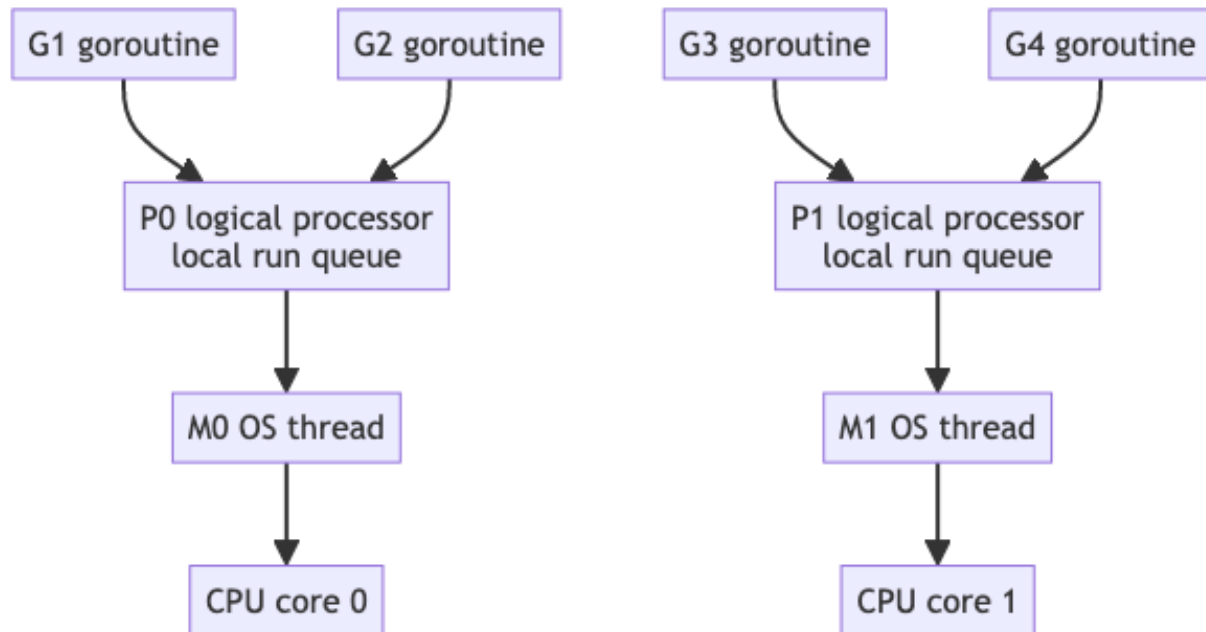
The scheduler: G, M, and P

The runtime contains a *scheduler* that decides which goroutine runs on which thread. It uses three kinds of objects. Learn these three letters; every discussion of Go scheduling uses them.

- **G — goroutine.** One unit of work: a stack, an instruction pointer, and a little bookkeeping.
- **M — machine.** One OS thread. Only an M can actually run code on a CPU. (Think pthread.)

- **P — processor.** A *logical* processor: the permission to run Go code, plus a local queue of ready goroutines. The number of P's is set by `GOMAXPROCS`, which defaults to the number of CPU cores.

To run a goroutine, an M must hold a P. So the number of goroutines running *at the same instant* equals the number of P's, which is `GOMAXPROCS`. Every other ready goroutine waits in a queue.



Each P owns a **local run queue**: a list of goroutines ready to run. There is also one **global run queue** shared by all P's. A P takes work from its own local queue first, because that needs no lock and is fast.

`GOMAXPROCS = 2` => two P's, each with its own local run queue.

```

P0 +-----+-----+-----+      runs on M0 -> CPU core 0
   |  G7  |  G3  |  G9  |
   +-----+-----+-----+

P1 +-----+-----+-----+      global run queue (shared, needs a lock)
   |      |      |      |      +-----+-----+
   |      |      |      |      |  G2  |  G5  |
   +-----+-----+-----+      +-----+-----+

empty: M1 has nothing to do
      |
      +-- work stealing: P1 takes half of P0's queue (G9),
          and also pulls from the global queue.
  
```

Work stealing

What if one P empties its queue while another P still has a long one? The idle P does not sit idle. It **steals** work: it takes about half of the goroutines from a busy P's local queue. It also occasionally checks the global queue. This keeps all CPU cores busy and balances load automatically, with no

work from you.

C vs Go. With raw pthreads you choose how many threads to run and how to feed them work — usually a thread pool plus a mutex-protected queue you write by hand. Go ships that whole design inside the runtime: P’s are the pool slots, the run queues are the work queue, and work stealing balances them, for free.

You can read these numbers at run time with the `runtime` package: `runtime.NumCPU()` is the cores the OS reports, `runtime.GOMAXPROCS(0)` is the number of P’s (by default the same value), and `runtime.NumGoroutine()` is how many goroutines are alive right now.

When does a goroutine yield? Scheduling points

A goroutine keeps its P until it stops or is taken away. Go uses both *cooperative* scheduling (the goroutine gives up the P itself) and *preemptive* scheduling (the runtime takes it away).

Cooperative points — where a goroutine voluntarily releases its P so another can run:

- **Channel operations** that block (Chapter 14 — Channels and select).
- **Blocking system calls**, such as reading a file. The runtime detaches the P from the blocked M and gives it to another M, so the other goroutines keep running while this one waits on the OS.
- **Network I/O.** Go has a *network poller* built on `epoll/kqueue/IOCP`. A goroutine that reads a socket is *parked*; when the OS reports the socket is ready, the poller makes it runnable again. One thread can wait for thousands of connections.
- Certain runtime calls: `time.Sleep`, garbage-collection points, and `runtime.Gosched` (a manual “let others run”).

Preemptive point — added in **Go 1.14**:

- **Asynchronous preemption.** A goroutine that runs a long loop with no function calls used to keep its P forever. Now the runtime can interrupt such a goroutine (using an OS signal) after about 10 milliseconds and reschedule it. You do not have to insert yield calls by hand.

Deep dive. Before Go 1.14, a tight loop like `for {}` with no calls could freeze the scheduler running on that P. Modern Go preempts it. You may still meet this on very old Go versions.

The life of a goroutine

main is special: when it returns, everything dies

There is one goroutine you did not start: `main`. When the `main` function returns, the **whole program exits immediately**, and every other goroutine is killed where it stands — no warning, no cleanup.

```
package main

import "fmt"

func main() {
    go fmt.Println("from the goroutine")
    fmt.Println("from main")
}
```

```

    // main returns here; the program exits at once.
}

```

Run it and you will usually see only:

```
from main
```

The goroutine often never gets the chance to print: `main` reached its end and the process exited. This is the single most common surprise for newcomers. The fix is to wait — for example with the `WaitGroup` shown earlier.

No goroutine IDs, and no killing from outside

Coming from `pthread`s, you expect a thread handle (`pthread_t`) and a thread id. Goroutines have **neither**. You cannot read a goroutine’s id, and you cannot cancel or kill another goroutine from the outside. There is no `pthread_cancel`.

A goroutine stops only when it **decides** to: when its function returns, or when it notices a signal you sent it. The standard signal is a closed channel or a cancelled `context.Context` (Chapter 15 — sync and context). The goroutine must check for that signal itself; cancellation is *cooperative*.

Rule of thumb. Design every goroutine so that something can tell it to stop, and so it actually checks. “Fire and forget” is how you leak goroutines.

Stack growth

A C thread gets a fixed stack (commonly 1–8 MB) chosen when it is created. Overrun it and you get a stack overflow and a crash.

A goroutine starts with about **2 KB** of stack. When it needs more, the runtime allocates a bigger, **contiguous** block, copies the old stack into it, fixes up the pointers, and continues. When the goroutine later uses less, the stack can shrink. Both growth and shrinkage are automatic.

goroutine needs more stack than it has:

before (2 KB)		after growth (4 KB, a NEW contiguous block)
+-----+		+-----+
frame f	copy	frame f
frame g	====>	frame g
(full!)		frame h <- room to grow
+-----+		+-----+
old block freed		pointers into the stack are updated by the runtime

C vs Go. In C you size the stack up front and hope it is enough. In Go the stack is elastic, so deep recursion that would overflow a small C stack simply grows. The price is that the runtime owns your stack layout: the address of a stack variable can change when the stack moves. You never notice, because Go updates every pointer for you.

Goroutine leaks

A goroutine that blocks forever never goes away. It keeps its stack and everything it references. That is a **goroutine leak** — the concurrent cousin of a memory leak.

The classic cause is waiting on a channel that no one will ever use:

```
package main

import "fmt"

func leak() {
    ch := make(chan int) // unbuffered channel (see Chapter 14)
    go func() {
        v := <-ch // blocks forever: nobody will ever send
        fmt.Println(v)
    }()
    // leak returns; the goroutine above is stuck and never goes away.
}

func main() {
    leak()
    fmt.Println("leak returned; the inner goroutine is now stuck forever")
}
```

`leak` returns, but the goroutine it started is parked on `<-ch` forever, because nothing will ever send. Call `leak` a thousand times and you slowly leak memory and goroutines. The program still exits fine here (because `main` returns), but a long-running server would bleed resources.

Rule of thumb. Never start a goroutine without knowing how it will stop.

Before you type `go`, answer one question: what makes this goroutine return? Usually the answer is “a channel is closed” or “a context is cancelled.”

Data races: shared memory is not safe by default

Goroutines run concurrently and share one address space, exactly as pthreads share a process’s memory. So the same danger applies: if two goroutines touch the same variable at the same time and at least one of them writes, the result is undefined. That is a **data race**, and it is a bug even when it seems to work.

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    var wg sync.WaitGroup
    counter := 0
    for range 1000 {
        wg.Add(1)
        go func() {
            defer wg.Done()
            counter++ // DATA RACE: many goroutines write the same memory
        }()
    }
    wg.Wait()
    fmt.Println(counter)
```

```

    }()
}
wg.Wait()
fmt.Println("counter =", counter) // rarely 1000
}

```

Every goroutine runs `counter++`, which is really *read, add one, write back* — three steps that can interleave. Updates are lost, so the final value is rarely 1000. C has the identical bug with pthreads and a shared `int`.

Go ships a **race detector**. Add `-race` to find races while the program runs:

```

go run -race .           # run with the detector on
go test -race ./...      # test with the detector on

```

On the program above it prints **WARNING: DATA RACE** and names the two conflicting lines. The detector has almost no false positives: if it reports a race, you have one. We use it heavily in Chapter 16 — Concurrency Patterns.

The fix is to *synchronize* access — with a channel (Chapter 14 — Channels and select), a `sync.Mutex`, or an atomic operation (Chapter 15 — sync and context). The Go motto is:

Do not communicate by sharing memory; instead, share memory by communicating.

In other words, prefer passing a value through a channel — handing off ownership — over many goroutines poking the same variable behind a lock. Chapter 14 builds on exactly this idea.

Key takeaways

- `go f(args)` starts a goroutine. The **arguments are evaluated immediately**; the function body runs concurrently, possibly later.
- Goroutines are cheap: a growable ~2 KB stack, managed by the Go runtime, not the OS. You can run hundreds of thousands. This is **M:N scheduling**.
- The scheduler uses **G** (goroutine), **M** (OS thread), and **P** (logical processor; the count is `GOMAXPROCS`, by default the number of CPUs). Each P has a local run queue; idle P's **steal** work.
- Goroutines yield at channel operations, blocking syscalls (the P moves to another M), network I/O (the poller), and via **asynchronous preemption** (Go 1.14+).
- When **main** returns, the program exits and **all** goroutines die. Wait with a `WaitGroup` or a channel.
- There are **no goroutine IDs** and no external kill; a goroutine must cooperate (a closed channel or a cancelled context).
- A goroutine blocked forever is a **leak**. Never start one without knowing how it stops.
- Shared memory needs synchronization; find data races with `-race`.

Watch out (gotchas for C programmers)

- **main returning kills every other goroutine instantly**. Output you expected may never appear. Wait for your goroutines.
- **Goroutine leaks**. A goroutine blocked on a channel no one will use stays forever and holds its memory. There is no garbage collector for stuck goroutines.

- **You cannot kill or signal a goroutine from outside.** There is no `id` and no `pthread_cancel`. Build in a stop signal (a channel or a context) and check it.
- **Unsynchronized shared variables are data races**, just as with pthreads. “It worked on my machine” means nothing. Run with `-race`.
- **Closures capture variables by reference.** Two goroutines sharing the same captured variable can race. (Loop variables are per-iteration since Go 1.22, so the old loop-capture bug is gone — but a shared accumulator is still a race.)
- **Do not assume any ordering.** Goroutines may run in any order, or not at all before `main` ends.

Interview questions

Q: What is a goroutine, and how is it different from an OS thread? A: A goroutine is a function managed by the Go runtime that runs concurrently. It is far lighter than an OS thread: it starts with about a 2 KB stack that grows and shrinks, while an OS thread reserves 1–2 MB up front. The runtime multiplexes many goroutines onto a few threads (M:N scheduling), so you can run hundreds of thousands of goroutines but only thousands of threads.

Q: Explain the G-M-P scheduler. A: G is a goroutine, M is an OS thread (machine), and P is a logical processor that holds a local run queue and is the permission to run Go code. The number of P’s is `GOMAXPROCS`, by default the number of CPU cores. An M must hold a P to run a G, so at most `GOMAXPROCS` goroutines run at the same instant. Each P runs goroutines from its own queue; an idle P steals work from another P or from the global queue.

Q: When does a goroutine give up the CPU? A: At blocking channel operations, at blocking system calls (the runtime hands the P to another M so other goroutines keep running), at network I/O handled by the poller, and at certain runtime calls. Since Go 1.14 the runtime can also asynchronously preempt a goroutine stuck in a long computation, so a tight loop no longer starves the others.

Q: What happens to other goroutines when `main` returns? A: The program exits immediately and all other goroutines are killed with no cleanup. To make sure work finishes, the main goroutine must wait — for example with a `sync.WaitGroup` or by receiving from a channel.

Q: How do you stop a goroutine, and why can goroutines leak? A: You cannot kill a goroutine from outside; it has no `id` and there is no cancel call. A goroutine stops only by returning, so you send it a signal it checks for — typically a closed channel or a cancelled `context.Context`. If a goroutine blocks forever (for example waiting on a channel nothing sends to), it never returns and leaks its stack and memory. Rule: never start a goroutine without knowing how it will stop.

Q: What is a data race, and how do you detect one in Go? A: A data race is two goroutines accessing the same memory at the same time with at least one writing, and no synchronization between them; the result is undefined. Go’s race detector finds them at run time — build, run, or test with the `-race` flag (`go run -race`, `go test -race`). Fix races with channels, a `sync.Mutex`, or atomic operations.

Try it

1. Run the “from main” example. You will usually see only `from main`. Now add a `sync.WaitGroup` (as in the preview) and watch the goroutine’s line appear every time.

2. Run the counter example with `go run -race .` and read the report: it names the exact line (`counter++`) and the two goroutines that collided.

Chapter 14 — Channels and select

What you'll learn. How channels move values safely between goroutines, the difference between unbuffered (a rendezvous) and buffered (a small queue) channels, how `close` and `range` work, and how `select` waits on several channels at once. All of it compared to the hand-built queues you would write with pthreads.

In C, a thread-safe queue is something you *build*: a ring buffer, a `pthread_mutex_t` to protect it, and one or two `pthread_cond_t` condition variables so threads can wait for “not empty” or “not full.” It is fiddly, and easy to get wrong.

Go gives you that queue as a built-in type: the **channel**. A channel is a typed, thread-safe *conduit* (a pipe) that carries values from one goroutine to another. The mutex and condition variables live inside it; you never see them. Channels are how goroutines (Chapter 13 — Goroutines and the Scheduler) talk to each other.

Making, sending, and receiving

`make(chan T)` builds a channel that carries values of type T:

```
ch := make(chan int) // a channel of int
```

There are three operations, all written with the arrow `<-`, which always points in the direction the data moves:

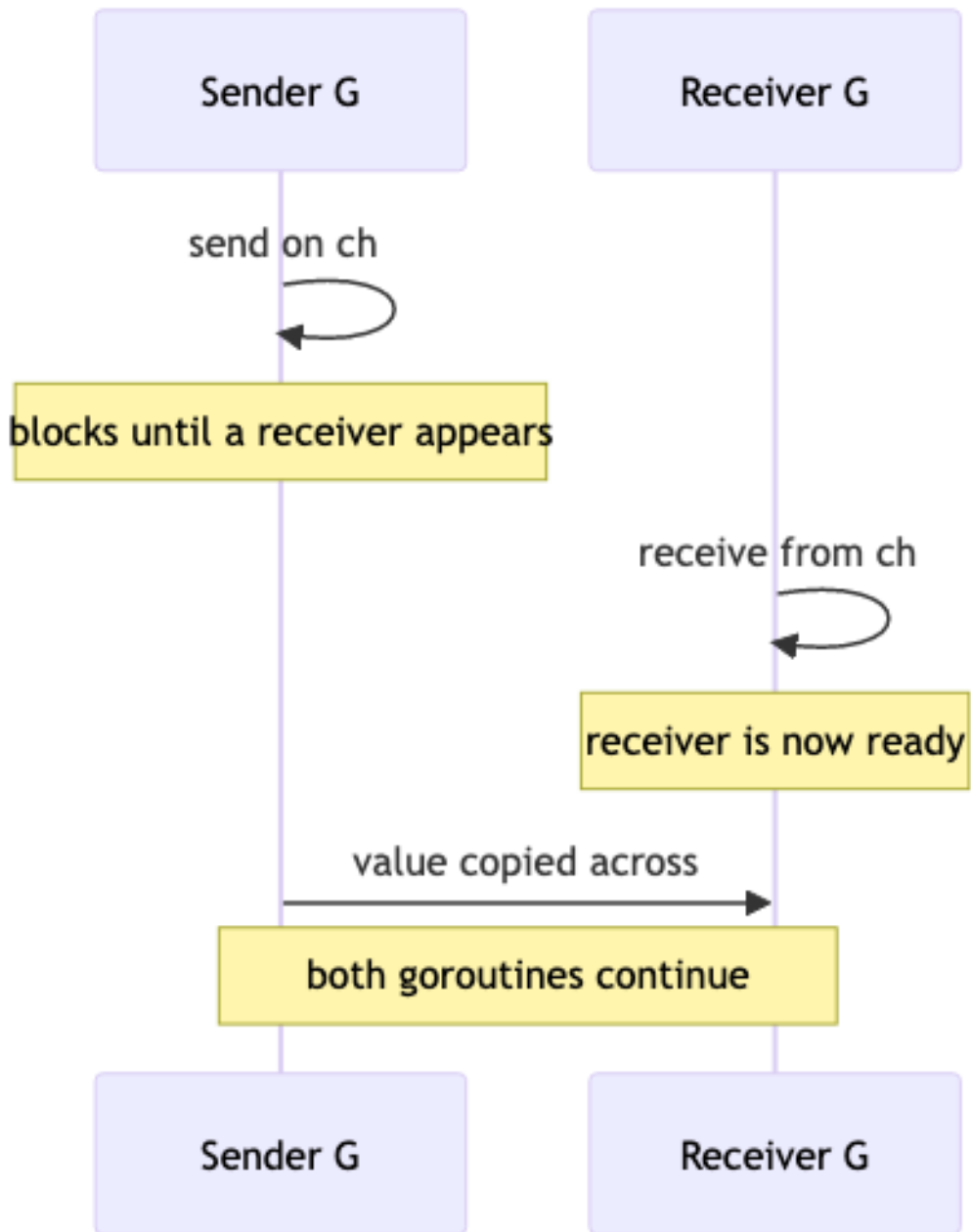
```
ch <- v           // SEND: put v into the channel
v := <-ch         // RECEIVE: take a value out of the channel
v, ok := <-ch     // RECEIVE with ok: ok is false if the channel is closed and empty
close(ch)         // CLOSE: announce that no more values will be sent
```

The arrow leans **toward** the channel on a send (`ch <- v`) and **away** from it on a receive (`<-ch`). Memorize that and the syntax reads itself.

C vs Go. The closest C analog is a hand-built bounded queue: a ring buffer plus a mutex plus condition variables. A channel is all of that in one value: type-safe, with the locking built in.

Unbuffered channels: a rendezvous

`make(chan T)` with no size is **unbuffered**. It stores nothing. A send blocks until another goroutine is ready to receive, and a receive blocks until another goroutine is ready to send. The two goroutines meet at the same instant and hand the value across. That meeting is a **rendezvous** (a French word for a planned meeting).



```
package main

import "fmt"

func main() {
    ch := make(chan int) // unbuffered
    go func() {
        ch <- 42 // blocks until main is ready to receive
    }()
    v := <-ch // blocks until the goroutine sends
```

```
    fmt.Println(v)
}
```

Because the send and the receive happen together, an unbuffered channel is also a **synchronization point**: once the exchange completes, you know both goroutines reached that line.

Watch out. A send on an unbuffered channel with no receiver blocks forever. If nothing else can run, the program deadlocks (see below).

Buffered channels: a small queue

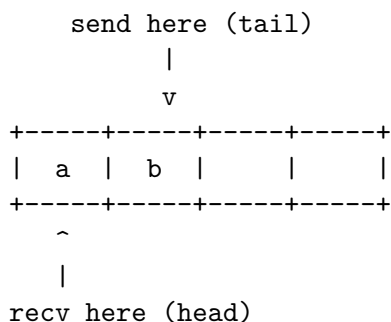
`make(chan T, n)` with capacity `n` is **buffered**. It can hold up to `n` values. A send blocks only when the buffer is full; a receive blocks only when it is empty. Up to the capacity, sends and receives need not happen at the same time — the channel is **asynchronous**.

```
package main

import "fmt"

func main() {
    ch := make(chan string, 2) // capacity 2
    ch <- "a"                  // does not block: buffer has room
    ch <- "b"                  // does not block: buffer now full
    fmt.Println("len", len(ch), "cap", cap(ch))
    fmt.Println(<-ch, <-ch)
}
```

`len(ch)` is how many values are waiting; `cap(ch)` is the capacity. Internally the buffer is a ring:
`make(chan int, 4)`: a buffered channel is a fixed-size ring buffer.



`len = 2` (items waiting) `cap = 4` (slots)
 send: write at tail, advance tail; blocks only when full
 recv: read at head, advance head; blocks only when empty
 the head and tail wrap around the ends, so it is a ring.

Rule of thumb. Use an unbuffered channel when you want a guarantee that the receiver actually took the value (a handoff). Use a small buffer to smooth bursts or to let a sender keep working while the receiver catches up. A buffer is not a fix for a logic bug; choose its size for a reason, not to silence a deadlock.

Closing a channel

`close(ch)` marks a channel as finished: no more values will be sent. Closing is a *message to receivers*, not a cleanup step. (Channels are garbage-collected like any value; you do not have to close one to free it.)

The two-value receive tells you whether the channel is still open, and `range` receives until the channel is closed and drained:

```
package main

import "fmt"

func main() {
    ch := make(chan int, 3)
    for i := range 3 {
        ch <- i
    }
    close(ch)

    for v := range ch { // stops when the channel is closed and drained
        fmt.Println("got", v)
    }

    v, ok := <-ch // ok is false; v is the zero value
    fmt.Println("after close:", v, ok)
}
```

This prints `got 0, got 1, got 2, then after close: 0 false`. The behavior of each operation depends on the channel's state:

Operation	nil channel	open channel	closed channel
send <code>ch <- v</code>	blocks forever	blocks (or buffers) until received	panics
receive <code><-ch</code>	blocks forever	blocks (or buffers) until a value	returns zero value at once, <code>ok == false</code>
<code>close(ch)</code>	panics	ok	panics

In words:

- **Only the sender closes**, and only when it is the only sender. A receiver must never close: it cannot know whether more sends are coming.
- **Sending on a closed channel panics** (panic: send on closed channel).
- **Closing a closed or a nil channel panics** (panic: close of closed channel, panic: close of nil channel).
- **Receiving from a closed channel never blocks**: it returns the zero value immediately with `ok == false`, and a `range` loop ends.

C vs Go. `close` is not `free()` or `fclose`. A channel holds no OS resource. Closing only

signals “no more values”; the channel’s memory is reclaimed by the garbage collector when nothing refers to it.

Direction types in APIs

A function parameter can restrict a channel to one direction. The direction is part of the type and checked by the compiler:

- `chan<- T` — **send-only**: you may only send (arrow points into the channel).
- `<-chan T` — **receive-only**: you may only receive (arrow points out).

```
package main

import "fmt"

// produce only sends; the type chan<- int enforces that.
func produce(out chan<- int, n int) {
    for i := range n {
        out <- i
    }
    close(out) // the sender closes
}

// consume only receives; the type <-chan int enforces that.
func consume(in <-chan int) {
    for v := range in {
        fmt.Println("consumed", v)
    }
}

func main() {
    ch := make(chan int)
    go produce(ch, 3)
    consume(ch)
}
```

A plain `chan T` converts to either restricted form automatically when you pass it. Direction types document intent and prevent mistakes: a producer cannot accidentally receive, and a consumer cannot accidentally close.

Rule of thumb. In public APIs, take and return directed channels (`<-chan T` to hand a stream to a caller, `chan<- T` to accept one). The data-flow direction becomes obvious, and the compiler enforces it.

select: waiting on several channels

A `select` statement waits on several channel operations at once. It is the channel version of C’s `select()/poll()` on file descriptors, but for channels. Each case is one operation — a receive, a send, or a timeout from `time.After`. The rules:

- `select` blocks until **one** of its cases can proceed.

- If several cases are ready at once, it picks **one uniformly at random**. This avoids starvation; never rely on order.
- A **default** case runs immediately when no other case is ready, which makes the **select non-blocking**.

```
package main

import (
    "fmt"
    "time"
)

func main() {
    ch := make(chan int)
    go func() {
        time.Sleep(50 * time.Millisecond)
        ch <- 7
    }()

    // Non-blocking check: default runs when nothing is ready.
    select {
    case v := <-ch:
        fmt.Println("ready:", v)
    default:
        fmt.Println("nothing yet")
    }

    // Wait, but not forever.
    select {
    case v := <-ch:
        fmt.Println("got:", v)
    case <-time.After(200 * time.Millisecond):
        fmt.Println("timed out")
    }
}
```

`time.After(d)` returns a channel that delivers one value after the duration `d`. Putting it in a `select` is the standard way to add a **timeout**.

The for-select loop

A goroutine that services channels usually runs a `for` loop around a `select`, until a “done” signal arrives:

```
package main

import "fmt"

func main() {
```

```

ticks := make(chan int)
done := make(chan struct{})

go func() {
    for i := range 3 {
        ticks <- i
    }
    close(done)
}()

for {
    select {
    case v := <-ticks:
        fmt.Println("tick", v)
    case <-done:
        fmt.Println("done")
        return
    }
}

```

This is the idiomatic shape of a long-lived worker: loop, **select** on the work channel and a stop channel, and **return** when stop fires. The empty struct `struct{}` carries no data and uses no memory — a common choice for a pure signal.

A nil channel blocks forever — and that is useful

A send or receive on a **nil** channel blocks forever (see the table above). That sounds useless, but it is the trick for **disabling a case** in a **select**: a nil channel is never ready, so its case is skipped. The classic use is merging streams. When one input closes, set its variable to nil so its case stops firing:

```

package main

import "fmt"

// merge reads from a and b until both are closed. When one closes, we set that
// variable to nil so its case is disabled (a nil channel never fires).
func merge(a, b <-chan int) {
    for a != nil || b != nil {
        select {
        case v, ok := <-a:
            if !ok {
                a = nil // disable this case
                continue
            }
            fmt.Println("from a:", v)
        case v, ok := <-b:
            if !ok {

```



```

        b = nil
        continue
    }
    fmt.Println("from b:", v)
}
}
fmt.Println("both channels drained")
}

func main() {
    a := make(chan int)
    b := make(chan int)
    go func() { a <- 1; a <- 2; close(a) }()
    go func() { b <- 10; close(b) }()
    merge(a, b)
}

```

Without the `a = nil` line, the closed channel `a` would be ready on every loop (returning zero values forever) and spin the CPU. Setting it to `nil` switches that case off cleanly.

Deadlocks

If every goroutine is blocked waiting on a channel, no one can make progress. The Go runtime detects this exact situation and aborts the program:

```
fatal error: all goroutines are asleep - deadlock!
```

The smallest example is a send with no possible receiver:

```

package main

func main() {
    ch := make(chan int) // unbuffered
    ch <- 1               // no receiver exists; this blocks forever
}

```

Common causes:

- An unbuffered **send with no receiver** (or a receive with no sender).
- Forgetting to **close** a channel that a **range** is reading, so the loop waits forever for a value that never comes.
- A cycle of goroutines each waiting on the other.

C vs Go. A pthreads deadlock usually just hangs, and you attach a debugger to learn why. Go's runtime can often tell when *all* goroutines are stuck and aborts with a clear message and stack traces. It cannot detect a *partial* deadlock, where some goroutines still run while others are wedged.

Share memory by communicating

Go's concurrency slogan is “**Do not communicate by sharing memory; instead, share memory by communicating.**” Rather than guard one variable with a lock and let many goroutines

poke it, you give the data to one goroutine at a time by sending it through a channel. Whoever holds the value owns it; ownership travels with the message.

A common shape is a *generator*: a goroutine produces values and hands them out on a receive-only channel, closing it when finished.

```
package main

import "fmt"

// generate returns a receive-only channel, sends 0..n-1 on it, then closes it.
func generate(n int) <-chan int {
    out := make(chan int)
    go func() {
        for i := range n {
            out <- i
        }
        close(out)
    }()
    return out
}

func main() {
    for v := range generate(5) {
        fmt.Println(v)
    }
}
```

So when do you reach for a channel versus a mutex?

Use a channel when	Use a sync.Mutex when
You pass data or ownership between goroutines	You protect one shared structure in place
You coordinate or signal (start, stop, done)	A short critical section just updates fields
You build a pipeline or a worker pool	A channel would be overkill (a counter, a map)

Both are idiomatic Go. Mutexes, atomics, and the memory model are in Chapter 15 — sync and context; pipelines, fan-in/fan-out, and worker pools are in Chapter 16 — Concurrency Patterns.

Key takeaways

- A channel is a typed, thread-safe conduit. `make(chan T)` is unbuffered; `make(chan T, n)` is buffered.
- Send `ch <- v`, receive `v := <-ch`, test with `v, ok := <-ch`, finish with `close(ch)`. The arrow points the way data moves.
- **Unbuffered = synchronous rendezvous:** sender and receiver meet, hand over the value, and synchronize. **Buffered = asynchronous** up to the capacity.
- **Only the sender closes.** Sending on a closed channel, or closing a closed or nil channel, **panics**. Receiving from a closed channel returns the zero value with `ok == false`, and `range` ends.

- Direction types `chan<- T` (send-only) and `<-chan T` (receive-only) document and enforce one-way use.
- `select` waits on several operations; a ready case is chosen at random; `default` makes it non-blocking; `time.After` adds a timeout.
- A **nil channel blocks forever** — use it to disable a `select` case.
- If all goroutines block, the runtime aborts with **all goroutines are asleep - deadlock!**.
- Prefer channels to move data and ownership; prefer a mutex to protect state in place.

Watch out (gotchas for C programmers)

- **Sending on a closed channel panics.** So does closing a channel twice, or closing a nil channel.
- **Send or receive on a nil channel blocks forever** — handy in `select`, deadly by accident (an uninitialized channel variable is nil).
- **An unbuffered send with no ready receiver blocks;** if nothing else can run, the program deadlocks.
- **Only the sender should close, exactly once.** A receiver that closes is a bug.
- **range ch never ends unless someone closes ch** — a frequent cause of a hung program.
- **select with a default never blocks.** In a tight for loop with `default` and nothing ready, it busy-spins and burns a CPU core.

Interview questions

Q: What is the difference between an unbuffered and a buffered channel? A: An unbuffered channel (`make(chan T)`) stores nothing; a send blocks until a receiver is ready and vice versa, so the goroutines rendezvous and synchronize. A buffered channel (`make(chan T, n)`) holds up to `n` values; a send blocks only when the buffer is full and a receive only when it is empty, so the sender and receiver need not meet at the same instant.

Q: What happens when you send to, receive from, or close a closed channel? A: Sending on a closed channel panics. Closing an already-closed channel panics (and closing a nil channel panics). Receiving from a closed channel never blocks: it returns the zero value immediately with `ok == false`, and a `range` over it ends. Rule: only the sole sender closes, exactly once.

Q: How does select work, and what does default do? A: `select` blocks until one of its channel cases can proceed; if several are ready it picks one uniformly at random. A `default` case runs immediately when no other case is ready, which makes the `select` non-blocking. Combined with `time.After` it implements timeouts.

Q: Why would you ever want a nil channel? A: Operations on a nil channel block forever, so a nil channel's case in a `select` is never chosen. You use that to disable a case dynamically — for example, after a source channel closes you set its variable to nil so the `select` stops choosing it instead of spinning on the closed channel.

Q: When does Go report “all goroutines are asleep - deadlock!”, and when can it not? A: The runtime aborts with that message when it detects that every goroutine is blocked, so no progress is possible — for example an unbuffered send with no receiver in a single-goroutine program. It cannot detect a partial deadlock: if even one goroutine can still run, the runtime sees no global deadlock even though part of the program is stuck.

Q: What does “share memory by communicating” mean in practice? A: Instead of many

goroutines reading and writing the same variable under a lock, you pass the data through a channel so only one goroutine holds it at a time; ownership moves with the value. This avoids many data races and often makes the design clearer. A mutex is still the right tool for protecting a shared structure in place.

Try it

1. Make an unbuffered channel and send to it from `main` with no goroutine receiving. Run it and read the `all goroutines are asleep - deadlock!` message. Then start a receiving goroutine and watch it work.
2. Build the `generate` generator and `range` over it. Then wrap the receive in a `select` with a `time.After` case so the consumer gives up if the stream stalls.

Chapter 15 — Synchronization and context

What you'll learn. When to use channels and when to use the `sync` package, how `sync.Mutex`, `RWMutex`, `WaitGroup`, `Once`, and atomics map to the pthreads tools you already know, the practical rules of Go's memory model, and how `context.Context` carries cancellation and deadlines through your program.

Chapter 13 — Goroutines and the Scheduler gave you cheap threads, and Chapter 14 — Channels and select gave you a safe way to pass values between them. This chapter fills the last gap: how to **protect shared state** and how to **stop work** that is no longer needed. If you have written multithreaded C with pthreads, almost everything here has a direct analog — usually a simpler one.

Two toolboxes: channels vs the sync package

Go gives you two ways to coordinate goroutines. Both are correct. The trick is picking the right one for the job.

Use this	When you want to	C analog
Channels	Hand a value from one goroutine to another, or signal “an event happened”	A hand-built thread-safe queue (mutex + condvar + ring buffer)
sync package	Let several goroutines touch the <i>same</i> variable safely	<code>pthread_mutex_t</code> , <code>pthread_rwlock_t</code> , <code>pthread_once</code>

Rule of thumb. Use **channels to transfer ownership** of data and to coordinate *who does what next*. Use a **mutex to protect shared state** that stays in one place and is read or written by many goroutines. If you find yourself building a queue out of a mutex and a slice, you probably wanted a channel; if you find yourself sending a pointer back and forth just to guard one counter, you probably wanted a mutex.

This chapter is about the second toolbox (`sync`) plus `context`. Channels were Chapter 14; the patterns that combine both are Chapter 16 — Concurrency Patterns.

sync.Mutex: the lock you already know

A **mutex** (“mutual exclusion”) is a lock. At most one goroutine can hold it at a time. While one goroutine holds the lock, every other goroutine that calls `Lock` waits. This is exactly `pthread_mutex_t`.

```
import "sync"

var (
    mu      sync.Mutex
    balance int
)

func deposit(n int) {
    mu.Lock()
    defer mu.Unlock() // runs when deposit returns, on every path
    balance += n
}
```

The same idea in C with pthreads needs more ceremony, and there is no `defer`, so you must place the unlock on every return path by hand:

```
#include <pthread.h>

pthread_mutex_t mu = PTHREAD_MUTEX_INITIALIZER; /* must initialize */
long balance;

void deposit(long n) {
    pthread_mutex_lock(&mu);
    balance += n;
    pthread_mutex_unlock(&mu); /* forget this on any path and you deadlock */
}
```

The idiom `mu.Lock(); defer mu.Unlock()` is the heart of safe Go locking. `defer` schedules the unlock to run when the function returns, no matter *how* it returns — normal return, early return, or even a panic. It is the clean version of C’s `goto cleanup` (see Chapter 6 — Functions for `defer`).

C vs Go. In C you must initialize a mutex (`pthread_mutex_init`, or the static `PTHREAD_MUTEX_INITIALIZER`) and destroy it with `pthread_mutex_destroy`. In Go, the zero value of a `sync.Mutex` is an unlocked, ready-to-use mutex. Just declare it. There is no init call and no destroy call; the garbage collector reclaims it.

Concept	C (pthreads)	Go	Note
Mutex type	<code>pthread_mutex_t</code>	<code>sync.Mutex</code>	Zero value is ready.
Initialize	<code>pthread_mutex_init</code> / <code>PTHREAD_MUTEX_INITIALIZER</code>	<i>(nothing)</i>	<code>var mu sync.Mutex</code> is usable.
Lock / unlock	<code>pthread_mutex_lock</code> / <code>_unlock</code>	<code>mu.Lock()</code> / <code>mu.Unlock()</code>	Pair the unlock with <code>defer</code> .
Destroy	<code>pthread_mutex_destroy</code>	<i>(nothing)</i>	GC handles it.
Reader/writer lock	<code>pthread_rwlock_t</code>	<code>sync.RWMutex</code>	<code>RLock</code> for readers.

Concept	C (pthreads)	Go	Note
Recursive lock	PTHREAD_MUTEX_RECURSIVE	None	Go mutexes are never reentrant.
Condition variable	pthread_cond_t	sync.Cond (rare); prefer channels	Channels usually replace it.

A mutex protects a critical section

The code between Lock and Unlock is the **critical section**: the region only one goroutine may execute at a time.

goroutine A	shared state (guarded by mu)	goroutine B
-----		-----
mu.Lock()	-----+	
balance += 100	only ONE goroutine is	
balance += 50	inside the box at a time	
mu.Unlock()	-----+	
		mu.Lock() <- blocks while A holds it
		read balance
		mu.Unlock()

A common and clean pattern is to **bundle the lock with the data it protects** in a struct, and use pointer receivers (see Chapter 10 — Structs and Methods) so every method locks the same mutex:

```
package main

import (
    "fmt"
    "sync"
)

// Counter is safe for use by many goroutines at once.
type Counter struct {
    mu sync.Mutex
    n  int64
}

func (c *Counter) Inc() {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.n++
}

func (c *Counter) Value() int64 {
    c.mu.Lock()
    defer c.mu.Unlock()
    return c.n
}
```

```

}

func main() {
    var c Counter
    var wg sync.WaitGroup
    for range 1000 { // integer range, Go 1.22+
        wg.Add(1)
        go func() {
            defer wg.Done()
            c.Inc()
        }()
    }
    wg.Wait()
    fmt.Println(c.Value()) // always 1000
}

```

Watch out. A `sync.Mutex` is **not reentrant**. In C, a default `pthread_mutex_t` also deadlocks if you lock it twice in the same thread, but you can opt into `PTHREAD_MUTEX_RECURSIVE`. Go has **no recursive option at all**. If a method that holds the lock calls another method that also tries to lock, the goroutine deadlocks against itself. The fix: split the locked logic into a small unexported helper that assumes the lock is already held, and call that.

Rule of thumb. Keep critical sections short, and **never send on a channel (or make a blocking call) while holding a mutex**. Snapshot what you need under the lock, unlock, then do the blocking work. Holding a lock across a blocking operation is a classic way to freeze a whole program.

`sync.RWMutex`: many readers or one writer

When data is read far more often than it is written, a plain mutex is wasteful: readers do not conflict with each other, yet a `Mutex` makes them wait in line. A **read/write mutex** lets *many* readers hold the lock at once, but a writer gets it alone. This is `pthread_rwlock_t`.

```

type Config struct {
    mu sync.RWMutex
    m  map[string]string
}

func (c *Config) Get(key string) string {
    c.mu.RLock() // shared read lock; many goroutines may hold it
    defer c.mu.RUnlock()
    return c.m[key]
}

func (c *Config) Set(key, val string) {
    c.mu.Lock() // exclusive write lock; blocks all readers and writers
    defer c.mu.Unlock()
    c.m[key] = val
}

```

```
}
```

Use `RLock/RLock` for read-only access and `Lock/Unlock` for writes. Reach for `RWMutex` only when you have measured a read-heavy workload; for low contention a plain `Mutex` is simpler and often faster.

Do not copy a mutex

This rule catches every C programmer at least once. A `sync.Mutex` must **not be copied** after first use. Copying it makes a second, independent lock, so two goroutines that think they share a lock actually do not — and your “protected” data is now a data race.

A struct that contains a mutex inherits this rule: do not copy the struct either. So pass it by pointer.

```
type Counter struct {
    mu sync.Mutex
    n  int
}

func bad(c Counter) { c.mu.Lock() /* ... */ } // BUG: c is a COPY; locks nothing useful
func good(c *Counter){ c.mu.Lock() /* ... */ } // correct: shares the one mutex
```

C vs Go. In C, copying a `pthread_mutex_t` by assignment is undefined behavior and nothing warns you. In Go, the `go vet` tool has a built-in `copylocks` check that **flags this at build time**: “assignment copies lock value.” Run `go vet ./...` (see Chapter 22 — Tooling) and it catches the mistake for you. The fix is to use a pointer receiver and pass `*Counter`, or never pass the owning struct by value.

`sync.WaitGroup`: wait for a set of goroutines

A `WaitGroup` lets one goroutine wait until a group of other goroutines have all finished. It is a counter with three operations:

- `Add(n)` — add `n` to the counter (you are about to start `n` goroutines).
- `Done()` — subtract one (a goroutine finished). `Done()` is just `Add(-1)`.
- `Wait()` — block until the counter reaches zero.

It is the rough equivalent of calling `pthread_join` on every thread, but with one call instead of a loop, and without needing to keep the thread handles.

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    var wg sync.WaitGroup
    for i := range 5 {
        wg.Add(1) // increment BEFORE starting the goroutine
    }
}
```



```

    go func() {
        defer wg.Done() // decrement when this goroutine returns
        fmt.Println("worker", i) // i is per-iteration (Go 1.22+); each goroutine sees its
    }()
}
wg.Wait() // blocks until all five call Done
fmt.Println("all workers finished")
}

```

Two rules make WaitGroup correct:

1. **Call Add before the go statement, not inside the goroutine.** If you call Add inside the new goroutine, Wait might run *before* that goroutine gets scheduled, see a zero counter, and return too early. That is a race; the race detector (Chapter 16) will flag it.
2. **Pass a *WaitGroup (a pointer), never a copy.** A WaitGroup must not be copied, for the same reason a mutex must not be. When a helper function runs the goroutines, give it wg *sync.WaitGroup.

```

func runWorkers(wg *sync.WaitGroup, jobs []Job) {
    for _, j := range jobs {
        wg.Add(1)
        go func() {
            defer wg.Done()
            process(j)
        }()
    }
}

```

Deep dive. Go 1.25 added a convenience method `wg.Go(func())` that does `Add(1)`, runs the function in a new goroutine, and calls `Done()` for you. On Go 1.26 you can write `wg.Go(func() { process(j) })` instead of the three-line pattern above. The manual `Add/go/defer Done()` form is still the one you will read most often, so learn it first.

sync.Once: run something exactly once

`sync.Once` guarantees a function runs **one time only**, even if many goroutines call it at once. It is Go's `pthread_once`. The classic use is lazy, thread-safe initialization (a singleton).

```

var (
    once    sync.Once
    client  *http.Client
)

func Client() *http.Client {
    once.Do(func() {
        client = &http.Client{Timeout: 5 * time.Second}
    })
    return client // every caller gets the same client; setup ran once
}

```

Every goroutine that reaches `once.Do` either runs the function (the first one) or blocks until the first one finishes, then continues. After that, `Do` is nearly free.

Deep dive. Go 1.21 added `sync.OnceFunc`, `sync.OnceValue`, and `sync.OnceValues`, which wrap this pattern in a closure. `get := sync.OnceValue(func() T { ... })` returns a function you can call many times; the body runs once and every call returns the cached result.

`sync.Pool`: reuse allocations (brief)

`sync.Pool` is a free list of temporary objects you can reuse to reduce garbage collection pressure (see Chapter 17 — Memory and the Garbage Collector). You `Get` an object, use it, and `Put` it back. The pool may drop anything in it at any time, so only store things that are safe to lose and to reuse.

```
var bufPool = sync.Pool{
    New: func() any { return new(bytes.Buffer) }, // 'any', not interface{}
}

func writeReport(w io.Writer) {
    buf := bufPool.Get().(*bytes.Buffer)
    buf.Reset()
    defer bufPool.Put(buf)
    // ... build the report in buf, then copy it to w ...
}
```

Use it only when profiling shows a real allocation hot spot. It is not a general object cache.

`sync/atomic`: lock-free counters and flags

For a single integer or pointer touched by many goroutines, a full mutex can be overkill. The `sync/atomic` package gives **atomic operations**: reads and writes that the hardware guarantees happen as one indivisible step. Modern Go provides **typed atomics** — small struct types with methods — which are clearer and safer than the old free functions.

```
package main

import (
    "fmt"
    "sync"
    "sync/atomic"
)

func main() {
    var hits atomic.Int64 // zero value is ready; holds 0
    var wg sync.WaitGroup
    for range 1000 {
        wg.Add(1)
        go func() {
            defer wg.Done()

```

```

        hits.Add(1) // atomic increment; no mutex needed
    }()
}
wg.Wait()
fmt.Println(hits.Load()) // always 1000
}

```

The common typed atomics are `atomic.Int32`, `atomic.Int64`, `atomic.Uint32`, `atomic.Uint64`, `atomic.Bool`, and `atomic.Pointer[T]` (generic and type-safe).

Concept	C11 / GCC	Go	Note
Atomic integer	<code>_Atomic int</code> , <code>atomic_int</code>	<code>atomic.Int64</code> , <code>atomic.Int32</code> , ...	A struct with methods.
Add and return	<code>atomic_fetch_add</code>	<code>v.Add(1)</code>	Returns the new value.
Load / store	<code>atomic_load /</code> <code>atomic_store</code>	<code>v.Load() /</code> <code>v.Store(x)</code>	
Compare-and-swap	<code>atomic_compare_exchange</code>	<code>CompareAndSwap(old, new)</code>	Returns a bool.
Atomic pointer	<code>_Atomic(T*)</code>	<code>atomic.Pointer[T]</code>	Generic, no casts.

Rule of thumb. Use an atomic when you are guarding **one** value with simple operations (a counter, a flag, a swappable pointer). Use a **mutex** when an update touches **several** fields that must stay consistent together — for example, moving money between two balances. You cannot make two atomic operations into one atomic transaction; a mutex can.

The Go memory model, in practice

A **data race** happens when two goroutines access the same memory at the same time, at least one of them writes, and there is no synchronization between them. A data race is a **bug** in Go: the result is undefined, just like in C, and the program may behave differently every run.

The memory model is defined in terms of **happens-before**: an ordering between events. If event A happens-before event B, then B sees the effects of A. The synchronization tools create these orderings for you:

- A send on a channel **happens-before** the matching receive completes.
- A `mu.Unlock()` **happens-before** the next `mu.Lock()` returns.
- The `go` statement that starts a goroutine **happens-before** the goroutine runs.
- A `wg.Wait()` that returns sees everything done before the matching `Done()` calls.

If two accesses to shared memory have **no** happens-before relationship and one is a write, you have a race. Plain reads and writes of ordinary variables provide *no* ordering — that is why an unguarded shared counter is broken even though `n++` “looks atomic.” (It is not: it is load, add, store.)

Rule of thumb. *If you have to wonder whether something is a data race, it is.* Do not reason your way to “it is probably fine.” Either add synchronization (a channel, a

mutex, or an atomic) or prove there is no shared write. And always test with the race detector: `go test -race ./...` (Chapter 16 explains it in detail).

C vs Go. C11 also defines a memory model with `_Atomic` and memory orders, and data races are undefined behavior there too. The big practical difference: Go ships a **race detector** in the standard toolchain (`-race`), so you can *catch* races in testing instead of guessing. There is no setup; it is one flag.

context.Context: cancellation, deadlines, and request values

A long C program that wants to cancel work threads it through by hand: a shared “stop” flag, a pipe to `select` on, a deadline you check in a loop. Go standardizes all of this in one type, **context.Context**. A context carries three things down a call tree:

1. A **cancellation signal** — a way to say “stop, the result is no longer needed.”
2. A **deadline or timeout** — cancel automatically at a certain time.
3. A small set of **request-scoped values** — data tied to one request, such as a trace ID.

The core of a context is one method: `Done() <-chan struct{}`. It returns a channel that is **closed** when the context is cancelled. Goroutines wait on it with `<-ctx.Done()`. After cancellation, `ctx.Err()` tells you why: `context.Canceled` or `context.DeadlineExceeded`.

Creating contexts

You never make a context from nothing; you start from a root and derive children.

```
ctx := context.Background() // the root: never cancelled, no deadline, no values
ctx := context.TODO()       // a placeholder when you have not wired a real ctx yet
```

From a parent you derive children that add behavior. Each `With...` returns a new context **and a cancel function**:

```
// Cancel manually when you decide the work is done.
```

```
ctx, cancel := context.WithCancel(parent)
```

```
// Cancel automatically after a duration.
```

```
ctx, cancel := context.WithTimeout(parent, 2*time.Second)
```

```
// Cancel automatically at a specific time.
```

```
ctx, cancel := context.WithDeadline(parent, deadline)
```

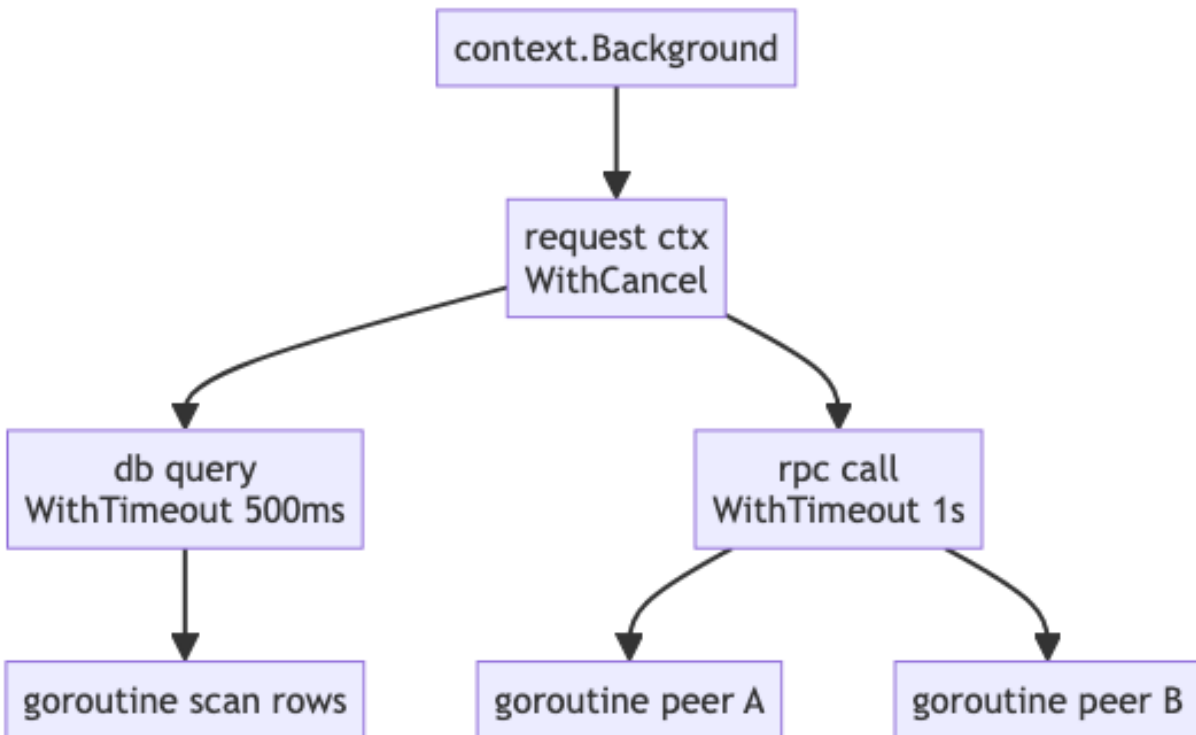
```
// Attach a request-scoped value (use sparingly; see below).
```

```
ctx := context.WithValue(parent, requestIDKey, "abc-123")
```

Watch out. You must **always** call **cancel**, even for `WithTimeout` and `WithDeadline` that look like they cancel themselves. Calling it releases the resources (timer, goroutine bookkeeping) tied to the context. The idiom is to **defer** it on the line right after creating the context: `defer cancel()`. Forgetting it leaks resources, and `go vet` will warn you.

Cancellation flows down a tree

Contexts form a tree. Cancelling a context cancels **all of its descendants**, but never its parent. This is how one timeout at the top of a request stops every goroutine the request started.



If the client disconnects and you cancel `root`, then `db`, `rpc`, and all four leaf goroutines see their `Done()` channel close and shut down. If only the `rpc` timeout fires, just `rpc`, `rpc1`, and `rpc2` are cancelled; the `db` branch keeps running.

Using a context

A worker selects on `ctx.Done()` alongside its real work, and returns as soon as the context is cancelled.

```
package main

import (
    "context"
    "fmt"
    "time"
)

// work simulates a job that takes 5 seconds, but gives up early if cancelled.
func work(ctx context.Context) error {
    select {
    case <-time.After(5 * time.Second):
        fmt.Println("work finished")
    }
```

```

    return nil
case <-ctx.Done():
    return ctx.Err() // context deadline exceeded, or context canceled
}
}

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 100*time.Millisecond)
    defer cancel() // ALWAYS; releases the timer

    if err := work(ctx); err != nil {
        fmt.Println("stopped early:", err) // stopped early: context deadline exceeded
    }
}

```

Conventions you must follow

The standard library and every Go team expect these rules:

- Pass `ctx` as the first parameter, named `ctx`: `func Fetch(ctx context.Context, url string) (...)`.
- **Do not store a Context in a struct.** Pass it explicitly to each call that needs it. A context describes one operation's lifetime; a struct outlives the operation.
- **Always defer `cancel()`** after a `With...` call.
- Use `context.WithValue` sparingly — only for request-scoped data that crosses API boundaries (a trace ID, an auth token), never for passing optional function arguments. Values are untyped (`any`) and invisible to the compiler, so overusing them turns clear function signatures into a guessing game. Use a private key type to avoid collisions:

```

type ctxKey int
const requestIDKey ctxKey = 0

ctx = context.WithValue(ctx, requestIDKey, "abc-123")
id, ok := ctx.Value(requestIDKey).(string) // type assertion back to string

```

Key takeaways

- Two valid toolboxes: **channels** transfer ownership and signal events; the **sync package** protects shared state in place.
- `sync.Mutex` is `pthread_mutex_t`, but its **zero value is ready** (no init, no destroy). Use `mu.Lock()`; `defer mu.Unlock()`.
- Go mutexes are **not reentrant** and must **not be copied**; go vet's `copylocks` check catches accidental copies.
- `sync.RWMutex` allows many readers or one writer; use it only for read-heavy data.
- `sync.WaitGroup` waits for a group of goroutines. Call `Add` **before** go, use `defer wg.Done()`, and pass a `*WaitGroup`.
- `sync.Once` runs setup exactly once (singletons); `sync.Pool` reuses allocations.
- `sync/atomic` typed atomics (`atomic.Int64`, `atomic.Bool`, ...) are best for a single counter or flag; a mutex is better when several fields must change together.

- A **data race** is a bug; rely on happens-before (channels, locks, atomics) and test with `-race`. “If you have to wonder whether it is a race, it is.”
- `context.Context` carries cancellation, deadlines, and request values down a call tree. Pass it first, never store it in a struct, and always `defer cancel()`.

Watch out (gotchas for C programmers)

- **Forgetting to unlock.** Always pair `Lock` with `defer Unlock`. An early return without unlocking freezes every other goroutine.
- **Copying a mutex (or a struct that holds one).** Two goroutines then lock two different mutexes and the data is unprotected. Use pointer receivers; run `go vet`.
- **Recursive locking deadlocks.** Go has no recursive mutex. A locked method that calls another locking method deadlocks. Extract a lock-free helper.
- **Calling `wg.Add` inside the goroutine.** `Wait` can return before the goroutine starts. Always `Add` before `go`.
- **Sending on a channel while holding a mutex.** Snapshot under the lock, unlock, then send. Otherwise you risk a deadlock.
- **Forgetting `defer cancel()`.** Even `WithTimeout` must be cancelled to free its timer; skipping it is a resource leak.
- **Storing `context.Context` in a struct or overusing `ctx.Value`.** Pass context as the first argument; keep values for request-scoped data only.

Interview questions

Q: When would you use a channel and when would you use a mutex? A: Use a channel to transfer ownership of data between goroutines or to signal that an event happened — anything that is really “pass this along” or “coordinate who acts next.” Use a mutex to protect shared state that stays in one place and is read or written by several goroutines, such as a counter or a cache. Both are idiomatic; pick the one that matches the problem.

Q: Why does Go not need `pthread_mutex_init`, and what is special about a mutex’s zero value? A: A `sync.Mutex` is designed so its zero value is a valid, unlocked mutex. Declaring `var mu sync.Mutex` (or embedding it in a struct) is enough to use it. There is no `init` and no `destroy` call; the garbage collector reclaims it. This is part of Go’s broader “make the zero value useful” philosophy.

Q: What is a data race, and how do you find one in Go? A: A data race is two goroutines accessing the same memory concurrently with no synchronization between them, where at least one access is a write. The result is undefined. You find races with Go’s built-in race detector by running `go test -race` or `go run -race`; it instruments memory accesses and reports the conflicting goroutines and stacks at runtime.

Q: When should you reach for `sync/atomic` instead of a mutex? A: Use an atomic when you guard a single value with simple operations — an integer counter, a boolean flag, or a swappable pointer. Use a mutex when one logical update spans several fields that must stay consistent together, because you cannot combine two atomic operations into a single atomic transaction.

Q: What problems does `context.Context` solve, and what are the rules for using it? A: It carries cancellation, deadlines/timeouts, and request-scoped values down a call tree so you can stop work that is no longer needed and bound how long it runs. Cancelling a context cancels all

of its descendants. The rules: pass it as the first parameter named `ctx`, never store it in a struct, always `defer cancel()` after a `With...` call, and use `ctx.Value` only for request-scoped data, not ordinary arguments.

Try it

Write a `SafeMap` type wrapping a `map[string]int` with a `sync.RWMutex`. Give it `Get`, `Set`, and `Inc` methods. Launch 100 goroutines that each call `Inc("x")` 1000 times, wait with a `WaitGroup`, and print the total. Run it with `go run -race .` and confirm the detector stays quiet. Then remove the locks and watch it both report a race and print the wrong total.

Chapter 16 — Concurrency Patterns

What you'll learn. The race detector and how to use it, plus the handful of goroutine-and-channel patterns you will reuse forever: pipelines, fan-out/fan-in, worker pools, semaphores, `errgroup`, rate limiting, and graceful shutdown — with the common mistakes spelled out.

You now have goroutines (Chapter 13 — Goroutines and the Scheduler), channels (Chapter 14 — Channels and select), and the `sync` package and `context` (Chapter 15 — Synchronization and context). This chapter assembles them into the standard shapes. Learn these and you will recognize the structure of almost any concurrent Go program. They are the Go answer to the ad-hoc thread-pool and producer/consumer code you would hand-roll with pthreads in C.

The race detector: your most important tool

A **data race** (Chapter 15) is two goroutines touching the same memory at once with no synchronization, where at least one writes. The result is undefined. The good news: Go ships a **race detector** in the standard toolchain. Turn it on with one flag.

```
go test -race ./...    # run the tests with race detection (do this in CI)
go run -race .         # run a program with race detection
go build -race -o app  # build an instrumented binary
```

What it does and does not do:

- It is a **dynamic** detector. It watches the memory accesses that **actually happen** while the program runs, and reports a race only when conflicting accesses really occur. It does *not* prove your code is race-free; it finds the races your test actually exercised. So feed it realistic, concurrent tests.
- When it sees a race it prints both goroutines' stacks and the variable, then the program exits non-zero — perfect for failing a CI build.
- **It is not free.** Expect roughly 2x–20x more CPU time and several times more memory. You run it in tests and CI, not usually in production.

Rule of thumb. Run `go test -race ./...` in continuous integration on every change. A race that reproduces once in a thousand runs in production will often show up immediately under `-race`, because the detector flags the *unsynchronized access*, not the rare bad interleaving.

A tiny race and its fix

Here is a classic broken counter. Many goroutines do `count++`, which is really *load, add, store* — three steps that can interleave.

```
// BROKEN: data race on count. Run with: go run -race .
package main

import (
    "fmt"
    "sync"
)

func main() {
    count := 0
    var wg sync.WaitGroup
    for range 1000 {
        wg.Add(1)
        go func() {
            defer wg.Done()
            count++ // RACE: unsynchronized read-modify-write
        }()
    }
    wg.Wait()
    fmt.Println(count) // not reliably 1000
}
```

Under `-race` this reports `DATA RACE` and points at the `count++` line. The fix is any synchronization tool from Chapter 15 (Synchronization and context). Here, an atomic counter is the simplest:

```
// FIXED: atomic counter. Run with: go run -race .
package main

import (
    "fmt"
    "sync"
    "sync/atomic"
)

func main() {
    var count atomic.Int64
    var wg sync.WaitGroup
    for range 1000 {
        wg.Add(1)
        go func() {
            defer wg.Done()
            count.Add(1) // atomic: no race
        }()
    }
}
```

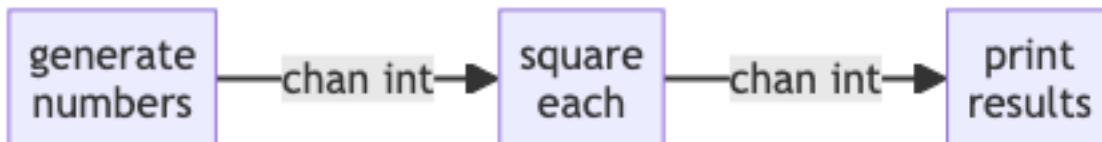
```

    wg.Wait()
    fmt.Println(count.Load()) // always 1000
}

```

Pipelines

A **pipeline** is a series of **stages** connected by channels. Each stage is a group of goroutines that receive values from an input channel, do one job, and send results to an output channel. It is the concurrent version of piping commands in a shell: `cmd1 | cmd2 | cmd3`.



The first stage is a **generator**: a function that returns a receive-only channel (`<-chan T`) and runs a goroutine that fills it, then **closes** it when done. Closing tells downstream stages “no more values,” so their `for range` loops end.

```

package main

import "fmt"

// gen sends the given numbers on a channel, then closes it.
func gen(nums ...int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out) // closing ends the receiver's range loop
        for _, n := range nums {
            out <- n
        }
    }()
    return out
}

// square reads ints from in, sends their squares to out, closes out at the end.
func square(in <-chan int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for n := range in { // ends when in is closed
            out <- n * n
        }
    }()
    return out
}

func main() {

```

```

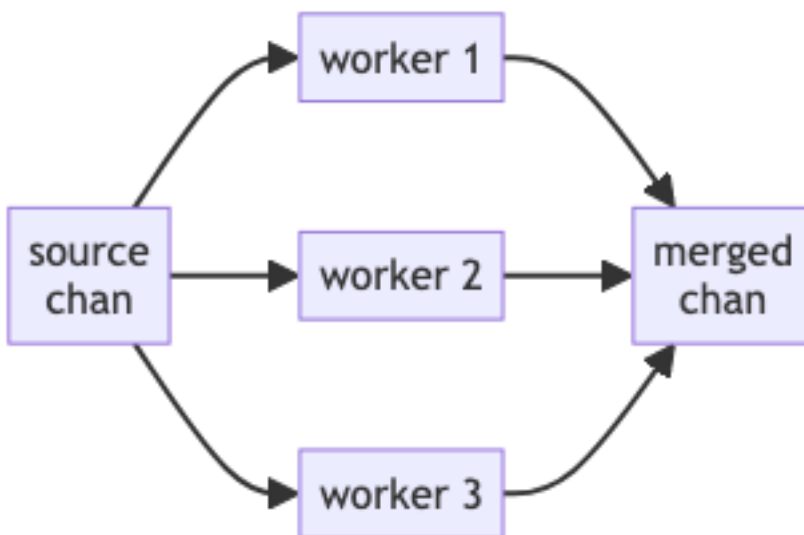
for n := range square(gen(2, 3, 4)) { // compose the stages
    fmt.Println(n) // 4, 9, 16
}
}

```

Mental model. A stage owns its output channel: it is the only one that sends on it, and it is the one that closes it. “The sender closes, never the receiver” is the rule from Chapter 14, and pipelines depend on it to terminate cleanly.

Fan-out and fan-in

When one stage is slow, run **several** copies of it reading from the same channel. Distributing work to multiple goroutines is **fan-out**. Merging their results back into one channel is **fan-in**.



Because Go channels are safe for many senders and many receivers, fan-out is just “start N goroutines that all **range** the same input channel.” Fan-in needs a little care: we use a `WaitGroup` to close the merged channel exactly once, *after* every worker has finished.

```

package main

import (
    "fmt"
    "sync"
)

func gen(nums ...int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for _, n := range nums {
            out <- n
        }
    }()
}

```

```

    return out
}

// merge fans in: it copies values from every input channel into one output channel.
func merge(ins ...<-chan int) <-chan int {
    out := make(chan int)
    var wg sync.WaitGroup
    for _, in := range ins {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for n := range in {
                out <- n
            }
        }()
    }
    go func() {
        wg.Wait() // wait for all copiers...
        close(out) // ...then close the merged channel exactly once
    }()
    return out
}

func square(in <-chan int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for n := range in {
            out <- n * n
        }
    }()
    return out
}

func main() {
    in := gen(2, 3, 4, 5, 6, 7, 8, 9)

    // Fan-out: two workers read the same channel.
    w1 := square(in)
    w2 := square(in)

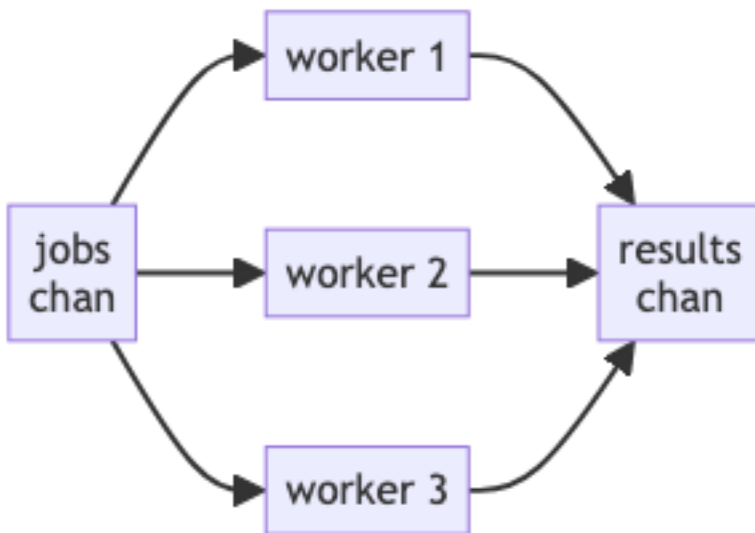
    // Fan-in: merge their outputs, then sum (order is not deterministic).
    sum := 0
    for n := range merge(w1, w2) {
        sum += n
    }
    fmt.Println(sum) // 284
}

```

Watch out. Results from fan-in arrive in **nondeterministic order**, because the workers race each other. If you need order, attach an index to each item and sort at the end, or do not fan out that stage.

Worker pool with bounded concurrency

A pipeline starts one goroutine per stage; a **worker pool** starts a *fixed* number of workers that pull jobs from a shared channel. This is how you cap concurrency: 1000 jobs, but only N running at once. It is the Go version of a pthread thread pool, but the “pool” is just a channel plus a few goroutines.



```
package main

import (
    "fmt"
    "sync"
)

func worker(id int, jobs <-chan int, results chan<- int, wg *sync.WaitGroup) {
    defer wg.Done()
    for j := range jobs { // pull jobs until the channel is closed
        results <- j * 2 // the "work"
    }
}

func main() {
    const numJobs = 9
    const numWorkers = 3

    jobs := make(chan int, numJobs)
    results := make(chan int, numJobs)
```

```

// Start a fixed number of workers (bounded concurrency).
var wg sync.WaitGroup
for w := 1; w <= numWorkers; w++ {
    wg.Add(1)
    go worker(w, jobs, results, &wg)
}

// Send all jobs, then close so workers' range loops end.
for j := 1; j <= numJobs; j++ {
    jobs <- j
}
close(jobs)

// Close results once every worker has returned.
go func() {
    wg.Wait()
    close(results)
}()

sum := 0
for r := range results { // drain until results is closed
    sum += r
}
fmt.Println("sum:", sum) // sum: 90
}

```

The shape to memorize: **create workers, feed a jobs channel, close jobs, wait, then close results**. The `WaitGroup` plus the “closer goroutine” is what lets the `for range results` loop end on its own.

Bounding parallelism with a semaphore channel

Sometimes you do not want a fixed pool; you want to spawn one goroutine per task but limit how many run at once — for example, “fetch 500 URLs, but at most 10 in flight.” A **buffered channel of empty structs** works as a counting semaphore.

`struct{}` is the zero-size empty struct, so `chan struct{}` carries no data, only the “slot taken / slot free” signal. The buffer size `n` is the limit.

```

package main

import (
    "fmt"
    "sync"
)

func main() {
    urls := make([]string, 50)
    for i := range urls {

```

```

    urls[i] = fmt.Sprintf("url-%d", i)
}

sem := make(chan struct{}, 10) // at most 10 concurrent
var wg sync.WaitGroup

for _, u := range urls {
    wg.Add(1)
    sem <- struct{}{} // acquire: blocks once 10 slots are taken
    go func() {
        defer wg.Done()
        defer func() { <-sem }() // release the slot
        _ = fetch(u)
    }()
}
wg.Wait()
fmt.Println("done")
}

func fetch(u string) error { return nil } // stand-in for real work

```

Sending to `sem` blocks when the buffer is full, so the loop pauses until a running goroutine releases a slot by receiving from `sem`. That is the entire limiter.

errgroup: goroutines that can fail

The `WaitGroup` patterns above ignore errors. Real work fails, and usually you want the **first** error and to **cancel** the rest. The package `golang.org/x/sync/errgroup` does exactly that. It is part of the `golang.org/x` modules — official, but a **separate dependency** you must add to `go.mod` with `go get golang.org/x/sync`.

`errgroup.WithContext` gives you a group and a context. `g.Go(func() error)` runs each task; if any returns a non-nil error, the group **cancels the context** so the others can stop, and `g.Wait()` returns that first error.

```

package main

import (
    "context"
    "errors"
    "fmt"

    "golang.org/x/sync/errgroup"
)

func main() {
    g, ctx := errgroup.WithContext(context.Background())

    urls := []string{"a", "b", "bad", "c"}

```

```

for _, u := range urls {
    g.Go(func() error { // each task is a func() error
        return fetch(ctx, u)
    })
}

if err := g.Wait(); err != nil { // first non-nil error, or nil
    fmt.Println("group failed:", err) // group failed: bad url: bad
} else {
    fmt.Println("all ok")
}
}

```

```

func fetch(ctx context.Context, u string) error {
    if u == "bad" {
        return errors.New("bad url: " + u)
    }
    select {
    case <-ctx.Done():
        return ctx.Err() // a sibling failed; stop early
    default:
        return nil
    }
}

```

`g.SetLimit(n)` even bounds concurrency, turning the group into a worker pool with error handling. `errgroup` is usually the cleanest answer when you fan out work that can fail.

Rate limiting

To do something at most `N` times per second, the standard-library tool is `time.Ticker`. A ticker sends the current time on its channel at a fixed interval; receive from it once per unit of work to pace a loop.

```

package main

import (
    "fmt"
    "time"
)

func main() {
    ticker := time.NewTicker(200 * time.Millisecond) // 5 per second
    defer ticker.Stop() // always stop a ticker to free it

    for i := range 3 {
        <-ticker.C // wait for the next tick before proceeding
        fmt.Println("request", i, "at", time.Now().Format("15:04:05.000"))
    }
}

```



```
}
```

Rule of thumb. For anything beyond a steady tick — bursts, a token bucket, or per-key limits — use `golang.org/x/time/rate` (another `golang.org/x` dependency). Its `rate.Limiter` supports `Wait`, `Allow`, and `Reserve`, and handles bursts correctly. Reach for it instead of building your own bucket.

Graceful shutdown: context + WaitGroup

Production services must stop cleanly: stop accepting new work, let in-flight work finish, then exit. The idiom combines the two tools from Chapter 15. A `context` broadcasts “shut down now”; a `WaitGroup` waits for the workers to drain.

```
package main
```

```
import (  
    "context"  
    "fmt"  
    "sync"  
    "time"  
)
```

```
// worker runs until the context is cancelled, doing one unit per tick.
```

```
func worker(ctx context.Context, id int, wg *sync.WaitGroup) {  
    defer wg.Done()  
    ticker := time.NewTicker(10 * time.Millisecond)  
    defer ticker.Stop()  
    for {  
        select {  
        case <-ctx.Done(): // shutdown signal: stop the loop and return  
            fmt.Printf("worker %d stopping: %v\n", id, ctx.Err())  
            return  
        case <-ticker.C:  
            // ... do one unit of work ...  
        }  
    }  
}
```

```
func main() {  
    ctx, cancel := context.WithCancel(context.Background())  
  
    var wg sync.WaitGroup  
    for id := range 3 {  
        wg.Add(1)  
        go worker(ctx, id, &wg)  
    }  
  
    // Run for a while, then signal shutdown. In a real server this would be an  
    // OS signal (see Chapter 23 - Building Web Services with net/http) via signal.NotifyConte
```

```

time.Sleep(50 * time.Millisecond)
cancel() // tell every worker to stop (cancellation propagates down the tree)
wg.Wait() // wait for all of them to finish in-flight work
fmt.Println("shutdown complete")
}

```

Cancellation **propagates**: cancelling the parent context closes every child's `Done()` channel, so a single `cancel()` stops the whole tree. Real servers get the root context from `signal.NotifyContext(ctx, os.Interrupt)`, which cancels it when the user presses Ctrl-C.

Common mistakes

These three account for most concurrency bugs in real Go code.

- **Leaking goroutines.** A goroutine that blocks forever on a channel never returns, and its stack memory is never freed. The usual cause: a goroutine sends to a channel no one will ever receive from (often because the receiver returned early). Always give a goroutine a way to exit — a closed channel, or `<-ctx.Done()`.

```

// LEAK: if the caller stops reading, this goroutine blocks on send forever.
func leaky() <-chan int {
    ch := make(chan int) // unbuffered
    go func() {
        for i := 0; ; i++ {
            ch <- i // blocks forever once nobody receives
        }
    }()
    return ch
}

```

The fix is to pass a `context.Context` and `select` on `ctx.Done()` in the sender, so it can return when the consumer is done.

- **Creating unbounded goroutines.** `for _, item := range hugeSlice { go handle(item) }` can launch millions of goroutines, exhausting memory and thrashing the scheduler. Bound it: use a worker pool, a semaphore channel, or `errgroup` with `SetLimit`.
- **Discarding errors from goroutines.** A bare `go doWork()` throws away any error `doWork` returns — and you cannot `return` an error out of a goroutine to its starter. Collect errors through a channel, or use `errgroup` so failures are not silently lost.

Watch out. A **full channel** is also a deadlock waiting to happen. Sending to an unbuffered channel blocks until someone receives; if that someone is the same goroutine, or is also blocked sending, the program deadlocks. The runtime detects a *total* deadlock and panics with **all goroutines are asleep - deadlock!**, but a *partial* one (only some goroutines stuck) just hangs. Run with `-race` and design clear ownership of who sends and who closes.

Recap: share memory by communicating

Go's guiding proverb for concurrency is:

Do not communicate by sharing memory; instead, share memory by communicating.

In C you typically do the opposite: threads share a buffer and you bolt a mutex on to guard it. Go says: prefer to **pass the data through a channel** so that at any moment only one goroutine owns it. Ownership moves with the value, and there is nothing to lock. Mutexes and atomics are still there (Chapter 15) for the cases where shared state really is the simplest design — but reach for channels first, and you will write fewer races.

Key takeaways

- The **race detector** (`-race`) is your safety net. It finds races that actually occur at runtime; run it in tests and CI. It costs CPU and memory, so it is not for production.
- A **pipeline** chains stages with channels; a **generator** returns `<-chan T` and closes it when done. The sender always closes the channel.
- **Fan-out** runs several workers on one input channel; **fan-in** merges their outputs, using a `WaitGroup` to close the merged channel once.
- A **worker pool** bounds concurrency with a fixed number of workers pulling from a jobs channel. A **semaphore channel** (`make(chan struct{}, n)`) bounds per-task goroutines.
- `golang.org/x/sync/errgroup` runs failable tasks, returns the first error, and cancels the rest via context (it is an `x/` module dependency).
- Rate-limit with `time.Ticker` for a steady rate, or `golang.org/x/time/rate` for bursts and token buckets.
- **Graceful shutdown** = a context to signal stop + a `WaitGroup` to drain workers.
- Avoid leaking goroutines, creating unbounded goroutines, and discarding goroutine errors. Prefer to **share memory by communicating**.

Watch out (gotchas for C programmers)

- **Goroutine leaks.** A goroutine blocked forever on a channel is never collected. Give every goroutine an exit path (`ctx.Done()` or a closed channel).
- **Unbounded goroutines.** `go` inside a loop over a huge collection can spawn millions. Bound concurrency with a pool, a semaphore, or `errgroup.SetLimit`.
- **Lost errors.** You cannot return an error from a goroutine to its starter. Use a channel or `errgroup`; never write a bare `go mightFail()`.
- **Deadlock on a full or unbuffered channel.** A send with no matching receive blocks forever. The runtime panics only on a *total* deadlock; partial ones hang.
- **Not running `-race`.** Concurrency bugs hide in timing. Make `go test -race ./...` part of CI so they surface early.

Interview questions

Q: What does the Go race detector do, and what are its limits? A: It instruments memory accesses at runtime and reports a data race when it observes two unsynchronized accesses to the same memory where at least one is a write, printing both goroutine stacks. You enable it with `-race` on `go test`, `go run`, or `go build`. Its limit is that it is dynamic: it only finds races on code paths that actually execute during the run, so it cannot prove the absence of races. It also adds significant CPU and memory overhead, so it is for testing and CI, not production.

Q: How do you bound the number of goroutines doing work at once? A: Two common ways. A worker pool starts a fixed number of goroutines that all read from one shared jobs channel, so concurrency equals the worker count. Or a semaphore channel — a buffered `chan struct{}` of size `n` — where each goroutine sends before starting and receives when done, capping in-flight work at `n`. `errgroup.SetLimit` provides the same bound with error handling built in.

Q: How does a pipeline terminate cleanly? A: Each stage owns its output channel and is the only sender; it closes that channel when its input is exhausted (typically with `defer close(out)`). Closing ends the downstream stage's `for range` loop, which lets that stage finish and close its own output, and so on down the line. The rule is that the sender closes, never the receiver.

Q: What causes a goroutine leak and how do you prevent it? A: A goroutine leaks when it blocks forever — usually sending on a channel no one will receive from, or receiving from one that will never be sent to or closed — so it never returns and its stack is never freed. Prevent it by giving every goroutine a guaranteed exit: `select on ctx.Done()`, ensure channels get closed, or use buffered channels and timeouts so a send or receive cannot block indefinitely.

Q: Explain “do not communicate by sharing memory; share memory by communicating.” A: It is Go's preference for passing data between goroutines through channels rather than having them share a variable guarded by a lock. When a value travels over a channel, ownership moves with it, so only one goroutine touches it at a time and there is no shared state to protect. Mutexes and atomics remain available for genuinely shared state, but channels are the first choice and eliminate many data races by design.

Try it

Build a concurrent URL checker. Read a list of URLs, fan them out to a worker pool of 8 goroutines that record each status, and collect the results. Add a `context.WithTimeout` so the whole run cannot exceed 5 seconds, and switch the pool to `errgroup` so the first hard failure cancels the rest. Run it with `go run -race .` and confirm the detector stays silent.

Chapter 17 — Memory and the Garbage Collector

What you'll learn. Where Go puts your values (stack or heap), how the compiler decides with *escape analysis*, how the garbage collector frees memory for you, and how to write code that allocates less — all compared to C's `malloc/free`.

In C you own every byte. You call `malloc` to get heap memory and `free` to give it back, and if you get the bookkeeping wrong you get a leak, a double-free, or a use-after-free. Go takes that whole job away. There is **no free**. A part of the Go runtime called the **garbage collector** (GC) finds memory you no longer use and reclaims it automatically.

This chapter explains what happens under the hood, so you can reason about performance the way you already do in C — just without the manual frees.

Stack vs heap

Two regions hold your data while a program runs. The terms mean the same thing they do in C.

- The **stack** is fast, scratch memory tied to a function call. Each call gets a **frame** (a block holding that call's local variables). Allocating is almost free: the runtime just moves a pointer.

When the function returns, its whole frame disappears at once. Nothing to free.

- The **heap** is memory that must outlive the call that created it. It is more expensive to manage. In C *you* manage it with `malloc/free`; in Go the **GC** manages it.

The big difference from C: **each goroutine has its own stack, and that stack grows automatically**. A goroutine is Go's lightweight thread (see Chapter 13 — Goroutines and the Scheduler). Its stack starts small (a few kilobytes) and, when a deep call chain needs more room, the runtime allocates a bigger stack and copies the frames over. You never set a stack size or get a “stack overflow” from normal code, as you might with a fixed C thread stack.

C vs Go. In C the stack has a fixed maximum (often 8 MiB per thread), set at thread creation. Deep recursion crashes with a stack overflow. In Go the goroutine stack is growable, so the same recursion just makes the stack bigger. The trade is that Go must be able to find and rewrite pointers when it moves a stack — which it can, because Go has no pointer arithmetic.

Escape analysis: the compiler decides stack or heap

Here is the key idea, and it surprises most C programmers: **you do not choose stack or heap in Go**. There is no `malloc`. You just write `x := 0` or `p := &T{}`, and the **compiler** decides where the value lives. This decision is called **escape analysis**.

The rule the compiler follows: a value can stay on the stack only if the compiler can prove its lifetime ends when the function returns. If the value's address **escapes** — outlives the function — it must go on the heap instead. A value escapes when:

- its address is **returned** from the function, or
- it is **stored into something on the heap** (a heap struct field, a slice, a map), or
- it is **captured by a closure** that outlives the function (see Chapter 6 — Functions), or
- it is **sent on a channel** (see Chapter 14 — Channels and `select`), or
- it is **stored in an interface** value whose dynamic type is large or escapes (see Chapter 11 — Interfaces).

The most important consequence: **returning the address of a local variable is safe in Go**. This is the exact opposite of the classic C bug.

```
/* BROKEN in C: 'c' lives in this frame, which is freed on return.  
The returned pointer dangles; using it is undefined behavior. */  
int *new_counter(void) {  
    int c = 0;  
    return &c;           /* do NOT do this in C */  
}  
  
// Safe in Go. The compiler sees that &c escapes, so it allocates c  
// on the heap. The pointer stays valid for as long as the caller holds it.  
func newCounter() *int {  
    c := 0  
    return &c  
}
```

In C, returning `&c` is a bug because the stack frame is gone after `return`. In Go it is normal and correct: escape analysis moves `c` to the heap, and the GC frees it later when nobody points to it

anymore. You write the *same* code; the compiler makes it safe.

Seeing the decisions

The compiler will print every escape decision. Pass `-gcflags=-m` to `go build`:

```
$ go build -gcflags=-m .
# ./main.go:11:2: moved to heap: x      <- escapes(): x is allocated on the heap
# (stays() prints nothing: its x stayed on the stack)
```

Here are the two functions that produce that output:

```
package main

import "fmt"

func stays() int {
    x := 42 // does NOT escape: copied out by value, dies with the frame
    return x
}

func escapes() *int {
    x := 42 // escapes: its address leaves the function
    return &x // safe; the compiler moves x to the heap
}

func main() {
    a := stays()
    b := escapes()
    fmt.Println(a, *b)
}
```

This ASCII picture shows what the runtime does at the moment `escapes` returns:

STACK (one per goroutine; grows/shrinks; freed on return)			HEAP (freed later, by the GC)	
+-----+			+-----+	
main frame				
a int = 42 <- stays here				
b *int	----->			
+-----+			+-----+	
escapes frame (already gone)			x = 42	
returned &x			moved here because	
			its address escaped	
+-----+			+-----+	
			^ reclaimed once no pointer refers to it	

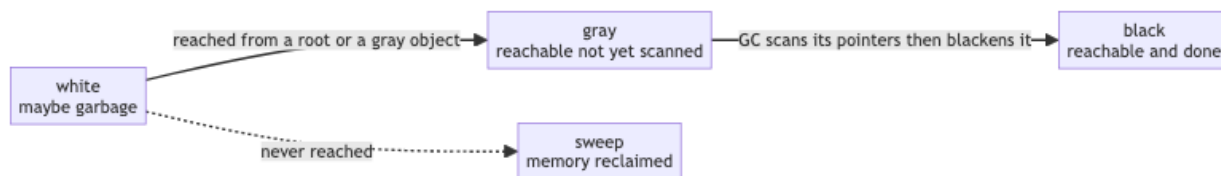
Mental model. Escape analysis is the compiler doing your `malloc/free` reasoning for you. “Does this outlive the call?” In C you answer that question and call `malloc` if the answer is yes. In Go the compiler answers it and allocates for you — and never gets the `free` wrong.

Rule of thumb. Do not contort your code to avoid the heap. Write clear code first. If a profiler later shows a hot allocation, *then* use `-gcflags=-m` to see why a value escapes and adjust.

The garbage collector

Go's GC is a **concurrent, tri-color, mark-and-sweep** collector. It is **non-generational** and **non-compacting**. Those words matter, so here is each one in plain English.

- **Mark-and-sweep.** The GC first *marks* every object that is still reachable (something points to it), then *sweeps* (reclaims) everything not marked.
- **Tri-color.** A bookkeeping scheme that lets marking run *while your program keeps running*. Objects are sorted into three sets: white, gray, and black (see the diagram below).
- **Concurrent.** Marking happens at the same time as your goroutines, on spare CPU. The GC only stops all goroutines (a **stop-the-world**, or STW, pause) for very short moments to start and finish a cycle. In modern Go these pauses are usually **well under a millisecond**.
- **Non-generational.** Unlike Java's HotSpot GC, Go does not keep separate “young” and “old” heaps. It scans the whole live heap each cycle.
- **Non-compacting.** The GC never moves a live heap object, so **a pointer's address never changes** for the life of the object. (Stacks *can* move, but the runtime fixes up those pointers itself.)



Marking starts from the **roots** — the values on every goroutine stack plus the global variables. Each root's targets turn gray. The GC scans a gray object's pointer fields (turning each target gray), then colors that object black. When no gray objects remain, every white object is unreachable garbage and gets swept. A small **write barrier** (extra code the compiler inserts around pointer writes) keeps the coloring correct even though your program is mutating pointers at the same time.

C vs Go. In C you decide *when* and *what* to free, at the cost of getting it right every time. In Go the GC decides, trading a little CPU and memory for the guarantee that you can never double-free, use-after-free, or leak through a forgotten **free**. You can still leak — but only by keeping references *alive* (see below), never by forgetting to release them.

Tuning: GOGC and GOMEMLIMIT

You rarely call the GC directly. Instead you tune *how often* it runs with two knobs.

GOGC sets the heap growth target as a percentage. The default is 100, which means: let the heap grow to about **100% larger** than the live size measured after the last GC before starting the next collection. Higher values run the GC **less** often (more memory, less CPU); lower values run it **more** often.

```
GOGC=100 ./server    # default: collect after the heap doubles
GOGC=200 ./server    # collect less often: more memory, less GC CPU
```

```
GOGC=off ./server    # disable the GC entirely (rare; for short batch jobs)
```

GOMEMLIMIT (added in Go 1.19) sets a **soft memory limit**. The runtime runs the GC more aggressively as total memory approaches the limit, to try to stay under it. It is *soft*: Go will blow past it rather than crash if the live set genuinely needs the memory, but it will spend a lot of CPU on GC trying not to. It is the best defense against running out of memory in a container.

```
GOMEMLIMIT=512MiB ./server    # keep total memory near 512 MiB if possible
```

You can set both from code via the `runtime/debug` package:

```
import "runtime/debug"

func tuneGC() {
    debug.SetGCPercent(200)           // same as GOGC=200
    debug.SetMemoryLimit(512 << 20) // 512 MiB, same as GOMEMLIMIT=512MiB
}
```

Rule of thumb. The common production setup is to **leave GOGC at 100 and set GOMEMLIMIT** to a bit below your container’s memory limit. That gives the GC room to be efficient while protecting you from an out-of-memory kill.

Forcing GC, finalizers, and cleanups

- `runtime.GC()` forces a full, blocking collection right now. Use it in tests or benchmarks, almost never in real code.
- `runtime.SetFinalizer(p, fn)` asks the runtime to call `fn(p)` *sometime* after `p` becomes unreachable. Finalizers are unreliable (no guaranteed timing, run at most once, and they *delay* reclamation). **Rarely use them.**
- `runtime.AddCleanup` (added in Go 1.24) is the modern, safer replacement. You attach a cleanup function and a copied argument; the argument must **not** be the object itself, so the cleanup cannot accidentally keep the object alive.

```
import (
    "fmt"
    "runtime"
)

type Handle struct{ id int }

func track(h *Handle) {
    // Runs after h is unreachable. Note we pass h.id by value, not h,
    // so this closure does not pin h in memory.
    runtime.AddCleanup(h, func(id int) {
        fmt.Println("cleaning up", id)
    }, h.id)
}
```

Watch out. Finalizers and cleanups are a backstop, not a **free**. Do not use them to release things that need timely closing (files, sockets, locks). For those, use an explicit `Close` method and `defer` (see Chapter 6 — Functions).

You cannot free, but you can help the GC

You never call `free`, but you can make the GC's job easier by **allocating less** and by **not keeping things alive longer than needed**.

Allocate less

- **Preallocate slices** when you know the size. `append` grows a slice by allocating a bigger backing array and copying — repeatedly — if you start from nothing. Giving it capacity up front turns many allocations into one (see Chapter 8 — Arrays, Slices, and Strings).

```
// Many reallocations as the slice grows from zero.
nums := []int{}
for i := range 1000 {
    nums = append(nums, i)
}

// One allocation: length 0, capacity 1000 reserved up front.
nums := make([]int, 0, 1000)
for i := range 1000 {
    nums = append(nums, i)
}
```

- **Reuse buffers** instead of allocating per call. A `sync.Pool` is a free list of temporary objects the GC may still reclaim under pressure.

```
import (
    "bytes"
    "sync"
)

var bufPool = sync.Pool{
    New: func() any { return new(bytes.Buffer) },
}

func render(data []byte) string {
    buf := bufPool.Get().(*bytes.Buffer)
    defer bufPool.Put(buf)
    buf.Reset() // a pooled buffer may still hold old bytes
    buf.Write(data)
    return buf.String()
}
```

Do not keep references alive

A Go “memory leak” is never a forgotten `free`. It is a pointer you forgot to *drop*. As long as anything reachable points at an object, the GC must keep it.

- **A small slice can pin a huge backing array.** Slicing does not copy; the small slice still references the whole original array, so none of it can be freed.

```

import "bytes"

// BUG: the returned slice shares data's backing array. If data is
// megabytes, all of it stays alive as long as the caller holds the result.
func firstLineLeaky(data []byte) []byte {
    i := bytes.IndexByte(data, '\n')
    if i < 0 {
        return data
    }
    return data[:i]
}

// FIX: copy the few bytes you need so the big array can be collected.
func firstLine(data []byte) []byte {
    i := bytes.IndexByte(data, '\n')
    if i < 0 {
        i = len(data)
    }
    out := make([]byte, i)
    copy(out, data[:i])
    return out
}

```

- **Ever-growing global maps.** A package-level cache that only inserts is a leak by another name. Use an eviction policy or a bounded cache, and remember `clear(m)` empties a map but keeps it allocated for reuse; set it to `nil` to let the GC reclaim it.

```

var cache = map[string][]byte{} // grows forever if entries are never deleted

```

- **Leaked goroutines.** A goroutine that blocks forever is never collected, and neither is anything it references. The classic cause is a send on a channel that no one will ever receive from (see Chapter 13 — Goroutines and the Scheduler).

```

func leak() {
    ch := make(chan int) // unbuffered
    go func() {
        ch <- 42 // blocks forever: no receiver, goroutine never exits
    }()
} // leak returns, but the goroutine (and ch) live on

```

Measuring memory

You do not have Valgrind, and you do not need it. Go's measurement tools are built in.

- `runtime.MemStats` reads live counters: bytes on the heap, number of GC cycles, and more.

```

import (
    "fmt"
    "runtime"
)

```

```
func printMem() {
    var m runtime.MemStats
    runtime.ReadMemStats(&m)
    fmt.Printf("HeapAlloc=%d KiB   NumGC=%d\n", m.HeapAlloc/1024, m.NumGC)
}
```

- **-benchmem** makes benchmarks report bytes and allocations per operation — the fastest way to compare two implementations (see Chapter 21 — Testing).

```
go test -bench=. -benchmem
# BenchmarkRender-8    5000000    240 ns/op    64 B/op    2 allocs/op
```

- **Heap profiles with pprof** show *which lines* allocate the most. This is the tool you reach for on a real service; it is covered in Chapter 22 — Tooling.

Value semantics reduce GC pressure

Go passes everything **by value** by default (see Chapter 7 — Pointers). Copying a small struct is cheap and, crucially, a value that never has its address taken can stay on the stack — so it costs the GC nothing. Two habits follow:

- Prefer small value types and pass them by value when they are small. A value that does not escape is free to allocate and free.
- Avoid taking pointers, boxing into interfaces, or capturing in closures unless you need to. Each of those can force a value onto the heap.

The compiler also **inlines** small functions (copies the body into the caller), which often removes a call and lets more values stay on the stack.

Deep dive. “Boxing” means storing a value inside an interface. Because an interface holds a type plus a pointer to the value, assigning a non-pointer to an interface usually copies the value to the heap. This is why `fmt.Println(x)` — whose parameter is `...any` — can allocate even for an `int`. It is rarely worth worrying about, but it explains surprise allocations in profiles.

Key takeaways

- Go has a **stack** and a **heap** like C, but **you do not choose** between them and there is **no free**. The compiler and the GC do that work.
- **Escape analysis** decides placement: a value whose address outlives the call goes on the heap. **Returning &local is safe** — the opposite of the C bug. See decisions with `go build -gcflags=-m`.
- Each goroutine has its own **growable stack**, so deep recursion does not crash.
- The GC is **concurrent, tri-color, mark-and-sweep, non-generational, and non-compacting**, with sub-millisecond stop-the-world pauses.
- Tune with **GOGC** (heap growth %, default 100) and **GOMEMLIMIT** (soft memory cap, Go 1.19+). Leaving `GOGC=100` and setting `GOMEMLIMIT` is the common production choice.
- Help the GC by allocating less (preallocate slices, reuse with `sync.Pool`) and by not keeping references alive (sub-slices, growing maps, leaked goroutines).
- Measure with `runtime.MemStats`, **-benchmem**, and **pprof** heap profiles.

Watch out (gotchas for C programmers)

- **There is no free, but you can still “leak.”** A leak in Go is a reference you failed to drop, not memory you failed to release. Look for pinned sub-slices, unbounded global maps, and stuck goroutines.
- **A small slice keeps its whole backing array alive.** Copy the bytes you need to let the big array be collected.
- **Hidden allocations exist.** Interface boxing, `string []byte` conversions, `append` growth, and closures can all allocate. Use `-benchmem` and `-gcflags=-m` to find them.
- **Finalizers are not destructors.** Their timing is not guaranteed and they may never run before exit. Use `defer` and explicit `Close` for real cleanup; prefer `runtime.AddCleanup` over `runtime.SetFinalizer` when you must.
- **Misusing the knobs hurts.** `GOGC=off` or a huge `GOGC` can exhaust memory; a tiny `GOGC` or a too-low `GOMEMLIMIT` burns CPU in constant GC. Change them only with measurements in hand.

Interview questions

Q: In Go, who decides whether a value goes on the stack or the heap, and how? A: The compiler decides, using escape analysis. If it can prove the value’s lifetime ends when the function returns, the value stays on the stack; if the value’s address escapes (it is returned, stored on the heap, captured by a surviving closure, sent on a channel, or boxed in an interface), it is allocated on the heap. You can inspect the decisions with `go build -gcflags=-m`.

Q: Is it safe to return the address of a local variable in Go? Why is that different from C? A: Yes, it is safe in Go. Escape analysis sees that the address outlives the function and allocates the variable on the heap, so the pointer stays valid and the GC frees it later. In C the local lives in the stack frame, which is freed on return, so the returned pointer dangles and dereferencing it is undefined behavior.

Q: Describe Go’s garbage collector in one sentence, then explain GOGC and GOMEMLIMIT. A: It is a concurrent, tri-color, mark-and-sweep collector that is non-generational and non-compacting, with very short stop-the-world pauses. `GOGC` (default 100) sets how much the heap may grow relative to the live set before the next collection — higher means less frequent GC and more memory. `GOMEMLIMIT` (Go 1.19+) is a soft total-memory cap that makes the GC run harder as memory approaches it, to avoid running out.

Q: There is no free in Go, so how can a Go program still leak memory? A: By keeping references alive. The GC frees only unreachable objects, so anything still reachable is retained. Common causes are a small slice pinning a large backing array, a global map or cache that only grows, and goroutines that block forever (which keep themselves and everything they reference alive).

Q: Name three ways to reduce allocations and GC pressure in a hot path. A: Preallocate slices and maps with a known capacity so `append` does not repeatedly reallocate; reuse temporary buffers with `sync.Pool`; and keep values small and on the stack by avoiding unnecessary pointers, interface boxing, and closures that capture them. Measure the effect with `go test -bench=. -benchmem` and heap profiles.

Try it

1. Put `stays/escapes/main` from this chapter in a file and run `go build -gcflags=-m ..`. Confirm only `escapes`'s variable is “moved to heap.”
2. Write a benchmark that builds a 10,000-element slice two ways — from `[]int{}` and from `make([]int, 0, 10000)` — and run `go test -bench=. -benchmem`. Compare the `allocs/op`.

Chapter 18 — Packages and Modules

What you'll learn. How Go ships and versions code with *modules* and `go.mod`, how import paths map to directories, how to add and upgrade dependencies safely, and how Go's reproducible, verified builds compare to C's `headers-and-1foo` model.

C has no standard package manager. You install a library with your OS package manager (or build it from source), point the compiler at its headers with `-I`, point the linker at its library with `-L` and `-l`, and track which version you have by hand or with a `Makefile` comment. Different machines easily end up with different versions, and “it builds on mine” is a real problem.

Go replaces all of that with **modules**: a built-in system for fetching, versioning, and verifying dependencies, so a build is **reproducible** (same inputs everywhere) and **verified** (cryptographically checked). This chapter builds on the package rules from Chapter 3 — Program Structure: Packages, Imports, and Visibility, and the commands from Chapter 2 — Installing Go and the `go` Command. We focus here on modules themselves.

A module is a tree of packages

Recall from Chapter 3 that **one directory is one package**. A **module** is the next level up: a *tree* of packages that are versioned, released, and downloaded together, with a single `go.mod` file at its root.

Every module has a **module path**: a globally unique name that is usually the location of its source repository, such as `github.com/acme/widget`. This path is both the module's identity and the **prefix for importing its packages**.

You create a module with `go mod init`, passing the module path:

```
mkdir widget && cd widget
go mod init github.com/acme/widget    # creates go.mod
```

C vs Go. There is no C equivalent of `go mod init`. The closest mindset is “this directory tree is a library with a name and a version.” In C the name and version live in your head, a `README`, or a distro package; in Go they live in `go.mod`, and the tool enforces them.

Anatomy of `go.mod`

`go.mod` is a small text file the tool reads and writes. You rarely edit it by hand. Here is an annotated example showing every directive you are likely to meet:

```
module github.com/acme/widget    <- the module path; also the import prefix
```

```

go 1.26                                <- minimum Go language version this module needs

toolchain go1.26.4                     <- exact toolchain to use; managed automatically

require (                               <- dependencies and their selected versions
    github.com/google/uuid v1.6.0
    golang.org/x/text v0.16.0 // indirect <- pulled in by a dependency, not by you
)

replace github.com/google/uuid => ../forks/uuid <- swap a dependency for a local copy

exclude golang.org/x/text v0.15.0      <- never select this known-bad version

```

- **module** declares the module path.
- **go** declares the minimum Go version; it also turns language features on or off to match that version.
- **toolchain** (Go 1.21+) names the exact toolchain; if you have an older Go, the tool downloads this one automatically (see Chapter 2).
- **require** lists dependencies with exact versions. The `// indirect` comment marks a dependency you do not import directly but that something you use needs.
- **replace** swaps one module for another path or version — most often a local directory during development. It affects *your* build only, not anyone who imports you.
- **exclude** forbids a specific version from being selected.

Next to `go.mod` sits `go.sum`. It is *not* a lock file of versions (that is `go.mod`'s job); it is a list of **cryptographic checksums**. Each dependency contributes two lines — a hash of its file tree and a hash of its `go.mod` — so the tool can verify on every build that the code in your module cache is byte-for-byte what it was when first downloaded.

```

github.com/google/uuid v1.6.0 h1:NIvaJDM... <hash of the module's files>
github.com/google/uuid v1.6.0/go.mod h1:Ti... <hash of that version's go.mod>

```

Watch out. Commit **both** `go.mod` and `go.sum`. Together they make a build reproducible and tamper-evident. Do not edit `go.sum` by hand; the tool maintains it.

Import paths

An **import path** is just the **module path** plus the **subdirectory** of the package inside the module. No headers, no search paths to configure.

```

github.com/acme/widget                module path (from go.mod)
    internal/store/                    subdirectory
        store.go      package store

import "github.com/acme/widget/internal/store" // module path + "/internal/store"

```

The last element of the path is usually, but not always, the package name (see Chapter 3). The standard library uses short paths with no domain (`fmt`, `net/http`); those resolve to `GOROOT`. Everything else resolves through the module cache (see Chapter 2 — Installing Go and the `go` Command).

Adding and updating dependencies

You do **not** hand-edit `require` lines. Two workflows keep `go.mod` correct.

Workflow 1 — write the import, then tidy. Add the import, then let the tool discover and record it:

```
import "github.com/google/uuid"

go mod tidy    # adds missing requires, drops unused ones, updates go.sum
```

Workflow 2 — go get a specific version. Use this to pin or upgrade:

```
go get github.com/google/uuid@v1.6.0    # add or pin an exact version
go get github.com/google/uuid@latest     # move to the newest release
go get -u ./...                          # upgrade dependencies to newer minor/patch
go get github.com/google/uuid@none       # remove the dependency
```

Since Go 1.17, `go get` only edits dependencies; it no longer installs program binaries (use `go install pkg@version` for tools — see Chapter 2).

Versions: semantic versioning and the v2 rule

Go uses **semantic versioning**: a version is `vMAJOR.MINOR.PATCH`, e.g. `v1.6.0`. The contract is: bumping **PATCH** fixes bugs, **MINOR** adds features compatibly, and **MAJOR** may break compatibility.

Go enforces this with **semantic import versioning**: a breaking new major version gets a **new import path**. For `v2` and above, the major version becomes a suffix on both the module path and every import path.

```
import "github.com/acme/widget"          // any v0 or v1 release
import "github.com/acme/widget/v2"       // v2.x.x - note the /v2 suffix
```

This is unusual to C eyes but powerful: `v1` and `v2` are *different paths*, so a single build can use both at once during a migration, with no “two versions of the same symbol” conflict.

C vs Go. In C, upgrading a shared library to an incompatible major version (a new `.so` soname) can break every program linked against it, and you juggle versions with symlinks and `LD_LIBRARY_PATH`. Go encodes the major version *in the import path*, so incompatible majors simply coexist.

Minimal Version Selection (brief)

When several modules in your build require different versions of the same dependency, Go uses **Minimal Version Selection (MVS)**: it picks the **highest version that any module explicitly requires** — and no higher. It does not grab “latest” behind your back. The result is deterministic: the same `go.mod` files always select the same versions, on every machine and every day, until you explicitly upgrade.

Integrity and proxies

Three environment settings control where code comes from and how it is verified (set and inspected with `go env` — see Chapter 2):

- **GOPROXY** — the download source. The default, `proxy.golang.org`, is a caching mirror run by Google. It makes downloads fast and keeps working even if the original repository disappears.
- **GOSUMDB** — the checksum database (`sum.golang.org`). The first time anyone fetches a module version, the tool records its hash here; later fetches are checked against it, so a tampered or swapped release is detected globally.
- **GOPRIVATE** — a list of module path patterns that should **bypass** the public proxy and checksum database. Set it for private company repositories.

```
go env GOPROXY GOSUMDB GOPRIVATE
```

```
GOPRIVATE=github.com/acme/* go build ./...    # fetch acme repos directly, no proxy
```

C vs Go. This is the part with no C analog at all. C has no checksum database verifying that the `libfoo` you fetched matches what everyone else fetched. Go's proxy plus checksum database give you reproducible *and* tamper-evident dependencies for free.

Special directories and directives

`internal/` packages

A package under a directory named **`internal/`** can be imported **only** by code rooted at **`internal/`**'s parent. It is Go's way to make a package public within your module but private to the outside world.

```
github.com/acme/widget/
  internal/store/      importable by widget code only
  api/                 importable by anyone
```

Anyone outside `github.com/acme/widget` who tries to import `.../widget/internal/store` gets a compile error. Use `internal/` freely to keep implementation details from leaking into others' code.

`replace` for local development and forks

The `replace` directive points a module path at a different location, most often a local checkout. It is how you develop a library and an app that uses it at the same time:

```
go mod edit -replace github.com/acme/widget=../widget
```

Use `replace` for short-term local work or to test a fork. For day-to-day multi-module development, prefer a workspace (below), which does the same thing without editing `go.mod`.

Vendoring

`go mod vendor` copies every dependency's source into a top-level `vendor/` directory committed with your code. The build then uses `vendor/` instead of the module cache.

```
go mod vendor          # write ./vendor with all dependencies
go build ./...         # uses ./vendor automatically when it exists
```

Vendoring guarantees a build with no network and an auditable copy of every dependency in your repository — useful in locked-down or air-gapped environments.

Workspaces for multi-module local dev

A **workspace** (go.work, Go 1.18+) lets you work on several modules together without touching any go.mod. It is the recommended way to edit a library and its consumer side by side (recap from Chapter 2):

```
go work init ./app ./lib    # builds in ./app now use your local ./lib
```

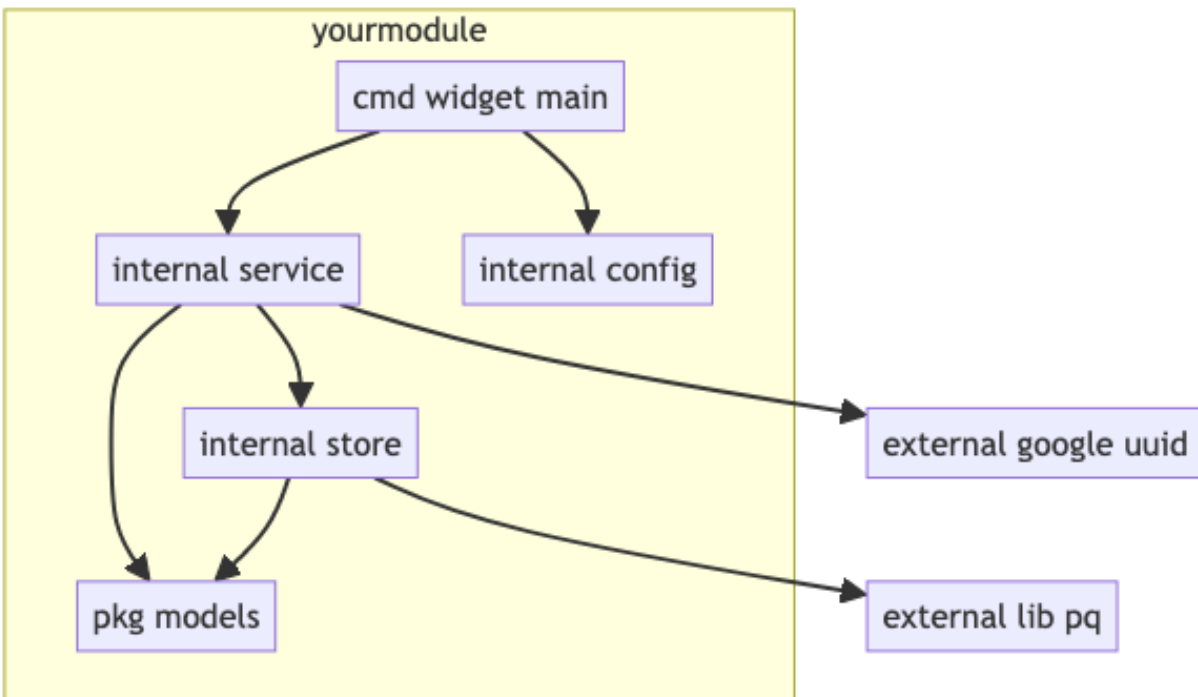
Rule of thumb. Reach for a **workspace** for ongoing local multi-module work; use **replace** for a one-off fork or pin; use **vendoring** when you need an in-repo, offline-buildable copy of all dependencies.

Project layout

Go has **no required project layout** — the only hard rule is “a directory is a package.” A few conventions are widespread:

- **cmd/** — one subdirectory per executable, each a **package main** (cmd/widget/main.go). Use it when a module ships more than one binary.
- **internal/** — private packages, enforced by the compiler as described above.
- **pkg/** — a (debated) place for library code meant to be imported by others. Many Go developers consider it unnecessary; do not add it just out of habit.

Here is how packages in a small module depend on each other. Arrows point from importer to imported, and the graph must be acyclic — circular imports are forbidden (see Chapter 3); break a cycle by extracting a shared package or by introducing an interface (see Chapter 11 — Interfaces).



Rule of thumb. Prefer **small, flat modules**. Start with a go.mod and a few packages in the repository root. Add cmd/, internal/, and submodules only when you actually

have multiple binaries or a real privacy boundary. Do not build a deep directory tree before you need one.

Publishing a module

There is no upload step and no central registry to push to. You **publish by pushing to a public repository and tagging a version**:

```
git tag v1.2.3
git push origin v1.2.3
```

The first time anyone runs `go get github.com/acme/widget@v1.2.3`, the proxy fetches that tag, records its checksum, and caches it. For a v2+ release, the module path in `go.mod` must end with `/v2` to match the import path.

Watch out. The module path **must match where the code actually lives** for `go get` to find it. If your repository is `github.com/acme/widget`, the module line must say exactly that. A mismatched path builds fine locally but is impossible for others to fetch.

Key takeaways

- A **module** is a versioned tree of packages with a `go.mod` at its root; its **module path** (e.g. `github.com/acme/widget`) is both its identity and its import prefix. Create one with `go mod init`.
- `go.mod` holds module, go, toolchain, require, replace, and exclude. `go.sum` holds **checksums** for reproducible, tamper-evident builds. Commit both.
- An **import path** is the module path plus the package's subdirectory.
- Add or change dependencies with `import + go mod tidy`, or `go get pkg@version` — never by hand-editing `go.mod`.
- Versions follow **semantic versioning**; **v2+ needs a /v2 path suffix**. MVS picks the highest explicitly-required version, making builds deterministic.
- `GOPROXY`, `GOSUMDB`, and `GOPRIVATE` control downloading and verification; set `GOPRIVATE` for company repositories.
- `internal/` limits visibility to your subtree; `replace`, vendoring, and workspaces support local/offline/fork development.
- There is **no required layout**; prefer small, flat modules. Publish by pushing and tagging `vX.Y.Z`.

Watch out (gotchas for C programmers)

- **The module path must match the repository** or `go get` cannot fetch it. This bites people who rename a repo without updating the module line.
- **v2 and above need the /v2 suffix** in both `go.mod`'s module line and every import path. Forgetting it is the most common module mistake.
- **Set GOPRIVATE for private modules**, or the tool will try the public proxy and checksum database and fail to fetch them.
- **Circular imports are a hard error** (see Chapter 3). Modules do not change that; the package dependency graph must stay acyclic.

- **internal/** is enforced by the compiler, not by convention. Code outside the parent of **internal/** simply cannot import it.
- Let the tool edit **go.mod/go.sum**. Use `go get`, `go mod tidy`, and `go mod edit`; hand-editing easily produces an inconsistent **go.sum**.

Interview questions

Q: What is the difference between a package and a module in Go? A: A package is one directory of **.go** files compiled together (Chapter 3). A module is a tree of packages versioned and released together, identified by a module path and described by a **go.mod** file at its root. You import packages; you version and download modules.

Q: What do go.mod and go.sum each do, and why commit both? A: **go.mod** records the module path, the Go version, and the exact dependency versions selected for the build. **go.sum** records cryptographic checksums of those dependencies so the tool can verify on every build that the downloaded code has not changed or been tampered with. Committing both makes the build reproducible and verifiable on any machine.

Q: How does Go handle an incompatible (major) version upgrade of a dependency? A: With semantic import versioning. Major versions **v2** and above carry a **/vN** suffix in the module path and in every import path, so different majors are different import paths. That lets a build depend on both **v1** and **v2** of the same library at once, which avoids the symbol-clash problems C has with incompatible shared-library sonames.

Q: When several modules require different versions of the same dependency, which version does Go pick? A: Minimal Version Selection picks the highest version that some module in the build explicitly requires, and no higher. It never silently jumps to “latest,” so the selection is deterministic and only changes when you upgrade on purpose.

Q: How do Go modules compare to managing C dependencies? A: C has no standard package manager: you install libraries via the OS or build them, pass **-I/-L/-l** to the toolchain, and track versions by hand, so builds differ across machines. Go modules give a built-in, versioned, reproducible, and cryptographically verified dependency system — **go.mod** plus **go.sum** plus a proxy and checksum database — with nothing to wire up manually.

Try it

1. Run `go mod init example.com/play`, add `import "github.com/google/uuid"` and a call to `uuid.NewString()` in `main`, then run `go mod tidy`. Inspect the new `require` line in **go.mod** and the two lines in **go.sum**.
2. Make a package under **internal/** and try to import it from a second, separate module on your machine. Watch the compiler refuse with an **internal** error.

Chapter 19 — Generics

What you'll learn. Why Go added generics (type parameters), how to write generic functions and types, how constraints describe the types you allow, and when to reach for generics instead of interfaces. Everything is compared to C's macros and **void ***.

A C programmer who wants one piece of code to work for many types has two old tools, and both hurt. You can write a **macro** with **#define**, which the preprocessor pastes in with no type checking

and no scope. Or you can use `void *` and `cast`, which throws away type safety and invites crashes. Go 1.18 added a third option, **generics**: real, type-checked code that works for many types. This chapter shows how to use them well — and when not to.

Life before generics

Say you want a `Max` function. Before generics, Go gave you three bad choices, all of which a C programmer will recognize.

1. Copy-paste, one function per type. Safe, but you maintain *N* near-identical copies:

```
func MaxInt(a, b int) int {
    if a > b {
        return a
    }
    return b
}

func MaxFloat64(a, b float64) float64 {
    if a > b {
        return a
    }
    return b
}
```

// ...and a third copy for string, and a fourth, and a fifth.

2. Use `any` and `type-assert`. One function, but you lose type safety and pay for boxing (wrapping each value in an interface). Mistakes blow up at run time, not compile time:

```
func Max(a, b any) any {
    // You cannot write a > b on values of type any. You must type-switch on
    // every case by hand, and the caller must cast the result back. Painful.
    return a
}
```

3. Code generation. Run a tool that writes the per-type copies for you. It works, but now your build has an extra step and generated files to manage.

Here is how the same problem looks in C, and why each path is unsafe:

```
/* A macro: the preprocessor pastes text. No type checking, no scope. */
/* MAX(i++, j) evaluates an argument twice -> bugs. */
#define MAX(a, b) ((a) > (b) ? (a) : (b))

/* A void* container: the compiler cannot check what you put in or take out. */
void stack_push(Stack *s, void *item);
void *stack_pop(Stack *s); /* caller must cast back and hope it is right */
```

Tool	Type-checked	Scoped	Performance	Notes
C <code>#define</code> macro	No	No (global text)	Inlined	Double-evaluates args; no types
C <code>void * + cast</code>	No	Yes	Pointer indirection	Crashes if you cast wrong
Go <code>any + assert</code>	At run time only	Yes	Boxing + checks	Errors surface late
Go generics	At compile time	Yes	No boxing	The modern answer

Mental model. A C macro is “find and replace before compiling”: no idea what a type is. A Go generic is a real function the compiler checks once and then specializes for each type you use it with. You get the reuse of a macro with the safety of a normal function.

Type parameters

A **generic function** declares one or more **type parameters** in square brackets after its name, before the ordinary parameters. Here is `Max` written once for every ordered type:

```
import "cmp"

func Max[T cmp.Ordered](a, b T) T {
    if a > b {
        return a
    }
    return b
}
```

Read it piece by piece, with C in mind:

- `[T cmp.Ordered]` declares a type parameter named `T`. `T` is a stand-in for a real type, chosen when the function is called. `cmp.Ordered` is its **constraint** — the set of types `T` is allowed to be (here, types that support `<`, `>`, and so on). More on constraints below.
- `(a, b T)` — both parameters and the return value are of that same type `T`.
- Inside, you may only use operations the constraint guarantees. Because `cmp.Ordered` permits ordering, `a > b` is legal.

Instantiation is choosing a concrete type for `T`. Usually you do nothing — Go **infers** `T` from the arguments:

```
fmt.Println(Max(3, 5))           // T inferred as int    -> 5
fmt.Println(Max(2.5, 1.5))       // T inferred as float64 -> 2.5
fmt.Println(Max("go", "c"))     // T inferred as string -> go
```

When inference is not possible, or you want to be explicit, name the type in brackets at the call site:

```
fmt.Println(Max[float64](1, 2)) // explicit: T is float64
```

C vs Go. C++ templates are the closest cousin, and Go’s syntax is similar. But Go checks a generic function once, against its constraint, and reports clear errors. A C++ template is checked only when instantiated, which is why a small mistake can produce pages of errors. Go’s constraints make the contract explicit up front.

Watch out. Go already has builtin `min` and `max` functions (since Go 1.21), so you do not need to write `Max` yourself. We use it here only because it is the simplest possible generic.

Constraints: describing the allowed types

A **constraint** is just an interface used in a type-parameter list. But constraint interfaces can say more than ordinary ones. An ordinary interface lists *methods*. A constraint interface may also list a **type set**: the exact set of types allowed.

Method constraints

If your code calls a method on `T`, require that method in the constraint:

```
import "strings"

type Stringer interface {
    String() string
}

func JoinAll[T Stringer](items []T) string {
    var b strings.Builder
    for _, it := range items {
        b.WriteString(it.String()) // allowed: the constraint guarantees String()
    }
    return b.String()
}
```

This looks like normal interface use, and it is — but because `T` is a type parameter, the slice `[]T` holds concrete values with no per-element boxing.

Type sets, unions, and ~

When your code uses **operators** (`+`, `<`, `==`) rather than methods, list the allowed types instead. `A` is a **union** (“any of these”):

```
type Integer interface {
    int | int8 | int16 | int32 | int64
}
```

There is a catch. `int | int64` means *exactly* `int` or `int64`. It does **not** include a named type whose underlying type is `int`, such as `type Celsius int`. The `~` token fixes this: `~int` means “any type whose **underlying type** is `int`.” (Underlying type is the built-in type a named type is defined from; see Chapter 4 — Types, Variables, and Constants.)

```
type Number interface {
    ~int | ~int64 | ~float64 // also matches type Celsius int, type Price float64, ...
}
```

```

}

func Sum[T Number](nums []T) T {
    var total T // zero value of T: 0 for numbers
    for _, n := range nums {
        total += n // allowed: every type in the set supports +
    }
    return total
}

```

Here is the type set drawn out:

constraint Number = the set of types whose underlying type is one of:

```

~int      --> int,      type Celsius int,   type Count int    ...
~int64    --> int64,   type Timestamp int64 ...
~float64  --> float64, type Price float64,  type Ratio float64 ...

```

A type `T` satisfies `Number` if `T` appears in this set.

The predeclared constraints

Three constraints are built in or one import away, and cover most needs:

- **any** — no restriction at all. `T any` means “any type whatsoever.” (`any` is just the modern spelling of `interface{}`.) Use it for containers that do not inspect their elements, like a generic stack.
- **comparable** — types that support `==` and `!=`. Required to use a `T` as a map key or to compare values. Note: this is *not* the same as `cmp.Ordered`; `comparable` allows equality, not `</>`.
- **cmp.Ordered** (from the `cmp` package) — types that support `<`, `<=`, `>`, `>=`: all integer, float, and string types. Use it for sorting, `min/max`, and range checks.

```

func Contains[T comparable](haystack []T, needle T) bool {
    for _, x := range haystack {
        if x == needle { // allowed: comparable guarantees ==
            return true
        }
    }
    return false
}

```

Constraint	Allows	Typical use
<code>any</code>	every type	containers that do not inspect elements
<code>comparable</code>	<code>==</code> , <code>!=</code>	map keys, membership tests
<code>cmp.Ordered</code>	<code><</code> , <code>></code> , <code><=</code> , <code>>=</code>	sorting, <code>min/max</code> , ranges
a method interface	the listed methods	algorithms that call behavior
a type-set interface	operators on those types	numeric/string math

Generic types

Types can have type parameters too. The classic case is a container. A generic `Stack` works for any element type with no `void *` and no casting:

```
package main

import (
    "errors"
    "fmt"
)

type Stack[T any] struct {
    items []T
}

func (s *Stack[T]) Push(item T) {
    s.items = append(s.items, item)
}

func (s *Stack[T]) Pop() (T, error) {
    var zero T // the zero value of T, for the error case
    if len(s.items) == 0 {
        return zero, errors.New("pop from empty stack")
    }
    last := len(s.items) - 1
    item := s.items[last]
    s.items = s.items[:last]
    return item, nil
}

func main() {
    var s Stack[int] // instantiate the type: T is int
    s.Push(10)
    s.Push(20)
    v, _ := s.Pop()
    fmt.Println(v) // 20
}
```

Two details worth noting:

- The receiver is written `(s *Stack[T])` — you repeat the type parameter, but you do **not** re-declare its constraint; that lives on the type.
- `var zero T` gives you the zero value of whatever `T` is (`0`, `""`, `nil`, an empty struct). It is the generic way to “return nothing meaningful.”

Watch out. A method cannot introduce its own new type parameter. Only functions and type declarations can. This does not compile:

```
// ILLEGAL: a method may not declare a new type parameter U.
```



```
func (s *Stack[T]) Map[U any](f func(T) U) *Stack[U] { /* ... */ }
```

Write Map as a top-level *function* instead: `func Map[T, U any](s *Stack[T], f func(T) U) *Stack[U]`.

Generics vs interfaces: pick the right tool

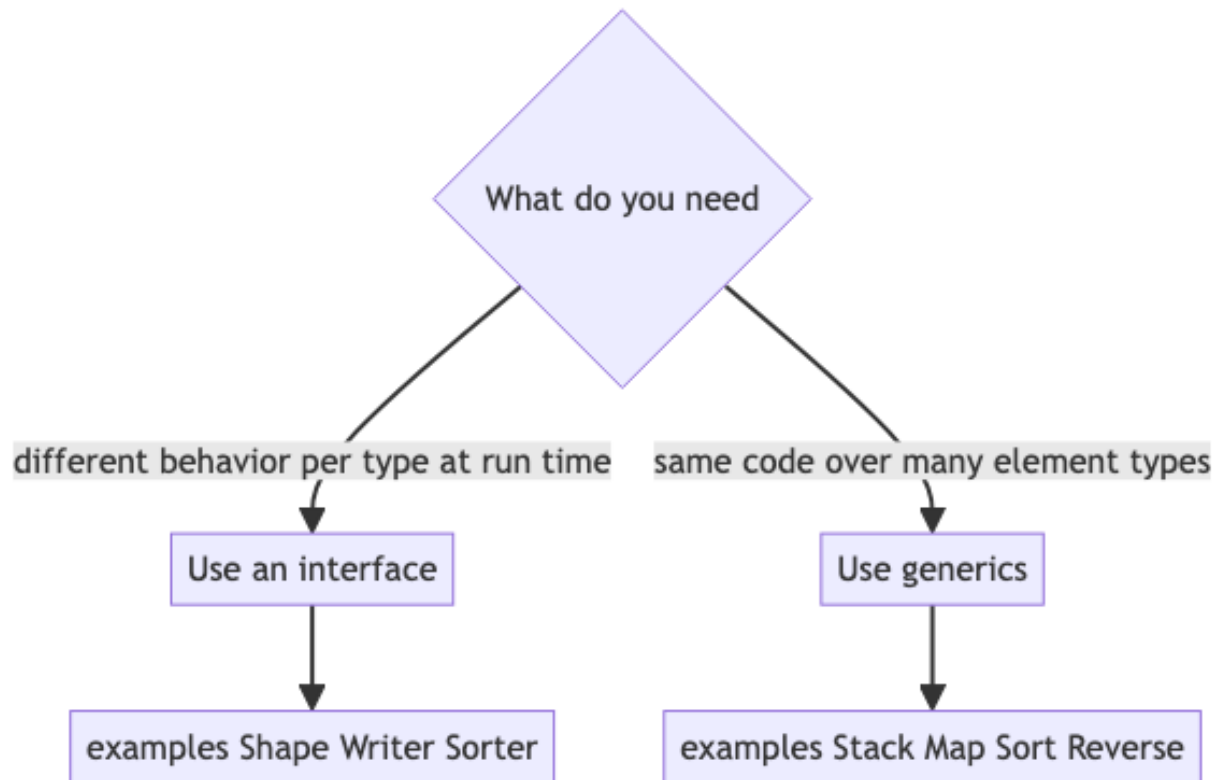
Generics and interfaces overlap, so it helps to know which fits.

Use an **interface** when you care about **behavior** and want **dynamic dispatch** — different concrete types behaving differently behind one method call, decided at run time. This is classic polymorphism (see Chapter 11 — Interfaces):

```
// One slice, many concrete types, dispatch decided at run time.
func TotalArea(shapes []Shape) float64 {
    var sum float64
    for _, s := range shapes {
        sum += s.Area() // each shape's own Area runs
    }
    return sum
}
```

Use **generics** when you care about **types**, not behavior — a container or algorithm that should work uniformly over many element types **without boxing**, keeping full type information:

```
// Same code for []int, []string, []float64 - no boxing, type preserved.
func Reverse[T any](s []T) {
    for i, j := 0, len(s)-1; i < j; i, j = i+1, j-1 {
        s[i], s[j] = s[j], s[i]
    }
}
```



Rule of thumb. “Write code, not types.” Reach for generics only when you are actually repeating the *same* logic across types. If a plain function, a slice of a concrete type, or one interface does the job, use that. Over-generic code is harder to read than the duplication it replaced.

The standard generic helpers

Go ships generic helpers so you rarely write the basics yourself. Three packages cover most slice and map work: `slices`, `maps`, and `cmp`.

```
package main

import (
    "fmt"
    "maps"
    "slices"
)

func main() {
    nums := []int{3, 1, 4, 1, 5, 9}

    slices.Sort(nums)           // sort in place: [1 1 3 4 5 9]
    fmt.Println(slices.Contains(nums, 4)) // true
    fmt.Println(slices.Index(nums, 5))    // 4 (index of first 5)
    fmt.Println(slices.Max(nums))         // 9
}
```

```

m := map[string]int{"a": 1, "b": 2, "c": 3}
keys := slices.Sorted(maps.Keys(m))    // collect keys, sorted: [a b c]
fmt.Println(keys)
}

```

A few useful members:

- **slices**: Sort, SortFunc, Contains, Index, Max, Min, Equal, Reverse, Insert, Delete, Collect, Sorted.
- **maps**: Keys, Values, Clone, Equal, Copy.
- **cmp**: Ordered (the constraint), Compare, Less.

Watch out. Since Go 1.23, `maps.Keys(m)` returns an **iterator**, not a slice. To get a slice, wrap it: `slices.Collect(maps.Keys(m))`, or `slices.Sorted(maps.Keys(m))` to get it sorted in one step (as above). This is different from the old experimental `maps` package that returned a slice.

A short performance note

Generics avoid the boxing cost of **any**: a `[]T` of `int` stores plain `ints`, not interface values, so there are no allocations or pointer chases per element. That is the main performance win over the old **any-and-assert** approach.

But generics are **not always literally zero-cost**. The Go compiler implements them with a mix of two techniques:

- **Stenciling** — generating a specialized copy of the code for a type (or for a group of types that share a memory shape), like a C++ template instantiation. This is as fast as hand-written code.
- **Dictionaries** — for some type groups, passing a hidden table of type information so one shared copy of the code can serve several types. This trades a little run-time indirection for smaller binaries.

Rule of thumb. Choose generics for clarity and type safety, not as a micro-optimization. They reliably beat **any-plus-boxing**, and they match hand-written code closely — but if a hot loop’s speed truly matters, measure it (see Chapter 17 — Memory and the Garbage Collector) rather than assuming.

Key takeaways

- Generics give type-checked code reuse, replacing C’s untyped `#define` macros and unsafe `void *` casts, and Go’s own **any-plus-type-assertion** workaround.
- A generic function declares type parameters in brackets: `func Max[T cmp.Ordered](a, b T) T`. The type is usually **inferred** from the arguments.
- A **constraint** is an interface used in a type-parameter list. It may list methods, or a **type set** using unions (`|`) and `~` for underlying types.
- The predeclared constraints are **any** (everything), **comparable** (`==`), and **cmp.Ordered** (`<`, `>`).
- **Types** can be generic too (type `Stack[T any] struct{...}`), but their **methods cannot add new type parameters**.

- Use **interfaces** for behavior and dynamic dispatch; use **generics** for containers and algorithms over many types without boxing. Do not overuse them.
- The `slices`, `maps`, and `cmp` packages provide ready-made generic helpers.

Watch out (gotchas for C programmers)

- **Do not overuse generics.** “Write code, not types.” If one concrete type or a single interface works, prefer it; needless type parameters hurt readability.
- **~ is about the underlying type.** `int` matches only `int`; `~int` also matches `type Celsius int`. Forgetting `~` makes your function reject users’ named types for no obvious reason.
- **Methods cannot be generic.** A method may use its receiver’s type parameters but cannot declare new ones. Make it a top-level function instead.
- **You cannot type-switch on a type parameter directly.** `switch x.(type)` where `x` has type `T` is a compile error. Convert first: `switch any(x).(type) { ... }`.
- **comparable is not cmp.Ordered.** `comparable` gives you `==/!=` only. You need `cmp.Ordered` to use `<` or `>`.
- **Inference is not magic.** If a type parameter does not appear in the ordinary parameters, Go cannot infer it; you must instantiate explicitly, e.g. `New[int]()`.

Interview questions

Q: Why were generics added to Go, and what did people use before? A: To get type-checked code reuse across many types. Before generics you copied the same function per type, used `any` (then `interface{}`) with type assertions — losing compile-time safety and paying for boxing — or ran a code generator. Generics give the reuse of a C macro with the safety of a normal, compiler-checked function.

Q: What is a constraint, and what can it contain? A: A constraint is an interface used in a type-parameter list that limits which types a type parameter may be. Besides methods, a constraint can list a **type set**: a union of types with `|`, optionally using `~T` to mean “any type whose underlying type is `T`.” The predeclared constraints are `any`, `comparable`, and `cmp.Ordered`.

Q: What does `~int` mean in a constraint, and how is it different from `int`? A: `int` allows only the exact type `int`. `~int` allows any type whose underlying type is `int`, including named types like `type Celsius int`. Use `~` when you want your generic code to accept users’ own types built on a primitive, not just the primitive itself.

Q: When should you choose an interface over generics? A: Use an interface when you want runtime polymorphism — different concrete types behaving differently behind one method call (dynamic dispatch). Use generics when you want the same logic over many element types with full type safety and no boxing, such as containers and algorithms. If you only need one behavior contract, an interface is simpler.

Q: Can a method have its own type parameters? How do you work around it? A: No. A method may use the type parameters of its receiver type but cannot introduce new ones. If you need an extra type parameter (for example a `Map` that changes element type), write a top-level generic function instead, such as `func Map[T, U any](s []T, f func(T) U) []U`.

Q: Are Go generics zero-cost? A: Not always, but they avoid interface boxing, which is the big win over `any`. The compiler uses stenciling (specialized copies, as fast as hand-written code) and

dictionaries (a shared copy with a hidden type table, adding small indirection) depending on the types. They are usually close to hand-written speed; measure a hot path rather than assuming.

Try it

1. Write `Filter[T any](s []T, keep func(T) bool) []T` that returns a new slice of the elements for which `keep` returns true. Use it on a `[]int` and a `[]string` with no changes to the function.
2. Add a `Number` constraint with `~int | ~float64` and write `Sum`. Define type `Celsius float64`, make a `[]Celsius`, and confirm `Sum` accepts it because you used `~float64` and not plain `float64`.

Chapter 20 — A Tour of the Standard Library

What you'll learn. The packages you will use every day: `fmt`, `strings`, `strconv`, `bytes`, `unicode`, `io`, `bufio`, `os`, `time`, `sort/slices`, `encoding/json`, `errors`, `math`, `math/rand/v2`, `crypto/rand`, `regexp`, `log/log/slog`, `flag`, `context`, and the generic `slices/maps/cmp` helpers. For each one you get a tiny, runnable example and the C equivalent.

Batteries included

C gives you a small standard library: `<stdio.h>`, `<string.h>`, `<stdlib.h>`, `<time.h>`, `<math.h>`, and a few more. Anything bigger — JSON, HTTP, hashing, compression — you find a third-party library, install it, and link it yourself.

Go ships a **large** standard library. It can parse JSON, serve HTTP, hash passwords, compress files, and talk to the network with **zero** outside dependencies. The first rule of Go is therefore simple:

Rule of thumb. Before you add a dependency, check the standard library. It is almost certainly there, it is well tested, and it will still work in ten years.

The documentation lives at <https://pkg.go.dev> (every package, searchable in your browser) and in your terminal with `go doc` (see Chapter 2 — Installing Go and the `go` Command). Read the docs; they are short and full of examples.

Job	C header	Go package
Formatted print	<code><stdio.h></code>	<code>fmt</code>
String functions	<code><string.h></code>	<code>strings</code> , <code>bytes</code>
Number text	<code><stdlib.h></code> (<code>atoi</code>)	<code>strconv</code>
Files and OS	<code><stdio.h></code> , <code><unistd.h></code>	<code>os</code> , <code>io</code> , <code>bufio</code>
Time and dates	<code><time.h></code>	<code>time</code>
Sorting	<code><stdlib.h></code> (<code>qsort</code>)	<code>sort</code> , <code>slices</code>
Math	<code><math.h></code>	<code>math</code>
Random numbers	<code><stdlib.h></code> (<code>rand</code>)	<code>math/rand/v2</code> , <code>crypto/rand</code>
Regular expressions	POSIX <code><regex.h></code>	<code>regexp</code>
Logging	<code><stdio.h></code> to <code>stderr</code>	<code>log</code> , <code>log/slog</code>
JSON	(none — third party)	<code>encoding/json</code>

Skim this chapter once to learn what exists, then use it as a reference for the packages you reach for most.

fmt — formatted I/O

fmt (“format”) is your `<stdio.h>`. It prints, scans, and builds strings.

```
fmt.Println("hello", 42)           // print args + spaces + newline
fmt.Printf("%s is %d\n", "age", 42) // C-style format string, to stdout
s := fmt.Sprintf("%05d", 42)       // build a string instead of printing: "00042"
fmt.Fprintln(os.Stderr, "oops")    // print to any io.Writer (here, stderr)
```

The **verbs** (Go’s name for format specifiers) you will use most:

Verb	Meaning	Example output
%v	default format for any value	{Alice 30}
%+v	struct with field names	{Name:Alice Age:30}
%#v	Go syntax (a literal you could paste)	main.User{Name:"Alice", Age:30}
%T	the value’s type	main.User
%d	integer, base 10	42
%s	string or []byte	hello
%q	double-quoted, escaped string	"hello\n"
%x	hexadecimal (numbers or bytes)	2a or 68656c6c66f
%p	pointer address	0xc000012345
%f %.2f	float, fixed precision	3.14
%t	boolean	true

%v is the safe default when you do not care about the exact shape. %+v and %#v are your debugging friends.

A type controls how it prints by implementing the **Stringer** interface: one method, `String()` string. `fmt` calls it for %v and %s.

```
type Color struct{ R, G, B uint8 }

func (c Color) String() string {
    return fmt.Sprintf("#%02x%02x%02x", c.R, c.G, c.B)
}

func main() {
    c := Color{255, 160, 0}
    fmt.Println(c)           // #ffa000 (uses String())
    fmt.Printf("%v\n", c)    // #ffa000
}
```

C vs Go. `fmt.Printf` looks like C’s `printf`, but it is **type-safe**: %d with a string argument is caught by go vet (Chapter 22 — Tooling), not turned into a crash. And

you do not pick the verb from the type by hand — %v prints anything. There is no %lld versus %d confusion; %d works for every integer width.

strings — text without <string.h>

Strings in Go are immutable (see Chapter 8 — Arrays, Slices, and Strings). The `strings` package gives you functions that *return new strings* instead of writing into a buffer.

```
strings.Contains("seafood", "foo")    // true
strings.HasPrefix("golang", "go")     // true
strings.HasSuffix("file.txt", ".txt") // true
strings.Split("a,b,c", ",")          // ["a" "b" "c"]
strings.Join([]string{"a", "b"}, "-") // "a-b"
strings.ReplaceAll("a.b.c", ".", "/") // "a/b/c"
strings.TrimSpace(" hi \n")           // "hi"
strings.Fields(" one two three ")     // ["one" "two" "three"] (split on whitespace)
strings.ToUpper("go")                 // "GO"
```

`strings.Cut` (Go 1.18) splits once at the first separator. It is the clean way to parse `key=value`:

```
key, val, found := strings.Cut("mode=fast", "=")
// key="mode", val="fast", found=true
```

To build a string piece by piece, use `strings.Builder`. It avoids the quadratic cost of `s = s + more` in a loop (each `+` copies the whole string).

```
var b strings.Builder
for i := range 3 {
    fmt.Fprintf(&b, "line %d\n", i)
}
result := b.String()
```

Task	C (<string.h>)	Go (strings)
Find substring	<code>strstr</code>	<code>strings.Contains</code> / <code>Index</code>
Compare	<code>strcmp</code>	<code>==</code> or <code>strings.Compare</code>
Length (bytes)	<code>strlen</code>	<code>len(s)</code>
Copy/concatenate	<code>strcpy</code> / <code>strcat</code>	<code>+</code> or <code>strings.Builder</code>
Split on delimiter	<code>strtok</code> (mutates!)	<code>strings.Split</code>

C vs Go. C's `strtok` writes null bytes into your buffer and keeps hidden state, so it is not reentrant. `strings.Split` returns a fresh slice of strings and touches nothing. There is no buffer to size, no off-by-one, no overflow.

strconv — numbers and strings

`strconv` (“string conversion”) turns text into numbers and back. This is C’s `atoi`, `strtol`, and `sprintf("%d”)`, but **every parse returns an error**.

```
n, err := strconv.Atoi("42")    // string -> int
s := strconv.Itoa(42)           // int -> string ("42")
```

```
i, err := strconv.ParseInt("ff", 16, 64) // base 16, fits in 64 bits -> 255
f, err := strconv.ParseFloat("3.14", 64) // -> 3.14
b, err := strconv.ParseBool("true")      // -> true

q := strconv.Quote("hi\n")               // -> `hi\n` (a safe, escaped literal)
```

Watch out. C's `atoi("abc")` silently returns 0; you cannot tell 0 from an error. Go's `strconv.Atoi("abc")` returns (0, `err`) — **always check the error**. Ignoring it is the single most common bug C programmers bring to Go.

```
n, err := strconv.Atoi(userInput)
if err != nil {
    return fmt.Errorf("bad number %q: %w", userInput, err)
}
use(n)
```

bytes — the []byte twin of strings

Every function in `strings` has a partner in `bytes` that works on `[]byte`. Use `bytes` when you already hold a mutable byte buffer (a network read, a file chunk) and do not want to convert to `string` first (a conversion copies).

```
bytes.Contains([]byte("seafood"), []byte("foo")) // true
bytes.HasPrefix(buf, []byte("HTTP/"))           // true
```

`bytes.Buffer` is a growable byte buffer that implements both `io.Reader` and `io.Writer` (see below). It is the idiomatic in-memory stream:

```
var buf bytes.Buffer
buf.WriteString("status: ")
fmt.Fprintf(&buf, "%d\n", 200)
io.Copy(os.Stdout, &buf) // drains the buffer to stdout
```

unicode and unicode/utf8

Go source and strings are UTF-8. These two packages classify and decode characters (called **runes** — a rune is one Unicode code point, an `int32`).

```
unicode.IsLetter('A') // true
unicode.IsDigit('7')  // true
unicode.IsSpace(' ')  // true
unicode.ToUpper('a')  // 'A'

utf8.RuneCountInString("héllö") // 5 runes (the string is 6 bytes)
utf8.ValidString("\xff")        // false (not valid UTF-8)
```

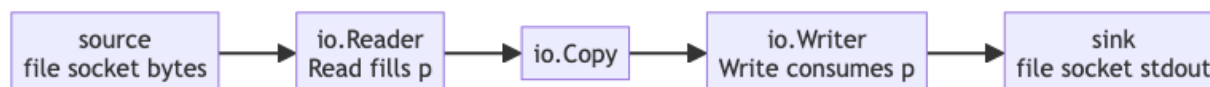
C vs Go. C's `<ctype.h>` (`isdigit`, `toupper`) works on one `char` (one byte) and assumes ASCII. Go's `unicode` works on full Unicode code points, so it is correct for the whole world's text, not just ASCII.

io — the Reader and Writer interfaces

io defines the two most important interfaces in Go. Almost everything that moves bytes implements one or both.

```
type Reader interface { Read(p []byte) (n int, err error) }
type Writer interface { Write(p []byte) (n int, err error) }
type Closer interface { Close() error }
```

A `Reader` fills a byte slice and tells you how many bytes it got. A `Writer` consumes a byte slice. Because files, network connections, `bytes.Buffer`, `strings.Reader`, and HTTP bodies all implement these, the same code works on all of them.



`io.Copy` connects any reader to any writer; `io.ReadAll` slurps a whole reader into memory; `io.EOF` is the sentinel error that means “no more data.”

```
n, err := io.Copy(os.Stdout, someReader) // stream src -> dst, return bytes copied
data, err := io.ReadAll(resp.Body)      // read everything into a []byte
```

C vs Go. `io.Reader`/`io.Writer` are the typed, safe version of C’s `read(2)`/`write(2)` and `FILE *`. A function that takes an `io.Reader` does not care whether the bytes come from a file, a socket, or a test string. This is why Go code is so easy to test: pass a `strings.Reader` instead of a real file.

bufio — buffered I/O and scanning

Raw `Read` calls can be small and frequent. `bufio` (“buffered I/O”) adds a buffer, and — most usefully — a `Scanner` that reads input one line (or word) at a time.

```
sc := bufio.NewScanner(os.Stdin)
for sc.Scan() { // true until EOF or error
    line := sc.Text() // the current line, without the newline
    fmt.Println(strings.ToUpper(line))
}
if err := sc.Err(); err != nil { // Scan() never returns the error itself
    log.Fatal(err)
}
```

A buffered writer batches small writes into big ones; remember to `Flush`:

```
w := bufio.NewWriter(os.Stdout)
defer w.Flush() // without this, buffered bytes are lost
fmt.Fprintln(w, "buffered line")
```

Watch out. `bufio.Scanner` has a default maximum token size of 64 KB. A longer line returns an error from `sc.Err()` (and `Scan` stops). For huge lines, grow the buffer with `sc.Buffer(make([]byte, 0, 1<<20), 1<<20)`, or use a `bufio.Reader` with `ReadString('\n')` instead.

To scan words instead of lines, set the split function: `sc.Split(bufio.ScanWords)`.

os — talking to the operating system

os is your interface to the process and the filesystem.

```
os.Args           // []string: program name + command-line arguments
os.Getenv("HOME") // an environment variable ("" if unset)
os.Exit(1)        // exit immediately with a status code (no deferred calls run)
```

Reading and writing whole files is one call each:

```
data, err := os.ReadFile("config.txt") // []byte, like reading the whole file
err = os.WriteFile("out.txt", data, 0o644) // create/overwrite with mode 0644
```

For streaming, open a file as an `*os.File`, which is an `io.Reader`, `io.Writer`, and `io.Closer`:

```
f, err := os.Open("big.log") // read-only
if err != nil {
    log.Fatal(err)
}
defer f.Close()
sc := bufio.NewScanner(f)
// ... scan f line by line ...
```

`os.Create` makes a new file for writing. `os.Stdin`, `os.Stdout`, and `os.Stderr` are the three standard streams, each an `*os.File`.

Task	C	Go (os)
Command-line args	<code>argc, argv</code>	<code>os.Args</code>
Environment	<code>getenv</code>	<code>os.Getenv</code> / <code>os.LookupEnv</code>
Open file	<code>fopen</code>	<code>os.Open</code> / <code>os.Create</code>
Read whole file	<code>fread</code> loop	<code>os.ReadFile</code>
Exit with status	<code>exit(1)</code> / <code>return 1</code>	<code>os.Exit(1)</code>

Watch out. `os.Exit` quits **right now**. Any deferred calls (Chapter 6 — Functions) do **not** run. Prefer returning an error up to `main` and exiting there; reserve `os.Exit` for the very top level.

time — dates, durations, and timers

`time` replaces `<time.h>`, and it is much friendlier. Two core types:

- `time.Time` — an instant (a point on the calendar/clock).
- `time.Duration` — a span of time, stored as nanoseconds (an `int64`).

```
start := time.Now()
time.Sleep(50 * time.Millisecond) // durations have units: no raw integers
elapsed := time.Since(start)      // a Duration
fmt.Println(elapsed)              // e.g. "50.1ms" (Duration has a nice String())

deadline := time.Now().Add(2 * time.Hour)
```

Durations are written with units, which kills a whole class of “is this seconds or milliseconds?” bugs: `5 * time.Second`, `100 * time.Millisecond`.

A `Timer` fires once; a `Ticker` fires repeatedly. Both deliver the time on a channel (see Chapter 14 — Channels and `select`):

```
t := time.NewTimer(time.Second)
<-t.C // blocks for one second

tick := time.NewTicker(time.Second)
defer tick.Stop() // a Ticker must be stopped or it leaks
for range 3 {
    <-tick.C // fires every second
}
```

The reference date: 2006-01-02 15:04:05

This surprises everyone. To format or parse a time, you do **not** write `%Y-%m-%d`. Instead you write an *example* of the layout using **one specific reference time**:

Mon Jan 2 15:04:05 MST 2006

Read it as a counting sequence: **01/02 03:04:05PM '06 -0700** — month is 1, day is 2, hour is 3, minute is 4, second is 5, year is 6, time zone is 7. You show Go the format by writing that reference time the way you want your output to look.

```
const layout = "2006-01-02 15:04:05" // year-month-day hour:minute:second (24h)

now := time.Now()
fmt.Println(now.Format(layout)) // e.g. 2026-03-14 09:30:00

t, err := time.Parse(layout, "2026-03-14 09:30:00")
if err != nil {
    log.Fatal(err)
}
fmt.Println(t.Year(), t.Month()) // 2026 March
```

Field	C strftime	Go layout token
4-digit year	%Y	2006
Month (zero-padded)	%m	01
Day	%d	02
Hour (24h)	%H	15
Minute	%M	04
Second	%S	05

Watch out. The numbers in the layout are not arbitrary — 15 really means “the hour,” 01 really means “the month.” If you write 2025 instead of 2006, parsing breaks in confusing ways. Memorize the reference date 01/02 03:04:05PM '06 -0700 or copy a known-good layout.

sort and the generic slices.Sort

The old way is the `sort` package. `sort.Ints`, `sort.Strings`, and the general `sort.Slice` (which takes a “less” function) still work and you will see them in older code.

```
xs := []int{3, 1, 2}
sort.Ints(xs) // [1 2 3]
```

```
people := []Person{{"Bob", 30}, {"Ann", 25}}
sort.Slice(people, func(i, j int) bool { return people[i].Age < people[j].Age })
```

Since Go 1.21 the **preferred** way is the generic `slices` package: it is type-safe and faster (no per-element interface boxing).

```
xs := []int{3, 1, 2}
slices.Sort(xs) // [1 2 3] - works for any ordered type
```

```
// SortFunc takes a comparison returning -1, 0, or +1 (like C's qsort comparator):
slices.SortFunc(people, func(a, b Person) int {
    return cmp.Compare(a.Age, b.Age)
})
```

C vs Go. `sort.Slice` and `slices.SortFunc` replace `qsort`. The big win: the comparator is a closure with **typed** values, not two `const void *` you must cast. `cmp.Compare` returns the same `-1/0/+1` your `qsort` comparator returned.

encoding/json — JSON in the standard library

This is the package C does not have at all. It converts Go values to JSON (**marshal**) and back (**unmarshal**), driven by struct field names and tags.

```
type User struct {
    Name  string `json:"name"`
    Email string `json:"email,omitempty"` // omit if empty string
    admin bool   // unexported: never appears in JSON
}

func main() {
    u := User{Name: "Alice"}
    data, _ := json.Marshal(u)
    fmt.Println(string(data)) // {"name":"Alice"} (Email omitted; admin invisible)

    var back User
    if err := json.Unmarshal([]byte(`{"name":"Bob","email":"b@x.com"}`), &back); err != nil {
        log.Fatal(err)
    }
    fmt.Printf("%+v\n", back) // {Name:Bob Email:b@x.com admin:false}
}
```

The **struct tag** `json:"name"` maps the Go field `Name` to the JSON key `name`. `omitempty` drops the field when it holds its zero value.

When you do not know the shape ahead of time, decode into `map[string]any` (the JSON object) or `[]any` (the JSON array). Numbers come back as `float64`.

```
var m map[string]any
json.Unmarshal([]byte(`{"n":1,"ok":true,"tags":["a","b"]}`), &m)
fmt.Println(m["n"], m["ok"]) // 1 true    (m["n"] is a float64)
```

For streams (a file, an HTTP body), use the **Encoder/Decoder** so you do not hold the whole document in memory:

```
enc := json.NewEncoder(os.Stdout)
enc.SetIndent("", " ") // pretty-print
enc.Encode(u)           // writes one JSON value + newline to the writer

dec := json.NewDecoder(os.Stdin)
var got User
dec.Decode(&got) // reads one JSON value from the reader
```

Watch out. `encoding/json` can only see **exported** (capitalized) fields, because it uses reflection and reflection cannot read unexported fields. An unexported field is silently skipped on both `marshal` and `unmarshal` — a classic “why is my field always empty?” bug (see Chapter 3 — Program Structure).

errors — working with error values

A recap of Chapter 12 — Errors. `errors` helps you build and inspect the error chains created by wrapping with `fmt.Errorf("...: %w", err)`.

```
var ErrNotFound = errors.New("not found")

err := fmt.Errorf("load config: %w", ErrNotFound) // wrap

errors.Is(err, ErrNotFound) // true - is this error (or its cause) ErrNotFound?

var perr *os.PathError
errors.As(err, &perr) // true if some error in the chain is a *os.PathError

both := errors.Join(err1, err2) // combine several errors into one (Go 1.20)

errors.Is replaces fragile string comparison (strings.Contains(err.Error(), "not found")).
errors.As extracts a specific error type from the chain.
```

math, math/rand/v2, and crypto/rand

`math` is `<math.h>`: `math.Sqrt`, `math.Pow`, `math.Floor`, `math.Abs`, `math.Max`/`math.Min` (for floats), constants like `math.Pi`, and limits like `math.MaxInt64`.

```
math.Sqrt(2)    // 1.414...
math.MaxInt32   // 2147483647
math.Inf(1)     // +Inf
```

For non-secure randomness use `math/rand/v2` (the modern version, Go 1.22). It seeds itself automatically — no `srand(time(NULL))` needed.

```
rand.IntN(6)           // a number in [0, 6)
rand.N(100)            // generic: [0, 100) for any integer type
rand.Float64()         // [0.0, 1.0)
rand.Shuffle(len(xs), func(i, j int) { xs[i], xs[j] = xs[j], xs[i] })
```

For anything security-sensitive (tokens, passwords, keys), use `crypto/rand`, which reads from the operating system's secure random source.

```
buf := make([]byte, 16)
if _, err := crand.Read(buf); err != nil { // crand is an alias for crypto/rand
    log.Fatal(err)
}
token := fmt.Sprintf("%x", buf) // 32 hex characters
```

Watch out. `math/rand/v2` is fast and predictable — **never** use it for passwords, session tokens, or keys. An attacker can reproduce its output. Use `crypto/rand` for secrets. This is the same `rand()` versus `/dev/urandom` distinction you know from C, but the package names make the choice explicit.

regexp — regular expressions

`regexp` implements RE2 syntax (safe: no catastrophic backtracking). Compile a pattern once, then reuse it.

```
re := regexp.MustCompile(`(\w+)@(\w+)\.com`) // MustCompile panics on a bad pattern
re.MatchString("a@b.com")                  // true
re.FindString("contact a@b.com now")        // "a@b.com"
re.FindStringSubmatch("a@b.com")            // ["a@b.com" "a" "b"] (full match + groups)
re.ReplaceAllString("a@b.com", "REDACTED")  // "REDACTED"
```

Rule of thumb. For fixed substrings, `strings.Contains` is far faster than a regular expression. Reach for `regexp` only when the pattern is truly variable.

log and structured log/slog

The classic `log` package writes timestamped lines to standard error:

```
log.Println("server starting") // 2026/03/14 09:30:00 server starting
log.Printf("listening on :%d", 8080)
log.Fatal("cannot bind")       // prints, then calls os.Exit(1)
```

Modern Go prefers `log/slog` (Go 1.21) for *structured* logging: each line is key/value pairs, easy for machines to parse and filter.

```
slog.Info("request handled", "method", "GET", "path", "/", "status", 200)
// 2026/03/14 09:30:00 INFO request handled method=GET path=/ status=200
```

Swap the **handler** to emit JSON, and set a minimum level:

```
logger := slog.New(slog.NewJSONHandler(os.Stdout, &slog.HandlerOptions{
    Level: slog.LevelDebug,
```

```

}))
slog.SetDefault(logger) // now plain slog.Info(...) uses this handler

logger.Info("user login", slog.String("user", "alice"), slog.Int("attempts", 1))
// {"time":"...", "level":"INFO", "msg":"user login", "user":"alice", "attempts":1}

```

C vs Go. In C you log with `fprintf(stderr, ...)` and grep unstructured text. `slog` gives every log line a level and typed key/value fields, so a log system can filter on `status=500` without a regex. Prefer `slog` for new services.

flag — command-line flags

`flag` parses `-name value` options. A short taste here; Chapter 24 — Building Command-Line Tools goes deep.

```

port := flag.Int("port", 8080, "port to listen on")
verbose := flag.Bool("v", false, "verbose output")
flag.Parse() // reads os.Args; remaining args are flag.Args()

fmt.Println(*port, *verbose) // flag values come back as pointers

```

context — cancellation and deadlines

A recap of Chapter 15 — Synchronization and context. A `context.Context` carries a cancellation signal and a deadline across function and goroutine boundaries. By convention it is the first argument of any function that does I/O or blocks.

```

ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
defer cancel() // always cancel to release resources

select {
case <-ctx.Done():
    fmt.Println("timed out:", ctx.Err()) // context deadline exceeded
case res := <-work:
    use(res)
}

```

The generic helpers: slices, maps, cmp

These three packages (Go 1.21) add the helpers that, in C, you wrote by hand for every type. They are **generic** (Chapter 19 — Generics), so one function works for `[]int`, `[]string`, or `[]YourType`.

`slices` operates on any slice:

```

slices.Contains(xs, 3)           // membership test
slices.Index(xs, 3)             // first index of 3, or -1
slices.Max(xs)                  // largest element
slices.Equal(a, b)              // element-by-element comparison
slices.Reverse(xs)              // in place
ys := slices.Clone(xs)          // shallow copy

```

maps operates on any map. Note that `maps.Keys` returns an **iterator** (`iter.Seq`), not a slice (Go 1.23) — combine it with `slices` to get a sorted slice:

```
m := map[string]int{"b": 2, "a": 1}
keys := slices.Sorted(maps.Keys(m)) // ["a" "b"] - collect + sort the iterator
mc := maps.Clone(m)                 // shallow copy of the map
maps.Equal(m, mc)                   // true
```

`cmp` provides comparison helpers for ordered types:

```
cmp.Compare(3, 5) // -1 (a<b -> -1, a==b -> 0, a>b -> +1)
cmp.Less(3, 5)    // true
name := cmp.Or(userName, "anonymous") // first non-zero value (Go 1.22)
```

C vs Go. In C you copy-paste a sort or a “contains” loop for each type, or fight `void *` and `qsort`. Go’s generic `slices/maps/cmp` give you one tested implementation that the compiler specializes for your exact type — type safety *and* no boilerplate.

Key takeaways

- Go’s standard library is large and high quality. Check it before adding a dependency. Docs are at <https://pkg.go.dev> and via `go doc`.
- `fmt` is type-safe `printf`. Learn `%v`, `%+v`, `%#v`, `%T`, `%q`; implement `String()` `string` to control how a type prints.
- `strings/bytes` return new values instead of mutating buffers; build strings with `strings.Builder`. `strconv` converts text and numbers and **always** returns an error to check.
- `io.Reader/io.Writer` are the universal byte interfaces; `bufio.Scanner` reads input line by line (mind the 64 KB token limit). `os` covers args, env, files, and the standard streams.
- `time` uses the reference layout 2006-01-02 15:04:05; durations carry units.
- Prefer the generic `slices.Sort/slices/maps/cmp` helpers over the older `sort` package and hand-written loops.
- `encoding/json` (de)serializes **exported** fields, guided by struct tags.
- Use `math/rand/v2` for speed, `crypto/rand` for secrets. Use `slog` for structured logging.

Watch out (gotchas for C programmers)

- **strconv errors are not optional.** Unlike `atoi`, a parse failure returns an error; ignoring it hides bad input.
- **The time layout is the literal reference date.** 2006, 01, 02, 15, 04, 05 are magic numbers, not arbitrary examples.
- **encoding/json ignores unexported fields.** Capitalize fields you want serialized; use tags to set the JSON key.
- **bufio.Scanner caps tokens at 64 KB by default.** Long lines error out unless you enlarge the buffer.
- **math/rand/v2 is not secure.** Use `crypto/rand` for anything an attacker must not predict.
- **maps.Keys returns an iterator, not a slice.** Wrap it with `slices.Sorted` or `slices.Collect` to get a slice.
- **os.Exit skips defer.** Deferred cleanup does not run; prefer returning an error to `main`.

Interview questions

Q: What is the difference between `fmt.Println`, `fmt.Printf`, and `fmt.Sprintf`? A: `Println` prints its arguments separated by spaces with a trailing newline. `Printf` prints using a format string with verbs like `%d` and `%s` and adds no newline unless you write `\n`. `Sprintf` is like `Printf` but returns the result as a `string` instead of printing it. There are also `Fprint*` variants that write to any `io.Writer`.

Q: Why does `strconv.Atoi` return an error when C's `atoi` does not? A: `atoi` returns 0 for invalid input, so you cannot distinguish the number zero from a parse failure. `strconv.Atoi` returns `(int, error)`, forcing you to detect and handle bad input explicitly, which prevents silent bugs.

Q: Explain Go's time layout string `2006-01-02 15:04:05`. A: Go formats and parses time by example. You write the one reference instant — Mon Jan 2 15:04:05 MST 2006 — in the shape you want. The tokens count up: month 1, day 2, hour 3 (or 15 for 24-hour), minute 4, second 5, year 6, zone 7. So `2006-01-02` means “four-digit year, dash, two-digit month, dash, two-digit day.”

Q: What do `io.Reader` and `io.Writer` give you that C's `FILE *` does not? A: They are small interfaces, so any source or sink of bytes — file, socket, in-memory buffer, HTTP body, test string — satisfies them. Functions written against `io.Reader`/`io.Writer` work on all of those without change, which makes code reusable and easy to test (pass a `strings.Reader` or `bytes.Buffer`).

Q: When do you use `math/rand/v2` versus `crypto/rand`? A: Use `math/rand/v2` for simulations, sampling, shuffling, and load generation, where speed matters and predictability is fine. Use `crypto/rand` for security: tokens, passwords, keys, nonces. `math/rand/v2` output is reproducible and must never protect a secret.

Q: Why does `encoding/json` ignore some struct fields? A: It uses reflection, which can only access exported (capitalized) fields. An unexported field is invisible to `marshal` and `unmarshal`. Capitalize the field and use a `json:"..."` tag to control the key name and options like `omitempty`.

Try it

1. Write a program that reads numbers from standard input with a `bufio.Scanner`, parses each with `strconv.Atoi` (handling errors), and prints the running total. Pipe a file into it: `go run . < numbers.txt`.
2. Define a `struct` with a few fields and `json:"..."` tags, marshal it with `json.MarshalIndent`, then unmarshal the bytes back and confirm the round trip. Lowercase one field name and watch it disappear from the JSON.

Chapter 21 — Testing

What you'll learn. How Go builds testing into the language and the `go` tool: `_test.go` files, table-driven tests with subtests, benchmarks (including the new `b.Loop`), fuzzing, example functions that double as documentation, setup/teardown helpers, golden files, and test doubles — all with no external framework.

C has no standard test framework; Go does

In C, testing is bring-your-own. You sprinkle `assert()`, write a `main` that calls your checks, or pull in a third-party framework (CUnit, Check, Unity). There is no single `make test` everyone agrees on, and the framework you pick is rarely the one the next project uses.

Go puts testing **in the box**. The `testing` package and the `go test` command ship with the toolchain. Every Go project tests the same way, so you can drop into any codebase and run `go test ./...` immediately (see Chapter 2 — Installing Go and the `go` Command).

Concept	C	Go
Test framework	Third party (CUnit, Check) or <code>assert.h</code>	Built in: <code>testing</code> + <code>go test</code>
Where tests live	Separate files/dirs you wire up	<code>*_test.go</code> beside the code
How to run	Custom <code>make test</code> target	<code>go test ./...</code>
Benchmarks	Hand-rolled timers	<code>func BenchmarkXxx, go test -bench</code>
Fuzzing	Separate tool (libFuzzer, AFL)	Built in: <code>func FuzzXxx, go test -fuzz</code>
Coverage	<code>gcov/lcov</code> setup	<code>go test -cover</code>

Tests live beside the code

A test file ends in `_test.go` and sits in the **same directory** as the code it tests. The `go build` command ignores these files, so test code never ships in your binary; `go test` compiles them in.

```
strutil/  
  reverse.go      package strutil    (the code)  
  reverse_test.go package strutil    (white-box tests)  
  export_test.go  package strutil_test (black-box tests, optional)
```

A test file may use one of two package clauses:

- **package strutil** — the *same* package as the code. The test can see unexported names (white-box testing). This is the common choice.
- **package strutil_test** — a separate test-only package that *imports* `strutil`. The test sees only the exported API, exactly as a real user would (black-box testing). Go allows this one exception to the “one package per directory” rule from Chapter 3 — Program Structure.

Here is the code we will test throughout the chapter:

```
// file: reverse.go  
package strutil  
  
// Reverse returns s with its runes (Unicode code points) in reverse order.  
func Reverse(s string) string {  
    r := []rune(s)  
    for i, j := 0, len(r)-1; i < j; i, j = i+1, j-1 {  
        r[i], r[j] = r[j], r[i]  
    }  
}
```

```
    return string(r)
}
```

Your first test

A test is a function named **TestXxx** (the name after **Test** must start with an uppercase letter) that takes one argument, ***testing.T**.

```
// file: reverse_test.go
package strutil

import "testing"

func TestReverse(t *testing.T) {
    got := Reverse("hello")
    want := "olleh"
    if got != want {
        t.Errorf("Reverse(%q) = %q, want %q", "hello", got, want)
    }
}
```

There are no assertion macros. You write a normal **if**, and if the result is wrong you call a method on **t** to report it. Run it:

```
go test           # run tests in the current package
go test ./...     # run every test in the module
```

t gives you two ways to report a failure, and the difference matters:

Method	Marks test failed?	Keeps going?	Use when
t.Error / t.Errorf	yes	yes	one of several independent checks
t.Fatal / t.Fatalf	yes	no (stops this test)	a precondition failed; later code would panic
t.Log / t.Logf	no	yes	extra detail (shown only with -v or on failure)

Rule of thumb. Use **t.Fatal** when continuing makes no sense — for example after **err != nil** from a setup step, because the next line would dereference a **nil**. Use **t.Error** to record a failed comparison and let the rest of the checks run, so you see *all* the problems at once.

A **helper** is a function that contains the actual checks. Call **t.Helper()** inside it so failures are reported at the *caller's* line, not deep inside the helper:

```
func assertEquals(t *testing.T, got, want string) {
    t.Helper() // report failures at the caller, not here
    if got != want {
        t.Errorf("got %q, want %q", got, want)
    }
}
```

```
}
}
```

Table-driven tests with subtests

The idiomatic Go test is **table-driven**: a slice of test cases (the “table”) looped over, with each case run as a **subtest** via `t.Run`. This is the single most important pattern in this chapter.

```
func TestReverse(t *testing.T) {
    tests := []struct {
        name string
        in   string
        want string
    }{
        {name: "empty", in: "", want: ""},
        {name: "ascii", in: "hello", want: "olleh"},
        {name: "palindrome", in: "level", want: "level"},
        {name: "unicode", in: "héllo", want: "olléh"},
    }

    for _, tc := range tests {
        t.Run(tc.name, func(t *testing.T) {
            got := Reverse(tc.in)
            if got != tc.want {
                t.Errorf("Reverse(%q) = %q, want %q", tc.in, got, tc.want)
            }
        })
    }
}
```

Why this pattern wins:

- **Adding a case is one line** in the table — no copy-pasted test function.
- `t.Run(name, ...)` gives each case its **own name**, so a failure says exactly which one broke:
--- FAIL: TestReverse/unicode.
- You can run a single case: `go test -run TestReverse/unicode`.

C vs Go. In C you would write a loop over an array of structs and `printf` the failures yourself, then remember to set the exit code. `t.Run` gives you named, individually-runnable, independently-failing subtests for free, and the framework tracks pass/fail.

Running tests

The flags you will use constantly:

```
go test ./...           # all packages
go test -v ./...        # verbose: show each test and its logs
go test -run TestReverse # only tests whose name matches the regexp
go test -run TestReverse/ascii # only that subtest
go test -race ./...      # build with the race detector (Chapter 22)
go test -count=1 ./...   # disable the test cache (force a real run)
```

Watch out. Go **caches** test results. If nothing the test depends on has changed, `go test` prints (cached) and does not re-run. That is great for speed but confusing when a test depends on something outside the build (a clock, a file, the network). Add `-count=1` to force a fresh run; it is the standard “turn off caching” idiom.

Coverage

Coverage shows which lines your tests exercise.

```
go test -cover ./...           # print a percentage per package
go test -coverprofile=cover.out ./... # write a detailed profile
go tool cover -func=cover.out      # per-function coverage in the terminal
go tool cover -html=cover.out      # open an annotated, colored view
```

C vs Go. This replaces the `gcc --coverage + gcov/lcov` dance. No extra compiler flags, no separate tool to install — coverage is part of `go test`.

Benchmarks

A benchmark is a function named **BenchmarkXxx** taking `*testing.B`. The framework runs your code many times and reports nanoseconds per operation.

The modern form (Go 1.24) uses `b.Loop()`:

```
func BenchmarkReverse(b *testing.B) {
    for b.Loop() { // runs the body enough times to get a stable measurement
        Reverse("the quick brown fox")
    }
}
```

`b.Loop()` is preferred because it keeps the loop count internal, prevents the compiler from optimizing the call away, and excludes one-time setup automatically. You will still see the older `b.N` form in existing code:

```
func BenchmarkReverseOld(b *testing.B) {
    for i := 0; i < b.N; i++ { // the framework chooses b.N
        Reverse("the quick brown fox")
    }
}
```

If a benchmark needs expensive setup, do it first and then call `b.ResetTimer()` so the setup is not counted:

```
func BenchmarkProcess(b *testing.B) {
    data := buildLargeInput() // not part of what we measure
    b.ResetTimer()
    for b.Loop() {
        Process(data)
    }
}
```

Run benchmarks (they do **not** run during a normal `go test`):

```
go test -bench=. # run all benchmarks
go test -bench=Reverse -benchmem # also report allocations per operation
```

Sample output (`-benchmem` adds the last two columns):

```
BenchmarkReverse-8    28412641    42.11 ns/op    16 B/op    1 allocs/op
```

That reads: ran ~28 million times, 42 ns each, 16 bytes and 1 allocation per call.

Fuzzing

A **fuzz test** feeds your code a flood of automatically generated inputs to find cases you did not think of. It is built into `go test` (Go 1.18). A fuzz target is named **FuzzXxx** and takes `*testing.F`.

```
import "unicode/utf8"

func FuzzReverse(f *testing.F) {
    f.Add("hello") // seed corpus: example inputs to start from
    f.Add("h      ")
    f.Add("")

    f.Fuzz(func(t *testing.T, s string) {
        rev := Reverse(s)
        doubled := Reverse(rev)
        // Property 1: reversing twice returns the original.
        if s != doubled {
            t.Errorf("Reverse(Reverse(%q)) = %q", s, doubled)
        }
        // Property 2: a valid UTF-8 string stays valid after reversing.
        if utf8.ValidString(s) && !utf8.ValidString(rev) {
            t.Errorf("Reverse(%q) produced invalid UTF-8: %q", s, rev)
        }
    })
}
```

You do not assert exact outputs (you cannot know them for random input). Instead you assert **properties** that must always hold. Run it:

```
go test -fuzz=FuzzReverse # fuzz until it finds a failure or you stop
go test -fuzz=FuzzReverse -fuzztime=30s # fuzz for 30 seconds, then stop
```

When fuzzing finds a crashing input, it **saves that input** under `testdata/fuzz/` and turns it into a permanent regression test. Without `-fuzz`, `go test` still runs the seed corpus as ordinary test cases.

C vs Go. In C you bolt on libFuzzer or AFL, write a separate harness, and build with special flags. Go fuzzing is just another function in your `_test.go` file and one command-line flag.

Example functions are tests *and* documentation

A function named **ExampleXxx** with a trailing `// Output:` comment is compiled, run, and checked — and it also appears on the package’s documentation page. One piece of code serves as a test and a usage example.

```
import "fmt"

func ExampleReverse() {
    fmt.Println(Reverse("hello"))
    // Output: olleh
}
```

`go test` runs it and fails if the printed text does not match the `// Output:` block exactly. If the order of lines is not deterministic, use `// Unordered output:` instead. An example with no `// Output:` comment is compiled (so it cannot rot) but not run.

Watch out. The match is **exact**, including whitespace. A stray trailing space or a missing newline makes the example fail. This strictness is the point: the docs cannot drift from the code.

Setup and teardown

Most tests need no global setup. When you do, use these tools instead of a hand-written `main`.

TestMain runs once for the whole package, around all its tests. Use it for expensive shared setup (a database container, a temp directory):

```
func TestMain(m *testing.M) {
    // setup before any test runs
    code := m.Run() // runs all Test/Benchmark/Fuzz functions; returns the exit code
    // teardown after all tests finish
    os.Exit(code)
}
```

Per-test helpers clean up automatically — no manual teardown to forget:

```
func TestSaveConfig(t *testing.T) {
    dir := t.TempDir() // temp dir, deleted after the test
    path := filepath.Join(dir, "config.json")
    t.Setenv("APP_ENV", "test") // env var, restored after the test
    t.Cleanup(func() { closeSomething() }) // runs at the end, even on failure

    if err := os.WriteFile(path, []byte("{}"), 0o644); err != nil {
        t.Fatal(err)
    }
    // ... assertions ...
}
```

- `t.TempDir()` returns a unique directory removed when the test ends.
- `t.Setenv(k, v)` sets an environment variable and restores the old value after.

- `t.Cleanup(fn)` registers teardown that runs even if the test fails — like `defer`, but tied to the test's lifetime.

testdata/ and golden files

The directory named `testdata/` is ignored by the Go tools, so it is the standard home for test fixtures: sample inputs and expected outputs. A **golden file** holds the expected output of a function; the test compares against it and a `-update` flag rewrites it when the output legitimately changes.

```
var update = flag.Bool("update", false, "update golden files")

func TestRender(t *testing.T) {
    got := Render(sampleInput)
    golden := filepath.Join("testdata", "render.golden")

    if *update { // run `go test -update` to regenerate
        if err := os.WriteFile(golden, got, 0o644); err != nil {
            t.Fatal(err)
        }
    }
    want, err := os.ReadFile(golden)
    if err != nil {
        t.Fatal(err)
    }
    if !bytes.Equal(got, want) {
        t.Errorf("output does not match %s (run go test -update to refresh)", golden)
    }
}
```

Test doubles via interfaces (no mocking framework)

Go does not need a mocking framework. Because interfaces are implicit (Chapter 11 — Interfaces), you write a small fake that satisfies the interface your code depends on. Accept an interface, pass a fake in the test.

```
// The code depends on an interface, not a concrete database.
type Store interface {
    Get(id string) (string, error)
}

func Greet(s Store, id string) (string, error) {
    name, err := s.Get(id)
    if err != nil {
        return "", err
    }
    return "Hello, " + name, nil
}

// The test supplies a fake Store - no real database needed.
```



```

type fakeStore struct{ data map[string]string }

func (f fakeStore) Get(id string) (string, error) {
    v, ok := f.data[id]
    if !ok {
        return "", errors.New("not found")
    }
    return v, nil
}

func TestGreet(t *testing.T) {
    s := fakeStore{data: map[string]string{"1": "Alice"}}
    got, err := Greet(s, "1")
    if err != nil {
        t.Fatal(err)
    }
    if got != "Hello, Alice" {
        t.Errorf("got %q", got)
    }
}

```

Testing HTTP with net/http/httpptest

For HTTP code, net/http/httpptest spins up a real server on a random local port, or records a handler's response in memory.

```

func TestPingServer(t *testing.T) {
    srv := httpptest.NewServer(http.HandlerFunc(
        func(w http.ResponseWriter, r *http.Request) {
            fmt.Fprintln(w, "pong")
        }
    ))
    defer srv.Close()

    resp, err := http.Get(srv.URL)
    if err != nil {
        t.Fatal(err)
    }
    defer resp.Body.Close()

    body, _ := io.ReadAll(resp.Body)
    if strings.TrimSpace(string(body)) != "pong" {
        t.Errorf("body = %q, want pong", body)
    }
}

```

To test a handler without any network, use httpptest.NewRecorder (it implements http.ResponseWriter) and httpptest.NewRequest:

```

func TestPingHandler(t *testing.T) {

```

```

req := httptest.NewRequest("GET", "/ping", nil)
rec := httptest.NewRecorder()

pingHandler(rec, req) // call the handler directly

if rec.Code != http.StatusOK {
    t.Errorf("status = %d, want 200", rec.Code)
}
}

```

Parallel tests

Call `t.Parallel()` at the top of a test to run it alongside other parallel tests. The runner pauses such tests, then runs them together, which speeds up an I/O-heavy suite.

```

func TestSlowA(t *testing.T) {
    t.Parallel()
    // ... independent work ...
}

```

Watch out. Parallel tests run **at the same time**, so they must not share mutable state (the same temp file, the same global variable). If two parallel tests touch one resource, you get flaky failures or a data race. Give each test its own `t.TempDir()` and avoid package-level mutable variables.

A note on testify

The most popular third-party test library is **testify**, which adds assertion helpers (`assert.Equal(t, want, got)`, `require.NoError(t, err)`) and mock support. It is widely used and fine to adopt. But you do not *need* it: the standard `if got != want { t.Errorf(...) }` style is idiomatic, dependency-free, and what the Go standard library itself uses. Learn the stdlib way first.

Key takeaways

- Testing is built in: `*_test.go` files beside the code, `func TestXxx(t *testing.T)`, and `go test`. No framework to install.
- Use `package foo` for white-box tests (see unexported names) or `package foo_test` for black-box tests (exported API only).
- `t.Error` records a failure and continues; `t.Fatal` stops the test. Mark helpers with `t.Helper()`.
- Prefer **table-driven tests** with `t.Run` subtests: easy to extend, named, individually runnable.
- Benchmarks use `func BenchmarkXxx(b *testing.B)` with `for b.Loop()` (Go 1.24); fuzz tests use `func FuzzXxx(f *testing.F)` and assert *properties*.
- `ExampleXxx` with `// Output:` is a test and documentation at once.
- `TestMain`, `t.Cleanup`, `t.TempDir`, and `t.Setenv` handle setup/teardown; `testdata/` holds fixtures and golden files.
- Fake dependencies with small interface implementations; use `net/http/httptest` for HTTP.

Watch out (gotchas for C programmers)

- **Test results are cached.** Use `-count=1` to force a re-run when a test depends on something outside the build.
- **t.Error vs t.Fatal.** `Fatal` only stops the *current* test/goroutine; prefer it after failed setup so later code does not panic.
- **Parallel tests share nothing safely.** `t.Parallel()` tests run concurrently; isolate their state or you get flaky races.
- **Example output must match exactly.** Trailing spaces and newlines count.
- **Benchmarks need -bench.** A plain `go test` skips them, so a broken benchmark can hide until you ask for it.
- **Fuzz corpus failures are saved to testdata/fuzz/.** Commit them — they become permanent regression tests.

Interview questions

Q: How do you write and run a basic test in Go? A: Put a function `func TestXxx(t *testing.T)` in a file ending in `_test.go` in the same package. Inside, call the code, compare the result, and report failures with `t.Errorf` (continue) or `t.Fatalf` (stop). Run it with `go test ./...`. There are no assertion macros; you use ordinary `if` statements.

Q: What is a table-driven test and why is it the preferred style? A: It is a slice of test-case structs (inputs and expected outputs) that you loop over, running each case as a subtest with `t.Run(name, ...)`. It minimizes duplication, names each case so failures are precise, lets you run one case with `-run`, and makes adding a case a one-line change.

Q: What is the difference between t.Error and t.Fatal? A: Both mark the test as failed. `t.Error/Errorf` records the failure and lets the test keep running, so you see all problems. `t.Fatal/Fatalf` records the failure and stops the current test immediately (it calls `runtime.Goexit`). Use `Fatal` when continuing would panic, such as after a setup error.

Q: Why might go test say “(cached)” and how do you force a re-run? A: Go caches test results keyed on the test binary and its inputs; if nothing changed it reuses the previous result. To force a real run, pass `-count=1`, which disables the cache. This matters for tests that read a clock, the filesystem, or the network.

Q: How does Go’s fuzzing work, and what do you assert in a fuzz test? A: You write `func FuzzXxx(f *testing.F)`, add seed inputs with `f.Add`, and call `f.Fuzz` with a function that takes `*testing.T` plus the fuzzed arguments. Go generates many inputs and runs the body. You assert *properties* that must always hold (for example, reversing twice yields the original), not exact outputs. Crashing inputs are saved as regression tests.

Q: How do you test code that depends on a database or an HTTP service without the real thing? A: Depend on an interface, then pass a small fake that implements it in the test — Go’s implicit interfaces make this trivial, so no mocking framework is required. For HTTP, use `net/http/httptest` to run a real local server or to record a handler’s response with `httptest.NewRecorder`.

Try it

1. Create `strutil/reverse.go` with the `Reverse` function and a `reverse_test.go` with the table-driven `TestReverse`. Run `go test -v`.
2. Add `BenchmarkReverse` using `for b.Loop()` and run `go test -bench=. -benchmem`.
3. Add `FuzzReverse` and run `go test -fuzz=FuzzReverse -fuzztime=15s`. Try introducing a bug (reverse bytes instead of runes) and watch the fuzzer catch a multi-byte UTF-8 input.

Chapter 22 — Tooling: Format, Vet, Lint, Race, Profile, Debug

What you’ll learn. The tools that keep Go code clean, correct, and fast: `gofmt/goimports`, `go vet`, `staticcheck`, `golangci-lint`, the race detector, `pprof` profiling, the execution tracer, the Delve debugger, escape/inlining analysis, `go generate`, `govulncheck`, and the `gopls` language server — each mapped to the C tool it replaces.

One toolbox instead of many

In C you assemble your quality tools from separate projects: `clang-format` to format, `cppcheck` or `clang-tidy` to lint, Valgrind to find memory and threading bugs, `gprof` to profile, and `gdb` to debug. Each has its own flags, its own quirks, and its own install.

Go builds most of this into the `go` command and `go tool` (see Chapter 2 — Installing Go and the `go` Command). A few extras (`staticcheck`, `golangci-lint`, `dlv`, `govulncheck`, `gopls`) are single-binary installs with `go install`. Here is the map a C programmer can keep on their desk:

Job	C tool	Go tool
Format source	<code>clang-format</code>	<code>gofmt</code> / <code>go fmt</code> , <code>goimports</code>
Static analysis (lint)	<code>cppcheck</code> , <code>clang-tidy</code>	<code>go vet</code> , <code>staticcheck</code> , <code>golangci-lint</code>
Data-race detection	Valgrind (Helgrind/DRD)	race detector (<code>-race</code>)
Memory errors	Valgrind (Memcheck)	(mostly gone: GC + bounds checks)
CPU / heap profiling	<code>gprof</code> , Callgrind	<code>pprof</code> (<code>go tool pprof</code>)
Execution timeline	—	<code>go tool trace</code>
Debugger	<code>gdb</code> , <code>lldb</code>	Delve (<code>dlv</code>)
Disassemble	<code>objdump</code>	<code>go tool objdump</code>
Vulnerability scan	(third party)	<code>govulncheck</code>
Editor intelligence	<code>clangd</code>	<code>gopls</code>

Mental model. Valgrind splits into two Go ideas. Its *memory* checking (use-after-free, leaks) is mostly unnecessary because Go has a garbage collector and bounds-checked slices. Its *threading* checking maps to Go’s built-in race detector.

Formatting: `gofmt`, `go fmt`, `goimports`

Go has exactly one official style, applied by `gofmt`. There is nothing to configure and no style argument to have. This was covered in Chapter 2; the key flags for tooling and CI are:

```
gofmt -l .      # LIST files that are not correctly formatted (prints nothing if clean)
gofmt -d .      # show a DIFF of what would change
gofmt -w .      # rewrite files in place
go fmt ./...    # wrapper: format every package in the module
```

`gofmt -l .` is the CI check: if it prints any filename, the build fails because someone committed unformatted code.

goimports is `gofmt` plus import management: it adds missing imports and removes unused ones while formatting. Many editors run it on save.

```
go install golang.org/x/tools/cmd/goimports@latest
goimports -w .    # format AND fix the import block
```

C vs Go. `clang-format` needs a `.clang-format` file and team agreement on 50 options. `gofmt` has zero options on purpose, so every Go file in the world looks the same. Do not fight it; configure your editor to run it on save.

Vetting: `go vet`

`go vet` is a static analyzer that ships with Go. It reports code that compiles but is probably wrong. It is the rough equivalent of `cppcheck`, but built in and zero-config.

```
go vet ./...
```

A sample of what it catches:

- **printf** — a format string that does not match its arguments, e.g. `fmt.Printf("%d", "hello")`. This is the check C programmers miss most, because C's `printf` is not type-checked at all.
- **copylocks** — copying a value that contains a `sync.Mutex` (copying a lock breaks it).
- **lostcancel** — the `cancel` function from `context.WithCancel` is never called (a resource leak).
- **unreachable** — code after a `return` that can never run.
- **structtag** — a malformed struct tag like `json:name` (missing quotes).

```
// go vet flags this: %d wants an integer, got a string.
fmt.Printf("count=%d\n", "three")
```

Rule of thumb. Run `go vet ./...` on every commit and in CI. It is fast, has almost no false positives, and `go test` already runs a subset of `vet` before your tests.

Linting: `staticcheck` and `golangci-lint`

`go vet` is deliberately conservative. For deeper analysis, add a linter.

staticcheck is the most respected single linter. It finds bugs, dead code, and simplifications `go vet` will not.

```
go install honnef.co/go/tools/cmd/staticcheck@latest
staticcheck ./...
```

golangci-lint is a *meta-linter*: it runs many linters (including `go vet` and `staticcheck`) in one fast pass, configured by a `.golangci.yml` file. It is the de-facto standard in CI.

```
# install a released binary (preferred - see golangci-lint.run for the current command)
brew install golangci-lint
```

```
golangci-lint run ./...      # run the configured set of linters
golangci-lint run --fix ./... # auto-fix what can be fixed
```

C vs Go. `golangci-lint` is like running `clang-tidy` with a curated set of checks, but it is fast (it caches and runs linters in parallel) and the whole ecosystem uses the same handful of linters, so findings are consistent across projects.

The race detector

A **data race** is two goroutines touching the same memory at the same time with at least one write and no synchronization (see Chapter 16 — Concurrency Patterns). The race detector finds these at runtime. Add `-race` to any run:

```
go test -race ./...  # the most common use
go run -race .
go build -race -o app .
```

When it sees a race, it prints both stacks — the read and the conflicting write — so you know exactly which two accesses collided.

Watch out. The race detector is a **dynamic** tool: it only finds races on code paths that actually execute during the run. A race in a rarely-taken branch is invisible until that branch runs. So run your *tests* with `-race` in CI to exercise as many paths as possible. Also note the cost: roughly 2–20× slower and several times more memory, so it is for testing, not production.

Profiling with pprof

`pprof` answers “where does the time (or memory) go?” — the job of `gprof` and Callgrind in C, but easier and built in. You collect a **profile** (a sample of where the program spends resources), then explore it with `go tool pprof`.

There are three ways to collect a profile.

1. **From a benchmark or test** — the quickest:

```
go test -bench=. -cpuprofile=cpu.prof -memprofile=mem.prof
```

2. **From a program with runtime/pprof:**

```
import (
    "os"
    "runtime/pprof"
)

func main() {
    f, err := os.Create("cpu.prof")
    if err != nil {
        panic(err)
    }
}
```

```

defer f.Close()

pprof.StartCPUProfile(f) // begin sampling
defer pprof.StopCPUProfile()

doExpensiveWork()
}

```

3. From a server with `net/http/pprof` — a blank import wires profiling endpoints onto the default HTTP mux at `/debug/pprof/`:

```

import (
    "net/http"
    _ "net/http/pprof" // registers /debug/pprof/ handlers as a side effect
)

func main() {
    go http.ListenAndServe("localhost:6060", nil) // profiling endpoint
    runServer()
}

```

Then collect a 30-second CPU profile from the live server:

```
go tool pprof http://localhost:6060/debug/pprof/profile?seconds=30
```

Once you have a profile, explore it interactively:

```
go tool pprof cpu.prof
```

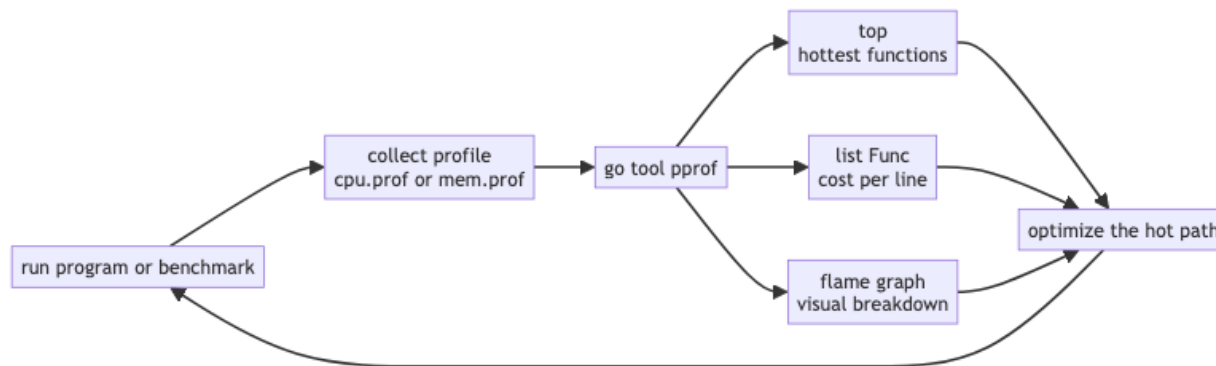
The commands you use inside `pprof`:

Command	What it shows
<code>top</code>	the functions using the most CPU/memory
<code>top -cum</code>	sorted by cumulative cost (function plus its callees)
<code>list Func</code>	annotated source of <code>Func</code> , cost per line
<code>web</code>	a visual call graph in your browser (needs Graphviz)
<code>peek Func</code>	callers and callees of <code>Func</code>

Or jump straight to a flame graph in the browser:

```
go tool pprof -http=:8080 cpu.prof # opens a web UI with a flame graph
```

The whole loop looks like this:



Watch out. A profile is only as good as the load you put on the program. Profiling an idle server or a trivial input tells you nothing. Drive **representative** work — a realistic benchmark or production-like traffic — before you trust the numbers. Profile first, optimize second: never guess.

The execution tracer: `go tool trace`

`pprof` tells you *where* time goes; the tracer tells you *what the runtime was doing over time* — goroutine scheduling, blocking, GC pauses, and channel waits. It is invaluable for concurrency problems that a CPU profile cannot show.

```
go test -trace=trace.out -bench=.
go tool trace trace.out  # opens an interactive timeline in the browser
```

Use it when goroutines are blocked or latency is spiky and you need to see who is waiting on whom.

The debugger: Delve (`dlv`)

`gdb` technically works on Go binaries, but it does not understand goroutines, channels, or the Go runtime well. The Go community debugger is **Delve** (`dlv`), which is Go-aware.

```
go install github.com/go-delve/delve/cmd/dlv@latest
```

Start a debug session — `dlv` compiles and runs your program under its control:

```
dlv debug ./cmd/server  # build and debug a program
dlv test ./...          # debug a test
dlv exec ./app          # debug an already-built binary
```

Inside the session, the commands mirror `gdb` but speak Go:

```
(dlv) break main.go:42      # set a breakpoint at a line
(dlv) break main.handleReq  # or at a function
(dlv) continue              # run to the next breakpoint
(dlv) next                  # step over one line
(dlv) step                  # step into a call
(dlv) print myVar           # inspect a variable
(dlv) goroutines             # LIST every goroutine (gdb cannot do this well)
(dlv) goroutine 7           # switch to goroutine 7
```


Action	gdb	Delve
Start	<code>gdb ./a.out</code>	<code>dlv exec ./app</code>
Breakpoint	<code>break file.c:42</code>	<code>break file.go:42</code>
Continue / step	<code>continue / next</code>	<code>continue / next</code>
Print variable	<code>print x</code>	<code>print x</code>
List threads/goroutines	<code>info threads</code>	<code>goroutines</code>

C vs Go. The big difference is **goroutines**: Delve lists and switches between goroutines, which **gdb** cannot do meaningfully for Go. For everyday work many Go programmers debug less than in C — fast compiles, **t.Log**, and the race detector catch most issues before a debugger is needed.

Compiler insight: `-gcflags=-m` and `go tool objdump`

To see whether a value is allocated on the stack or escapes to the heap (Chapter 17 — Memory and the Garbage Collector), ask the compiler with `-m`:

```
go build -gcflags=-m ./...
```

```
func newCounter() *int {
    n := 0    // the compiler reports: moved to heap: n
    return &n // returning its address forces n onto the heap
}
```

The output also notes which functions are **inlined** (can inline `newCounter`). This is how you confirm an optimization actually happened, rather than guessing.

To read the generated assembly for a single function, use `objdump`:

```
go build -o app .
go tool objdump -s 'main\newCounter' app # disassemble one function by regexp
```

C vs Go. This is your `objdump -d` and your `gcc -fdump-style` insight, but Go’s escape analysis output answers the question a C programmer actually cares about: “did this allocate, and where?”

`go generate`

`go generate` runs commands written as special comments, so generated code (such as **Stringer** methods or mocks) is reproducible. It never runs on its own — you invoke it (see Chapter 2).

```
//go:generate stringer -type=Color
type Color int

go generate ./... # runs the directives; commit the generated files
```

`govulncheck` — vulnerability scanning

`govulncheck` checks your dependencies against the official Go vulnerability database — and, crucially, reports only vulnerabilities your code **actually reaches**, not every advisory for every imported module.

```
go install golang.org/x/vuln/cmd/govulncheck@latest
govulncheck ./...
```

Run it in CI to catch a known-vulnerable dependency before it ships. There is no standard C equivalent shipped with the compiler.

gopls — the language server

gopls (“Go please”) is the official **language server**: the background program your editor talks to for autocomplete, go-to-definition, find-references, inline errors, and rename-across-the-project. It is the Go counterpart to clangd.

```
go install golang.org/x/tools/gopls@latest
```

You rarely run `gopls` by hand; your editor (VS Code, Neovim, GoLand) launches it and uses it through the Language Server Protocol. Installing it is usually all you do.

Key takeaways

- One toolbox: `gofmt/goimports` (format), `go vet + staticcheck + golangci-lint` (lint), `-race` (data races), `pprof` (CPU/heap), `go tool trace` (scheduling), `dlv` (debug), `govulncheck` (CVEs), `gopls` (editor).
- `gofmt` is non-negotiable: `gofmt -l .` in CI fails on unformatted code.
- `go vet` catches `printf` mismatches, copied locks, lost context cancels, and more — run it everywhere. `golangci-lint` bundles the deeper linters.
- The race detector (`go test -race`) is dynamic: it only sees races on executed paths, so run your tests with it in CI.
- Profile with `pprof` under representative load: collect `cpu.prof/mem.prof`, then `go tool pprof` with `top`, `list`, and a flame graph.
- Delve (`dlv`) is the Go-aware debugger; unlike `gdb` it understands goroutines.
- `-gcflags=-m` shows escape analysis and inlining; `govulncheck` scans dependencies; `gopls` powers your editor.

Watch out (gotchas for C programmers)

- **The race detector only finds races it observes.** A clean `-race` run does not prove the absence of races; it proves none happened on the paths you ran.
- **`-race` is for testing, not production.** It slows the program several-fold and uses much more memory.
- **A profile under unrealistic load is misleading.** Drive representative work before optimizing, and let the data — not intuition — pick the hot path.
- **`gofmt` is enforced on teams.** Unformatted code fails CI; set your editor to format on save.
- **Run `go vet` and a linter in CI, not just locally.** Findings that only you see locally do not protect the team.
- **`gdb` is a poor fit for Go.** Use Delve so you can inspect goroutines and the runtime.

Interview questions

Q: How does Go’s tooling replace the separate C tools `clang-format`, `cppcheck`, `valgrind`, `gprof`, and `gdb`? **A:** `gofmt/goimports` replace `clang-format`; `go vet`, `staticcheck`, and

`golangci-lint` replace `cppcheck/clang-tidy`; the built-in race detector (`-race`) replaces Valgrind's threading checks while the garbage collector and bounds checks remove most of Valgrind's memory job; `pprof` replaces `gprof/Callgrind`; and Delve (`dlv`) replaces `gdb`, adding goroutine awareness. Most ship with the `go` command; the rest are one `go install` away.

Q: What is the difference between `go vet` and a linter like `staticcheck`? A: `go vet` ships with Go and is intentionally conservative — it reports likely bugs (`printf` mismatches, copied locks, lost context cancels) with almost no false positives. `staticcheck` is a third-party linter that goes further: dead code, simplifications, and subtler correctness issues. `golangci-lint` runs many linters, including both, in one pass and is the usual CI choice.

Q: What are the limitations of the race detector? A: It is dynamic, so it only detects races on code paths that actually run during that execution; untaken branches are not checked. It also adds significant CPU and memory overhead, so it is used during testing (ideally in CI with good coverage) rather than in production. A clean run is evidence, not proof, of race-freedom.

Q: Walk through profiling a slow Go program with `pprof`. A: Collect a profile under representative load — from a benchmark (`go test -cpuprofile=cpu.prof`), a program (`runtime/pprof`), or a server (`net/http/pprof`). Open it with `go tool pprof cpu.prof`, run `top` to see the hottest functions, `list Func` to see cost per line, and `web` or a flame graph (`-http=:8080`) to visualize the call graph. Optimize the hot path, then re-profile to confirm.

Q: Why use Delve instead of `gdb` for Go? A: Delve understands the Go runtime — goroutines, channels, and Go's calling conventions — so it can list and switch between goroutines, print Go values correctly, and set breakpoints by Go function name. `gdb` works on the raw binary but misrepresents goroutines and runtime state, making it unreliable for concurrent Go programs.

Q: What does `go build -gcflags=-m` tell you? A: It prints the compiler's escape-analysis and inlining decisions: which variables “escape to the heap” (and why) and which functions can be inlined. A C programmer uses it to confirm whether a value is stack- or heap-allocated and to verify that a small function was inlined, instead of guessing.

Try it

1. Write a tiny program with a `printf` bug (`fmt.Printf("%d", "hi")`) and run `go vet ./....`. Watch it flag the mismatch the compiler accepted.
2. Add a CPU-heavy function, benchmark it with `go test -bench=. -cpuprofile=cpu.prof`, then run `go tool pprof -http=:8080 cpu.prof` and read the flame graph.
3. Run `go build -gcflags=-m .` on a function that returns `&localVar` and confirm the compiler reports “moved to heap.”

Chapter 23 — Building Web Services with `net/http`

What you'll learn. How to build a real HTTP/JSON service with Go's standard library and **no third-party code**: handlers, the modern method-and-wildcard router, a complete REST API, middleware, timeouts, graceful shutdown, and the HTTP client.

In C, writing an HTTP server is a project of its own. You pull in a library such as `libmicrohttpd` or `libevent`, learn its callback model, juggle sockets and buffers, and write a lot of code before the first request even works.

Go takes the opposite approach. The `net/http` package in the standard library is **production-grade**: real companies serve real traffic with it directly, no framework on top. You can write a complete web service with **zero dependencies** — no package to fetch, no build step to add, just `import "net/http"`. This chapter builds one service from the first line to a clean shutdown, then writes a client to call it.

Mental model. `net/http` is both the web *server* library and the HTTP *client* library in one package. The same `Request` and `Response` types describe the message on both sides of the wire.

The smallest server

Here is a complete server. It answers every request with the same line of text.

```
package main

import (
    "fmt"
    "net/http"
)

func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintln(w, "hello from Go")
    })
    http.ListenAndServe(":8080", mux) // blocks; serves until the process dies
}
```

Run it and call it:

```
go run .
curl localhost:8080      # -> hello from Go
```

Three pieces do the work:

- `http.ServeMux` is a *request multiplexer* — a router. It looks at each request and decides which function handles it. (“mux” is short for multiplexer.)
- `HandleFunc` registers a function for a pattern (here `/`, the catch-all).
- `http.ListenAndServe` opens a TCP socket on the address and runs the server loop. It blocks forever unless it fails.

Watch out. `ListenAndServe` returns an **error**. The snippet above ignores it to stay short, but real code must check it — we do that at the end of the chapter.

Handlers: the `http.Handler` interface

Everything that answers a request is an `http.Handler`. That is a one-method interface (interfaces are covered in Chapter 11 — Interfaces):

```
type Handler interface {
    ServeHTTP(w http.ResponseWriter, r *http.Request)
```

```
}
```

A handler reads the request `r` and writes the reply through `w`. To answer a request, you implement `ServeHTTP`.

Writing a whole type with a method is heavy when all you have is a function. So `net/http` gives you an **adapter**, `http.HandlerFunc`: a function type that *is* a `Handler`.

```
type HandlerFunc func(w http.ResponseWriter, r *http.Request)

// A HandlerFunc satisfies Handler by calling itself.
func (f HandlerFunc) ServeHTTP(w http.ResponseWriter, r *http.Request) {
    f(w, r)
}
```

So any plain function with the right signature becomes a `Handler` simply by converting it: `http.HandlerFunc(myFunc)`. The `mux.HandleFunc(pattern, fn)` method does that conversion for you.

C vs Go. In a C HTTP library you register a *callback*: a function pointer plus a `void *user_data` to carry state. Go's `http.Handler` is the same idea made type-safe. The interface replaces the function pointer; the handler's own fields, or a closure, replace `user_data`.

Mental model. A `Handler` answers one request. A `ServeMux` is a `Handler` that picks *other* handlers. Middleware (below) is a `Handler` that wraps *another* handler. They all share one shape, so they snap together.

Modern routing: method and wildcard patterns

Old Go code matched only on path prefixes and parsed the URL by hand. Since **Go 1.22**, a `ServeMux` pattern can include an **HTTP method** and **named wildcards**. This is now the default, idiomatic way to route, and it is what we use in this book.

A pattern reads as "METHOD /path/{name}":

```
mux := http.NewServeMux()
mux.HandleFunc("GET /tasks", listTasks)      // method + fixed path
mux.HandleFunc("POST /tasks", createTask)
mux.HandleFunc("GET /tasks/{id}", getTask)   // {id} is a wildcard segment
mux.HandleFunc("DELETE /tasks/{id}", deleteTask)
```

Inside a handler you read a wildcard by name with `r.PathValue`:

```
func getTask(w http.ResponseWriter, r *http.Request) {
    id := r.PathValue("id") // the text that matched {id}, always a string
    // ...
}
```

Rules worth knowing:

- The method must be uppercase and followed by a space: `GET /x`.
- `{id}` matches exactly one path segment. `{rest...}` (with three dots) matches the remainder of the path, slashes included.

- A trailing `{}` means “match this path exactly,” so `/tasks/{}` will not also swallow `/tasks/anything`.
- If the path matches but the **method** does not, `ServeMux` replies 405 Method Not Allowed for you. You do not write that check.
- The more specific pattern wins, so `GET /tasks/{id}` and `GET /tasks/today` can both be registered without conflict.

C vs Go. With a C library you usually match a path prefix yourself and then `strtok` the URL to pull out the id. Go’s wildcard router does the parsing and hands you `r.PathValue("id")` directly — no manual string surgery, no off-by-one bugs.

The request and the response

A handler receives a `*http.Request` (the incoming message) and an `http.ResponseWriter` (where you write the reply).

Common things you read from `*http.Request`:

```
r.Method           // "GET", "POST", ...
r.URL.Path         // "/tasks/7"
r.URL.Query().Get("limit") // ?limit=10 -> "10" (empty string if missing)
r.Header.Get("Content-Type")
r.PathValue("id")   // a {id} wildcard from the route
r.Body              // an io.ReadCloser streaming the request body
r.Context()         // the request's context (cancellation, deadlines)
```

The body is an `io.Reader` — a stream of bytes (see Chapter 8 — Arrays, Slices, and Strings). You usually hand it straight to a JSON decoder. On the **server** you do not have to close `r.Body`; the server does it for you.

To send a reply, `http.ResponseWriter` has three parts, used in this order:

```
w.Header().Set("Content-Type", "application/json") // 1. headers
w.WriteHeader(http.StatusCreated)                 // 2. status code (201)
w.Write([]byte(`{"ok":true}`))                     // 3. body
```

The order is mandatory. Set headers first, then call `WriteHeader` with the status, then write the body. The first call to `Write` *implies* `WriteHeader(http.StatusOK)` if you have not called it yourself. Once body bytes start flowing, the status and headers are already on the wire and **cannot change**.

Use the named constants — `http.StatusOK` (200), `http.StatusNotFound` (404), and so on — rather than bare numbers. They read better and prevent typos.

Watch out. Calling `w.WriteHeader(404)` *after* you have written body bytes does nothing useful (and logs a warning). The status went out with the first byte of the body. Setting headers vs writing the body in the wrong order is the single most common `net/http` mistake.

A complete JSON REST API

Now the real thing: an in-memory store of **tasks** with four endpoints.

Method and path	Does	Success status
GET /tasks	list all tasks	200 OK
POST /tasks	create a task from a JSON body	201 Created
GET /tasks/{id}	fetch one task	200 OK
DELETE /tasks/{id}	delete one task	204 No Content

The program is built up across the next few sections — the data and store, the JSON helpers, the handlers, the logging middleware, and finally `main`. Collected in that order into one `main.go` (the `slow` example in the *Context and timeouts* section is a separate illustration, not part of it), it compiles and runs.

The data and the store

```
// Command tasksd is a small JSON REST API for a list of tasks.
package main

import (
    "context"
    "encoding/json"
    "errors"
    "log"
    "net/http"
    "os"
    "os/signal"
    "strconv"
    "sync"
    "syscall"
    "time"
)

// Task is one to-do item. Only EXPORTED (capitalized) fields are encoded to
// JSON. The `json:"..."` struct tags set the key names used on the wire.
type Task struct {
    ID      int    `json:"id"`
    Title   string `json:"title"`
    Done    bool   `json:"done"`
}

// store holds tasks in memory. HTTP handlers run in separate goroutines, so a
// mutex must guard every read and write of the shared map and counter.
type store struct {
    mu      sync.Mutex
    tasks   map[int]Task
    nextID  int
}

func newStore() *store {
```

```

    return &store{tasks: make(map[int]Task), nextID: 1}
}

func (s *store) list() []Task {
    s.mu.Lock()
    defer s.mu.Unlock()
    // Pre-allocate with length 0 so an empty store encodes to [] and not null.
    out := make([]Task, 0, len(s.tasks))
    for _, t := range s.tasks {
        out = append(out, t)
    }
    return out
}

func (s *store) create(title string) Task {
    s.mu.Lock()
    defer s.mu.Unlock()
    t := Task{ID: s.nextID, Title: title}
    s.tasks[t.ID] = t
    s.nextID++
    return t
}

func (s *store) get(id int) (Task, bool) {
    s.mu.Lock()
    defer s.mu.Unlock()
    t, ok := s.tasks[id]
    return t, ok
}

func (s *store) remove(id int) bool {
    s.mu.Lock()
    defer s.mu.Unlock()
    if _, ok := s.tasks[id]; !ok {
        return false
    }
    delete(s.tasks, id)
    return true
}

```

Watch out. `encoding/json` can only see **exported** fields — those that start with a capital letter (see Chapter 3 — Program Structure: Packages, Imports, and Visibility). A lowercase field is silently skipped on encode and decode. The `json:"id"` tag sets the JSON key name; without a tag the key would be the Go field name `ID`.

Concurrency: handlers run in parallel

This is the point a C programmer must internalize first:

Watch out. The Go server runs **each request in its own goroutine**. Two requests can run your handler at the *same time* on different CPU cores. Any shared state — here the `tasks` map and the `nextID` counter — must be protected. A Go map is **not** safe for concurrent writes; an unprotected one triggers a fatal “concurrent map writes” crash, not a silent corruption.

That is why every store method takes `s.mu.Lock()` first. The `sync.Mutex` (see Chapter 15 — Synchronization and context) makes each operation atomic. In C with `pthread`s you would reach for a `pthread_mutex_t` for exactly the same reason; the hazard is identical, only the spelling differs.

JSON helpers

Two small helpers centralize the response format:

```
// writeJSON sets the header, then the status, then the body. Order matters:  
// once the body is written the status code is locked in.
```

```
func writeJSON(w http.ResponseWriter, status int, v any) {  
    w.Header().Set("Content-Type", "application/json")  
    w.WriteHeader(status)  
    _ = json.NewEncoder(w).Encode(v)  
}
```

```
func writeError(w http.ResponseWriter, status int, msg string) {  
    writeJSON(w, status, map[string]string{"error": msg})  
}
```

`json.NewEncoder(w).Encode(v)` writes `v` as JSON straight to the response, with no intermediate buffer. The decode side is the mirror image: `json.NewDecoder(r.Body).Decode(&in)` reads JSON from the request body into a struct.

Watch out. A nil slice encodes to JSON null, not `[]`. The `list` method returns `make([]Task, 0, ...)`, so an empty list serializes to `[]`, which is what clients expect.

The handlers

```
func (s *store) handleList(w http.ResponseWriter, r *http.Request) {  
    writeJSON(w, http.StatusOK, s.list())  
}  
  
func (s *store) handleCreate(w http.ResponseWriter, r *http.Request) {  
    var in struct {  
        Title string `json:"title"`  
    }  
    if err := json.NewDecoder(r.Body).Decode(&in); err != nil {  
        writeError(w, http.StatusBadRequest, "invalid JSON body")  
        return  
    }  
    if in.Title == "" {  
        writeError(w, http.StatusBadRequest, "title is required")  
        return  
    }
```

```

    }
    writeJSON(w, http.StatusCreated, s.create(in.Title))
}

func (s *store) handleGet(w http.ResponseWriter, r *http.Request) {
    id, err := strconv.Atoi(r.PathValue("id")) // the {id} wildcard, as a string
    if err != nil {
        writeError(w, http.StatusBadRequest, "id must be a number")
        return
    }
    t, ok := s.get(id)
    if !ok {
        writeError(w, http.StatusNotFound, "no such task")
        return
    }
    writeJSON(w, http.StatusOK, t)
}

func (s *store) handleDelete(w http.ResponseWriter, r *http.Request) {
    id, err := strconv.Atoi(r.PathValue("id"))
    if err != nil {
        writeError(w, http.StatusBadRequest, "id must be a number")
        return
    }
    if !s.remove(id) {
        writeError(w, http.StatusNotFound, "no such task")
        return
    }
    w.WriteHeader(http.StatusNoContent) // 204: success, no body
}

```

Notice the shape of every handler: validate the input, call a store method, then write a response. Errors return early with a JSON body and the right status. `strconv.Atoi` converts the `{id}` text to an `int` (see Chapter 20 — A Tour of the Standard Library). The `main` function that wires the routes and runs the server comes after middleware and shutdown, below.

Middleware

A **middleware** is a function that takes a `Handler` and returns a new `Handler` wrapping it. The signature is always the same:

```
func(http.Handler) http.Handler
```

The wrapper does work before and/or after, then calls the inner handler's `ServeHTTP`. Here is request logging:

```

// logging is middleware. It wraps a handler, does work before and after, and
// calls the next handler in between. The signature func(http.Handler)
// http.Handler is what lets you chain many middlewares together.
func logging(next http.Handler) http.Handler {

```

```

    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        next.ServeHTTP(w, r)
        log.Printf("%s %s took %s", r.Method, r.URL.Path, time.Since(start))
    })
}

```

You apply it by wrapping the mux. Because each middleware has the same input and output type, you **chain** them like the layers of an onion:

```

handler := logging(auth(rateLimit(mux)))
// request flows in this order:
//   logging -> auth -> rateLimit -> mux -> your handler
// and the responses unwind back out the same way.

```

Mental model. Middleware is Go’s version of a “filter chain.” In C you might push a request through an array of function pointers; here each layer is a `Handler` that holds the next one and calls it.

Rule of thumb. Keep each middleware to one concern — logging, authentication, compression, recovering from a panic — and compose them. Do not write one giant wrapper that does everything.

Context and timeouts

Every request carries a **context** (`context.Context`; see Chapter 15 — Synchronization and context). Get it with `r.Context()`. The server cancels this context when the client disconnects or the request ends. Use it to stop slow work — a database query, an outbound call — once the caller has gone away. This standalone handler (it would need its own `import "fmt"`) shows the idea:

```

// slow does some work but gives up early if the client disconnects.
func slow(w http.ResponseWriter, r *http.Request) {
    ctx := r.Context() // cancelled when the client goes away
    select {
    case <-time.After(2 * time.Second): // pretend this is a slow query
        fmt.Fprintln(w, "done")
    case <-ctx.Done(): // the client disconnected; stop working
        return
    }
}

```

Separately, set **timeouts on the server itself** so a slow or malicious client cannot tie up resources forever. You do this by building an `http.Server` value instead of calling the bare `http.ListenAndServe`:

```

srv := &http.Server{
    Addr:      ":8080",
    Handler:   logging(mux),
    ReadTimeout: 5 * time.Second,
    WriteTimeout: 10 * time.Second,
    IdleTimeout: 120 * time.Second,
}

```

```
}
```

Field	Limits	Protects against
ReadTimeout	time to read the whole request	clients that send very slowly
WriteTimeout	time to write the whole response	clients that read very slowly
IdleTimeout	time a keep-alive connection may sit idle	idle connections piling up

Watch out. A bare `http.ListenAndServe(addr, h)` has **no timeouts at all**. A single client that opens a connection and sends one byte per minute (a “slowloris” attack) can exhaust your server. Always build an `http.Server{}` with explicit timeouts for anything exposed to a network.

Graceful shutdown

When you deploy a new version, or a container is told to stop, you want in-flight requests to finish rather than be cut off mid-response. The pattern: catch the signal, then call `srv.Shutdown(ctx)`.

```
func main() {
    s := newStore()

    // Go 1.22+ routing: each pattern carries a method and may use {wildcards}.
    mux := http.NewServeMux()
    mux.HandleFunc("GET /tasks", s.handleList)
    mux.HandleFunc("POST /tasks", s.handleCreate)
    mux.HandleFunc("GET /tasks/{id}", s.handleGet)
    mux.HandleFunc("DELETE /tasks/{id}", s.handleDelete)

    srv := &http.Server{
        Addr:         ":8080",
        Handler:       logging(mux), // wrap the whole mux in one middleware
        ReadTimeout:  5 * time.Second,
        WriteTimeout: 10 * time.Second,
        IdleTimeout:  120 * time.Second,
    }

    // NotifyContext cancels ctx when the process gets SIGINT (Ctrl-C) or SIGTERM.
    ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
    defer stop()

    // Run the server in its own goroutine so main can wait for the signal.
    go func() {
        log.Printf("listening on %s", srv.Addr)
        if err := srv.ListenAndServe(); err != nil && !errors.Is(err, http.ErrServerClosed) {
            log.Fatalf("server error: %v", err)
        }
    }()
}
```

```

<-ctx.Done() // block here until a signal arrives
log.Println("shutting down")

// Give in-flight requests up to 10 seconds to finish, then stop.
shutdownCtx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
defer cancel()
if err := srv.Shutdown(shutdownCtx); err != nil {
    log.Fatalf("forced shutdown: %v", err)
}
log.Println("stopped cleanly")
}

```

`signal.NotifyContext` returns a context that is cancelled on `Ctrl-C` (`SIGINT`) or `SIGTERM` (the signal a container runtime or `systemd` sends). The server runs in a goroutine; `main` waits on `ctx.Done()`. `srv.Shutdown` then stops accepting new connections and waits for active handlers to return, up to the deadline you pass.

Watch out. `ListenAndServe` returns `http.ErrServerClosed` after a clean Shutdown. That is the *normal* path, not a failure. Check for it with `errors.Is` (see Chapter 12 — Errors) so you do not log a phantom error.

The HTTP client

`net/http` is also a client. The quick form is one call:

```

resp, err := http.Get("http://localhost:8080/tasks")
if err != nil {
    log.Fatal(err)
}
defer resp.Body.Close() // REQUIRED: always close the body
body, _ := io.ReadAll(resp.Body)

```

Watch out. Two traps live on the client side:

1. **Always defer `resp.Body.Close()`.** If you forget, the underlying TCP connection cannot be reused or freed — a slow leak that only bites under load.
2. **`http.Get`, `http.Post`, and `http.DefaultClient` have NO timeout.** A hung server will hang your program forever. For anything real, build your own `http.Client` with a `Timeout`.

The production form gives you a timeout and a context:

```

// Command taskget fetches the task list from the server and prints it.
package main

import (
    "context"
    "fmt"
    "io"
    "log"

```

```

    "net/http"
    "time"
)

func main() {
    // Always give a client a timeout. http.DefaultClient (used by http.Get and
    // http.Post) has NO timeout, so a slow server can hang your program forever.
    client := &http.Client{Timeout: 10 * time.Second}

    // A context lets you cancel the request from elsewhere: a deadline, Ctrl-C,
    // or a parent request that was itself cancelled.
    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()

    req, err := http.NewRequestWithContext(ctx, http.MethodGet, "http://localhost:8080/tasks",
    if err != nil {
        log.Fatal(err)
    }
    req.Header.Set("Accept", "application/json")

    resp, err := client.Do(req)
    if err != nil {
        log.Fatal(err)
    }
    defer resp.Body.Close() // ALWAYS close the body, or you leak connections.

    body, err := io.ReadAll(resp.Body)
    if err != nil {
        log.Fatal(err)
    }
    fmt.Printf("status %d\n%s", resp.StatusCode, body)
}

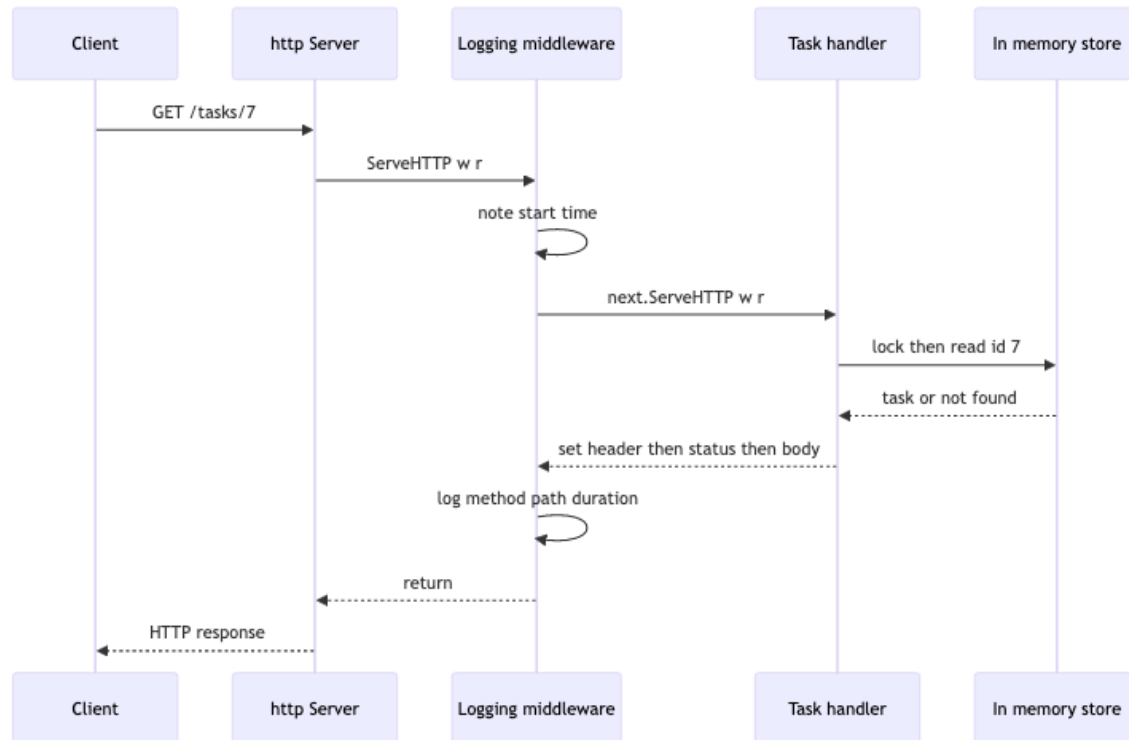
```

`http.NewRequestWithContext` ties the request to a context, so a deadline or a cancellation stops it. `client.Do(req)` sends it. The `Timeout` on the client is a backstop covering the whole exchange: connect, send, and receive.

Rule of thumb. Create one `http.Client` and reuse it across requests. It pools and reuses TCP connections under the hood. A fresh client per request defeats that pool.

How a request flows

This is the lifecycle of one `GET /tasks/7` request as it passes through the logging middleware to the handler and back.



Key takeaways

- `net/http` is a production-grade server **and** client in the standard library. You can ship a real service with zero dependencies.
- A **Handler** is a one-method interface (`ServeHTTP`). `http.HandlerFunc` adapts a plain function into one. A `ServeMux` routes to handlers.
- Use Go 1.22 routing: patterns carry a method and {wildcards}, e.g. `"GET /tasks/{id}"`, and you read them with `r.PathValue("id")`.
- Send the reply in order: headers, then `WriteHeader(status)`, then body. After the body starts, status and headers are fixed.
- Encode and decode JSON with `json.NewEncoder(w)` and `json.NewDecoder(r.Body)`. Only exported fields are included; tags name the keys.
- Handlers run concurrently — one goroutine each — so guard shared state with a `sync.Mutex`.
- Middleware is `func(http.Handler) http.Handler`; chain layers to compose behavior.
- Set `ReadTimeout`, `WriteTimeout`, and `IdleTimeout` on an `http.Server`, and shut down cleanly with `signal.NotifyContext` plus `srv.Shutdown`.
- On the client, always defer `resp.Body.Close()` and always set a `Timeout`.

Watch out (gotchas for C programmers)

- **Handlers run concurrently.** Each request gets its own goroutine; unguarded shared state races, and a concurrent map write is a hard crash, not a warning.
- **Order of writes is fixed.** Setting a header or status *after* writing body bytes silently does nothing.
- **Only exported struct fields are serialized.** A lowercase field vanishes from JSON; use exported fields with `json:"..."` tags.

- **A nil slice marshals to null, not []**. Pre-allocate an empty slice when a client expects an empty array.
- **http.DefaultClient and http.Get have no timeout**. Use a custom `http.Client{Timeout: ...}` for anything real.
- **Not closing resp.Body leaks connections**. Always defer `resp.Body.Close()` after checking the error.
- **A bare ListenAndServe has no server timeouts**, leaving you open to slowloris-style attacks. Build an `http.Server` with timeouts set.
- **http.ErrServerClosed is normal** after Shutdown; do not treat it as a failure.

Interview questions

Q: What is the http.Handler interface, and how does http.HandlerFunc relate to it? A: `http.Handler` has one method, `ServeHTTP(w http.ResponseWriter, r *http.Request)`. Anything implementing it can answer requests. `http.HandlerFunc` is a function type with a `ServeHTTP` method that just calls the function, so a plain function with the right signature becomes a `Handler` by converting it. This is an adapter from a function to an interface value.

Q: How does Go 1.22 routing differ from older ServeMux usage? A: Patterns can now include an HTTP method and named wildcards, such as `"GET /tasks/{id}"`. The mux matches on both method and path, returns 405 when the path matches but the method does not, and exposes captured segments through `r.PathValue("id")`. Older code matched only path prefixes and parsed the URL by hand.

Q: Why must shared state in handlers be synchronized, and how? A: The server serves each request in its own goroutine, so handlers can run in parallel on multiple cores. Reads and writes of shared data (maps, counters, caches) therefore race. Protect them with a `sync.Mutex` (or `sync.RWMutex`), or confine the state to a single goroutine and communicate over channels. An unprotected concurrent map write crashes the program.

Q: Why must you set headers and the status code before writing the body? A: The response is streamed. The first `Write` flushes the status line and headers (defaulting the status to 200 if you have not set one). After that the status and headers are already sent, so later calls to `Header().Set` or `WriteHeader` have no effect.

Q: Name two client-side mistakes with net/http and how to avoid them. A: First, forgetting `defer resp.Body.Close()`, which leaks the TCP connection; always close the body after checking the error. Second, using `http.Get` or `http.DefaultClient`, which have no timeout, so a stuck server hangs you forever; use a custom `http.Client{Timeout: ...}` and pass a context with `http.NewRequestWithContext`.

Q: How do you shut a Go HTTP server down gracefully? A: Create the listener with `signal.NotifyContext` so a context is cancelled on `SIGINT` or `SIGTERM`. Run `ListenAndServe` in a goroutine. When the context is done, call `srv.Shutdown(ctx)` with a timeout; it stops accepting new connections and waits for in-flight handlers to finish. Treat the `http.ErrServerClosed` returned by `ListenAndServe` as the normal end, not an error.

Try it

1. Save the task API as `main.go`, run `go run .`, and exercise it with curl: `curl -X POST localhost:8080/tasks -d '{"title":"buy milk"}'`, then `curl localhost:8080/tasks`,

- then `curl localhost:8080/tasks/1`, then `curl -X DELETE localhost:8080/tasks/1`.
2. Ask for a missing task (`curl -i localhost:8080/tasks/999`) and confirm you get 404 with a JSON error body.
 3. Send a wrong method (`curl -i -X PUT localhost:8080/tasks/1`) and watch the mux return 405 Method Not Allowed without any code from you.
 4. Press `Ctrl-C` while a request is in flight and watch the “stopped cleanly” log line print after the request completes.

Chapter 24 — Building Command-Line Tools

What you’ll learn. How to write a real command-line program in Go: reading arguments and flags, git-style subcommands, the `stdin/stdout/stderr` discipline, exit codes, config precedence, `Ctrl-C` handling, and how to ship a single static binary.

Go is one of the best languages for command-line tools, and the reasons line up with what a C programmer cares about. A Go program compiles to **one static binary** that starts instantly and runs on a target machine with nothing else installed (see Chapter 1 — Why Go for a C Programmer). That is why so many of the tools you use every day — `kubectl`, `docker`, `gh`, `terraform`, `hugo` — are written in Go.

In C you start from `int main(int argc, char **argv)` and `getopt`, and you build everything else by hand. Go gives you the same low-level access through `os.Args`, plus a small standard-library `flag` package that covers most tools without any dependency. This chapter builds a complete `grep`-like tool and shows the pieces along the way.

Arguments: `os.Args`

`os.Args` is a slice of strings: the program name followed by its arguments. It is Go’s `argv`, and `len(os.Args)` is its `argc`.

```
package main

import (
    "fmt"
    "os"
)

func main() {
    fmt.Println("program:", os.Args[0]) // like argv[0]
    fmt.Println("args:", os.Args[1:])  // the rest, like argv[1..argc-1]
}
```

Concept	C	Go
Argument list	<code>int argc, char **argv</code>	<code>os.Args []string</code>
Program name	<code>argv[0]</code>	<code>os.Args[0]</code>
Argument count	<code>argc</code>	<code>len(os.Args)</code>
Option parsing	<code>getopt, getopt_long</code>	the <code>flag</code> package
Exit with status	<code>exit(2)</code> or <code>return 2</code>	<code>os.Exit(2)</code> or return a code

Concept	C	Go
Standard streams	<code>stdin, stdout, stderr</code>	<code>os.Stdin, os.Stdout, os.Stderr</code>
Read environment	<code>getenv("X")</code>	<code>os.Getenv("X")</code>
Signal handling	<code>signal, sigaction</code>	the <code>os/signal</code> package

Exit codes and the `os.Exit` trap

A C program returns an exit status from `main` or calls `exit(code)`. In Go, `main` returns nothing; you set the status with `os.Exit(code)`. By convention 0 means success and any non-zero value means failure.

But there is a sharp trap:

Watch out. `os.Exit` terminates the process **immediately** and does **not run deferred functions** (see Chapter 6 — Functions). Any `defer file.Close()` or `defer buf.Flush()` is skipped, so buffers may not flush and temp files may not be removed.

```
func main() {
    defer fmt.Println("cleanup") // this line NEVER runs
    fmt.Println("working")
    os.Exit(1) // exits now, skipping every deferred call
}
```

Rule of thumb. Keep `main` tiny: have it call a helper such as `func run() int` that does the work with normal `defers` and `returns`, then `os.Exit(run())`. By the time `os.Exit` fires, every `defer` in `run` has already executed. The `minigrep` program below uses exactly this shape.

The `flag` package

The `flag` package is Go's `getopt`. You declare each flag, call `flag.Parse`, then read the values. Each `flag.String/Int/Bool` returns a **pointer** to the variable it will fill.

```
package main

import (
    "flag"
    "fmt"
    "strings"
)

func main() {
    name := flag.String("name", "world", "who to greet")
    count := flag.Int("count", 1, "how many times to greet")
    shout := flag.Bool("shout", false, "print in uppercase")
    flag.Parse() // parses os.Args[1:]; handles -h and reports bad flags

    msg := "hello, " + *name
    if *shout {
```

```

    msg = strings.ToUpper(msg)
}
for range *count { // integer range, Go 1.22+
    fmt.Println(msg)
}
if extra := flag.Args(); len(extra) > 0 {
    fmt.Println("ignored positional args:", extra)
}
}

go run . -name Ada -count 2 -shout
# HELLO, ADA
# HELLO, ADA
go run . -h # prints auto-generated usage from the descriptions

```

Key points:

- The third argument to each `flag.*` call is the **default**; it is used when the flag is absent.
- `flag.Parse` stops at the first non-flag argument. Call it **before** `flag.Args()`.
- `flag.Args()` returns the leftover **positional** arguments as `[]string`.
- `flag` builds the `-h/--help` usage text for you from the descriptions.

C vs Go. Go's `flag` differs from GNU `getopt_long` in ways that surprise people:

- A flag uses a **single dash** by convention (`-count`), but `flag` accepts a double dash too (`--count`); both mean the same thing.
- You **cannot bundle** short booleans: `-abc` is one flag named `abc`, not `-a -b -c`.
- The value forms `-count 2` and `-count=2` are both fine. Booleans use `-shout` or `-shout=false`.
- Use a bare `--` to mark the end of flags, so later dash-leading words are treated as positional arguments.

The forms the `flag` package accepts, at a glance:

Form	Meaning
<code>-flag</code>	boolean flag, set to true
<code>-flag=false</code>	boolean flag, set to false
<code>-flag value</code>	flag with a value, space-separated
<code>-flag=value</code>	flag with a value, equals sign
<code>--flag</code>	same as <code>-flag</code> ; the double dash is accepted
<code>--</code>	end of flags; everything after is positional
<code>word</code>	a positional argument, read with <code>flag.Args()</code>

Custom flags with `flag.Value`

For a flag that the standard types do not cover — say one that may be **repeated** — implement the `flag.Value` interface. It needs two methods, `String() string` and `Set(string) error`, then you register it with `flag.Var`.

```

// tags collects repeated -tag values. A type satisfies the flag.Value
// interface by providing String() and Set() methods.

```

```

type tags []string

func (t *tags) String() string { return strings.Join(*t, ",") }
func (t *tags) Set(v string) error { *t = append(*t, v); return nil }

func main() {
    var tagList tags
    flag.Var(&tagList, "tag", "add a tag; may be repeated")
    flag.Parse()
    fmt.Printf("tags = %v\n", []string(tagList)) // -tag a -tag b -> tags = [a b]
}

```

flag calls Set once for each -tag on the command line, so the slice accumulates every value.

Subcommands (git-style)

Tools like git commit and go build use **subcommands**: the first argument selects a mode, and each mode has its own flags. The flag package supports this with flag.NewFlagSet, which creates an independent flag set you parse yourself. You switch on os.Args[1] and parse the remaining arguments.

```

// A git-style CLI with subcommands: `tool add ...` and `tool list ...`.
// Each subcommand has its own independent set of flags.
package main

import (
    "flag"
    "fmt"
    "os"
)

func main() {
    if len(os.Args) < 2 {
        fmt.Fprintln(os.Stderr, "usage: tool <add|list> [flags]")
        os.Exit(2)
    }

    switch os.Args[1] { // dispatch on the first argument
    case "add":
        fs := flag.NewFlagSet("add", flag.ExitOnError)
        priority := fs.Int("priority", 1, "task priority")
        _ = fs.Parse(os.Args[2:]) // parse only the args AFTER the subcommand
        fmt.Printf("add: priority=%d items=%v\n", *priority, fs.Args())
    case "list":
        fs := flag.NewFlagSet("list", flag.ExitOnError)
        all := fs.Bool("all", false, "include finished tasks")
        _ = fs.Parse(os.Args[2:])
        fmt.Printf("list: all=%v\n", *all)
    default:

```

```

        fmt.Fprintf(os.Stderr, "tool: unknown subcommand %q\n", os.Args[1])
        os.Exit(2)
    }
}

go run . add -priority 5 milk bread    # add: priority=5 items=[milk bread]
go run . list -all                     # list: all=true

```

The flag set is parsed with `flag.ExitOnError`, so a bad flag prints usage and exits. Each subcommand sees only `os.Args[2:]`, so `add` and `list` can reuse the same flag names without clashing.

Mental model. Think of the default flag functions (`flag.Int`, and so on) as one built-in flag set named after the program. `flag.NewFlagSet` makes more of them, one per subcommand.

Reading standard input

A good CLI tool works in a pipeline. Read from `os.Stdin` so users can pipe data in, and fall back to it when no file is named. The easiest reader is `bufio.Scanner`, which splits the stream into lines.

```

sc := bufio.NewScanner(os.Stdin)
for sc.Scan() {
    line := sc.Text() // the current line, without the trailing newline
    fmt.Println(strings.ToUpper(line))
}
if err := sc.Err(); err != nil { // Scan returns false on EOF and on error
    fmt.Fprintln(os.Stderr, "read error:", err)
}

echo "hello" | go run .    # piping works because we read os.Stdin

```

Keep stdout and stderr separate

This discipline matters more in Go pipelines than many C programmers expect:

- **Real output** (the data) goes to **stdout** (`os.Stdout`).
- **Diagnostics** — logs, warnings, errors, progress — go to **stderr** (`os.Stderr`).

That way a user can pipe your data into another tool while still seeing errors on the terminal, and `2>/dev/null` hides only the noise.

```

fmt.Fprintln(os.Stdout, result)           // data
fmt.Fprintln(os.Stderr, "warn:", note)    // diagnostics

```

For high-volume output, wrap stdout in a `bufio.Writer` so you do not make a syscall per line. You **must flush** it, or buffered bytes are lost.

```

out := bufio.NewWriter(os.Stdout)
defer out.Flush() // flush the buffer before the program ends
for _, line := range lines {
    fmt.Fprintln(out, line)
}

```

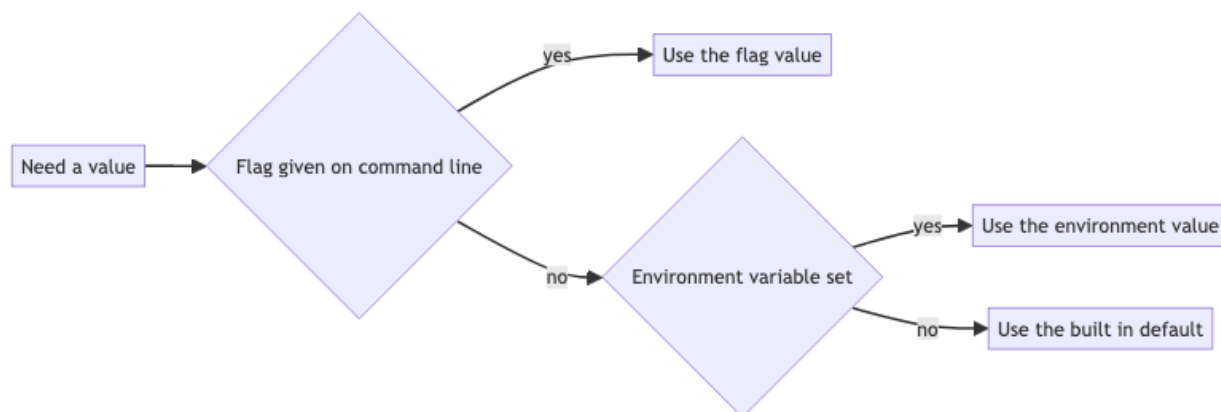
Watch out. A buffered writer plus `os.Exit` is a classic data-loss bug: `os.Exit` skips the deferred `Flush`, so the last buffered lines never print. Flush explicitly before exiting, or use the `run() int` pattern.

Configuration precedence

Tools usually accept settings from several places. The common, predictable order is **flags beat environment variables, which beat built-in defaults**.

// precedence: an explicit flag wins, then the environment, then the default.

```
func resolve(flagVal, envKey, def string) string {
    if flagVal != "" {
        return flagVal
    }
    if v := os.Getenv(envKey); v != "" {
        return v
    }
    return def
}
```



Rule of thumb. Document the precedence in your `-h` output. Users must know whether `MYTOOL_HOST` or `-host` wins when both are set.

Handling Ctrl-C

To clean up on `Ctrl-C`, listen for the interrupt signal. The simplest modern way is `signal.NotifyContext`, which returns a context (see Chapter 15 — Synchronization and context) that is cancelled when the signal arrives.

```
ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt)
defer stop() // release the signal handler when done
```

// Pass ctx into long-running work; it is cancelled on the first Ctrl-C.

```
<-ctx.Done()
```

```
fmt.Fprintln(os.Stderr, "interrupted, cleaning up")
```

C vs Go. In C you install a handler with `signal()` or `sigaction()` and set a volatile `sig_atomic_t` flag from it. Go delivers signals as values on a channel, and

`signal.NotifyContext` wraps that into a context you can pass through your call tree — no global flag, no `async-signal-safety` worries.

A complete tool: `minigrep`

Here is the whole tool. It prints lines that match a pattern, reads the files named on the command line (or standard input when none are given), and follows `grep`'s exit-code convention: 0 if a line matched, 1 if none, 2 on error.

```
// Command minigrep prints lines that match a pattern. It reads the files named
// on the command line, or standard input when no files are given. It follows
// grep's exit-code convention: 0 if a line matched, 1 if none, 2 on error.
package main

import (
    "bufio"
    "flag"
    "fmt"
    "io"
    "os"
    "regexp"
)

func main() {
    os.Exit(run())
}

// run holds the real work and returns an exit code. main stays tiny so os.Exit
// (which does NOT run deferred functions) is called in exactly one place, after
// every defer inside run has already executed.
func run() int {
    ignoreCase := flag.Bool("i", false, "ignore case")
    number := flag.Bool("n", false, "print line numbers")
    invert := flag.Bool("v", false, "print lines that do NOT match")
    flag.Usage = func() {
        fmt.Fprintln(os.Stderr, "usage: minigrep [-i] [-n] [-v] PATTERN [FILE...]")
        flag.PrintDefaults()
    }
    flag.Parse() // must be called before flag.Args()

    args := flag.Args() // the positional arguments left after the flags
    if len(args) < 1 {
        flag.Usage()
        return 2
    }

    pattern := args[0]
    if *ignoreCase {
```

```

    pattern = "(?i)" + pattern
}
re, err := regexp.Compile(pattern)
if err != nil {
    fmt.Fprintf(os.Stderr, "minigrep: bad pattern: %v\n", err)
    return 2
}

// Buffer stdout for speed and flush exactly once, at the end.
out := bufio.NewWriter(os.Stdout)
defer out.Flush()

files := args[1:]
matched := false

if len(files) == 0 {
    matched = grep(re, *invert, *number, os.Stdin, "", out)
} else {
    for _, name := range files {
        f, err := os.Open(name)
        if err != nil {
            fmt.Fprintf(os.Stderr, "minigrep: %v\n", err) // errors -> stderr
            continue
        }
        prefix := ""
        if len(files) > 1 {
            prefix = name + ":" // show the filename when scanning many files
        }
        if grep(re, *invert, *number, f, prefix, out) {
            matched = true
        }
        f.Close()
    }
}

if matched {
    return 0
}
return 1
}

func grep(re *regexp.Regexp, invert bool, number bool, r io.Reader, prefix string, out *bufio.Writer) bool {
    matched := false
    sc := bufio.NewScanner(r) // Scanner splits into lines by default
    line := 0
    for sc.Scan() {
        line++
        // `== invert` flips the test when -v is set.

```



```

    if re.MatchString(sc.Text()) == invert {
        continue
    }
    matched = true
    if number {
        fmt.Fprintf(out, "%s%d:%s\n", prefix, line, sc.Text()) // data -> stdout
    } else {
        fmt.Fprintf(out, "%s%s\n", prefix, sc.Text())
    }
}
if err := sc.Err(); err != nil {
    fmt.Fprintf(os.Stderr, "minigrep: read error: %v\n", err)
}
return matched
}

```

Try it:

```

go build -o minigrep .
printf 'apple\nbanana\ncherry\n' | ./minigrep -n an    # 2:banana
./minigrep -v an fruit.txt                            # lines without "an"
echo $?                                                # 0 if matched, else 1

```

This one small program exercises everything in the chapter: flags, positional arguments via `flag.Args()`, stdin-or-file input, the stdout/stderr split, buffered output with a flushed `defer`, meaningful exit codes, and the `os.Exit(run())` pattern that keeps `defer` working.

Popular libraries (and when you need them)

The standard `flag` package is enough for most tools. Reach for a framework only when you have many subcommands and want generated help, shell completion, and GNU-style long options.

Library	Use it for	Notable users
spf13/cobra	rich nested subcommands, help, shell completion	kubect!, docker, gh, hugo
spf13/viper	config files, env, and remote config merged together	often paired with cobra
spf13/pflag	GNU-style <code>--long</code> flags; near drop-in for <code>flag</code>	the cobra stack
urfave/cli	a lighter command framework	many smaller tools

The spf13 trio (cobra + viper + pflag) is the de facto stack behind the big cloud-native tools. urfave/cli is a simpler alternative.

Rule of thumb. Start with the standard `flag` package. Move to cobra when you find yourself hand-rolling a large subcommand dispatcher, nested commands, or completion scripts — not before. Extra dependencies are a cost.

Distributing your tool

This is where Go shines for CLIs. A pure-Go program builds into one static binary with no runtime to install (see Chapter 1 — Why Go for a C Programmer).

```
go build -o minigrep .           # one binary in the current dir
GOOS=linux GOARCH=arm64 go build -o minigrep . # cross-compile; no cross-toolchain
CGO_ENABLED=0 go build -o minigrep .         # force a fully static binary
go install example.com/cmd/minigrep@latest   # build and drop it into $GOBIN
```

- **Cross-compilation** needs only two environment variables, `GOOS` and `GOARCH`. You can build a Linux/arm64 binary on a Mac with no extra tools.
- `go install` compiles a program and places it in your `GOBIN` directory (by default `~/go/bin`), like a package manager that builds from source. Add that directory to your `PATH` (see Chapter 18 — Packages and Modules).
- Pure-Go binaries are static by default. Importing C through `cgo` can pull in a dynamic `libc`; `CGO_ENABLED=0` keeps the binary fully static.

Key takeaways

- `os.Args` is Go's `argv`; `os.Args[0]` is the program name and `len(os.Args)` is `argc`.
- Set the exit status with `os.Exit(code)`. `os.Exit` does **not** run deferred functions, so keep `main` tiny and use the `os.Exit(run())` pattern.
- The `flag` package covers most needs: `flag.String/Int/Bool`, `flag.Parse`, positional args via `flag.Args()`, auto usage, and custom `flag.Value` types.
- Build git-style subcommands with `flag.NewFlagSet` per command, switching on `os.Args[1]` and parsing `os.Args[2:]`.
- Read `os.Stdin` with `bufio.Scanner` so your tool works in pipelines. Send data to `stdout` and diagnostics to `stderr`; flush a buffered writer before exit.
- A common config precedence is flags, then environment variables, then defaults.
- Handle Ctrl-C with `signal.NotifyContext`.
- Ship one static binary; cross-compile with `GOOS/GOARCH`; install with `go install`. Use `cobra`/`viper` only when the `stdlib` `flag` is not enough.

Watch out (gotchas for C programmers)

- **`os.Exit` skips deferred functions.** Cleanup and buffer flushes do not run. Return from `main`, or call `os.Exit(run())` so all defers fire first.
- **Call `flag.Parse()` before `flag.Args()`.** Positional arguments are empty until parsing happens.
- **Do not mix data and logs on `stdout`.** Diagnostics belong on `stderr` so output stays pipeable.
- **Flush buffered output.** A `bufio.Writer` loses its last bytes if you forget to `Flush`, especially when combined with `os.Exit`.
- **`flag` is not `getopt`.** No bundling of short flags (`-abc` is one flag), and long flags use a single dash by convention (though `--` is accepted).
- **Use meaningful exit codes.** 0 for success and distinct non-zero codes for distinct failures; scripts depend on them.

Interview questions

Q: Why is `os.Exit` dangerous, and what is the idiomatic way to set an exit code? A: `os.Exit` ends the process immediately and skips all deferred functions, so file closes and buffer flushes never run. The idiomatic fix is to keep `main` trivial and put the logic in a helper like `func run() int` that uses normal `defer` and `return`, then call `os.Exit(run())`. Every `defer` runs before the process exits.

Q: How do you parse flags and read positional arguments with the standard library? A: Declare flags with `flag.String`, `flag.Int`, or `flag.Bool` (each returns a pointer), set `flag.Usage` if you want custom help, then call `flag.Parse()`. After parsing, `flag.Args()` returns the leftover non-flag arguments. You must call `flag.Parse()` before `flag.Args()`.

Q: How do you implement git-style subcommands in Go without a framework? A: Create a separate `flag.FlagSet` per subcommand with `flag.NewFlagSet`. Switch on `os.Args[1]` to choose the subcommand, then call `fs.Parse(os.Args[2:])` to parse just that command's flags. Each set has its own flags and usage, so names do not collide between subcommands.

Q: Why separate `stdout` and `stderr`, and how does Go express them? A: `Stdout` carries the program's real output; `stderr` carries diagnostics (logs, warnings, errors, progress). Keeping them apart lets users pipe data downstream while still seeing errors, and lets them silence noise with `2>/dev/null`. In Go they are `os.Stdout` and `os.Stderr`, both `io.Writers` you target with `fmt.Fprintln` or a `bufio.Writer`.

Q: When should you use `cobra/viper` instead of the standard `flag` package? A: Use `flag` for most tools; it has no dependencies and handles single commands and simple subcommands well. Move to `cobra` (often with `pflag` and `viper`) when you need many nested subcommands, generated help and shell completion, GNU-style long options, or layered configuration from files and the environment. Adding those libraries is a real cost, so do it only when the `stdlib` stops paying off.

Try it

1. Build `minigrep` and confirm the exit codes: `./minigrep apple fruit.txt; echo $?` (expect 0) versus `./minigrep zzz fruit.txt; echo $?` (expect 1).
2. Pipe data in with no file argument: `cat /etc/hosts | ./minigrep -n local`. Confirm it reads `stdin` and prints line numbers.
3. Redirect only the data: `./minigrep -n local /etc/hosts > out.txt`. Confirm diagnostics for a missing file still appear on the terminal: `./minigrep x nope.txt 2>/dev/null`.

Chapter 25 — Idioms and Style

What you'll learn. How Go code *should* be written: one canonical format, Go's naming rules, the house style for errors, the “accept interfaces, return concrete types” rule, useful zero values, the comma-ok idiom, constructors, the functional options pattern, composition over inheritance, and the small habits that make Go code read the same everywhere.

In C, every team invents its own style: where braces go, tabs versus spaces, how to name things, when to use `goto`. Code reviews argue about it for years. Go ends those arguments on purpose. The language ships with one formatter, one set of naming rules, and a small list of community

conventions that almost everyone follows. This chapter is that list. Learning it is how you stop writing “C in Go” and start writing code a Go team will recognize as their own.

The gofmt culture: one canonical format

Go comes with a tool, `gofmt`, that rewrites your source into the one official format. Indentation is **tabs**. Braces go on the **same line**. Spacing, alignment, and import order are all fixed. You do not choose; the tool decides.

```
gofmt -w .      # format every .go file in this tree, in place
go fmt ./...    # the same thing through the go command
```

C vs Go. In C you pick a style (K&R, Allman, GNU) and argue about it. In Go there is exactly one style and `gofmt` enforces it. Nobody reviews formatting, because it cannot vary. This sounds small; on a team it is huge.

The payoff is that *all* Go code looks alike. When you open an unfamiliar file, the shape is already familiar, so you spend your attention on the logic. Editors run `gofmt` on save. Continuous-integration pipelines reject unformatted code. Treat the format as fixed, like the rules of arithmetic. (More on the toolchain in Chapter 22 — Tooling.)

Naming

`gofmt` fixes layout, but it cannot choose your names. Go’s naming rules are short and strict, and they differ from common C habits.

MixedCaps, never snake_case

Go uses **MixedCaps** (also called camelCase) for multi-word names, not underscores. This is a strong convention, not a tool rule, but every Go codebase follows it.

```
// Go style
var maxRetries int
func parseConfig() {}

// not Go style (C habit)
var max_retries int
func parse_config() {}
```

Capitalization is visibility, not decoration

Recall from Chapter 3 — Program Structure: Packages, Imports, and Visibility: an **uppercase first letter exports** a name (other packages can see it); a lowercase first letter keeps it **unexported** (package-private). So the case of the first letter is *meaning*, not taste. Name a thing lowercase unless other packages truly need it; a small exported surface is easier to maintain.

Acronyms stay all one case

Keep initialisms like URL, ID, HTTP, API, and SQL in a single case — all caps when exported, all lowercase when not. Do not write `Url` or `Http`.

```

type URL struct{ /* ... */ }    // not Url
func parseURL(s string) {}      // not parseUrl
var userID int                  // not userId
func ServeHTTP() {}             // not ServeHttp

```

Getters have no Get prefix

A method that returns a field is named after the field, with no **Get**. Use **Set** for the setter, because a setter *does* something.

```

type Person struct{ name string }

func (p *Person) Name() string    { return p.name } // not GetName
func (p *Person) SetName(n string) { p.name = n }   // Set is fine

```

Avoid stutter

The package name is already part of every name you use from it, so do not repeat it. Callers write `http.Server`, so the type is **Server**, not **HTTPServer**.

You want	Stutters (avoid)	Idiomatic
an HTTP server	<code>http.HTTPServer</code>	<code>http.Server</code>
a byte buffer	<code>bytes.BytesBuffer</code>	<code>bytes.Buffer</code>
make an error	<code>errors.NewError</code>	<code>errors.New</code>

Short receiver names

A method’s receiver (the value before the method name) gets a **short** name, usually one or two letters from the type — not **self** or **this**. Use the *same* receiver name on every method of that type.

```

// good: short and consistent
func (s *Server) Start() {}
func (s *Server) Stop() {}

// not Go style
func (self *Server) Start() {} // no self
func (this *Server) Stop() {}  // no this

```

Interfaces often end in -er

A one-method interface is usually named for the method plus **-er**: **Reader** (**Read**), **Writer** (**Write**), **Stringer** (**String**), **Closer** (**Close**). This reads well and signals “this is a small behavior.” (Interfaces are Chapter 11 — Interfaces.)

Error-handling style

Errors in Go are ordinary values you check with `if err != nil` (Chapter 12 — Errors). A few style rules keep error handling clean.

Handle an error once. Either handle it here, or return it to the caller — not both. The most common mistake is to **log it and also return it**: now every layer logs the same failure and the log fills with duplicates.

```
// bad: logs AND returns - the caller will log it again
if err != nil {
    log.Printf("save failed: %v", err)
    return err
}

// good: add context and return; let the top of the stack decide what to do
if err != nil {
    return fmt.Errorf("saving user %d: %w", id, err)
}
```

Add context with %w. `fmt.Errorf` with the `%w` verb *wraps* the original error, so callers can still inspect it with `errors.Is` and `errors.As` while getting a readable chain like `saving user 7: writing row: disk full`.

Error strings are lowercase and have no trailing punctuation. Because your error is often wrapped inside another sentence, a leading capital or a trailing period reads wrong in the middle of a chain.

```
errors.New("file not found")    // good
errors.New("File not found.")   // bad: capital + period
```

Use sentinel and typed errors for errors callers must detect. A *sentinel* is a predefined error value compared with `errors.Is`. A *typed* error is a custom type inspected with `errors.As` to read extra fields.

```
var ErrNotFound = errors.New("not found") // sentinel

type ValidationError struct{ Field string } // typed
func (e *ValidationError) Error() string { return "invalid field: " + e.Field }
```

Accept interfaces, return concrete types

This is one of Go's most quoted rules, and it shapes good API design.

- **Accept interfaces** as function parameters. Then any type with the right methods can be passed in, including a fake for tests. The classic example is the standard library: functions take `io.Reader` or `io.Writer`, not a concrete file.
- **Return concrete types** (usually a struct or `*struct`). The caller gets the full, documented type with all its methods, not a narrow interface that hides them.

A second rule goes with it: **define the interface in the package that *uses* it, not the package that *implements* it**. The consumer knows exactly which methods it needs, so it declares a small interface for just those. The producer simply returns its concrete type and never imports the consumer.

```
// package report (the CONSUMER) defines the small interface it needs.
type Store interface {
```

```

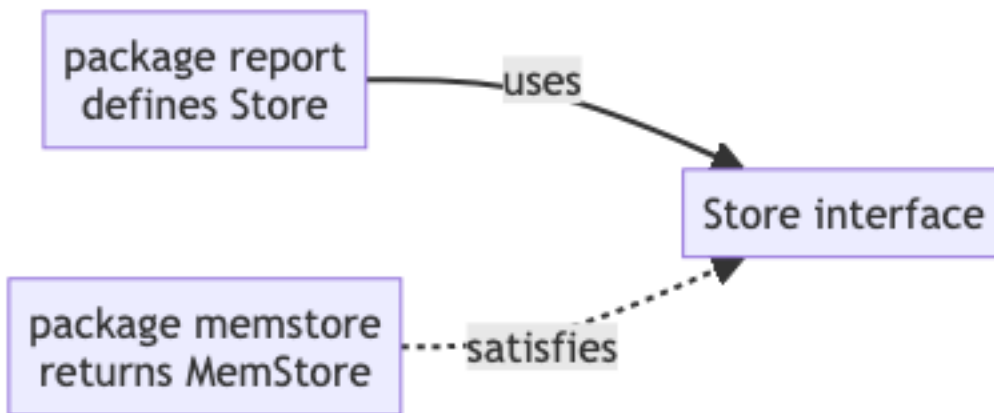
    Get(id string) ([]byte, error) // just the one method report uses
}

func Build(s Store, id string) ([]byte, error) {
    return s.Get(id) // report does not care which concrete Store this is
}

// package memstore (the PRODUCER) returns a CONCRETE type.
func New() *MemStore { return &MemStore{data: map[string][]byte{}} }

```

Rule of thumb. Keep interfaces small: one to three methods. The biggest, most reused interfaces in Go (`io.Reader`, `io.Writer`) have a single method. A large interface is hard to implement and hard to fake in tests.



The arrow of *dependency* points from the consumer to a tiny interface it owns; the producer just happens to satisfy it. No import points back, so packages stay loosely coupled.

Design so the zero value is useful

Every Go variable starts at its **zero value** (`0`, `"`, `false`, `nil`) with no initializer (Chapter 4 — Types, Variables, and Constants). Good Go types are designed so that this zero value is already usable, with no constructor call.

```

package main

import (
    "bytes"
    "fmt"
    "sync"
)

func main() {
    var mu sync.Mutex // zero value is an unlocked, ready mutex
    mu.Lock()
    mu.Unlock()

    var buf bytes.Buffer // zero value is an empty, ready buffer

```

```

buf.WriteString("hello")
fmt.Println(buf.String())

var nums []int // nil slice: len 0, but append and range work fine
nums = append(nums, 1, 2, 3)
fmt.Println(len(nums), nums)
}

```

C vs Go. In C a freshly declared `struct` holds garbage and you must `memset` or initialize it. In Go the zero value is defined and, for well-designed types, immediately useful. A `sync.Mutex`, a `bytes.Buffer`, and a `nil` slice all work out of the box. (One exception: a `nil` map can be *read* but not *written* — see Chapter 26 — Gotchas for C Programmers (the checklist).)

The comma-ok idiom

Several Go operations return a second boolean (`ok`) that says “did this succeed?” This is the **comma-ok** idiom, and you will use it constantly.

```

v, ok := m["key"]    // ok is false if the key is absent (map; Chapter 9 - Maps)
s, ok := x.(string)  // ok is false if x does not hold a string (type assertion)
v, ok := <-ch         // false if the channel is closed (Chapter 14 - Channels and select)

```

Use the two-value form whenever a missing or wrong value is a normal case, not an error. The single-value form (`v := m["key"]`) is fine when the zero value is an acceptable answer.

Constructors: the NewX convention

Go has no special constructor syntax. By convention, a function named `New` or `NewType` builds and returns a value, doing any setup the zero value cannot.

```

type Client struct {
    baseURL string
    timeout time.Duration
}

func NewClient(baseURL string) *Client {
    return &Client{baseURL: baseURL, timeout: 30 * time.Second}
}

```

If a package builds **one** main type, name the function just `New`; callers read `pkg.New()` (for example `list.New()`). If a package builds several types, name each after its type: `NewClient`, `NewServer`. Return a concrete type, as above.

The functional options pattern

Often a constructor needs many optional settings with sensible defaults. C reaches for a big config struct or a long argument list. Go’s idiomatic answer is **functional options**: variadic functions that each set one field. This keeps the common call short, makes every option self-documenting, and lets you add new options later without breaking callers.


```

package main

import (
    "fmt"
    "time"
)

// Server is configured through functional options.
type Server struct {
    addr    string
    port    int
    timeout time.Duration
    maxConn int
}

// Option sets one field of a Server. It is just a function.
type Option func(*Server)

// WithPort returns an Option that sets the port.
func WithPort(p int) Option {
    return func(s *Server) { s.port = p }
}

// WithTimeout returns an Option that sets the timeout.
func WithTimeout(d time.Duration) Option {
    return func(s *Server) { s.timeout = d }
}

// WithMaxConn returns an Option that sets the connection limit.
func WithMaxConn(n int) Option {
    return func(s *Server) { s.maxConn = n }
}

// NewServer applies sensible defaults, then each option in order.
func NewServer(addr string, opts ...Option) *Server {
    s := &Server{
        addr:    addr,
        port:    8080,           // default
        timeout: 30 * time.Second, // default
        maxConn: 100,           // default
    }
    for _, opt := range opts {
        opt(s)
    }
    return s
}

func main() {

```

```

    // The caller sets only what differs from the defaults.
    s := NewServer("localhost", WithPort(9000), WithTimeout(5*time.Second))
    fmt.Printf("%+v\n", *s)
}

```

Rule of thumb. Reach for functional options when a constructor has several optional settings, especially in a library others will call. For two or three required fields, a plain `NewX(a, b)` is simpler — do not over-engineer.

Composition over inheritance

Go has **no classes and no inheritance**. Instead you build a bigger type by **embedding** a smaller one: write a type (or interface) inside a struct with no field name, and its fields and methods are *promoted* to the outer type (Chapter 10 — Structs and Methods).

```

package main

import "fmt"

type Engine struct{ Power int }

func (e Engine) Start() string { return fmt.Sprintf("start %d hp", e.Power) }

// Car embeds Engine. Car gets Engine's fields and methods, no rewriting.
type Car struct {
    Engine // embedded: no field name
    Brand  string
}

func main() {
    c := Car{Engine: Engine{Power: 200}, Brand: "Mazda"}
    fmt.Println(c.Brand, c.Power) // c.Power is promoted from Engine
    fmt.Println(c.Start())        // c.Start() is promoted too
}

```

C vs Go. This is closer to putting one struct inside another in C and reaching through it, but Go *promotes* the inner names so you skip the extra `.Engine` step. It is composition (a Car *has* an Engine), not inheritance (a Car *is* an Engine). There is no virtual override; promoted methods are just forwarded.

Avoid premature abstraction

C programmers often reach for a layer of indirection early. Go culture pushes the other way: write the simple, concrete thing first, and add an interface or generic only when a second real use appears.

Rule of thumb. “A little copying is better than a little dependency.” If you need a five-line helper that lives in another package, copying it can be wiser than importing a whole dependency for it. Fewer dependencies means faster builds and less to break.

Do not add an interface “in case” you swap implementations later. An interface with one implementation is just extra indirection. Add it when the second implementation (or the test fake) actually

arrives.

Libraries return errors; they do not panic

A `panic` unwinds the stack and, if not recovered, crashes the program (Chapter 12 — Errors). In **library** code, do not panic for ordinary failures — return an **error** so the caller decides what to do. Reserve **panic** for truly unrecoverable states or clear programmer bugs (for example, an impossible switch case).

```
// bad in a library: caller cannot recover gracefully
func MustLoad(path string) []byte {
    b, err := os.ReadFile(path)
    if err != nil {
        panic(err)
    }
    return b
}

// good: return the error
func Load(path string) ([]byte, error) {
    return os.ReadFile(path)
}
```

The `MustX` naming (like `regexp.MustCompile`) is the one accepted exception: it panics on error and is meant only for package-level variables and tests, where a failure should stop the program at startup.

Keep concurrency simple and document ownership

Goroutines and channels are powerful but easy to misuse (Chapters 13–16). A few style habits prevent most trouble.

- Start a goroutine only when you can say **who stops it** and **who closes the channel**. Only the sender closes a channel; document that.
- Document **ownership** of shared data: which goroutine may touch which variable, and which mutex guards it. Unclear ownership is the root of most data races.
- Prefer the simplest tool that works: a `sync.Mutex` around shared state is often clearer than an elaborate channel dance.
- Always test concurrent code with the race detector: `go test -race ./...` (Chapter 16 — Concurrency Patterns).

Pass `context.Context` first; never store it

A `context.Context` carries cancellation, deadlines, and request-scoped values (Chapter 15 — Synchronization and context). The convention is firm: it is the **first parameter** of a function, named `ctx`, and you **do not store it in a struct**.

```
// good: ctx is the first parameter
func (c *Client) Fetch(ctx context.Context, url string) ([]byte, error) {
    // ... pass ctx down to the next call
    return nil, nil
}
```

```

}

// bad: storing a context in a struct hides its lifetime and leaks it
type Bad struct {
    ctx context.Context // anti-pattern; pass ctx per call instead
}

```

A context belongs to one operation. Storing it in a long-lived struct keeps a finished request's deadline and values alive, which causes subtle bugs. Pass it through the call chain instead.

Where the rules are written down

You do not have to memorize all of this. Three sources are the canonical references, and Go programmers cite them in code review:

- **Effective Go** — the original guide to idiomatic Go.
- **Go Code Review Comments** — the community checklist of common review feedback (naming, error strings, receivers, and more).
- **Google Go Style Guide** — a thorough, modern style guide used at scale.

Links to all three are in the References section at the end of the book. When in doubt, run `gofmt` and `go vet`, then check these guides.

Key takeaways

- `gofmt` defines one canonical format. Do not argue about layout; let the tool decide and move on.
- Names use MixedCaps, acronyms stay one case (URL, ID, HTTP), getters drop `Get`, types avoid stutter (`http.Server`), and receivers are short (`s`, `c`).
- Handle each error once: add context with `%w` and return it, or handle it — not both. Error strings are lowercase with no trailing punctuation.
- Accept small interfaces (1–3 methods), defined in the consumer package; return concrete types from constructors.
- Design types so the zero value works (`sync.Mutex`, `bytes.Buffer`, nil slice).
- Use comma-ok for maps, type assertions, and channel receives; use `NewX` constructors; use functional options for configurable constructors.
- Prefer composition (embedding) over inheritance; avoid premature abstraction.
- Libraries return errors instead of panicking; concurrency stays simple with clear ownership; `context.Context` is the first parameter and is never stored.

Watch out (gotchas for C programmers)

- **Stutter names.** Do not write `http.HTTPServer` or `errors.NewError`; the package name already qualifies the type.
- **Get prefix on getters.** Use `Name()`, not `GetName()`. Keep `Set` for setters.
- **Overusing interfaces and `any`.** An interface with one implementation, or passing `any` everywhere, throws away Go's type safety. Add abstraction when a second use appears, not before.
- **Ignoring errors.** Do not write `v, _ := f()` to silence a real error, and do not both log and return the same error.

- **Panicking in a library.** Return an `error`; reserve `panic` for unrecoverable bugs and `MustX` startup helpers.
- **Over-engineering.** A little copying beats a little dependency. Simple, concrete code is the Go default.

Interview questions

Q: What does “accept interfaces, return concrete types” mean, and why is it good practice? A: Functions should take interface parameters so callers can pass any type with the right methods (including test fakes), but constructors should return the concrete struct so callers get the full, documented type. It keeps callers flexible and implementations honest. The matching rule is to define the interface in the package that *uses* it, kept small, so dependencies point one way and packages stay loosely coupled.

Q: Why do getters in Go not start with `Get`, and what about setters? A: Go convention names a getter after the value it returns, so a field `name` is read by a method `Name()`. The `Get` prefix adds noise and reads worse at the call site (`p.Name()` versus `p.GetName()`). Setters keep a `Set` prefix (`SetName`) because a setter performs an action rather than just naming a value.

Q: Describe the functional options pattern and when to use it. A: You define a type `Option func(*T)` and provide `WithX` functions that each return an `Option` setting one field. The constructor takes `opts ...Option`, applies defaults, then runs each option. It gives short calls, self-documenting settings, and lets a library add options later without breaking callers. Use it when a constructor has several optional settings; for a couple of required fields, a plain `NewX(a, b)` is simpler.

Q: How should errors be wrapped and worded in idiomatic Go? A: Add context with `fmt.Errorf("doing X: %w", err)`, which wraps the original so callers can still use `errors.Is` and `errors.As`. Handle an error once — either return it (with context) or handle it, never both, and never log-and-return. Error strings are lowercase and have no trailing punctuation, because they are often embedded in a larger message.

Q: Why does “make the zero value useful” matter, and give examples? A: Because every Go variable starts at its zero value, a type whose zero value is already usable needs no constructor and is harder to misuse. `sync.Mutex` is an unlocked, ready mutex; `bytes.Buffer` is an empty, ready buffer; a `nil` slice supports `len`, `range`, and `append`. Designing for a useful zero value makes APIs simpler and safer.

Try it

1. Take a struct you would normally configure with many constructor arguments and rewrite it using the functional options pattern. Add a `WithTimeout` option and confirm old call sites that do not pass it still compile and use the default.
2. Run `go vet ./...` on a small package, then read three entries from the Go Code Review Comments guide (see References) and fix any names that stutter or use a `Get` prefix.

Chapter 26 — Gotchas for C Programmers (the checklist)

What you’ll learn. The traps that bite C programmers most often when they write Go, gathered into one checklist. For each trap you get: what it is, why it surprises you

coming from C, and the fix with a tiny snippet — plus a pointer to the chapter that teaches it in full.

This is the chapter to keep open while you write your first Go. Most of these are not bugs in Go; they are places where Go's rules differ from C's, so your C instinct points the wrong way. Read it once now, then come back to it whenever something behaves in a way you did not expect. Each item is short on purpose.

Arrays, slices, and strings

These are the single biggest source of surprises. The full story is in Chapter 8 — Arrays, Slices, and Strings.

Arrays are value types

In C, an array name *decays* to a pointer to its first element, so passing an array to a function passes a pointer — the function can change the original. In Go, an array is a **value**. Assigning it, or passing it to a function, **copies the whole array**.

```
a := [3]int{1, 2, 3}
b := a    // copies all three elements
b[0] = 99
// a is still {1, 2, 3}; b is {99, 2, 3}
```

Watch out. A `[1000000]int` passed by value copies a megabyte every call. In Go you almost never use arrays directly. Use a **slice** (`[]int`), which is a small header (pointer, length, capacity) and is cheap to pass.

Slice aliasing: sub-slices share the backing array

A slice is a view into a backing array. Two slices can point at the **same** array. Slicing (`s[1:3]`) does not copy; it makes another view. So writing through one slice can change another.

```
s := []int{1, 2, 3, 4}
t := s[:2]           // t shares s's backing array
t = append(t, 99)    // there is spare capacity, so this writes into s[2]
// s is now {1, 2, 99, 4} - t's append overwrote a neighbor
```

<code>s := []int{1,2,3,4}</code>	backing array:	<code>[1 2 3 4]</code>	<code>len=4 cap=4</code>
<code>t := s[:2]</code>	t	same array <code>[1 2 3 4]</code>	<code>len=2 cap=4</code>
<code>t = append(t, 99)</code>	fits in cap, so it writes index 2:		
		<code>[1 2 99 4]</code>	
	s[2] changed too - that is aliasing.		

Watch out. `append` either writes into spare capacity (mutating shared data) or allocates a new array and copies (leaving the old slice behind). You cannot tell which from the call. Two rules keep you safe: **always assign the result** (`s = append(s, x)`), and if you need an independent copy, make one explicitly.

```
indep := make([]int, len(s))
copy(indep, s)    // indep has its own backing array
```

To force `append` to allocate (so it never touches a shared array), cut the capacity with a *three-index slice*: `t := s[:2:2]`.

A small slice can pin a big array (a leak)

Because a slice keeps its whole backing array alive, a tiny slice of a huge buffer keeps the **entire** buffer in memory. The garbage collector cannot free it.

```
func firstLine(huge []byte) []byte {
    return huge[:80] // keeps all of `huge` alive, maybe megabytes
}
```

Watch out. This is a real memory leak even with a garbage collector (Chapter 17 — Memory and the Garbage Collector). The fix is to **copy** the part you need so the big array can be freed:

```
func firstLine(huge []byte) []byte {
    out := make([]byte, 80)
    copy(out, huge[:80]) // out does not reference `huge`
    return out
}
```

A string index is a byte, not a character

A Go string is an immutable, read-only sequence of **bytes**, encoded in UTF-8, with **no NUL terminator**. Indexing gives one *byte*, and `len` counts *bytes*, not characters.

```
s := "héllö"
fmt.Println(len(s))           // 6, not 5 (é is two bytes in UTF-8)
fmt.Println(s[0])             // 104, the byte 'h' (a byte, not a rune)
for i, r := range s {         // range decodes UTF-8: r is a rune (code point)
    _ = i
    _ = r
}
runes := []rune(s)            // convert to runes to index by character
fmt.Println(len(runes))       // 5
```

C vs Go. In C a string is `char*`, NUL-terminated, and `strlen` walks until the `\0`. A Go string carries its length, has no terminator, and is bytes of UTF-8. To edit a string, convert to `[]byte` or `[]rune`, change that, and convert back — the string itself cannot be modified in place.

```
char s[] = "hi";
s[0] = 'H'; /* legal in C: strings are mutable arrays */

s := "hi"
// s[0] = 'H'           // does NOT compile: strings are immutable
b := []byte(s)
b[0] = 'H'
s = string(b)           // "Hi"
```

Maps

See Chapter 9 — Maps.

Writing to a nil map panics

A map's zero value is `nil`. You can **read** a nil map (every key returns the zero value), but **writing** to it panics. You must create it with `make` (or a literal) first.

```
var m map[string]int
_ = m["x"]      // fine: returns 0
// m["x"] = 1 // PANIC: assignment to entry in nil map

m = make(map[string]int) // now writable
m["x"] = 1
```

Watch out. This bites when a struct has a map field you forgot to initialize. The zero struct compiles and reads fine, then panics the first time you assign to the map. Initialize map fields in your `NewX` constructor.

Map iteration order is random

Ranging over a map visits keys in an **unspecified, deliberately randomized** order that changes from run to run. C has no built-in map, but programmers expect ordered or stable containers. Go guarantees the opposite.

```
keys := make([]string, 0, len(m))
for k := range m {
    keys = append(keys, k)
}
slices.Sort(keys) // sort if you need a stable order
```

Map values are not addressable

You cannot take the address of a map element, so you cannot assign to a field of a struct stored *by value* in a map. C lets you write `table[i].x = 1` freely.

```
type P struct{ X int }
m := map[string]P{"a": {}}
// m["a"].X = 1           // does NOT compile: m["a"] is not addressable
v := m["a"]               // copy out
v.X = 1
m["a"] = v                // write back
// or store pointers: map[string]*P, then m["a"].X = 1 works
```

Numbers

See Chapter 4 — Types, Variables, and Constants.

No implicit numeric conversions

C silently promotes and converts between numeric types. Go does **not**: mixing types is a compile error. You must convert on purpose.

```
var i int = 5
var f float64 = 2.0
// x := i * f           // does NOT compile: mismatched types
x := float64(i) * f     // explicit conversion required
_ = x
```

C vs Go. This removes a whole class of silent C bugs (lost precision, sign surprises), at the cost of more `int64(x)`-style noise. The compiler will never guess what you meant.

int is 64-bit on 64-bit platforms

In C, `int` is almost always 32 bits, even on a 64-bit machine. In Go, `int` is the machine word: **64 bits on a 64-bit platform**, 32 bits on a 32-bit platform. It is *not* guaranteed to be 32 bits.

```
var n int           // 64 bits on a typical machine today
fmt.Println(unsafe.Sizeof(n)) // 8
```

Watch out. When a size must be fixed (file formats, wire protocols, hashing), use a sized type: `int32`, `int64`, `uint8`, and so on. Do not assume `int` is 32 bits the way you might in C.

Integer division truncates; signed overflow wraps

Integer division truncates toward zero, as in C. The difference is **overflow**: in C, signed integer overflow is *undefined behavior*; in Go it is **defined** — it wraps around using two's complement.

```
fmt.Println(7 / 2) // 3 (truncated)
var x int8 = 127
x++           // defined: wraps to -128 (not UB)
fmt.Println(x) // -128
```

C vs Go. A C compiler may *assume* signed overflow never happens and optimize on that assumption, so overflowing code can do anything. Go promises wraparound, so the behavior is portable and predictable (though usually still a bug you want to avoid).

Do not compare floats with ==

Floating-point math is not exact, so equality rarely holds. This is true in C too, but it surprises people who expect `0.1 + 0.2` to equal `0.3`.

```
x, y := 0.1, 0.2
sum := x + y
fmt.Println(sum == 0.3) // false
fmt.Println(math.Abs(sum-0.3) < 1e-9) // true: compare within a tolerance
```

Declarations, scope, and visibility

:= can shadow an outer variable (especially **err**)

:= always declares a **new** variable in the current block. Inside an **if** or **for** block, **x, err := f()** creates a *new err*, hiding the outer one. The famous case is checking the inner **err** while the outer stays stale.

```
data, err := load()           // outer err
if err == nil {
    data, err := parse(data) // BUG: := makes a NEW data and err, shadowing both
    _ = data
    _ = err                  // this err is checked; the outer one is not updated
}
// outer `data` is unchanged here - probably not what you wanted
```

Watch out. Use **=** (not **:=**) when you mean to assign to an existing variable. **go vet** does not flag shadowing by default; the optional **shadow** analyzer does (Chapter 22 — Tooling). Blocks in **if**, **for**, and **switch** are new scopes (Chapter 5 — Control Flow).

Unused imports and unused local variables are compile errors

In C these are warnings at most. In Go they **stop the build**. This feels harsh on day one and keeps code clean forever (Chapter 1 — Why Go for a C Programmer).

```
import "fmt"    // if unused: "imported and not used" - build fails
func f() {
    x := 1       // if unused: "declared and not used" - build fails
}
```

Watch out. The escape hatch is the blank identifier **_**: **_ = x** to silence an unused variable on purpose, or **import _ "pkg"** for an import you need only for its side effects (Chapter 3 — Program Structure: Packages, Imports, and Visibility). Unused *function arguments* and *package-level variables* are allowed.

Capitalization is visibility, not style

An identifier starting with an **uppercase** letter is exported (visible to other packages); **lowercase** is package-private. This is a language rule, not a convention (Chapter 3).

```
type User struct {
    Name string // exported: visible everywhere
    email string // unexported: only this package
}
```

Watch out. Renaming **Total** to **total** is not cosmetic — it removes the name from every other package and breaks their code. It also hides the field from reflection-based libraries (see the JSON gotcha below).

Control flow

See Chapter 5 — Control Flow.

switch does not fall through

This is the **opposite** of C. Each **case** ends by itself; there is no implicit fall-through and you do not write **break**. If you actually want C's behavior, say so with the **fallthrough** keyword.

```
switch x {
case 1:
    fmt.Println("one") // does NOT continue into case 2
case 2:
    fmt.Println("two")
    fallthrough        // explicit: now continue into the next case
case 3:
    fmt.Println("three")
}
```

Watch out. A forgotten **break** is a classic C bug. In Go the danger is reversed: if you *relied* on fall-through, you must add **fallthrough** explicitly.

range gives a copy; the range expression runs once

The loop value from **range** is a **copy** of each element. Writing to it does not change the slice. Also, the thing you range over is evaluated **once**, before the loop starts.

```
type Big struct{ N int }
items := []Big{{1}, {2}}
for _, item := range items {
    item.N = 99 // changes the COPY; items is unchanged
}
for i := range items {
    items[i].N = 99 // correct: index into the slice to modify in place
}

s := []int{1, 2, 3}
for range s {
    s = append(s, 0) // loops exactly 3 times; the length is read once
}
```

Loop-variable capture (historical — fixed since Go 1.22)

You will see old tutorials warn that a goroutine or closure created in a **for** loop captures the *same* loop variable, so all of them see the final value. **That bug is fixed.** Since Go 1.22, each iteration gets its **own** copy of the loop variable, so capturing it is safe.

```
// Go 1.22+ (this book targets 1.26): each i is per-iteration, so this is correct.
for i := range 3 {
    go func() { fmt.Println(i) }() // prints 0, 1, 2 (in some order)
}
```

Watch out. This only matters when reading **old code** or tutorials. Before Go 1.22 the fix was to add **i := i** at the top of the loop body. You no longer need it, and leaving it in is harmless. (Chapter 13 — Goroutines and the Scheduler.)

Functions and defer

See Chapter 6 — Functions.

defer: when arguments and bodies run

`defer` schedules a call to run when the **function** returns. Two surprises: the deferred call's **arguments are evaluated immediately**, at the `defer` line; and defers run at *function* exit, not at the end of the enclosing block.

```
func f() {
    x := 10
    defer fmt.Println("x was", x) // 10 is captured now, even though x changes
    x = 20
    // at return, this prints "x was 10"
}
```

Watch out. A `defer` inside a loop does **not** run at the end of each iteration — every deferred call piles up until the function returns. Opening many files in a loop with `defer f.Close()` can exhaust file descriptors. Close inside the loop, or move the body into its own function.

```
for _, name := range names {
    func() {
        f, err := os.Open(name)
        if err != nil {
            return
        }
        defer f.Close() // runs at the end of THIS inner function, each iteration
        // ... use f
    }()
}
```

Watch out. `os.Exit` (and a fatal `log.Fatal`, which calls it) terminates the program **immediately and skips all deferred calls**. Do not rely on `defer` for cleanup right before `os.Exit`.

Returning a pointer to a local is safe

This is the **opposite** of C, and it is worth saying loudly. In C, returning the address of a local variable is a bug: the stack frame is gone and the pointer dangles. In Go it is perfectly safe — the compiler's *escape analysis* sees that the variable outlives the function and allocates it on the heap, and the garbage collector frees it later (Chapter 7 — Pointers; Chapter 17 — Memory and the Garbage Collector).

```
int *bad(void) {
    int n = 0;
    return &n;    /* BUG in C: dangling pointer to a dead stack frame */
}

func good() *int {
    n := 0
```

```
    return &n    // SAFE in Go: n escapes to the heap, GC frees it later
}
```

Pointers and interfaces

Dereferencing a nil pointer panics (it is defined)

Pointers exist in Go (`*T`, `&x`), but there is no pointer arithmetic. Dereferencing `nil` does not crash with undefined behavior as in C — it raises a **defined**, recoverable panic.

```
var p *int
// _ = *p // PANIC: runtime error: invalid memory address or nil pointer dereference
```

C vs Go. In C, dereferencing `NULL` is undefined behavior — it might segfault, might corrupt memory, might appear to work. In Go it always panics with a clear message, which you can even **recover** from (Chapter 12 — Errors).

The typed-nil interface trap

This one fools experienced programmers. An interface value holds **two** parts: a type and a value (Chapter 11 — Interfaces). An interface is **nil only when both parts are nil**. If you put a nil *pointer* into an interface, the interface is **not** nil, because it knows the type.

```
type myErr struct{}
func (*myErr) Error() string { return "boom" }

func bad() error {
    var p *myErr = nil
    return p    // returns an interface holding (type=*myErr, value=nil)
}

// bad() == nil is FALSE, even though the pointer inside is nil!
```

Watch out. Never declare a typed error variable and return it; return the literal `nil` for “no error”, or return a real error value. This trap most often appears as “my function clearly returns nil but `err != nil` is true.”

What you can compare with ==

You can compare with `==` (and use as map keys): booleans, numbers, strings, pointers, channels, **structs and arrays whose fields/elements are all comparable**, and interfaces. You **cannot** compare slices, maps, or functions — except to `nil`.

```
type Point struct{ X, Y int }
fmt.Println(Point{1, 2} == Point{1, 2}) // true: structs compare field by field

var s1, s2 []int
// s1 == s2    // does NOT compile (slices are not comparable)
fmt.Println(s1 == nil) // the only comparison allowed: against nil
fmt.Println(slices.Equal(s1, s2)) // use slices.Equal for element-wise comparison
```

C vs Go. In C, `==` on structs is not allowed at all and comparing arrays compares pointers. In Go, `==` on a comparable struct compares every field — handy, but it *panics at runtime* if the struct contains an interface holding an uncomparable value (like a slice).

Concurrency

See Chapters 13–15.

Goroutines: main exiting, leaks, and no external kill

`go f()` starts a lightweight thread called a *goroutine*. Three surprises:

- When `main` returns, the program exits and **all goroutines are killed** instantly, even mid-work. They are not joined for you.
- A goroutine blocked forever (on a channel that never receives, say) is a **leak**: it sits in memory until the program ends.
- You **cannot kill a goroutine** from outside. It must return on its own. The idiom is to signal it to stop with a `context.Context` or a channel (Chapter 15 — Synchronization and context).

```
func main() {
    go fmt.Println("might not print")
    // main returns immediately; the goroutine may never run.
    // To wait, use a sync.WaitGroup or a channel.
}
```

Watch out. Sharing memory between goroutines without synchronization is a **data race** and is undefined. Always test with the race detector: `go test -race ./...` or `go run -race .` (Chapter 16 — Concurrency Patterns).

Channels: closed, nil, and unbuffered

Channels are typed pipes between goroutines (Chapter 14 — Channels and select). The rules trip up newcomers:

- **Sending on a closed channel panics.** (Receiving from one is fine and returns the zero value with `ok == false`.)
- **A nil channel blocks forever** — both send and receive. A `var ch chan int` you forgot to `make` will hang the goroutine.
- **An unbuffered send blocks until someone receives.** If no one ever does, and no other goroutine can run, the runtime detects the deadlock and crashes.
- **Only the sender should close a channel**, and only once. Closing twice, or closing as the receiver, panics.

```
ch := make(chan int) // unbuffered
// ch <- 1           // deadlock if no other goroutine receives

close(ch)
// ch <- 1           // PANIC: send on closed channel
v, ok := <-ch        // fine: v == 0, ok == false (closed and drained)
```

```
_ = v
_ = ok
```

Do not copy a struct that contains a `sync.Mutex`

A `sync.Mutex` must not be copied after first use — a copy has its own lock state, so two goroutines can each “hold” their own copy and the mutex protects nothing. The trap is a **value receiver** on a method, which copies the whole struct (including the mutex) on every call.

```
type Counter struct {
    mu sync.Mutex
    n   int
}

// func (c Counter) Inc() { ... } // BUG: value receiver copies the mutex
func (c *Counter) Inc() {         // fix: pointer receiver, no copy
    c.mu.Lock()
    c.n++
    c.mu.Unlock()
}
```

Watch out. `go vet` catches this with its `copylocks` check and reports something like “*Inc passes lock by value: Counter contains sync.Mutex*”. Run `go vet ./...` and never copy a value that contains a lock (Chapter 22 — Tooling; Chapter 15 — Synchronization and context).

Serialization

`encoding/json` only sees exported fields

The standard `encoding/json` package uses reflection, which can only read **exported** (capitalized) fields. An unexported field is silently skipped on both `marshal` and `unmarshal` (Chapter 20 — A Tour of the Standard Library; Chapter 3 for visibility).

```
type User struct {
    Name  string `json:"name"` // included; tag renames the key
    email string    // NOT serialized: unexported
}

b, _ := json.Marshal(User{Name: "Ann", email: "secret"})
fmt.Println(string(b)) // {"name":"Ann"}
```

Watch out. If a field “disappears” from your JSON, check its capitalization first. Export every field you want serialized, and use a struct tag to control the key name. The same rule applies to `encoding/xml`, `gob`, and most other reflection-based encoders.

Key takeaways

- Arrays are values (copied); strings are immutable UTF-8 bytes. Use slices for dynamic data, but remember sub-slices **share** a backing array and **append** may mutate a neighbor or reallocate.
- A nil map is read-only — writing panics; map order is random; map values are not addressable.

- Go has no implicit numeric conversions; `int` is word-sized (64-bit on 64-bit); signed overflow **wraps** (defined), unlike C's undefined behavior; never compare floats with `==`.
- `:=` can shadow (watch `err`); unused imports and locals are **compile errors**; capitalization is visibility.
- `switch` does not fall through; `range` yields a copy and evaluates its expression once; loop-variable capture is **fixed** since Go 1.22.
- `defer` evaluates arguments immediately and runs at function return; defers in a loop accumulate; `os.Exit` skips them. Returning `&local` is **safe** in Go.
- A nil dereference panics (defined); a nil pointer inside an interface makes the interface **non-nil** (the typed-nil trap); you can `==` structs/arrays but not slices/maps/functions.
- Goroutines die when `main` exits, can leak, and cannot be killed externally; channel misuse (send-on-closed, nil, unbuffered with no receiver) panics or deadlocks; do not copy a `sync.Mutex`; `encoding/json` sees only exported fields.

Watch out (the one-line checklist)

Scan this list before you ship; each line is a trap explained above.

- Passing an array copies it — use a slice.
- Sub-slices alias; always `s = append(s, x)`; `copy` to get an independent slice.
- A small slice pins its whole backing array — `copy` out to release the rest.
- `len(string)` is bytes; `string[i]` is a byte; strings are immutable UTF-8.
- Writing a nil map panics — `make` it first; map order is random.
- No implicit numeric conversions; `int` is 64-bit here; signed overflow wraps.
- Do not compare floats with `==`.
- `:=` can shadow `err`; unused imports/locals fail the build.
- Capitalization decides exported vs unexported.
- `switch` has no fall-through; `range` value is a copy.
- Loop capture is fixed since Go 1.22 (only old code needs `i := i`).
- `defer` args evaluate now; defers run at function return; loops accumulate them; `os.Exit` skips them.
- Returning `&local` is safe (escape analysis).
- nil dereference panics; typed-nil makes an interface non-nil.
- `==` works on structs/arrays, not slices/maps/functions.
- `main` returning kills all goroutines; they can leak; you cannot kill one externally; use `-race`.
- Send-on-closed panics; nil channel blocks forever; unbuffered send needs a receiver; only the sender closes.
- Never copy a `sync.Mutex` (value receivers do this — `go vet` catches it).
- `encoding/json` serializes only exported fields.

Interview questions

Q: A colleague slices a request body to keep its first 16 bytes and the server's memory keeps growing. What is happening and how do you fix it? A: A slice keeps its entire backing array alive, so `body[:16]` pins the whole (possibly large) body in memory and the garbage collector cannot free it. Fix it by copying the bytes you need into a fresh slice — `out := make([]byte, 16); copy(out, body[:16])` — so the original backing array can be collected.

Q: Why might a function that “returns nil” still fail an `err != nil` check? A: The typed-

nil interface trap. An interface value stores a type and a value, and is nil only when both are nil. If the function declares a typed pointer (for example `var p *MyError`) and returns it, the returned `error` interface holds a non-nil type with a nil value, so it is not equal to nil. Return the literal `nil` for success instead of a typed nil pointer.

Q: How does Go's `switch` differ from C's, and how do `for`/`range` loop variables behave in modern Go? A: Go's `switch` does not fall through; each case ends on its own, and you opt into C-style fall-through with the `fallthrough` keyword. As of Go 1.22, each loop iteration gets its own copy of the loop variable, so capturing it in a goroutine or closure is safe — the old “all closures see the last value” bug is gone and the `i := i` workaround is no longer needed.

Q: Why does writing to a map sometimes panic, and reading never does? A: A map's zero value is nil. Reading a nil map is defined and returns the element type's zero value, but writing to a nil map panics because there is no underlying hash table to store into. You must create the map first with `make` or a composite literal. This commonly bites when a struct has a map field that was never initialized.

Q: Give three ways concurrency in Go surprises a C programmer. A: First, when `main` returns the whole program exits and all goroutines are killed immediately — they are not joined for you. Second, you cannot kill a goroutine from outside; it must return on its own, so you signal cancellation with a context or channel, and a goroutine blocked forever leaks. Third, channel rules cause panics or deadlocks: sending on a closed channel panics, a nil channel blocks forever, and an unbuffered send with no receiver deadlocks. Shared memory without synchronization is a data race, so test with `-race`.

Q: Why is returning the address of a local variable safe in Go but a bug in C? A: In C, a local lives in the function's stack frame, which is reclaimed on return, so a returned pointer dangles. In Go, the compiler performs escape analysis: if a local's address outlives the function, the compiler allocates it on the heap instead of the stack, and the garbage collector frees it when nothing references it. So `return &n` is a normal, safe idiom in Go.

Chapter 27 — Interview Questions and Answers

This chapter is a bank of real Go interview questions with short, correct answers. It also doubles as a fast review of the whole book.

How to use this. Cover the answer, say your own answer out loud, then check. If an answer surprises you, jump to the chapter named in it and read that section. Aim to explain each idea in two or three sentences, the way you would on a call. Where it helps, the answer compares Go to C, because that is how you already think.

Language basics

Q: What is a “zero value”, and why does Go have one? A: Every variable in Go is given a defined default value when you declare it without an initializer. Numbers become 0, booleans `false`, strings `""`, and pointers, slices, maps, channels, functions, and interfaces become `nil`. In C an uninitialized local holds garbage; in Go there is no garbage. Good Go types are designed so the zero value is already useful (an empty `sync.Mutex` is unlocked; a `nil` slice is an empty slice you can append to). See Chapter 4 — Types, Variables, and Constants.

Q: What is the difference between `:=` and `var`? A: `var x int = 5` declares a variable, and the type can be explicit or inferred. `x := 5` is short variable declaration: it declares *and* initializes and infers the type, and it only works inside a function. Use `var` for package-level variables, for the zero value (`var buf []byte`), or when you want to name the type. Use `:=` for ordinary local variables. They are not interchangeable: `:=` cannot appear at package scope, and it must introduce at least one new variable on its left.

Q: How do you make an enum in Go, and what is `iota`? A: Go has no `enum` keyword. You use a `const` block with `iota`, a counter that starts at 0 and increases by one for each line in the block.

```
type Color int

const (
    Red    Color = iota // 0
    Green           // 1
    Blue           // 2
)
```

This is like C's `enum { Red, Green, Blue }`, but `Color` is a distinct named type, so the compiler stops you from mixing it with a plain `int` by accident.

Q: Why does Go have no implicit numeric conversions? A: To stop silent bugs. In C, `int` and `double` mix freely, and a narrowing conversion can lose data quietly. In Go you must convert on purpose: `float64(i) / float64(n)`. Even `int` and `int32` are different types and will not mix. This is more typing but removes a whole class of “where did that rounding come from?” bugs.

Q: What is the difference between an array and a slice? A: An array `[N]T` has a fixed length that is *part of its type*, and it is a value: assigning or passing it copies all elements. A slice `[]T` is a small header that points at a backing array and has a length and a capacity; passing a slice copies the header, not the data. In practice you use slices almost everywhere and rarely write a fixed array. See Chapter 8 — Arrays, Slices, and Strings.

Q: Why does Go have only one loop keyword? A: Simplicity and one obvious way. `for` covers every case: a three-part C-style loop, a condition-only loop (`for cond {}` replaces `while`), an infinite loop (`for {}`), a `for range` over collections, and `for i := range n` to count from 0 to `n-1`. There is no `while` and no `do/while`. See Chapter 5 — Control Flow.

Q: What does `any` mean, and how is it different from `interface{}`? A: `any` is a built-in alias for `interface{}`, the empty interface, which every type satisfies. They are identical; `any` is just the modern, readable spelling and is preferred since Go 1.18. It is the closest thing Go has to C's `void *`, but it is type-safe: you must use a type assertion or type switch to get the concrete value back out.

Slices and strings

Q: What is a slice “header”, exactly? A: A slice is a three-word struct: a pointer to the first element in a backing array, an `int` length (`len`), and an `int` capacity (`cap`). This is a “fat pointer”. When you pass a slice to a function, you copy these three words, so the function sees the same backing array but its own copy of the header.

```
slice []int -> [ ptr | len | cap ]
```

```

      |
      v
backing array  [ 10 | 20 | 30 | 40 ]

```

Q: What is the difference between length and capacity? A: Length is how many elements the slice currently has and can index. Capacity is how many elements fit in the backing array starting from the slice's first element, before a new array must be allocated. `append` uses capacity to decide whether it can grow in place or must allocate. You read them with `len(s)` and `cap(s)`.

Q: What does `append` actually do? A: `append(s, x)` adds `x` after the last element. If `len < cap`, it writes into the existing backing array and returns a slice with `len+1`. If there is no room, it allocates a bigger backing array, copies the elements, writes `x`, and returns a slice that points at the new array. Because the return value may point somewhere new, you must always write `s = append(s, x)`.

Q: Explain the slice aliasing bug. A: Two slices can share one backing array. Writing through one, or appending when there is spare capacity, can change the other. A re-slice like `b := a[:2]` keeps `a`'s capacity, so `append(b, x)` can overwrite `a`'s elements. To get an independent copy, use `copy`, or cut capacity with a full slice expression `a[low:high:max]`.

Q: What does the built-in `copy` do? A: `copy(dst, src)` copies `min(len(dst), len(src))` elements and returns that count. It is the safe way to duplicate slice data so two slices no longer share memory. It is Go's `memmove`: it handles overlapping regions correctly.

Q: What is the difference between a `byte` and a `rune`? A: `byte` is an alias for `uint8`: one byte of raw data. `rune` is an alias for `int32` and holds one Unicode code point (one “character”). A `string` is bytes encoded as UTF-8, so one `rune` can be 1 to 4 bytes. Indexing `s[i]` gives a `byte`; ranging for `_, r := range s` gives `runes`.

Q: Are Go strings mutable? How are they stored? A: No, a `string` is immutable: a read-only slice of bytes, stored as UTF-8. You cannot assign to `s[i]`. To edit text, convert to `[]byte` (or `[]rune`), change it, and convert back; each conversion copies. `len(s)` returns the number of *bytes*, not characters, so an emoji adds several to `len`. See Chapter 8 — Arrays, Slices, and Strings.

Maps

Q: What is the “comma-ok” idiom for maps? A: Reading a missing key returns the value type's zero value, which you cannot tell apart from a real zero. The two-result form tells you which it is:

```
v, ok := m["key"] // ok is true only if the key is present
```

The same `, ok` form appears for channel receives and type assertions.

Q: What happens when you read from or write to a `nil` map? A: Reading from a `nil` map is safe and returns the zero value. Writing to a `nil` map *panics* at run time. The zero value of a map is `nil`, so you must create it with `make(map[K]V)` or a map literal before you store anything.

Q: Why is map iteration order random? A: Go deliberately randomizes the order each time you range over a map. This stops programs from accidentally depending on an order that the implementation never promised. If you need order, copy the keys into a slice and `sort` it.

Q: Why can't I write `m[k].field = x` when the value is a struct? A: Map values are not

addressable, so you cannot take their address or assign to a field in place. You must replace the whole value:

```
p := m[k]
p.field = x
m[k] = p
```

This is unlike C, where a struct stored in a hash table you wrote could be edited through a pointer. With a map of *pointers* (`map[K]*T`) you can edit fields directly.

Q: Are maps safe for concurrent use? A: No. Concurrent writes (or a write at the same time as any read) cause a fatal run-time error, “concurrent map writes”, which you cannot recover from. Protect a map with a `sync.Mutex/sync.RWMutex`, or use `sync.Map` for some read-heavy cases. See Chapter 9 — Maps.

Structs and interfaces

Q: When should a method use a value receiver versus a pointer receiver? A: Use a pointer receiver (`func (s *S) M()`) when the method must change the receiver, or when the struct is large and you want to avoid copying, or when some methods already need a pointer (keep them consistent). Use a value receiver for small, immutable types. A value receiver gets a copy, so changes to it are lost. See Chapter 10 — Structs and Methods.

Q: What is a method set, and why does it matter for interfaces? A: The method set of type `T` includes only methods with a value receiver. The method set of `*T` includes methods with value *and* pointer receivers. So if a method has a pointer receiver, only `*T` satisfies an interface that lists it; a plain `T` value does not. This is the usual reason for “cannot use `x` (type `T`) as type `I`” errors.

Q: What does “implicit interface satisfaction” mean? A: A type satisfies an interface just by having the required methods. There is no `implements` keyword and no declaration linking them, unlike C++ or Java. This is “structural typing”: if it has the methods, it fits. The check happens at compile time where the concrete value is used as the interface.

Q: How is an interface value represented in memory? A: As a two-word pair: a pointer to type information (which concrete type and its method table) and a pointer to the value (or the value itself if it fits). This is like a C struct that holds both a `vtable` pointer and a `void *data`, but the compiler builds and type-checks it for you. The zero value of an interface is `nil`, meaning both words are `nil`.

Q: What is the “nil interface” trap? A: An interface is `nil` only when *both* its type word and value word are `nil`. If you put a typed `nil` pointer into an interface, the interface holds (type=`*T`, value=`nil`), which is *not* equal to `nil`. So a function that returns a typed `nil` error will fail the `if err != nil` check. Return a literal `nil`, not a `nil` typed pointer.

Q: What is the empty interface used for? A: `any` (the empty interface) can hold a value of any type, so it is used for general containers, printing (`fmt.Println(args ...any)`), and decoding unknown JSON. You retrieve the real value with a type assertion `v, ok := x.(int)` or a type switch. Prefer concrete types or generics when you can; `any` loses compile-time type checks.

Q: How is embedding different from inheritance? A: Embedding puts one type inside another without a field name, and the outer type “promotes” the inner type’s fields and methods

so you can call them directly. It is *composition*, not inheritance: there is no base class, no virtual override, and no “is-a” relationship for type checking. You get code reuse; polymorphism comes from interfaces instead. See Chapter 11 — Interfaces.

Errors

Q: How does Go handle errors, and why not exceptions? A: Errors are ordinary values of the built-in `error` interface, returned as the last result and checked with `if err != nil`. There are no exceptions for normal failures. The reason is clarity: the error path is visible in the code, not hidden in invisible unwinding, so you cannot forget a failure case silently. See Chapter 12 — Errors.

Q: What do `%w`, `errors.Is`, and `errors.As` do? A: `fmt.Errorf("...: %w", err)` wraps an error, keeping the original inside for later inspection. `errors.Is(err, target)` reports whether `target` is anywhere in the wrap chain (use it to compare against sentinel errors like `io.EOF`). `errors.As(err, &target)` finds the first error in the chain of a given concrete type and assigns it, so you can read its fields.

Q: What is the difference between `panic` and returning an error? A: An error is for expected, recoverable problems (file not found, bad input); the caller decides what to do. `panic` is for programmer mistakes or impossible states (index out of range, nil dereference); it unwinds the stack and crashes unless a deferred `recover` stops it. Do not use `panic` for ordinary control flow.

Q: When is it acceptable to call `panic`? A: At program startup when a required condition is impossible to continue from (a missing config that you cannot run without), in truly “this should never happen” internal checks, and inside a package when you `recover` at the boundary and turn it back into an error. Library code that callers depend on should return errors, not `panic`.

Q: What does `recover` do, and where must it be called? A: `recover` stops a panic and returns the panic value, but it only works when called directly inside a deferred function. Outside `defer` it returns `nil` and does nothing. The common use is at an API boundary or per-request handler to keep one bad request from crashing the whole server.

Q: How do you define a custom error type? A: Define a type with an `Error() string` method, which satisfies the `error` interface.

```
type NotFoundError struct{ Key string }

func (e *NotFoundError) Error() string {
    return "not found: " + e.Key
}
```

Callers can then use `errors.As` to detect it and read `Key`.

Concurrency — goroutines and channels

Q: How is a goroutine different from an OS thread? A: A goroutine is a function scheduled by the Go runtime, not the OS. It starts with a tiny stack (about 2 KB) that grows and shrinks on demand, while an OS thread reserves megabytes. You can run hundreds of thousands of goroutines; the runtime multiplexes them onto a small number of OS threads. Starting one is `go f()`. See Chapter 13 — Goroutines and the Scheduler.

Q: What is the G-M-P scheduler model? A: G is a goroutine, M is a machine (an OS thread), and P is a processor (a scheduling context that owns a queue of runnable goroutines). An M must hold a P to run Gs. The number of Ps is `GOMAXPROCS`, which caps how many goroutines run in parallel. Idle Ps use “work stealing” to take Gs from busy Ps, which balances load.

Q: What is `GOMAXPROCS` and what is its default? A: It is the maximum number of OS threads that may execute Go code at the same time, which sets the real parallelism. By default it is the number of logical CPUs available. Since Go 1.25 the runtime is container-aware: on Linux it also honors the cgroup CPU limit and lowers `GOMAXPROCS` to match, and it adjusts if the limit changes.

Q: What is the difference between a buffered and an unbuffered channel? A: An unbuffered channel (`make(chan T)`) has no storage: a send blocks until a receiver is ready, so it is a rendezvous and also a synchronization point. A buffered channel (`make(chan T, n)`) holds up to `n` values; a send blocks only when the buffer is full, and a receive blocks only when it is empty. See Chapter 14 — Channels and Select.

Q: What does `select` do? A: `select` waits on several channel operations at once and runs the one that is ready first; if several are ready it picks one at random. A `default` case makes it non-blocking (do something else if no channel is ready). It is Go’s version of `select/poll` on file descriptors, but for channels.

Q: What are the rules for closing a channel? A: Only the sender should close a channel, and only once. Sending on a closed channel panics; closing an already-closed channel panics; closing a `nil` channel panics. Receiving from a closed channel never blocks: it returns the zero value immediately, and `v, ok := <-ch` sets `ok` to `false`. You do not have to close every channel; close to signal “no more values” to receivers.

Q: What are the common causes of a deadlock? A: All goroutines are blocked with no one able to make progress: sending on an unbuffered channel with no receiver, receiving from a channel that no one will send on or close, every goroutine waiting on a `WaitGroup` that never reaches zero, or two goroutines each holding one lock and waiting for the other. If the *whole* program is blocked, the runtime detects it and crashes with “all goroutines are asleep - deadlock!”.

Concurrency — synchronization and patterns

Q: When should I use a mutex versus a channel? A: Use a `sync.Mutex` to protect shared state that several goroutines read and write (a cache, a counter). Use a channel to *transfer ownership* of data or to coordinate steps between goroutines (a pipeline, a worker pool). The slogan is “share memory by communicating”, but a mutex is simpler and faster for plain shared state. See Chapter 15 — Sync and Context.

Q: What is a `sync.WaitGroup` for? A: It waits for a set of goroutines to finish. Call `wg.Add(n)` before starting them, `wg.Done()` (usually via `defer`) inside each one, and `wg.Wait()` to block until the counter reaches zero. It is like `pthread_join` for a group, without holding each thread handle.

```
var wg sync.WaitGroup
for _, item := range items {
    wg.Add(1)
    go func() {
        defer wg.Done()
        process(item)
    }()
}
```

```
    }()
}
wg.Wait()
```

Q: What is context, and how is it used for cancellation? A: A `context.Context` carries a deadline, a cancellation signal, and request-scoped values across API boundaries. A parent creates one with `context.WithCancel` or `context.WithTimeout` and passes it as the first argument. Worker code selects on `ctx.Done()` and stops when it is closed. It is the standard way to cancel a tree of goroutines (for example, when an HTTP client disconnects).

Q: What is the race detector and how do you use it? A: It is a tool built into the toolchain that finds data races: unsynchronized access to the same memory from two goroutines where at least one is a write. Run your tests or program with the `-race` flag (`go test -race ./...`, `go run -race .`). It adds overhead, so use it in testing and CI, not production. See Chapter 16 — Concurrency Patterns.

Q: What is a worker pool and why use one? A: It is a fixed number of goroutines that pull jobs from a shared channel. It limits how much work runs at once, which bounds memory and CPU use, instead of spawning one goroutine per job without limit. Producers send jobs and close the channel; workers range over it and exit when it is closed.

Q: What are fan-out and fan-in? A: Fan-out: start several goroutines that read from the same input channel, to spread work across CPUs. Fan-in: merge several input channels into one output channel so a single consumer can read all results. Together they form a pipeline stage that scales.

Q: What is a goroutine leak, and how do you avoid it? A: A goroutine leak is a goroutine that blocks forever and is never cleaned up, so it holds memory and resources for the life of the program. The usual cause is a send or receive with no partner (for example, writing to a channel whose reader has gone away). Avoid it by giving every goroutine a clear way to exit: close the channel, or pass a `context` and select on `ctx.Done()`.

Runtime and memory

Q: What is the difference between the stack and the heap in Go? A: Each goroutine has its own stack for local variables and call frames; it is fast and freed automatically when a function returns. The heap holds values that must outlive the function that created them; it is managed by the garbage collector. Unlike C, *you* do not choose: the compiler decides, and there is no `malloc/free`. See Chapter 17 — Memory and the Garbage Collector.

Q: What is escape analysis? A: It is the compiler step that decides whether a variable can live on the stack or must “escape” to the heap. A value escapes if it is still reachable after the function returns: it is returned by pointer, stored in a global, captured by a goroutine, or placed in an interface that outlives the call. You can see the decisions with `go build -gcflags=-m`.

Q: Is it safe to return a pointer to a local variable in Go? A: Yes. This is undefined behavior in C, but in Go it is safe and common. Escape analysis sees that the local outlives the function, so it allocates that variable on the heap, and the garbage collector frees it once nothing points to it.

```
func newCounter() *int {
    n := 0    // escapes to the heap
```



```
    return &n // safe in Go; would be a dangling pointer in C
}
```

Q: How does Go’s garbage collector work? A: It is a concurrent, tri-color, mark-and-sweep collector. It runs alongside your program and uses a write barrier so it can mark live objects while goroutines keep running; the stop-the-world pauses are tiny (usually well under a millisecond). It does not move objects (no compaction) and is not generational. Go 1.26 ships the new “Green Tea” collector by default, which scans small objects with better cache locality and CPU scaling.

Q: What do GOGC and GOMEMLIMIT control? A: GOGC (default 100) sets how much the heap may grow before the next collection: 100 means collect when live data has doubled. Raising it trades memory for fewer GC cycles; GOGC=off disables the GC. GOMEMLIMIT sets a *soft* memory limit (default effectively off); the runtime collects more aggressively as memory approaches it, which helps avoid out-of-memory kills in containers.

Q: How do you reduce allocations in a hot path? A: Preallocate slices and maps with a capacity (`make([]T, 0, n)`); reuse buffers with `sync.Pool`; pass and store values instead of pointers when they are small so they stay on the stack; use `strings.Builder` instead of `+` in a loop; and avoid putting values into `any`/interfaces needlessly. Measure first with `go test -bench` and `pprof`; do not guess. See Chapter 17 — Memory and the Garbage Collector.

Modules and tooling

Q: What is the difference between a module and a package? A: A package is one directory of `.go` files compiled together and is the unit of *imports*. A module is a collection of packages versioned and released together; it is the unit of *dependencies* and is defined by a `go.mod` file at its root. Roughly: a package is like a C library’s translation units; a module is like the whole versioned library you depend on. See Chapter 18 — Packages and Modules.

Q: What are `go.mod` and `go.sum`? A: `go.mod` declares the module path, the Go version, and the required dependencies and their versions. `go.sum` records cryptographic checksums of those dependencies so the build can verify it downloaded exactly the expected code. Commit both. You rarely edit them by hand; `go get` and `go mod tidy` maintain them.

Q: What do `gofmt` and `go vet` do? A: `gofmt` rewrites source into the one canonical format (tabs, brace placement, spacing), so style is never debated and diffs stay small. `go vet` runs static checks for likely mistakes that compile but are probably wrong, such as bad `Printf` format verbs, lost struct tags, or unreachable code. Run both in CI.

Q: What is a table-driven test? A: A test where the cases are a slice of structs and a loop runs each one, usually with `t.Run` so each row is a named subtest. It is the idiomatic Go pattern because adding a case is one line and a failure names the exact row.

```
func TestAbs(t *testing.T) {
    tests := []struct {
        name string
        in    int
        want  int
    }{
        {"positive", 3, 3},
        {"negative", -3, 3},
    }
```



```

        {"zero", 0, 0},
    }
    for _, tc := range tests {
        t.Run(tc.name, func(t *testing.T) {
            if got := abs(tc.in); got != tc.want {
                t.Errorf("abs(%d) = %d, want %d", tc.in, got, tc.want)
            }
        })
    }
}

```

Q: How do you write and read a benchmark? A: Write `func BenchmarkXxx(b *testing.B)` and loop `for b.Loop() { ... }` (or the older `for range b.N`). Run it with `go test -bench=.` `-benchmem`. The output reports nanoseconds per operation and, with `-benchmem`, bytes and allocations per operation, which is what you optimize against. See Chapter 21 — Testing.

Predict the output

These are classic puzzles. Read the code, decide the output, then check the answer and the reason. Each program is complete and runnable.

1. Defer order

```

package main

import "fmt"

func main() {
    for i := 0; i < 3; i++ {
        defer fmt.Println(i)
    }
}

```

Output:

```

2
1
0

```

Why: `defer` pushes each call onto a stack. The argument `i` is evaluated *now* (so 0, 1, 2 are captured), but the calls run in last-in-first-out order when `main` returns.

2. Append aliasing

```

package main

import "fmt"

func main() {
    a := []int{1, 2, 3}
    b := a[:2]
    b = append(b, 99)
}

```

```
    fmt.Println(a, b)
}
```

Output:

```
[1 2 99] [1 2 99]
```

Why: `b` has `len 2` but `cap 3`, sharing `a`'s backing array. `append` has spare capacity, so it writes 99 into index 2 of that shared array, overwriting `a[2]`. Both slices now show 99.

3. Goroutines and the loop variable

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    var wg sync.WaitGroup
    for i := 0; i < 3; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            fmt.Print(i)
        }()
    }
    wg.Wait()
}
```

Output: some ordering of the digits 0, 1, and 2, for example 012 or 201.

Why: Since Go 1.22 each iteration has its own `i`, so each goroutine prints its own value. Before Go 1.22 all goroutines shared one `i` and this famously printed 333. The order is not fixed because goroutines run concurrently.

4. The nil interface that is not nil

```
package main

import "fmt"

type MyError struct{}

func (*MyError) Error() string { return "boom" }

func find() error {
    var e *MyError // a nil pointer
    return e       // returned as an interface
}
```

```
func main() {
    fmt.Println(find() == nil)
}
```

Output:

false

Why: `find` returns an `error` interface holding (type=`*MyError`, value=`nil`). An interface equals `nil` only when both its type and value are `nil`; here the type is set, so the result is not `nil`. Fix it by returning a literal `nil`.

5. Map iteration order

```
package main

import "fmt"

func main() {
    m := map[int]string{1: "a", 2: "b", 3: "c"}
    for k := range m {
        fmt.Print(k)
    }
}
```

Output: a random ordering of 1, 2, 3, such as 231; it changes between runs.

Why: Go randomizes map iteration order on purpose so code cannot depend on it. To print in order, copy the keys into a slice and `sort.Ints` it first.

6. Named return changed by defer

```
package main

import "fmt"

func double() (result int) {
    defer func() {
        result *= 2
    }()
    return 5
}

func main() {
    fmt.Println(double())
}
```

Output:

10

Why: `return 5` first assigns 5 to the named return value `result`. Then the deferred function runs *before* the function truly returns, doubling `result` to 10. A deferred closure can read and modify named return values.

Design questions

These ask you to sketch a small system out loud. Give the data structures and the flow; you do not need full code.

Sketch a worker pool.

```
func workerPool(jobs <-chan int, results chan<- int, workers int) {
    var wg sync.WaitGroup
    for range workers { // start a fixed number of workers
        wg.Add(1)
        go func() {
            defer wg.Done()
            for j := range jobs { // ends when jobs is closed
                results <- j * j
            }
        }()
    }
    go func() {
        wg.Wait()
        close(results) // close results once every worker is done
    }()
}
```

Key points to say: a fixed number of goroutines bounds concurrency; the producer closes `jobs` to signal “no more work”; each worker `ranges` over `jobs`; a `WaitGroup` tracks the workers; a separate goroutine closes `results` after `Wait` so the consumer’s `range results` can end cleanly.

Sketch a rate limiter.

```
// Allow at most one event every 200 milliseconds.
ticker := time.NewTicker(200 * time.Millisecond)
defer ticker.Stop()
for req := range requests {
    <-ticker.C // wait for the next tick (a "token")
    go handle(req)
}
```

Key points: a `time.Ticker` produces tokens at a steady rate, and you take one before each request. To allow short bursts, use a buffered “token bucket” channel that a ticker refills up to its capacity. In production, prefer the standard `golang.org/x/time/rate.Limiter`, which implements a token bucket with burst support.

Sketch an LRU cache.

- Goal: `O(1)` `Get` and `Put`, and evict the least-recently-used entry when full.
- Structures: a `map[Key]*list.Element` for `O(1)` lookup, plus a doubly linked list (`container/list`) that orders entries from most-recently used at the front to least-recently used at the back.
- `Get`: find the element in the map, move it to the front, return its value.
- `Put`: if the key exists, update its value and move it to the front; otherwise insert at the front. If the size now exceeds the capacity, remove the back element and delete its key from the

- map.
- For concurrent use, guard both structures with one `sync.Mutex`.

Chapter 28 — Capstone Projects

What you'll learn. How to tie the whole book together by building three complete, runnable programs: a `wc`-style command-line tool, a JSON REST API, and a bounded concurrent fetcher. Each project lists the chapters it draws on, so you can also use this chapter as a guided review.

You have met every part of Go on its own. Now we assemble those parts into real programs. Each project below is small enough to read in one sitting but complete enough to run, modify, and ship. We build them the way you would at work: start with a clear goal, write idiomatic code, explain every section, run it, and then list ways to extend it.

All three programs live in this book's `examples/` module so you can run them directly. The module path is `goforc/examples` and it targets Go 1.26.

```
cd examples           # from the root of this book
go vet ./...          # static checks: should print nothing
go build ./...        # compiles all three programs

examples/
  go.mod              module goforc/examples
  linecount/main.go   Project 1 - the CLI tool
  taskapi/main.go     Project 2 - the JSON REST API
  workerpool/main.go  Project 3 - the concurrent fetcher
```

Each program is a separate `package main` in its own directory, so each builds into its own binary (see Chapter 3 — Program Structure: Packages, Imports, and Visibility). The standard library is the only dependency; there is nothing to download.

Project 1 — a `wc`-style line counter

Goal. Build `linecount`, a small clone of the Unix `wc` tool. It counts lines, words, and bytes. It reads the files named on the command line, or standard input when you name no files. Flags `-l`, `-w`, and `-c` choose which counts to print. Bad files go to standard error and set a non-zero exit code, exactly as a good Unix tool should.

This is the kind of program a C programmer writes first in any language. In C you would loop with `getchar`, track state by hand, and print with `printf`. The Go version is similar in spirit but safer: no buffer to size, no manual `fopen` error codes, and the file is always closed for you.

Here is the data flow. Input comes from standard input or from files; the counter turns each input into a small `counts` value; the formatter prints the selected fields to standard output; any open error goes to standard error.

```
stdin      count()      counts{lines, words, bytes}
files      (bufio)
```

```

        open err      format + selection

        stderr
    exit code 1        stdout

```

Types and imports

We start with the package comment, the imports, and the small value types. A `counts` holds the three totals. A `selection` records which of them the user asked to see. Keeping these as plain structs makes the rest of the code read like prose.

```

// Command linecount is a small wc-like tool. It counts lines, words, and
// bytes from files named on the command line, or from standard input when no
// files are given.
//
// Usage:
//
// linecount [-l] [-w] [-c] [file ...]
//
// With no flags it prints lines, words, and bytes, like wc. With one or more of
// -l, -w, -c it prints only the requested counts, in that fixed order.
package main

import (
    "bufio"
    "errors"
    "flag"
    "fmt"
    "io"
    "os"
    "strings"
)

// counts holds the three totals for one input.
type counts struct {
    lines int64
    words int64
    bytes int64
}

// add accumulates another set of counts into c. It is used to build the grand
// total across several files.
func (c *counts) add(o counts) {
    c.lines += o.lines
    c.words += o.words
    c.bytes += o.bytes
}

```

```
// selection records which counts the user asked to see.
type selection struct {
    lines, words, bytes bool
}
```

add has a **pointer receiver** (`c *counts`) because it changes the value it is called on (see Chapter 10 — Structs and Methods). We use `int64` for the totals so a very large file cannot overflow the count the way a 32-bit `int` might in C.

The core: counting bytes, words, and lines

The heart of the tool reads one byte at a time and updates the three counts. A `bufio.Reader` wraps the input so the byte-at-a-time loop is still fast: it reads in large blocks under the hood and hands you bytes from its buffer.

```
// isSpace reports whether b is an ASCII whitespace byte. This is the same set
// C's isspace recognizes in the default "C" locale: space, tab, newline,
// vertical tab, form feed, and carriage return.
func isSpace(b byte) bool {
    switch b {
    case ' ', '\t', '\n', '\v', '\f', '\r':
        return true
    default:
        return false
    }
}

// count reads everything from r and returns the line, word, and byte totals.
// A line is counted at each newline byte, and a word is a run of non-space
// bytes, exactly like wc. bufio.Reader buffers the underlying reads, so the
// byte-at-a-time loop is still fast.
func count(r io.Reader) (counts, error) {
    br := bufio.NewReader(r)
    var c counts
    inWord := false
    for {
        b, err := br.ReadByte()
        if err != nil {
            // io.EOF is the normal, expected end of input, not a failure.
            if errors.Is(err, io.EOF) {
                return c, nil
            }
        }
        return c, err
    }
    c.bytes++
    if b == '\n' {
        c.lines++
    }
}
```

```

        if isSpace(b) {
            inWord = false
        } else if !inWord {
            inWord = true
            c.words++
        }
    }
}

```

Two ideas here are pure Go and worth pausing on:

- **count accepts an `io.Reader`, not a file.** `io.Reader` is the one-method interface “something I can read bytes from” (see Chapter 11 — Interfaces). A file satisfies it, and so does standard input, and so does a string or a network socket. We write the logic once and feed it any source. In C you would pass a `FILE *` or a file descriptor; the Go interface is broader and checked at compile time.
- **End of input is an error value, not a magic return.** `ReadByte` returns `io.EOF` when there is nothing left. We test it with `errors.Is` (see Chapter 12 — Errors), treat it as success, and return the totals. C’s `getchar` returns the special value `EOF` (an `int`, not a `char`); Go makes the end-of-file signal an explicit **error** you check by name.

C vs Go. In C, “is this a word boundary?” usually means calling `isspace` from `<ctype.h>`. Our `isSpace` lists the same six ASCII characters on purpose, so `linecount` splits words the same way `wc` does in the default locale. We do not call `unicode.IsSpace` on a single byte, because a byte is not a rune: a multibyte UTF-8 character would be misread.

Opening files and formatting output

`countFile` opens a file, counts it, and guarantees it is closed with `defer` (see Chapter 6 — Functions). `format` turns a `counts` plus a `selection` into one aligned output line using a `strings.Builder`, which assembles a string without repeated allocations (see Chapter 8 — Arrays, Slices, and Strings).

```

// countFile opens name, counts it, and always closes the file.
func countFile(name string) (counts, error) {
    f, err := os.Open(name)
    if err != nil {
        // os.Open returns a *fs.PathError that already includes the file name
        // and the reason ("open foo: no such file or directory"), so we return
        // it unchanged.
        return counts{}, err
    }
    defer f.Close()
    return count(f)
}

// format renders the selected counts followed by an optional name. Counts are
// right-aligned in a fixed width so columns line up, similar to wc.
func format(c counts, sel selection, name string) string {

```



```

var sb strings.Builder
writeField := func(n int64) {
    if sb.Len() > 0 {
        sb.WriteByte(' ')
    }
    fmt.Fprintf(&sb, "%7d", n)
}
if sel.lines {
    writeField(c.lines)
}
if sel.words {
    writeField(c.words)
}
if sel.bytes {
    writeField(c.bytes)
}
if name != "" {
    sb.WriteByte(' ')
    sb.WriteString(name)
}
return sb.String()
}

```

`writeField` is a **closure**: a small function that captures and writes to the surrounding `sb` (see Chapter 6 — Functions). It adds a leading space before every field except the first, so the columns line up no matter which flags were chosen.

Wiring it together in main

`main` parses the flags, decides which counts to show, and then runs one of two paths: read standard input when there are no file arguments, or loop over the named files and also print a grand total when there is more than one.

```

func main() {
    lineFlag := flag.Bool("l", false, "print the line count")
    wordFlag := flag.Bool("w", false, "print the word count")
    byteFlag := flag.Bool("c", false, "print the byte count")
    flag.Parse()

    // If the user named no counters, show all three (the wc default).
    if !*lineFlag && !*wordFlag && !*byteFlag {
        *lineFlag, *wordFlag, *byteFlag = true, true, true
    }
    sel := selection{lines: *lineFlag, words: *wordFlag, bytes: *byteFlag}

    files := flag.Args()

    // No file arguments: read standard input and print only the counts.
    if len(files) == 0 {

```

```

    c, err := count(os.Stdin)
    if err != nil {
        fmt.Fprintf(os.Stderr, "linecount: stdin: %v\n", err)
        os.Exit(1)
    }
    fmt.Println(format(c, sel, ""))
    return
}

// One or more files: count each, print its line, and keep a running total.
// A bad file is reported on stderr; we continue and exit non-zero at the end.
exit := 0
var total counts
for _, name := range files {
    c, err := countFile(name)
    if err != nil {
        fmt.Fprintf(os.Stderr, "linecount: %v\n", err)
        exit = 1
        continue
    }
    total.add(c)
    fmt.Println(format(c, sel, name))
}
if len(files) > 1 {
    fmt.Println(format(total, sel, "total"))
}
os.Exit(exit)
}

```

Notice three Unix habits that Go makes easy (see Chapter 24 — CLI Tools):

- **Normal output to stdout, errors to stderr.** `fmt.Println` writes to standard output; `fmt.Fprintf(os.Stderr, ...)` writes to standard error. This lets a caller redirect them separately, just like in C.
- **A meaningful exit code.** A missing file sets `exit = 1`, but we keep going and still count the good files, then call `os.Exit(1)` at the end. Recall from Chapter 1 — Why Go for a C Programmer that `main` does not return an `int`; you set the status with `os.Exit`.
- **One reader, many sources.** Because `count` takes an `io.Reader`, the `stdin` path and the file path call the very same function.

Run it

`cd examples`

```

echo 'hello world' | go run ./linecount      # read standard input
go run ./linecount go.mod                   # one file
go run ./linecount -l -w go.mod             # only lines and words
go run ./linecount go.mod linecount/main.go # several files plus a total
go run ./linecount nope.txt                 # a missing file

```

Sample output (the counts depend on the files):

```
3      4      32 go.mod
3      4      32 go.mod
167    674    4063 linecount/main.go
170    678    4095 total
```

The missing file prints to standard error and the process exits with status 1:

```
linecount: open nope.txt: no such file or directory
```

Concepts it exercises

- Chapter 1 — Why Go for a C Programmer: program shape, `os.Exit` for status.
- Chapter 3 — Program Structure: Packages, Imports, and Visibility: `package main`.
- Chapter 6 — Functions: closures, `defer` for closing files.
- Chapter 8 — Arrays, Slices, and Strings: `strings.Builder`, bytes vs runes.
- Chapter 10 — Structs and Methods: the `counts` type and its pointer receiver.
- Chapter 11 — Interfaces: writing to an `io.Reader` works for files and stdin.
- Chapter 12 — Errors: `errors.Is` with the `io.EOF` sentinel.
- Chapter 20 — The Standard Library Tour: `flag`, `bufio`, `io`, `os`, `fmt`.
- Chapter 24 — CLI Tools: flags, stdin/stdout/stderr, exit codes.

Extend it

- Add a `-` argument that means “read standard input”, as real `wc` does.
- Add a `-m` flag for **rune** (character) count, decoding UTF-8 with `bufio.Reader.ReadRune`. Show how it differs from the byte count on non-ASCII text.
- Print counts with a width based on the largest number so columns are tight.
- Add a `-L` flag that reports the length of the longest line.
- Write table-driven tests for `count` using `strings.NewReader` (see Chapter 21 — Testing).

Project 2 — a JSON REST API for tasks

Goal. Build `taskapi`, an HTTP service that stores a list of “tasks” in memory. It speaks JSON and supports four operations: list all tasks, create one, fetch one by id, and delete one. It uses the modern `net/http` router (Go 1.22+) that matches on method and path, a logging middleware, a mutex to protect the shared store, sensible server timeouts, and graceful shutdown when the process receives an interrupt signal.

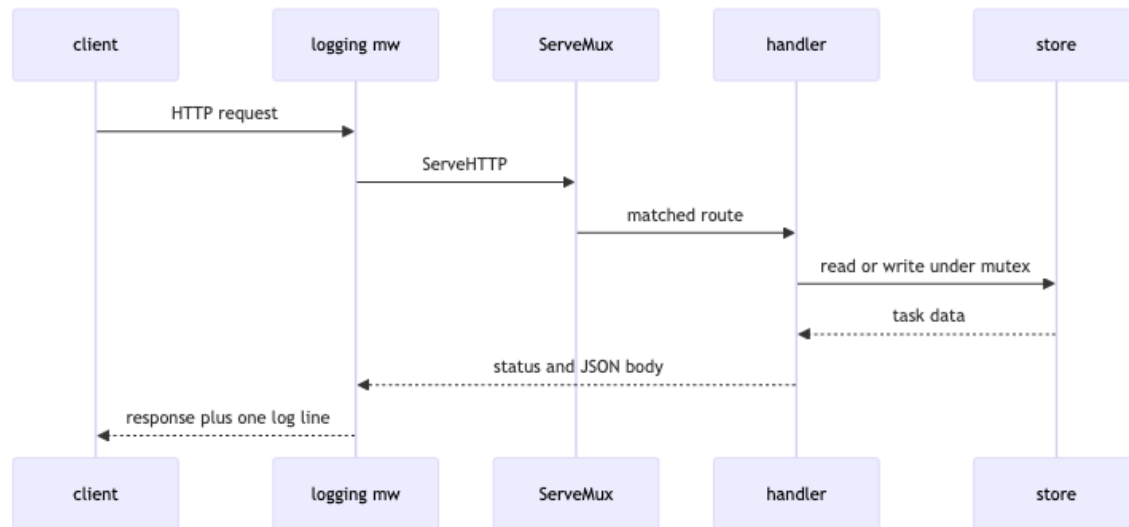
This is the bread-and-butter Go program: a small, dependency-free network service. A C programmer is often surprised that a production-shaped HTTP/JSON server needs no framework and no third-party libraries — the standard library is enough (see Chapter 23 — Web Services).

The REST routes:

Method	Path	Meaning	Success status
GET	<code>/tasks</code>	list all tasks	200 OK
POST	<code>/tasks</code>	create a task from a JSON body	201 Created

Method	Path	Meaning	Success status
GET	/tasks/{id}	fetch one task	200 OK
DELETE	/tasks/{id}	delete one task	204 No Content

Every request flows through the logging middleware, then the router, then a handler, which reads or writes the shared store:



Types and imports

The `Task` is the data we serialize. Only **exported** (capitalized) fields are visible to `encoding/json`, so each field starts with an uppercase letter, and a struct tag sets the lowercase JSON name (see Chapter 3 — Program Structure: Packages, Imports, and Visibility, and Chapter 10 — Structs and Methods).

```
// Command taskapi is a small in-memory REST service for "tasks". It shows the
// Go 1.22 net/http router (method and path patterns like "GET /tasks/{id}"),
// JSON encoding and decoding, a logging middleware, a mutex-protected store
// (handlers run concurrently, one goroutine per request), server timeouts, and
// graceful shutdown driven by an OS signal.
```

```
//
// Try it:
//
// go run ./taskapi
// curl -s localhost:8080/tasks
// curl -s -XPOST localhost:8080/tasks -d '{"title":"buy milk"}'
// curl -s localhost:8080/tasks/1
// curl -s -XDELETE localhost:8080/tasks/1 -i
```

```
package main
```

```
import (
    "cmp"
```

```

"context"
"encoding/json"
"errors"
"fmt"
"log"
"net/http"
"os"
"os/signal"
"slices"
"strconv"
"strings"
"sync"
"syscall"
"time"
)

// Task is one to-do item. The struct tags control the JSON field names; only
// exported (capitalized) fields are encoded, so every field we want in the
// output must start with an uppercase letter.
type Task struct {
    ID      int64  `json:"id"`
    Title   string `json:"title"`
    Done    bool   `json:"done"`
}

```

The store: shared state behind a mutex

Here is the most important concurrency point in the whole project, and the one C programmers must internalize about Go web servers:

Watch out. `net/http` serves **each request in its own goroutine**. Two requests can run your handlers at the exact same time. Any state they share — here, the map of tasks — must be protected, or you have a data race. A race in Go is just as undefined as a race in C.

The `store` wraps a map and a counter with a `sync.Mutex` (see Chapter 15 — Sync and Context). Each method locks, does its work, and unlocks with `defer`. The mutex guarantees only one goroutine touches the map at a time.

```

// store is an in-memory set of tasks guarded by a mutex. Every HTTP handler
// runs in its own goroutine, so two requests can hit the store at the same
// time. The mutex makes each operation safe: only one goroutine touches the
// map at once.
type store struct {
    mu      sync.Mutex
    nextID  int64
    tasks   map[int64]Task
}

```

```

func newStore() *store {
    return &store{tasks: make(map[int64]Task), nextID: 1}
}

// list returns all tasks sorted by ID. Map iteration order is random in Go, so
// we sort to give callers a stable, predictable order.
func (s *store) list() []Task {
    s.mu.Lock()
    defer s.mu.Unlock()
    out := make([]Task, 0, len(s.tasks))
    for _, t := range s.tasks {
        out = append(out, t)
    }
    slices.SortFunc(out, func(a, b Task) int { return cmp.Compare(a.ID, b.ID) })
    return out
}

func (s *store) create(title string) Task {
    s.mu.Lock()
    defer s.mu.Unlock()
    t := Task{ID: s.nextID, Title: title}
    s.tasks[t.ID] = t
    s.nextID++
    return t
}

func (s *store) get(id int64) (Task, bool) {
    s.mu.Lock()
    defer s.mu.Unlock()
    t, ok := s.tasks[id]
    return t, ok
}

func (s *store) delete(id int64) bool {
    s.mu.Lock()
    defer s.mu.Unlock()
    if _, ok := s.tasks[id]; !ok {
        return false
    }
    delete(s.tasks, id)
    return true
}

```

Two Go details a C programmer should note:

- **Map iteration order is random** (see Chapter 9 — Maps). To return a stable list, we copy the values into a slice and sort with `slices.SortFunc` using `cmp.Compare`, the modern, overflow-safe way to compare two ordered values.
- **get returns (Task, bool)** — the “comma, ok” idiom. The boolean says whether the id

existed, which is cleaner than returning a sentinel value or a null pointer as you might in C.

Handlers and JSON helpers

We give the handlers access to the store by making them **methods on an app struct**. This avoids global variables and is the idiomatic way to inject dependencies (see Chapter 25 — Idioms and Style). Two small helpers, `writeJSON` and `writeError`, keep every response consistent.

```
// app bundles the dependencies the handlers need. Methods on *app become our
// HTTP handlers, which is a clean way to give them access to the store without
// global variables.
type app struct {
    store *store
}

// writeJSON sets the content type, writes the status line, and encodes v. Once
// WriteHeader has run the status is already sent, so an encode failure can only
// be logged, not turned into a different HTTP error.
func writeJSON(w http.ResponseWriter, status int, v any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    if err := json.NewEncoder(w).Encode(v); err != nil {
        log.Printf("write json: %v", err)
    }
}

// writeError sends a JSON object like {"error":"..."} with the given status.
func writeError(w http.ResponseWriter, status int, msg string) {
    writeJSON(w, status, map[string]string{"error": msg})
}

// parseID reads the {id} path segment and converts it to an int64.
func parseID(r *http.Request) (int64, error) {
    return strconv.ParseInt(r.PathValue("id"), 10, 64)
}

func (a *app) listTasks(w http.ResponseWriter, r *http.Request) {
    writeJSON(w, http.StatusOK, a.store.list())
}

func (a *app) createTask(w http.ResponseWriter, r *http.Request) {
    var in struct {
        Title string `json:"title"`
    }

    // MaxBytesReader caps the body so a huge request cannot exhaust memory.
    // DisallowUnknownFields rejects bodies with stray fields.
    dec := json.NewDecoder(http.MaxBytesReader(w, r.Body, 1<<20))
    dec.DisallowUnknownFields()
```

```

    if err := dec.Decode(&in); err != nil {
        writeError(w, http.StatusBadRequest, "invalid JSON body: "+err.Error())
        return
    }
    if strings.TrimSpace(in.Title) == "" {
        writeError(w, http.StatusBadRequest, "title is required")
        return
    }
    t := a.store.create(in.Title)
    w.Header().Set("Location", fmt.Sprintf("/tasks/%d", t.ID))
    writeJSON(w, http.StatusCreated, t)
}

func (a *app) getTask(w http.ResponseWriter, r *http.Request) {
    id, err := parseID(r)
    if err != nil {
        writeError(w, http.StatusBadRequest, "id must be an integer")
        return
    }
    t, ok := a.store.get(id)
    if !ok {
        writeError(w, http.StatusNotFound, "no task with that id")
        return
    }
    writeJSON(w, http.StatusOK, t)
}

func (a *app) deleteTask(w http.ResponseWriter, r *http.Request) {
    id, err := parseID(r)
    if err != nil {
        writeError(w, http.StatusBadRequest, "id must be an integer")
        return
    }
    if !a.store.delete(id) {
        writeError(w, http.StatusNotFound, "no task with that id")
        return
    }
    w.WriteHeader(http.StatusNoContent)
}

```

A few things worth highlighting:

- `r.PathValue("id")` reads the `{id}` wildcard from the route pattern. This is the Go 1.22 router feature: the path variable is parsed for you, so you do not split the URL by hand.
- `json.NewDecoder(...).Decode(&in)` streams and parses the request body into a struct. `http.MaxBytesReader` caps the body size so a malicious client cannot make you allocate gigabytes, and `DisallowUnknownFields` rejects bodies that carry fields you did not expect.
- **Validation returns a clear 400.** An empty title or a non-numeric id produces a JSON error and the right status code, never a panic.

Middleware: logging every request

Middleware is a function that wraps a handler and returns a new handler. It runs before and after the inner handler. Ours logs one line per request: the method, the path, the status code, and how long the handler took.

To see the status code, we wrap `http.ResponseWriter` in a tiny type that records whatever `WriteHeader` is called with. This is **embedding** (see Chapter 10 — Structs and Methods): `statusWriter` gets all the `ResponseWriter` methods for free and overrides just `WriteHeader`.

```
// statusWriter wraps http.ResponseWriter so the logging middleware can see the
// status code that a handler chose. ResponseWriter has no getter for it, so we
// record it as it passes through WriteHeader.
type statusWriter struct {
    http.ResponseWriter
    status int
}

func (w *statusWriter) WriteHeader(code int) {
    w.status = code
    w.ResponseWriter.WriteHeader(code)
}

// logging is middleware: it takes a handler and returns a new handler that logs
// one line per request after the inner handler runs.
func logging(h http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        sw := &statusWriter{ResponseWriter: w, status: http.StatusOK}
        h.ServeHTTP(sw, r)
        log.Printf("%s %s -> %d (%s)", r.Method, r.URL.Path, sw.status, time.Since(start))
    })
}
```

Mental model. Middleware is like a Russian nesting doll of handlers. The outermost wrapper runs first on the way in and last on the way out. You can stack several (logging, authentication, compression) by wrapping again and again. Each layer satisfies the same `http.Handler` interface, so they compose freely (see Chapter 11 — Interfaces).

main: routes, timeouts, and graceful shutdown

`main` registers the routes on a `ServeMux`, wraps it in the logging middleware, configures an `http.Server` with timeouts, and then runs the server while waiting for a shutdown signal.

```
func main() {
    app := &app{store: newStore()}

    // The patterns include the HTTP method and a named wildcard {id}. The
    // router (Go 1.22+) matches on both, so we do not parse the method by hand.
    mux := http.NewServeMux()
```

```

mux.HandleFunc("GET /tasks", app.listTasks)
mux.HandleFunc("POST /tasks", app.createTask)
mux.HandleFunc("GET /tasks/{id}", app.getTask)
mux.HandleFunc("DELETE /tasks/{id}", app.deleteTask)

srv := &http.Server{
    Addr:      ":8080",
    Handler:   logging(mux),
    // Timeouts protect the server from slow or stuck clients. Without them
    // a single client could hold a connection open forever.
    ReadHeaderTimeout: 5 * time.Second,
    ReadTimeout:       10 * time.Second,
    WriteTimeout:      10 * time.Second,
    IdleTimeout:       60 * time.Second,
}

// NotifyContext returns a context that is canceled when the process gets
// SIGINT (Ctrl-C) or SIGTERM. That cancellation is our shutdown trigger.
ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
defer stop()

// Run the server in its own goroutine so main can wait for the signal. The
// channel is buffered so this goroutine never blocks even if nobody reads.
errCh := make(chan error, 1)
go func() {
    log.Printf("listening on %s", srv.Addr)
    errCh <- srv.ListenAndServe()
}()

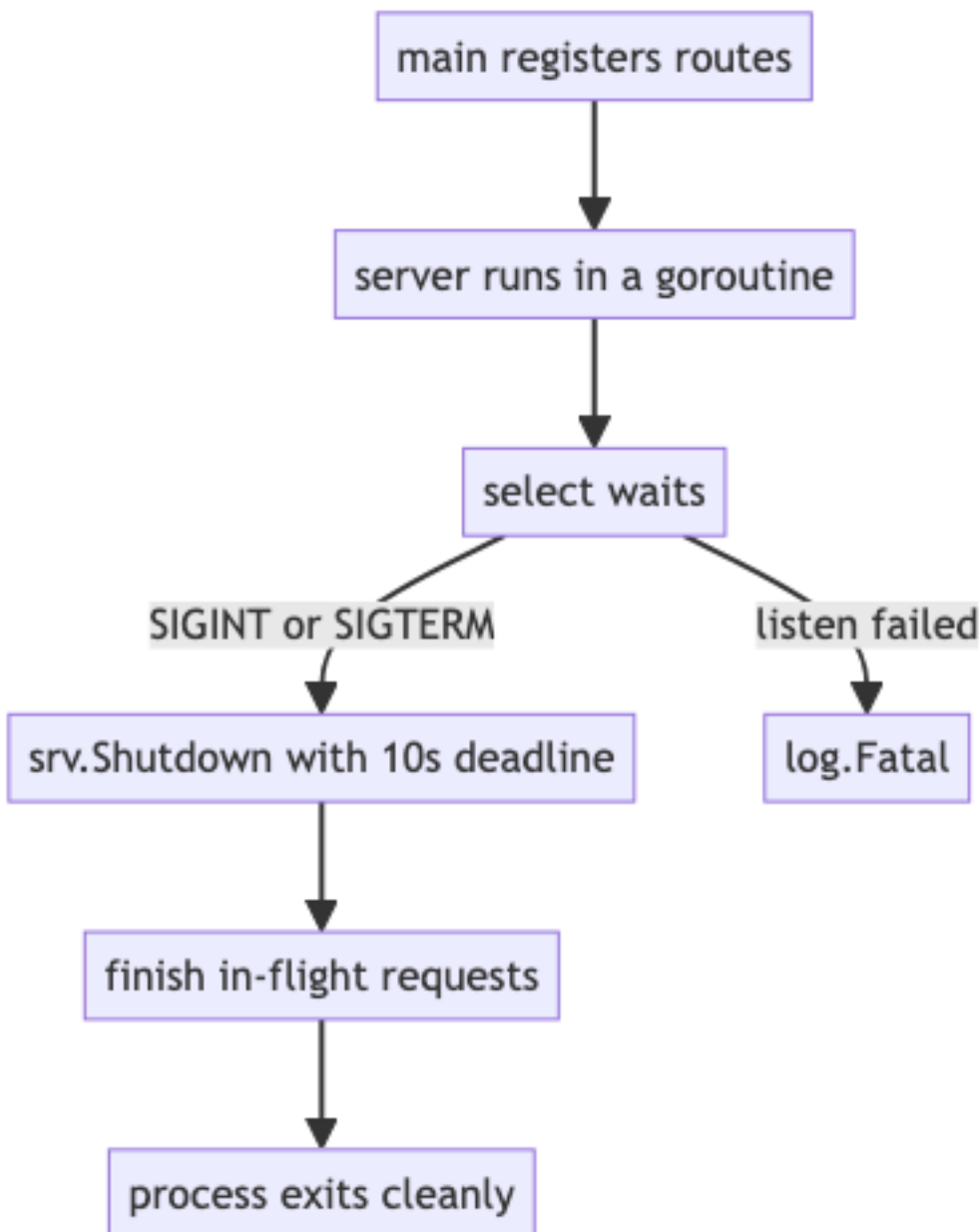
select {
case err := <-errCh:
    // ListenAndServe failed to even start (for example, port in use).
    if err != nil && !errors.Is(err, http.ErrServerClosed) {
        log.Fatalf("server error: %v", err)
    }
case <-ctx.Done():
    log.Println("shutdown signal received")
}

// Give in-flight requests up to 10 seconds to finish, then stop. Shutdown
// makes ListenAndServe above return http.ErrServerClosed, the clean case.
shutdownCtx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
defer cancel()
if err := srv.Shutdown(shutdownCtx); err != nil {
    log.Printf("graceful shutdown failed: %v", err)
    return
}
log.Println("server stopped cleanly")

```

}

This is the standard, production-shaped server loop. Trace the control flow:



The pieces that make it correct:

- **Timeouts.** `ReadHeaderTimeout`, `ReadTimeout`, `WriteTimeout`, and `IdleTimeout` stop a slow client from holding a connection open forever. The default `http.Server` has none of these; always set them.
- **`signal.NotifyContext`.** This turns an OS signal into a canceled `context.Context` (see Chapter 15 — Sync and Context). When you press Ctrl-C, `ctx.Done()` fires and the `select` wakes up. C handles signals with `signal(2)` and a handler function; Go funnels them into

the same context and channel machinery you already use elsewhere.

- **srv.Shutdown.** It stops accepting new connections and waits for in-flight requests to finish, up to the deadline. This is *graceful*: no request is cut off mid-response.
- **The buffered error channel.** `errCh` has capacity 1 so the server goroutine can always send its final error and exit, even after we have moved on to shutdown. This prevents a goroutine leak.

C vs Go. In C you would install a `SIGTERM` handler, set a global `volatile sig_atomic_t` flag, and poll it in your accept loop. Go gives you the same result with a context that integrates with `select`, channels, and every standard-library call that takes a `context.Context`. The shutdown signal flows through the same plumbing as request cancellation and timeouts.

Run it

Start the server in one terminal:

```
cd examples
go run ./taskapi
# 2026/06/13 01:25:50 listening on :8080
```

Drive it from another terminal with `curl`:

```
curl -s localhost:8080/tasks
# []
```

```
curl -s -i -XPOST localhost:8080/tasks -d '{"title":"buy milk"}'
# HTTP/1.1 201 Created
# Location: /tasks/1
# {"id":1,"title":"buy milk","done":false}
```

```
curl -s localhost:8080/tasks/1
# {"id":1,"title":"buy milk","done":false}
```

```
curl -s localhost:8080/tasks/999
# {"error":"no task with that id"} (HTTP 404)
```

```
curl -s -XDELETE localhost:8080/tasks/1 -i
# HTTP/1.1 204 No Content
```

```
curl -s -XPOST localhost:8080/tasks -d '{"oops":true}'
# {"error":"invalid JSON body: json: unknown field \"oops\""} (HTTP 400)
```

The server prints one log line per request, then shuts down cleanly on Ctrl-C:

```
2026/06/13 01:26:41 POST /tasks -> 201 (34.083µs)
2026/06/13 01:26:41 GET /tasks/999 -> 404 (6.625µs)
2026/06/13 01:26:41 DELETE /tasks/1 -> 204 (1.167µs)
^C
2026/06/13 01:27:55 shutdown signal received
2026/06/13 01:27:55 server stopped cleanly
```

Concepts it exercises

- Chapter 3 — Program Structure: Packages, Imports, and Visibility: exported fields and JSON.
- Chapter 9 — Maps: the in-memory store and random iteration order.
- Chapter 10 — Structs and Methods: methods on `*app` and `*store`, embedding.
- Chapter 11 — Interfaces: `http.Handler`, `http.ResponseWriter`, middleware.
- Chapter 12 — Errors: `errors.Is` with `http.ErrServerClosed`.
- Chapter 13 — Goroutines and the Scheduler: one goroutine per request.
- Chapter 15 — Sync and Context: `sync.Mutex`, `context`, `signal.NotifyContext`.
- Chapter 20 — The Standard Library Tour: `encoding/json`, `net/http`, `log`.
- Chapter 23 — Web Services: routing, middleware, timeouts, graceful shutdown.

Extend it

- Add PUT `/tasks/{id}` to update a task's title or `done` flag.
- Swap `sync.Mutex` for `sync.RWMutex` so reads can run in parallel, and measure the difference under load (see Chapter 15 — Sync and Context).
- Replace the map with a SQLite or Postgres store behind the same method set, so the handlers do not change.
- Add a `/healthz` route and a request-id middleware that tags each log line.
- Write tests with `net/http/httptest` to call the handlers without a real network (see Chapter 21 — Testing).

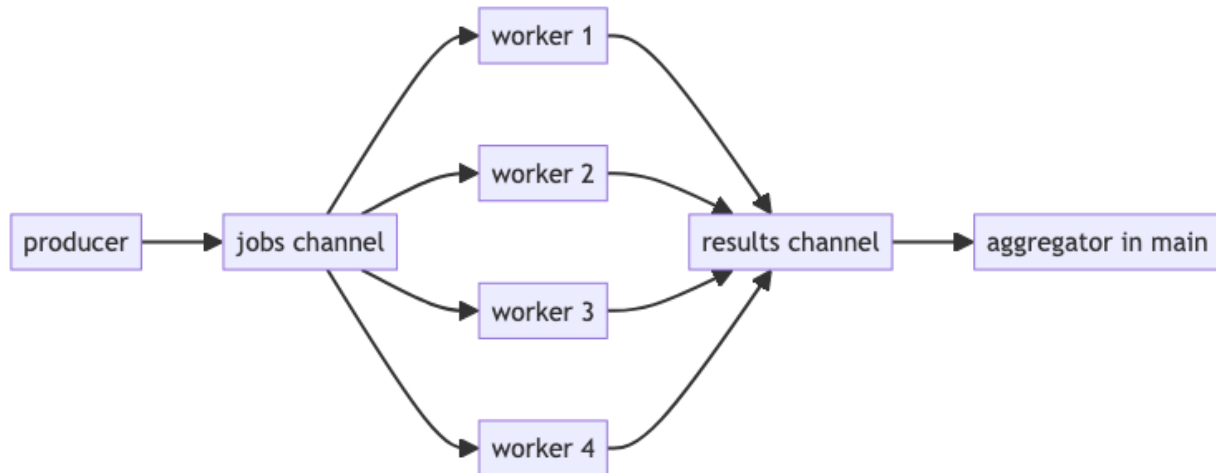
Project 3 — a bounded concurrent fetcher

Goal. Build `workerpool`, a program that runs many jobs through a **fixed, bounded** number of workers. Each job fetches one URL. The whole batch shares one deadline through a **context**. Results are aggregated at the end. The program terminates cleanly with no leaked goroutines and passes the race detector.

To keep the program **offline and deterministic**, it does not hit the real network. It starts an in-process test server with `net/http/httptest`, then fetches from that. The program therefore runs the same way on any machine, with no internet and no third-party modules.

Deep dive. In real code you might reach for `golang.org/x/sync/errgroup`, which combines a `WaitGroup` with shared error handling and context cancellation (see Chapter 16 — Concurrency Patterns). We build the pool by hand here, with only the standard library, so every moving part is visible. Once you understand this version, `errgroup` is a short step.

A worker pool is a **fan-out, fan-in** pattern. One producer fans work *out* to many workers over a jobs channel; the workers fan their results back *in* over a results channel; one aggregator reads them:



Mental model. Think of a fixed crew of workers and a conveyor belt of tasks. No matter how many tasks pile up, only the crew size determines how many run at once. That bound is the whole point: it caps memory, open sockets, and load on whatever you are calling. The unbounded alternative — `go fetch(url)` for every URL — can launch ten thousand requests at once and fall over.

Types and imports

```

// Command workerpool runs many jobs through a fixed, bounded set of workers.
// Each job fetches one URL. To stay fully offline and deterministic, the URLs
// point at an in-process test server (net/http/httptest) started by the program
// itself, so there is no real network and the run is repeatable on any machine.
//
// It demonstrates a bounded worker pool, a context timeout that cancels all
// work at once, result aggregation, and clean termination with no leaked
// goroutines. Run it under the race detector to confirm it is data-race free:
//
// go run -race ./workerpool
package main

import (
    "cmp"
    "context"
    "fmt"
    "io"
    "log"
    "net/http"
    "net/http/httptest"
    "slices"
    "sync"
    "time"
)

```

```

// Job is one unit of work: fetch one URL.
type Job struct {
    ID    int
    URL   string
}

// Result is the outcome of one Job. Err is nil on success.
type Result struct {
    JobID    int
    Status   int
    Bytes    int
    Err      error
}

```

Each `Result` carries its own `Err` field. This is the concurrent way to handle errors: a worker never stops the others, it just records what happened and the aggregator decides what to do (see Chapter 12 — Errors).

The job: a context-aware HTTP GET

`fetch` performs one request that respects the context. If the batch deadline passes or the context is canceled, the in-flight request is aborted and `Do` returns an error.

```

// fetch performs one HTTP GET that respects the context. If the context is
// canceled or its deadline passes, the in-flight request is aborted and Do
// returns an error. It reports the status code and how many body bytes arrived.
func fetch(ctx context.Context, client *http.Client, job Job) Result {
    req, err := http.NewRequestWithContext(ctx, http.MethodGet, job.URL, nil)
    if err != nil {
        return Result{JobID: job.ID, Err: err}
    }
    resp, err := client.Do(req)
    if err != nil {
        return Result{JobID: job.ID, Err: err}
    }
    defer resp.Body.Close()
    n, err := io.Copy(io.Discard, resp.Body)
    if err != nil {
        return Result{JobID: job.ID, Status: resp.StatusCode, Err: err}
    }
    return Result{JobID: job.ID, Status: resp.StatusCode, Bytes: int(n)}
}

```

Watch out. Always pass a context into outbound requests with `http.NewRequestWithContext`, and always `defer resp.Body.Close()`. Forgetting to close the body leaks the connection — the Go equivalent of forgetting to `close(fd)` in C. We discard the body with `io.Copy(io.Discard, ...)` because we only want its size, but we still must read and close it.

The worker: pull, do, send, and never block forever

A worker loops: take a job, do it, send the result. The two `select` statements are the key to **no goroutine leaks**. A worker can always make progress or return; it can never get stuck waiting on a channel that will never be served.

```
// worker pulls jobs until the jobs channel is closed or the context is  
// canceled, sends a Result for each, and then returns. Selecting on ctx.Done in  
// both the receive and the send guarantees the worker never blocks forever, so  
// it cannot leak.
```

```
func worker(ctx context.Context, client *http.Client, jobs <-chan Job, results chan<- Result, wg sync.WaitGroup) {  
    defer wg.Done()  
    for {  
        select {  
        case <-ctx.Done():  
            return  
        case job, ok := <-jobs:  
            if !ok {  
                return // jobs channel closed: no more work  
            }  
            res := fetch(ctx, client, job)  
            select {  
            case results <- res:  
            case <-ctx.Done():  
                return  
            }  
        }  
    }  
}
```

Read the two `selects` carefully (see Chapter 14 — Channels and Select):

- The **outer select** waits for either a cancellation or the next job. The `job, ok := <-jobs` form detects a *closed* channel: when the producer closes `jobs`, `ok` becomes `false` and the worker returns.
- The **inner select** sends the result, but also watches `ctx.Done()`. Without it, a canceled batch could leave a worker blocked forever trying to send into a channel that the aggregator has stopped reading. That stuck goroutine would be a leak.

C vs Go. The channel directions in the signature — `jobs <-chan Job` (receive only) and `results chan<- Result` (send only) — are checked by the compiler. A worker that tried to send into `jobs` would not compile. C has no way to express “this thread may only read from this queue”; Go bakes it into the type.

main: fan out, fan in, aggregate

`main` starts the test server, builds the job list, creates the channels, starts a fixed number of workers, feeds the jobs, and aggregates the results. Three helper goroutines keep the data flowing: the workers, a producer, and a closer.


```

func main() {
    // In-process server: the whole program is self-contained and offline. The
    // handler writes a short, fixed body so every run produces the same totals.
    srv := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintf(w, "result for %s\n", r.URL.Path)
    }))
    defer srv.Close()

    const (
        numJobs      = 20
        numWorkers = 4 // the bound: at most this many requests are ever in flight
    )

    jobList := make([]Job, numJobs)
    for i := range numJobs {
        jobList[i] = Job{ID: i, URL: fmt.Sprintf("%s/work/%d", srv.URL, i)}
    }

    // One deadline for the whole batch. If the work ran long, ctx would be
    // canceled and every worker and the producer would stop promptly.
    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()

    // A shared client is safe for concurrent use by many goroutines.
    client := &http.Client{Timeout: 2 * time.Second}

    jobs := make(chan Job)
    results := make(chan Result)

    // Start a fixed number of workers. This is what makes the pool bounded:
    // adding more jobs does not add more concurrency.
    var wg sync.WaitGroup
    for range numWorkers {
        wg.Add(1)
        go worker(ctx, client, jobs, results, &wg)
    }

    // Producer: feed every job, then close the jobs channel. Selecting on
    // ctx.Done means a cancellation cannot leave this goroutine stuck sending.
    go func() {
        defer close(jobs)
        for _, job := range jobList {
            select {
            case jobs <- job:
            case <-ctx.Done():
                return
            }
        }
    }
}

```

```

}()

// Closer: when every worker has returned, close results so the range below
// ends cleanly. This is the standard fan-in finish.
go func() {
    wg.Wait()
    close(results)
}()

// Aggregate. Results arrive in whatever order workers finish, so collect
// them first and sort by job ID for stable, repeatable output.
var collected []Result
for res := range results {
    collected = append(collected, res)
}
slices.SortFunc(collected, func(a, b Result) int { return cmp.Compare(a.JobID, b.JobID) })

var ok, failed, totalBytes int
for _, res := range collected {
    if res.Err != nil {
        failed++
        log.Printf("job %d failed: %v", res.JobID, res.Err)
        continue
    }
    ok++
    totalBytes += res.Bytes
}
fmt.Printf("done: %d ok, %d failed, %d jobs total, %d bytes\n", ok, failed, len(collected))
if err := ctx.Err(); err != nil {
    fmt.Printf("note: context ended early: %v\n", err)
}
}

```

The shutdown choreography is the part worth memorizing, because it is the same in every Go pipeline (see Chapter 16 — Concurrency Patterns):

1. The **producer** sends all jobs, then `close(jobs)`.
2. Each **worker** sees the closed `jobs` channel, finishes its last job, and returns, which runs `wg.Done()`.
3. The **closer** goroutine is blocked in `wg.Wait()`; when the last worker returns, it wakes and runs `close(results)`.
4. The **aggregator** for `res := range results` ends because `results` is now closed, and `main` returns.

Watch out. The rule is: **the sender closes a channel, never the receiver, and only once.** Here the producer owns `jobs` and the closer owns `results`. If two goroutines closed the same channel, or a worker tried to close `results` while another was still sending, the program would panic. Closing is how you say “no more values are coming,” which is exactly what ends a `range` over a channel.

Because every goroutine has a guaranteed exit — workers on closed `jobs` or `ctx.Done`, the producer on the last job or `ctx.Done`, the closer after `wg.Wait` — there are **no leaks**. The race detector confirms there are no data races either, because the only shared state is passed through channels and the `WaitGroup`.

Run it

```
cd examples
go run ./workerpool
# done: 20 ok, 0 failed, 20 jobs total, 390 bytes

go run -race ./workerpool
# done: 20 ok, 0 failed, 20 jobs total, 390 bytes
```

The output is the same every run: 20 jobs succeed, none fail, and the total body size is fixed because the test server returns a deterministic response. The `-race` build runs the same program with the race detector turned on (see Chapter 22 — Tooling); a clean exit means no goroutine wrote shared memory without synchronization.

Rule of thumb. Run anything concurrent with `go test -race` and `go run -race` during development. The detector finds real bugs that are otherwise invisible until production, and it has almost no false positives. C has nothing this good built in; the closest tools are ThreadSanitizer and Helgrind, which Go’s detector is in fact derived from.

Concepts it exercises

- Chapter 11 — Interfaces: `io.Reader`/`io.Writer`, `http.Handler`.
- Chapter 12 — Errors: per-job errors carried in the `Result`.
- Chapter 13 — Goroutines and the Scheduler: workers, producer, closer.
- Chapter 14 — Channels and Select: directioned channels, `select`, closing.
- Chapter 15 — Sync and Context: `sync.WaitGroup`, `context.WithTimeout`.
- Chapter 16 — Concurrency Patterns: bounded worker pool, fan-out/fan-in.
- Chapter 20 — The Standard Library Tour: `net/http`, `net/http/httptest`, `io`.
- Chapter 22 — Tooling: the `-race` detector.

Extend it

- Make the test server fail or stall for some paths, and confirm the aggregator reports the failures without crashing.
- Add a small random delay in the handler and a short context deadline, and watch the `note: context ended early` message appear as some jobs are canceled.
- Replace `numWorkers` with `runtime.NumCPU()` for CPU-bound work, or make it a `-workers` flag (see Chapter 24 — CLI Tools).
- Rewrite the pool using golang.org/x/sync/errgroup and compare the two; notice how much of the bookkeeping it removes (see Chapter 16 — Concurrency Patterns).
- Swap the HTTP job for a pure computation (for example, counting primes) to see the pool saturate every core.

Key takeaways

- Real Go programs are built from the same small pieces you learned one chapter at a time: interfaces like `io.Reader`, error values, structs with methods, goroutines, channels, and the standard library.
- A good CLI tool reads from `io.Reader` sources, writes normal output to `stdout` and errors to `stderr`, and signals failure with `os.Exit` — no framework needed.
- A production-shaped HTTP service fits in one file: the Go 1.22 router matches on method and path, JSON encode/decode is built in, a mutex protects shared state, server timeouts and `srv.Shutdown` make it robust, and `signal.NotifyContext` turns a signal into a context cancellation.
- A bounded worker pool caps concurrency with a fixed number of workers, shares one deadline through a context, aggregates results, and terminates with no leaked goroutines because every goroutine has a guaranteed exit.
- The standard library alone is enough to build all three. You can ship each as a single static binary with no dependencies to install.

Watch out (gotchas for C programmers)

- **HTTP handlers run concurrently.** `net/http` serves each request in its own goroutine; shared state needs a mutex or you have a data race, which is just as undefined as in C.
- **A byte is not a character.** In the line counter we test ASCII whitespace bytes on purpose; calling a rune function on a single byte misreads UTF-8.
- **Close a channel from the sender, exactly once.** Closing from a receiver, or twice, panics. Closing is the signal that ends a `range` over a channel.
- **Watch for goroutine leaks.** A goroutine blocked forever on a channel never dies and never gets garbage collected. Give every goroutine an exit, usually by selecting on `ctx.Done()` alongside the channel operation.
- **Always set server timeouts and close response bodies.** The zero-value `http.Server` has no timeouts, and an unclosed `resp.Body` leaks a connection.
- **main has no return value.** Use `os.Exit` for status codes, but remember that `os.Exit` skips defers — flush and close before you call it.

Interview questions

Q: An HTTP handler in Go reads and writes a shared map. What is wrong and how do you fix it? A: Each request runs in its own goroutine, so two requests can touch the map at the same time, which is a data race (and concurrent map access can crash the program). Protect the map with a `sync.Mutex` or `sync.RWMutex`, or move the state behind a single owning goroutine that you talk to over channels. Run tests with `-race` to catch the problem.

Q: How do you shut down an HTTP server without dropping in-flight requests? A: Listen for the OS signal, usually with `signal.NotifyContext`, which cancels a context on `SIGINT` or `SIGTERM`. When that context is done, call `srv.Shutdown(ctx)` with a bounded deadline. `Shutdown` stops accepting new connections and waits for active requests to finish, while `ListenAndServe` returns `http.ErrServerClosed`, which you treat as the clean case.

Q: What makes a worker pool “bounded,” and why does that matter? A: A fixed number of worker goroutines read from a shared jobs channel, so at most that many jobs run at once no matter how many are queued. It matters because launching one goroutine per job can

open thousands of sockets or allocate huge amounts of memory and overload whatever you call. The bound caps resource use and gives you predictable load.

Q: In a channel-based pipeline, who closes a channel, and what happens if you get it wrong? A: The sender closes the channel, and only once, to signal that no more values are coming; that is what ends a `for range` over the channel. Closing from a receiver, closing twice, or sending after close all cause a panic. When several goroutines send, use a `sync.WaitGroup` and a single closer goroutine that waits for all senders to finish, then closes.

Q: How do you make sure a concurrent program has no goroutine leaks? A: Give every goroutine a guaranteed way to exit. Workers should select on `ctx.Done()` as well as on their channels, both when receiving work and when sending results, so a cancellation can never strand them. Producers should select on `ctx.Done()` while sending. Confirm with the race detector and, if needed, by checking `runtime.NumGoroutine()` before and after the work.

Q: Why does the line counter take an `io.Reader` instead of a file path or `*os.File`? A: Depending on the smallest interface that does the job makes the function reusable and testable. `count` works with a file, with `os.Stdin`, with a `strings.Reader` in a unit test, or with a network connection — all of which satisfy `io.Reader`. This is the Go idiom “accept interfaces, return structs,” and it is far more flexible than tying the code to one concrete source.

Appendix A — C to Go Cheat Sheet

A fast lookup table for translating C into Go. Keep it open while you work. For the full story behind each row, follow the chapter pointers. Code is shown as `c` and `go`.

Types

C	Go	Note
<code>char</code>	<code>byte</code> (alias for <code>uint8</code>)	C <code>char</code> signedness is platform-defined; Go <code>byte</code> is unsigned.
<code>signed char</code>	<code>int8</code>	
<code>unsigned char</code>	<code>byte</code> / <code>uint8</code>	
<code>short</code> / <code>unsigned short</code>	<code>int16</code> / <code>uint16</code>	
<code>int</code> / <code>unsigned int</code>	<code>int32</code> / <code>uint32</code> , or Go <code>int</code>	C <code>int</code> is usually 32-bit; Go <code>int</code> is the machine word (32 or 64-bit).
<code>long</code> / <code>long long</code>	<code>int64</code> (pick a fixed size)	C widths vary by platform; Go's are fixed.
<code>unsigned long long</code>	<code>uint64</code>	
<code>size_t</code>	<code>int</code>	Go uses signed <code>int</code> for lengths and indices.
<code>ptrdiff_t</code>	<code>int</code>	
<code>intptr_t</code> / <code>uintptr_t</code>	<code>uintptr</code>	pointer-sized integer; rarely needed.
<code>float</code> / <code>double</code>	<code>float32</code> / <code>float64</code>	<code>float64</code> is Go's default float.

C	Go	Note
<code>_Bool / bool</code>	<code>bool</code>	not an integer; no implicit <code>int bool</code> .
<code>void *</code>	any or <code>unsafe.Pointer</code>	any for a type-safe value; <code>unsafe.Pointer</code> for raw bytes.
<code>T *</code>	<code>*T</code>	pointer; no pointer arithmetic .
<code>T name[N]</code>	<code>[N]T</code>	fixed array; a value (copied on assignment).
<code>T *</code> used as buffer	<code>[]T</code> (slice)	the everyday dynamic array (ptr, len, cap).
<code>struct {...}</code>	<code>struct {...}</code>	similar layout; Go adds methods and visibility.
<code>union {...}</code>	(none)	use a struct, an interface , or unsafe to overlay.
<code>enum {...}</code>	<code>type X int + const / iota</code>	a distinct named type.
<code>char * (text)</code>	<code>string</code> (immutable) / <code>[]byte</code> (mutable)	string is UTF-8 bytes.
code point	<code>rune</code> (alias for <code>int32</code>)	one Unicode character.
<code>int (*)(int)</code>	<code>func(int) int</code>	functions are first-class values.
<code>_Complex</code>	<code>complex64 / complex128</code>	built in.

Rule of thumb. Reach for the fixed-size type (`int32`, `uint64`) when you are matching a C ABI or a wire format; use plain `int` for ordinary counts and indices.

Declarations

Thing	C	Go
Variable, typed	<code>int x = 5;</code>	<code>var x int = 5</code>
Variable, inferred	<code>int x = 5;</code>	<code>x := 5</code> (inside a function)
Uninitialized	<code>int x; (garbage)</code>	<code>var x int</code> (zero value 0)
Multiple	<code>int a = 1, b = 2;</code>	<code>a, b := 1, 2</code>
Constant	<code>#define N 10 / const int N = 10;</code>	<code>const N = 10</code>
Enum group	<code>enum { A, B, C };</code>	<code>const (A = iota; B; C)</code>
Fixed array	<code>int a[10];</code>	<code>var a [10]int</code>
Array literal	<code>int a[] = {1, 2, 3};</code>	<code>a := [...]int{1, 2, 3}</code>
Slice literal	(manual)	<code>s := []int{1, 2, 3}</code>
Pointer	<code>int *p;</code>	<code>var p *int</code>
Address / deref	<code>&x / *p</code>	<code>&x / *p</code> (same)
Function	<code>int add(int a, int b) {...}</code>	<code>func add(a, b int) int {...}</code>
Function pointer	<code>int (*fp)(int, int);</code>	<code>var fp func(int, int) int</code>
Struct type	<code>struct P { int x, y; };</code>	<code>type P struct { X, Y int }</code>

Thing	C	Go
Distinct type	<code>typedef int Id;</code> (no teeth)	<code>type Id int</code> (no implicit conversion)
True alias	(same <code>typedef</code>)	<code>type Id = int</code> (Go 1.9+)
File-private global	<code>static int x;</code>	<code>var x int</code> (lowercase = unexported)
Function-local persistent	<code>static int n;</code> inside func	a package-level <code>var n int</code>
Declared elsewhere	<code>extern int x;</code>	<code>import</code> the package; use <code>pkg.X</code>
Public symbol	non- <code>static</code> in a header	Capitalized name (<code>Foo</code>)
Private symbol	<code>static</code>	lowercase name (<code>foo</code>)

See Chapter 3 — Program Structure: Packages, Imports, and Visibility and Chapter 4 — Types, Variables, and Constants.

Operators and keywords that differ

C	Go	Note
<code>p->field</code> <code>(*p).field</code> <code>cond ? a : b</code>	<code>p.field</code> <code>p.field</code> (none)	Go auto-dereferences; no <code>-></code> . same result. no ternary ; use <code>if/else</code> or a tiny helper.
<code>x++</code> , <code>++x</code> (as a value)	<code>x++</code> (a statement , not an expression)	no <code>++x</code> ; cannot use inside an expression.
<code>a, b</code> (comma operator)	(none)	use separate statements.
<code>if (x = f())</code> (assign in expr)	<code>if x := f(); x != 0 {}</code>	assignment is not an expression; use the init clause.
<code>(int)x</code> (cast)	<code>int(x)</code> (conversion)	the type is used like a function.
<code>&</code> , <code>\ </code> , <code>^</code> , <code><<</code> , <code>>></code>	same	<code>^</code> is also unary NOT.
<code>~x</code> (bitwise NOT)	<code>^x</code>	unary <code>^</code> .
(no operator)	<code>x &^ y</code>	bit clear (AND NOT).
<code>sizeof x</code>	<code>unsafe.Sizeof(x)</code>	see the Memory table.
<code>goto label;</code>	<code>goto label</code>	exists, but rare; prefer defer and labeled break .

Control flow

C	Go	Note
<code>while (c) { }</code> <code>do { } while (c);</code>	<code>for c { }</code> <code>for { ...; if !c { break } }</code>	one loop keyword. no <code>do/while</code> .
<code>for (i = 0; i < n; i++)</code>	<code>for i := 0; i < n; i++ { }</code>	no parentheses; braces required.
<code>count 0..n-1</code> <code>for (;;) </code>	<code>for i := range n { }</code> <code>for { }</code>	integer range, Go 1.22+. infinite loop.

C	Go	Note
walk an array <code>switch</code> falls through	<code>for i, v := range s { }</code> <code>switch</code> breaks after each case	index and value. use fallthrough to continue to the next case.
<code>switch</code> on integers only	<code>switch</code> on any comparable type	cases may be expressions; <code>switch { }</code> replaces an if/else chain.
<code>if (c) { }</code>	<code>if c { }</code>	braces required; init clause <code>if x := f(); c { }</code> .
<code>break / continue</code>	same, plus labels	break Outer, continue Outer.
<code>goto cleanup;</code>	<code>defer cleanup()</code>	the idiomatic replacement.

See Chapter 5 — Control Flow.

Memory

C	Go	Note
<code>malloc(sizeof *p)</code> <code>malloc(n * sizeof(T))</code> <code>calloc(n, sizeof(T))</code> <code>realloc(p, m)</code>	<code>p := new(T)</code> or <code>p := &T{}</code> <code>s := make([]T, n)</code> <code>make([]T, n)</code> <code>s = append(s, ...) /</code> <code>slices.Grow(s, m)</code> (nothing)	returns a zeroed <code>*T</code> . slice of <code>n</code> zero values. already zeroed (zero value). append grows the backing array.
<code>free(p)</code>		the garbage collector reclaims it.
<code>sizeof(T)</code> <code>sizeof(a)/sizeof(a[0])</code> choose stack vs heap	<code>unsafe.Sizeof(T{})</code> <code>len(a)</code> (compiler chooses)	size in bytes, at compile time. length is part of the value. escape analysis; returning &local is safe .
<code>memcpy(d, s, n)</code>	<code>copy(d, s)</code>	copies <code>min(len(d), len(s))</code> elements.
<code>memmove(d, s, n)</code> <code>memset(p, 0, n)</code>	<code>copy(d, s)</code> <code>clear(s)</code>	copy handles overlap. also empties a map; or rely on the zero value.
array decays to a pointer	slice keeps <code>len</code> and <code>cap</code>	no decay; access is bounds-checked.

See Chapter 7 — Pointers and Chapter 17 — Memory and the Garbage Collector.

Strings

C	Go	Note
<code>char *</code> / <code>char[]</code>	<code>string</code> (immutable) / <code>[]byte</code> (mutable)	<code>string</code> is UTF-8 bytes.

C	Go	Note
<code>strlen(s)</code>	<code>len(s)</code>	byte count, O(1); not the number of characters.
character count <code>s[i]</code>	<code>utf8.RuneCountInString(s)</code> <code>s[i]</code> (a byte)	counts runes (code points). one byte; for <code>_, r := range s</code> gives runes.
<code>strcmp(a, b) == 0</code> ordering	<code>a == b</code> <code>a < b, strings.Compare(a, b)</code>	<code>==</code> compares contents. lexical byte order.
<code>strcpy(d, s)</code>	<code>d = s / copy(dst, src)</code> for <code>[]byte</code>	string assignment is a cheap copy of the header.
<code>strcat(a, b)</code> build incrementally <code>strchr / strstr</code>	<code>a + b</code> <code>strings.Builder</code> <code>strings.IndexByte / strings.Index</code>	for loops, use a builder. avoids O(n ²) reallocation. returns index or -1.
<code>strtok</code>	<code>strings.Split / strings.Fields</code>	returns a <code>[]string</code> .
<code>toupper / case</code> <code>printf(fmt, ...)</code>	<code>strings.ToUpper</code> <code>fmt.Printf(fmt, ...)</code>	Unicode-aware. format verbs checked by <code>go vet</code> .
<code>sprintf(buf, ...)</code>	<code>s := fmt.Sprintf(...)</code>	returns a new string; no buffer sizing.
<code>snprintf(...)</code> <code>fprintf(stderr, ...)</code>	<code>fmt.Sprintf(...)</code> <code>fmt.Fprintf(os.Stderr, ...)</code>	no overflow to guard. writes to any <code>io.Writer</code> .
<code>puts(s)</code> <code>atoi(s) / strtol</code>	<code>fmt.Println(s)</code> <code>strconv.Atoi(s) / strconv.ParseInt</code>	adds a newline. returns (value, error).

See Chapter 8 — Arrays, Slices, and Strings.

C header to Go package

C header / function	Purpose	Go package(s)
<code><stdio.h></code>	print, files, buffered I/O	<code>fmt, os, bufio</code>
<code><stdlib.h> atoi, strtol</code>	parse numbers	<code>strconv</code>
<code><stdlib.h> malloc, free</code>	memory	built in (make, new) + the GC
<code><stdlib.h> qsort, bsearch</code>	sort, search	<code>sort, slices</code>
<code><stdlib.h> rand</code>	randomness	<code>math/rand/v2, crypto/rand</code>
<code><stdlib.h> getenv, exit</code>	environment, exit	<code>os.Getenv, os.Exit</code>
<code><string.h></code>	string and memory ops	<code>strings, bytes</code>
<code><ctype.h></code>	character classes	<code>unicode</code>
<code><math.h></code>	math functions	<code>math</code>
<code><time.h></code>	clocks and dates	<code>time</code>
<code><pthread.h></code>	threads, mutexes, condvars	<code>goroutines (go), sync, channels</code>

C header / function	Purpose	Go package(s)
<errno.h>	error codes	error values, errors (syscall.Errno for raw codes)
<assert.h>	assertions	none built in; use tests and panic
<signal.h>	signals	os/signal
<unistd.h>, <fcntl.h>	files, syscalls	os , io , syscall
<stdint.h>	fixed-width integers	built in (int32 , uint64 , ...)
<stdbool.h>	booleans	built in (bool)
<stdarg.h>	variadic args	variadic parameters ... T

See Chapter 20 — The Standard Library Tour.

Idioms

Error handling (return codes / **errno** → **error** values):

```
int fd = open(path, O_RDONLY);
if (fd < 0) { /* inspect errno */ return -1; }
/* use fd */
close(fd);

f, err := os.Open(path)
if err != nil {
    return err
}
defer f.Close()
// use f
```

Cleanup (goto cleanup → defer, which runs in last-in-first-out order):

```
FILE *a = fopen(pa, "r");
if (!a) goto done;
FILE *b = fopen(pb, "r");
if (!b) goto close_a;
/* work */
fclose(b);
close_a:
fclose(a);
done:
return;

a, err := os.Open(pa)
if err != nil {
    return err
}
defer a.Close()
b, err := os.Open(pb)
if err != nil {
```

```

    return err
}
defer b.Close()
// work; a and b close automatically, b before a

```

A struct of function pointers → an interface (a vtable you build by hand → one the compiler builds and type-checks):

```

typedef struct {
    int  (*Read)(void *self, char *buf, int n);
    int  (*Close)(void *self);
} Reader;

type Reader interface {
    Read(p []byte) (n int, err error)
    Close() error
}

```

Any type with `Read` and `Close` methods satisfies `Reader` automatically; there is no `implements` keyword and no manual wiring. See Chapter 11 — Interfaces.

The go command

Task	Command
Run the package here	<code>go run .</code>
Build a binary	<code>go build -o app .</code>
Build everything	<code>go build ./...</code>
Install a tool	<code>go install example.com/cmd@latest</code>
Run tests	<code>go test ./...</code>
Verbose tests	<code>go test -v ./...</code>
Tests with the race detector	<code>go test -race ./...</code>
Coverage	<code>go test -cover ./...</code>
Benchmarks	<code>go test -bench=. -benchmem</code>
Format in place	<code>gofmt -w .</code> (or <code>go fmt ./...</code>)
Static checks	<code>go vet ./...</code>
New module	<code>go mod init example.com/app</code>
Tidy dependencies	<code>go mod tidy</code>
Upgrade one dependency	<code>go get example.com/pkg@latest</code>
List dependencies	<code>go list -m all</code>
Read docs	<code>go doc fmt.Println</code>
Show settings	<code>go env</code>
Cross-compile	<code>GOOS=linux GOARCH=arm64 go build .</code>
Scan for vulnerabilities	<code>govulncheck ./...</code>

See Chapter 2 — Installing Go and the go Command.

Handy one-liners

```
b, err := os.ReadFile("f.txt")           // read a whole file
err = os.WriteFile("f.txt", b, 0o644)    // write a whole file
in, _ := io.ReadAll(os.Stdin)           // read all of stdin
parts := strings.Split(s, ",")           // split on a separator
s = strings.Join(parts, ",")             // join with a separator
n, err := strconv.Atoi("42")             // string -> int
s = strconv.Itoa(42)                     // int -> string
s = fmt.Sprintf("%s=%d", name, n)        // format into a string
slices.Sort(nums)                        // sort a slice in place
ok := slices.Contains(nums, x)           // membership test
keys := slices.Collect(maps.Keys(m))      // map keys -> slice (Go 1.23+)
time.Sleep(100 * time.Millisecond)      // pause
```

Appendix B — Glossary

Every important Go term in this book, defined in one to three plain sentences, with a C comparison where it helps. Terms are grouped by first letter.

B

backing array — The fixed-size array in memory that a slice points into. Several slices can share one backing array, which is why writing through one slice can change another. It is like the heap block a C dynamic array lives in, except the garbage collector owns its lifetime.

buffered channel — A channel with room for `n` values, created with `make(chan T, n)`. A send blocks only when the buffer is full; a receive blocks only when it is empty. Think of it as a fixed-size, thread-safe queue.

build cache — A local cache of compiled packages and test results that makes repeat builds fast. The `go` tool manages it; clear it with `go clean -cache`. A plain C/Make workflow has no built-in equivalent.

C

capacity — For a slice, the number of elements that fit in its backing array starting from the slice's first element; read it with `cap(s)`. `append` uses it to decide whether to grow in place or allocate a new array.

channel — A typed, thread-safe conduit that passes values between goroutines, created with `make(chan T)`. Send with `ch <- v` and receive with `v := <-ch`. It replaces the hand-built mutex-plus-condition-variable queue you would write in C.

closure — A function value that captures and keeps a reference to variables from the surrounding scope; those variables live as long as the closure does. C has function pointers but no closures, so you pass a separate `void *` context argument instead.

comparable — A built-in constraint matching types whose values work with `==` and `!=` (numbers, strings, booleans, pointers, channels, and structs or arrays of such). It is used as a generic constraint, for example for map key types.

constraint — In generics, an interface that limits which types a type parameter accepts and which operations are allowed, such as `[T comparable]` or `[T constraints.Ordered]`. It is the type-safe replacement for C’s untyped macros.

context — A `context.Context` value that carries a cancellation signal, a deadline, and request-scoped values across function calls. By convention it is the first argument, named `ctx`. It is the standard way to cancel a tree of goroutines.

D

deadlock — A state where goroutines are all blocked waiting on one another and none can proceed. If the entire program is stuck, the runtime detects it and aborts with a “deadlock” message, which a C pthread deadlock would not do.

defer — A statement that schedules a function call to run when the surrounding function returns, in last-in-first-out order. It is used for cleanup such as closing files or unlocking mutexes, and it replaces C’s `goto cleanup` pattern.

dynamic dispatch — Selecting which concrete method to call at run time based on the value stored in an interface. The interface holds a pointer to a method table, much like a C++ vtable or a C struct of function pointers.

E

embedding — Placing a type inside a struct or interface without a field name, so the outer type promotes the inner type’s fields and methods. It gives reuse by composition, not inheritance: there is no base class and no method overriding.

escape analysis — The compiler step that decides whether a variable can live on the stack or must “escape” to the heap. A value escapes if it is still reachable after its function returns. It is why returning a pointer to a local variable is safe in Go.

exported / unexported — An identifier starting with an uppercase letter is exported (visible outside its package), like a non-`static` symbol in a C header. A lowercase identifier is unexported (package-private), like `static`.

F

fat pointer — An informal name for a multi-word value carrying a pointer plus extra data. A slice is a fat pointer (pointer, length, capacity); an interface is a fat pointer (type, value). A C pointer is a single word.

G

garbage collector (GC) — The runtime component that automatically frees memory the program can no longer reach, so you never call `free`. Go’s is a concurrent, tri-color, mark-and-sweep collector with very short pauses; Go 1.26 enables the “Green Tea” design by default. It removes use-after-free and double-free bugs.

generics — Type parameters that let one function or type work for many types while staying type-safe, added in Go 1.18. They replace C’s `void *` containers and preprocessor macros, for example `func Max[T constraints.Ordered](a, b T) T`.

GMP — The Go scheduler’s model: G is a goroutine, M is a machine (an OS thread), and P is a processor (a scheduling context with a run queue). An M needs a P to run Gs, and the number of Ps is `GOMAXPROCS`.

gofmt — The standard tool that rewrites Go source into one official format (tabs, brace placement, spacing). Because everyone runs it, formatting is never debated and diffs stay small. C has no single official formatter.

GOMAXPROCS — The maximum number of OS threads that may run Go code at the same time, which sets real parallelism. It defaults to the number of logical CPUs; since Go 1.25 the runtime also honors Linux cgroup CPU limits.

GOPATH — An older environment variable naming the workspace directory for Go code and downloads. Since Go modules (Go 1.11+) it no longer controls project layout and mostly holds the module cache and installed tools.

GOROOT — The directory where the Go toolchain and standard library are installed. You rarely set it by hand; the installer and the `go` tool already know where it is.

goroutine — A function running concurrently, scheduled by the Go runtime rather than the OS, started with `go f()`. It begins with a tiny stack (about 2 KB) that grows on demand, so you can run hundreds of thousands of them. It is far cheaper than an OS thread.

H

happens-before — The ordering rule in the Go memory model that defines when one goroutine’s write is guaranteed visible to another goroutine’s read. If two accesses are not ordered this way and at least one writes, you have a data race. The C11 memory model has the same concept.

I

interface — A type that lists a set of method signatures; any type with those methods satisfies it automatically, with no `implements` keyword. At run time an interface value is a (type, value) pair. It is Go’s type-safe version of a C struct of function pointers.

iota — A counter usable inside a `const` block that starts at 0 and increases by one per line, used to build enumerations. `const ( A = iota; B; C )` yields 0, 1, 2. It replaces a C `enum` or a run of `#defines`.

M

method set — The set of methods callable on a type, which determines the interfaces it satisfies. The method set of `T` has value-receiver methods only; the method set of `*T` has both value- and pointer-receiver methods. This is why a pointer-receiver method requires `*T` to satisfy an interface.

module — A collection of related packages versioned and released together, defined by a `go.mod` file at its root. It is the unit of dependency management, roughly like a whole versioned library you depend on.

module cache — The local directory where downloaded module versions are stored and shared across projects, verified against `go.sum`. Clear it with `go clean -modcache`. C has no built-in dependency cache.

mutex — A mutual-exclusion lock from the `sync` package that lets only one goroutine touch shared data at a time. Its zero value is an unlocked, ready-to-use mutex, with no init call unlike `pthread_mutex_init`. Lock with `mu.Lock()` and `defer mu.Unlock()`.

MVS — Minimal Version Selection, the algorithm Go uses to pick dependency versions. It chooses the lowest version that satisfies all requirements, which keeps builds reproducible. Many other package managers instead pick the newest allowed version.

N

nil — The zero value of pointers, slices, maps, channels, functions, and interfaces, meaning “no value” or “not set”. It is like C’s `NULL`, but typed and broader: a `nil` slice is a valid empty slice you can append to, and a `nil` map is safe to read.

P

package — One directory of `.go` files compiled together, and the unit of imports and visibility; every file begins with `package name`. It replaces C’s header-plus-translation-unit model, so there are no `#includes`.

panic — A run-time error that unwinds the stack and crashes the program unless a deferred `recover` stops it. It is for programmer mistakes and impossible states (index out of range, nil dereference), not for ordinary errors. It resembles an unhandled exception or `abort`.

pointer — A value holding the address of another value, declared `*T`, taken with `&x`, and read with `*p`. Go has pointers but **no pointer arithmetic**, and the GC manages what they point to, so dangling pointers and manual frees do not happen.

R

race condition — A bug where two goroutines access the same memory without synchronization and at least one writes, giving undefined results. Find races by building or testing with the `-race` flag. It is the same hazard as unsynchronized pthreads in C.

receiver — The special parameter of a method, written before the method name, that the method acts on, as in `func (r Rectangle) Area() float64`. A value receiver gets a copy; a pointer receiver (`*Rectangle`) can modify the original. It is the explicit `self` or `this`.

recover — A built-in that stops a panic and returns its value, but only when called directly inside a deferred function. It turns a crash into a handled error at an API boundary; outside `defer` it does nothing.

rune — An alias for `int32` that holds one Unicode code point (one character). Ranging over a string yields runes, decoding the UTF-8 bytes. A C `char` is one byte, which is not one character for non-ASCII text.

S

select — A statement that waits on several channel operations and runs whichever is ready first, choosing at random among ties. A `default` case makes it non-blocking. It is like `select/poll` on file descriptors, but for channels.

semantic import versioning — Go’s rule that a module’s major version 2 or higher must appear in its import path, such as `example.com/lib/v2`. This lets two major versions of a library coexist in one build without conflict.

slice — Go’s dynamic array: a header (pointer, length, capacity) that views a backing array, written `[]T`. It grows with `append` and is passed by copying its small header. It is the everyday replacement for C’s manual `malloc/realloc` arrays.

slice header — The three-word value that represents a slice: a pointer to the backing array, an `int` length, and an `int` capacity. Passing a slice copies this header, not the elements it points to.

stop-the-world — A brief phase where the garbage collector pauses every goroutine to do work that needs a consistent view of memory. Modern Go keeps these pauses well under a millisecond by doing most GC work concurrently.

struct tag — A string literal attached to a struct field, written in backticks after the field, that carries metadata libraries read by reflection, such as a JSON field name like `json:"name"`. C has no equivalent; you would hand-write serialization code.

T

type assertion — Extracting the concrete value from an interface, as in `s, ok := x.(string)`, which succeeds if `x` holds a `string`. The `, ok` form avoids a panic when the type does not match. It is like a checked downcast.

type parameter — A placeholder type in a generic function or type, written in square brackets with a constraint, as in `func Map[T, U any](...)`. It is filled in with a real type at the call site.

type switch — A `switch` that branches on the concrete type inside an interface: `switch v := x.(type) { case int: ...; case string: ... }`. It is the idiomatic way to handle a value that could be one of several types.

V

vendoring — Copying a project’s dependencies into a local `vendor/` directory so builds use those copies instead of the module cache or the network. It helps with reproducible or offline builds, like checking third-party source into your repository.

vet — `go vet`, a static analysis tool that reports likely mistakes that still compile, such as wrong `Printf` verbs or misused struct tags. Run it in CI alongside the tests.

W

WaitGroup — A `sync.WaitGroup` counter that waits for a group of goroutines to finish: `Add` before starting them, `Done` (via `defer`) inside each, and `Wait` to block until the count reaches zero. It is like joining a group of threads.

work stealing — A scheduler technique where an idle processor (P) takes runnable goroutines from another P’s queue, balancing load across CPUs without central coordination.

Z

zero value — The defined default a variable receives when declared without an initializer: 0 for numbers, **false** for booleans, "" for strings, and **nil** for pointers, slices, maps, channels, and interfaces. There is no uninitialized garbage as in C, and good Go types make the zero value useful.

References and Further Reading

Where to go next. These are the resources this book draws on and the ones most useful to a C programmer learning Go. Links are to the official or canonical page; where no stable link exists, the resource is named so you can search for it.

Official documentation

- **The Go website** — <https://go.dev> — the home for downloads, documentation, and the blog. Start here.
- **A Tour of Go** — <https://go.dev/tour/> — an interactive, in-browser introduction. The fastest way to get hands-on with the syntax.
- **Effective Go** — https://go.dev/doc/effective_go — how to write idiomatic Go. Essential reading once the syntax feels comfortable.
- **The Go Programming Language Specification** — <https://go.dev/ref/spec> — the precise language definition. Short and readable, like the C standard but far smaller.
- **Standard library reference (pkg.go.dev)** — <https://pkg.go.dev> — searchable docs for the standard library and every public module. Your daily reference.
- **The Go Blog** — <https://go.dev/blog/> — official articles on language features, releases, and design decisions.
- **Go Code Review Comments** — <https://go.dev/wiki/CodeReviewComments> — a checklist of common review feedback. A concise list of “how Go is supposed to look.”
- **Google Go Style Guide** — <https://google.github.io/styleguide/go/> — Google’s internal style guide, made public. Opinionated and practical.
- **The Go Memory Model** — <https://go.dev/ref/mem> — the exact rules for when one goroutine’s writes are visible to another. The Go equivalent of the C11 memory model.
- **The Go FAQ** — <https://go.dev/doc/faq> — answers to “why does Go do it this way?”, including many questions a C or C++ programmer will ask.

Books

- **The Go Programming Language**, Alan A. Donovan and Brian W. Kernighan — <https://www.gopl.io> — the definitive book, often called “the K&R of Go.” Kernighan co-wrote *The C Programming Language*, so the style will feel familiar.
- **Learning Go**, Jon Bodner (O’Reilly) — a thorough, modern introduction that covers generics and recent versions. A good cover-to-cover first book.
- **Concurrency in Go**, Katherine Cox-Buday (O’Reilly) — a focused, deep treatment of goroutines, channels, and concurrency patterns.
- **100 Go Mistakes and How to Avoid Them**, Teiva Harsanyi (Manning) — a catalog of real traps and their fixes. Excellent for moving past “writing C in Go.”

Practice

- **Go by Example** — <https://gobyexample.com> — short, annotated programs for each language feature. Great for “show me the code” lookups.
- **Exercism Go track** — <https://exercism.org/tracks/go> — graded exercises with human mentorship, free.
- **Gophercises** — <https://gophercises.com> — small, practical coding projects that build real skills.

Tools

- **gopls** — <https://pkg.go.dev/golang.org/x/tools/gopls> — the official language server that powers editor features (completion, jump-to-definition, refactoring).
- **staticcheck** — <https://staticcheck.dev> — a powerful static analyzer that catches bugs and suspicious code beyond `go vet`.
- **golangci-lint** — <https://golangci-lint.run> — a fast runner that bundles many linters with one config. The common choice for CI.
- **Delve** — <https://github.com/go-delve/delve> — the Go debugger (`d1v`), with editor integrations. The rough equivalent of `gdb` for Go.
- **govulncheck** — <https://pkg.go.dev/golang.org/x/vuln/cmd/govulncheck> — scans your code and dependencies for known vulnerabilities, reporting only those you actually call.

Community

- **r/golang** — <https://www.reddit.com/r/golang/> — active discussion, news, and questions.
- **Gophers Slack** — <https://invite.slack.golangbridge.org> — a large community workspace with channels for beginners, specific packages, and more.
- **Go Time podcast** — <https://changelog.com/gotime> — interviews and discussion on Go practice, tooling, and community.

Especially for C programmers

These four pieces explain the parts of Go that surprise C programmers most. Read them early.

- **Go Slices: usage and internals** — <https://go.dev/blog/slices-intro> — what a slice is and how it sits on top of an array. Directly addresses the C “array vs pointer” mental model.
- **Arrays, slices (and strings): the mechanics of ‘append’** — <https://go.dev/blog/slices> — exactly how `append` grows a slice and why the aliasing traps happen. Read it before you trust `append`.
- **Share Memory By Communicating** — <https://go.dev/blog/codelab-share> — the idea behind channels, contrasted with the shared-memory-plus-locks model you know from `pthread`s.
- **Effective Go** — https://go.dev/doc/effective_go — the single best document for turning correct-but-C-like code into idiomatic Go.